



嵌入式 Linux
应用及驱动开发
Cortex-A53 FS6818 部分
(V1.1)

华清远见教育集团 • 研发中心



170-9108-5953



2306275952/2668462267



<http://dev.hqyj.com>



support@farsight.com.cn



前言

本书从嵌入式 Linux 的基础知识、系统环境搭建到综合应用，共分三个层次深入浅出地为读者拨开萦绕于嵌入式 Linux 这个概念的重重迷雾，引领读者渐渐步入嵌入式的世界，帮助探索者实实在在地把握第三次 IT 科技浪潮的方向。本书的特色如下：

(1) 重基础，适合教学。

(2) 重素质，全面讲解。本书在一般性教材的基础上，对嵌入式系统的软硬件开发环境进行了大量的讲解，可以让读者更进一步、更全面地了解嵌入式的开发过程。

(3) 重实践，与实际项目相结合。本书每个原理讲解都对应着丰富的实验内容，并且原理讲解和实验内容一一对应，使得每个知识都可以被深刻的理解运用，本书附带的光盘中给出了参考设计代码和文档。

(4) 重应用。书中的实例对时下经常使用的功能、设备、器材进行讲解和说明，力求教材所涉及的内容能紧跟行业实际应用的需要。

本书共分 19 个章节，每个章节下面包含一个对应此章节内容的实验。

第 1 章 嵌入式 Linux 主机开发环境搭建

第 2 章 嵌入式 Linux 主机调试环境搭建

第 3 章 目标机调试环境搭建

第 4 章 Linux 系统编译

第 5 章 Linux 基础操作

第 6 章 Linux Shell 编程

第 7 章 GCC 编译

第 8 章 Linux 系统 Makefile 编写实验

第 9 章 嵌入式文件 I/O 编程实验

第 10 章 嵌入式 Linux 多任务编程实验

第 11 章 Linux 系统进程间通信实验

第 12 章 嵌入式 Linux 多线程编程实验

第 13 章 嵌入式 Linux 网络编程实验

第 14 章 Linux 设备驱动开发



目 录

前 言	I
目 录	- 1 -
第 1 章 嵌入式 Linux 主机开发环境搭建	- 8 -
1.1 安装前准备	- 9 -
1.2 安装 VMware Player	- 11 -
1.3 运行开发环境	- 17 -
1.3.1 解压虚拟机镜像	- 17 -
1.3.2 打开虚拟机	- 19 -
1.3.3 配置优化虚拟机	- 22 -
1.3.4 启动虚拟机	- 27 -
1.3.5 设置 ROOT 密码	- 30 -
第 2 章 嵌入式 Linux 主机调试环境搭建	- 31 -
2.1 配置主机网络环境	- 31 -
2.1.1 实验目的	- 31 -
2.1.2 实验平台	- 31 -
2.1.3 实验原理	- 31 -
2.1.4 实验步骤	- 31 -
2.2 Linux 系统配置 NFS 实验	- 32 -
2.2.1 实验目的	- 32 -
2.2.2 实验平台	- 32 -
2.2.3 实验原理	- 32 -
2.2.4 实验步骤	- 32 -
2.3 交叉编译环境配置实验	- 33 -
2.3.1 实验目的	- 33 -
2.3.2 实验平台	- 33 -
2.3.3 实验原理	- 33 -
2.3.4 实验步骤	- 34 -
第 3 章 目标机调试环境搭建	- 35 -
3.1 连接开发平台	- 35 -
3.2 设置串口调试工具	- 36 -
3.2.1 错误处理	- 40 -
3.3 目标机启动 Linux 系统	- 40 -
3.3.1 通过 eMMC 方式启动 Linux	- 40 -



3.3.2	通过 NFS 挂载方式启动 Linux.....	- 43 -
第 4 章	Linux 系统编译	- 44 -
4.1	准备源码	- 44 -
4.1.1	拷贝内核源码	- 44 -
4.1.2	解压内核源码	- 44 -
4.1.3	拷贝 ext4 文件系统制作工具.....	- 44 -
4.2	编译源码	- 45 -
4.2.1	编译 Bootloader 源码.....	- 45 -
4.2.2	编译内核源码	- 45 -
4.2.3	Linux 文件系统镜像制作	- 47 -
第 5 章	Linux 基础操作	- 48 -
5.1	Linux 基本命令	- 48 -
5.1.1	用户系统相关命令	- 49 -
5.1.2	文件目录相关命令	- 55 -
5.1.3	压缩打包相关命令	- 67 -
5.1.4	比较合并文件相关命令.....	- 71 -
5.1.5	网络相关命令	- 76 -
5.2	Linux 编辑器 vi 的使用.....	- 79 -
5.2.1	vi 的模式	- 80 -
5.2.2	vi 的基本流程	- 80 -
5.2.3	vi 的各模式功能键	- 82 -
第 6 章	Linux Shell 编程.....	- 85 -
6.1	实验原理	- 85 -
6.2	实验目的	- 85 -
6.3	实验平台	- 85 -
6.4	实验步骤	- 85 -
6.4.1	拷贝代码	- 85 -
6.4.2	执行代码	- 86 -
6.4.3	实验结果	- 86 -
6.5	相关代码	- 86 -
第 7 章	GCC 编译	- 88 -
7.1	实验原理	- 88 -
7.2	实验目的	- 89 -
7.3	实验平台	- 89 -
7.4	实验步骤	- 89 -



7.4.1	拷贝代码	- 89 -
7.4.2	编译代码	- 90 -
7.4.3	执行代码	- 90 -
7.4.4	实验结果	- 90 -
7.5	相关代码	- 90 -
第 8 章	Linux 系统 Makefile 编写实验.....	- 91 -
8.1	实验原理	- 91 -
8.1.1	Makefile 基本结构	- 91 -
8.1.2	Makefile 变量	- 92 -
8.1.3	Makefile 规则	- 95 -
8.1.4	Make 管理器的使用	- 96 -
8.2	实验目的	- 96 -
8.3	实验平台	- 96 -
8.4	实验步骤	- 97 -
8.4.1	拷贝代码	- 97 -
8.4.2	执行代码	- 97 -
8.5	相关代码	- 99 -
第 9 章	嵌入式文件 I/O 编程实验.....	- 101 -
9.1	实验原理	- 101 -
9.1.1	系统调用	- 101 -
9.1.2	用户编程接口 (API)	- 101 -
9.1.3	系统命令	- 101 -
9.1.4	虚拟文件系统 (VFS)	- 102 -
9.1.5	Linux 中文件及文件描述符	- 103 -
9.1.6	标准 I/O 编程	- 104 -
9.2	标准 IO 实验	- 110 -
9.2.1	实验目的	- 110 -
9.2.2	实验平台	- 110 -
9.2.3	实验步骤	- 110 -
9.2.4	实验现象	- 111 -
9.2.5	实验代码	- 111 -
9.3	Linux 系统文件目录操作编程实验	- 111 -
9.3.1	实验目的	- 111 -
9.3.2	实验平台	- 111 -
9.3.3	实验步骤	- 111 -



9.3.4	实验现象	- 112 -
9.3.5	实验代码	- 112 -
第 10 章	嵌入式 Linux 多任务编程实验	- 117 -
10.1	使用 ps 命令查看进程信息	- 117 -
10.1.1	实验内容	- 117 -
10.1.2	实验平台	- 117 -
10.1.3	实验原理	- 117 -
10.1.4	实验步骤	- 117 -
10.2	使用 proc 文件系统查看进程信息	- 117 -
10.2.1	实验目的	- 117 -
10.2.2	实验平台	- 117 -
10.2.3	实验原理	- 118 -
10.2.4	实验内容	- 118 -
10.3	使用 fork、exit 和 exec 系统调用编写多进程程序	- 119 -
10.3.1	实验目的	- 119 -
10.3.2	实验平台	- 119 -
10.3.3	实验原理	- 119 -
10.3.4	实验步骤	- 119 -
10.3.5	实验现象	- 120 -
10.3.6	实验代码	- 120 -
10.4	Linux 系统守护进程实验	- 121 -
10.4.1	实验目的	- 121 -
10.4.2	实验平台	- 121 -
10.4.3	实验原理	- 121 -
10.4.4	实验步骤	- 122 -
10.4.5	实验现象	- 123 -
10.4.6	实验代码	- 124 -
第 11 章	Linux 系统进程间通信实验	- 126 -
11.1	实验原理	- 126 -
11.1.1	管道通信	- 127 -
11.1.2	无名管道系统调用	- 127 -
11.1.3	标准流管道	- 129 -
11.1.4	有名管道 (FIFO)	- 130 -
11.1.5	信号通信	- 131 -
11.1.6	信号量	- 138 -



11.1.7	共享内存	- 143 -
11.1.8	消息队列	- 145 -
11.2	Linux 系统无名管道通信实验	- 147 -
11.2.1	实验目的	- 147 -
11.2.2	实验平台	- 147 -
11.2.3	实验步骤	- 147 -
11.2.4	实验现象	- 148 -
11.2.5	实验代码	- 148 -
11.3	Linux 系统有名管道通信实验	- 149 -
11.3.1	实验目的	- 149 -
11.3.2	实验平台	- 149 -
11.3.3	实验内容	- 149 -
11.3.4	实验步骤	- 150 -
11.3.5	实验现象	- 151 -
11.3.6	实验代码	- 151 -
11.4	Linux 系统信号机制实验	- 155 -
11.4.1	实验目的	- 155 -
11.4.2	实验平台	- 155 -
11.4.3	实验内容	- 155 -
11.4.4	实验步骤	- 155 -
11.4.5	实现现象	- 156 -
11.4.6	实验代码	- 156 -
11.5	Linux 系统共享内存通信实验	- 157 -
11.5.1	实验目的	- 157 -
11.5.2	实验平台	- 158 -
11.5.3	实验内容	- 158 -
11.5.4	实验步骤	- 158 -
11.5.5	实验现象	- 159 -
11.5.6	实验代码	- 159 -
11.6	Linux 系统消息队列实验	- 163 -
11.6.1	实验目的	- 163 -
11.6.2	实验平台	- 163 -
11.6.3	实验内容	- 163 -
11.6.4	实验步骤	- 163 -
11.6.5	实验现象	- 164 -



11.6.6	实验代码	- 165 -
第 12 章	嵌入式 Linux 多线程编程实验	- 169 -
12.1	实验目的	- 169 -
12.2	实验平台	- 169 -
12.3	实验原理	- 169 -
12.3.1	线程基本编程	- 169 -
12.3.2	线程之间的同步与互斥	- 171 -
12.3.3	线程属性	- 174 -
12.4	实验步骤	- 176 -
12.5	实验现象	- 177 -
12.6	实验代码	- 177 -
第 13 章	嵌入式 Linux 网络编程实验	- 179 -
13.1	实验原理	- 179 -
13.1.1	TCP/IP 的分层模型	- 179 -
13.1.2	TCP/IP 分层模型特点	- 180 -
13.1.3	TCP/IP 核心协议	- 182 -
13.1.4	套接字 (socket) 概述	- 185 -
13.1.5	地址及顺序处理	- 185 -
13.1.6	套接字编程	- 190 -
13.2	Linux 系统 tcp 网络协议编程实验	- 196 -
13.2.1	实验目的	- 196 -
13.2.2	实验平台	- 196 -
13.2.3	实验内容	- 196 -
13.2.4	实验步骤	- 196 -
13.2.5	实验现象	- 197 -
13.2.6	实验代码	- 197 -
13.3	Linux 系统 udp 网络协议编程实验	- 201 -
13.3.1	实验目的	- 201 -
13.3.2	实验平台	- 201 -
13.3.3	实验内容	- 201 -
13.3.4	实验步骤	- 201 -
13.3.5	实验现象	- 202 -
13.3.6	实验代码	- 202 -
13.4	Linux 系统 select IO 复用实验	- 205 -
13.4.1	实验目的	- 205 -



13.4.2	实验平台	- 205 -
13.4.3	实验内容	- 205 -
13.4.4	实验步骤	- 205 -
13.4.5	实验现象	- 206 -
13.4.6	实验代码	- 207 -
第 14 章	Linux 设备驱动开发.....	- 211 -
14.1	内核模块	- 211 -
14.1.1	最简单的内核模块	- 211 -
14.1.2	内核模块程序结构	- 212 -
14.1.3	模块的编译	- 212 -
14.1.4	内核模块的相关工具	- 214 -
14.1.5	实验一 简单的内核模块实验.....	- 214 -
14.2	字符设备驱动	- 216 -
14.2.1	字符设备驱动框架	- 217 -
14.2.2	实验二 字符设备驱动编写.....	- 220 -
14.3	GPIO 驱动开发	- 222 -
14.3.1	什么是 GPIO?	- 222 -
14.3.2	如何操作 GPIO?	- 222 -
14.3.3	实验三 LED 灯驱动开发实验.....	- 224 -
14.4	PWM 定时器.....	- 227 -
14.5	platform 设备驱动.....	- 229 -
14.5.1	平台设备	- 230 -
14.5.2	平台驱动	- 232 -
14.5.3	ADC 驱动编写	- 233 -
14.6	I2C 总线驱动	- 235 -
14.6.1	MMA8451 三轴加速度传感器实验.....	- 240 -
14.6.2	ZLG7290 驱动开发.....	- 242 -
14.6.3	ZLG72128 驱动开发.....	- 246 -



第 1 章 嵌入式 Linux 主机开发环境搭建

华清远见开发环境是基于 Ubuntu 12.04 LTS 64-bit 操作系统搭建的，使用 [VMware Player](#)（**免费版**）作为虚拟机工具软件（用户也可以使用 VMware 公司所提供的**付费版**虚拟机软件 [VMware Workstation](#) 代替 VMware Player）。本开发环境可用作嵌入式 Linux 和 Android 的编译与开发。

本开发环境在 Ubuntu 12.04 64-bit LTS 基础上，安装了编译调试 Bootloader、Linux、Android 系统所需要的工具和依赖的库，用户可以在无需额外操作的基础上，直接使用本开发环境进行嵌入式的学习和工作。

本开发环境在 Ubuntu 12.04 64-bit 基础上，安装配置了如下工具：

- 将 GCC、G++编译器版本从 4.6 降至 4.4；
- 安装了 Android 编译所需要的工具和库（source.android.com）；
- 安装 SUN JAVA JDK 6；
- 安装内核编译所依赖的工具包；
- 解决了 libncurses 32 位和 64 位不能同时安装导致编译 Android 和配置内核软件冲突的问题；
- 安装制作安卓文件系统 yaffs2 格式 mkyaffs 工具；
- 添加了常用的 arm-linux 交叉工具链，版本号为 4.3.2、4.4.6、4.5.1；
- 安装 Vim、Ctags；
- 安装 Vim 常用插件；
- 安装配置 TFTP；
- 安装配置 NFS 网络文件系统服务；
- 安装 SSH 工具网络服务程序；
- 安装 Kmit 串口调试工具；
- 安装 Sogou 输入法；
- 关闭 Ubuntu 更新提示；

说明 1：Ubuntu 用户名为“linux”，主机名为“ubuntu64”，默认密码为“1”

说明 2：以上安装过的软件和库，用户在不明确的前提下，切勿再次安装！（比如 source.android.com 要求的、tftp 服务等等），如果因为重复安装导致源码编译错误，请重新解压开发环境镜像。

下图是使用华清远见开发环境编译源码：



1.1 安装前准备

【主机配置】

华清远见开发环境是基于 Ubuntu 12.04 LTS 64-bit 操作系统搭建的，使用 VMware Player 免费版作为虚拟机工具软件。用作 Linux 和 Android 的编译与开发。所以建议开发主机硬件配置越高越好，配置越高则开发效率则越高。

配置	参数
CPU	Intel/AMD CPU; 主频 2GHz 或者更多; 双核或者更多; 支持 EM64T (INTEL)或者 X86_64(AMD)指令; 支持 Intel Virtualization Technology (32 位操作系统必须)
内存	内存 2G 以上 (Windows XP / 7 / 8 32 位); 内存 4G 以上 (Windows 7 / 8 64 位);
硬盘	虚拟机存放硬盘分区剩余空间 80G 以上; 推荐使用虚拟机挂载物理 ext 分区 (无 Linux 基础者慎用)
USB	支持 USB2.0 或者更高; 2 路或者更多;
网卡	至少一路以太网卡;



串口	RS232 串口（也可以用 USB 转串口替代）
操作系统	Windows XP 或者更新； 32 位/64 位操作系统；

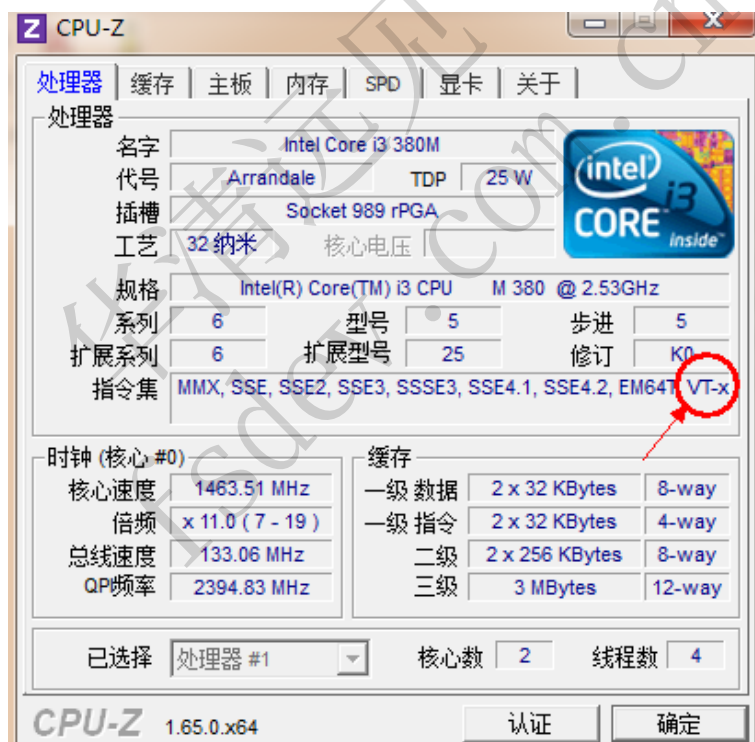
【Intel Virtualization Technology（32 位操作系统）】

此选项为用户主机操作系统为 32 位必须具备，如果主机操作系统为 64 位，则可忽略此部。

华清远见开发环境是基于 Ubuntu 12.04 LTS 64-bit 操作系统搭建的，使用 VMware Player 作为虚拟机工具软件。所以在如果主机为 32 位操作系统，那么系统必须支持英特尔虚拟化技术（Intel Virtualization Technology）才可以通过 VMware Player 工具使用 64 位操作系统。

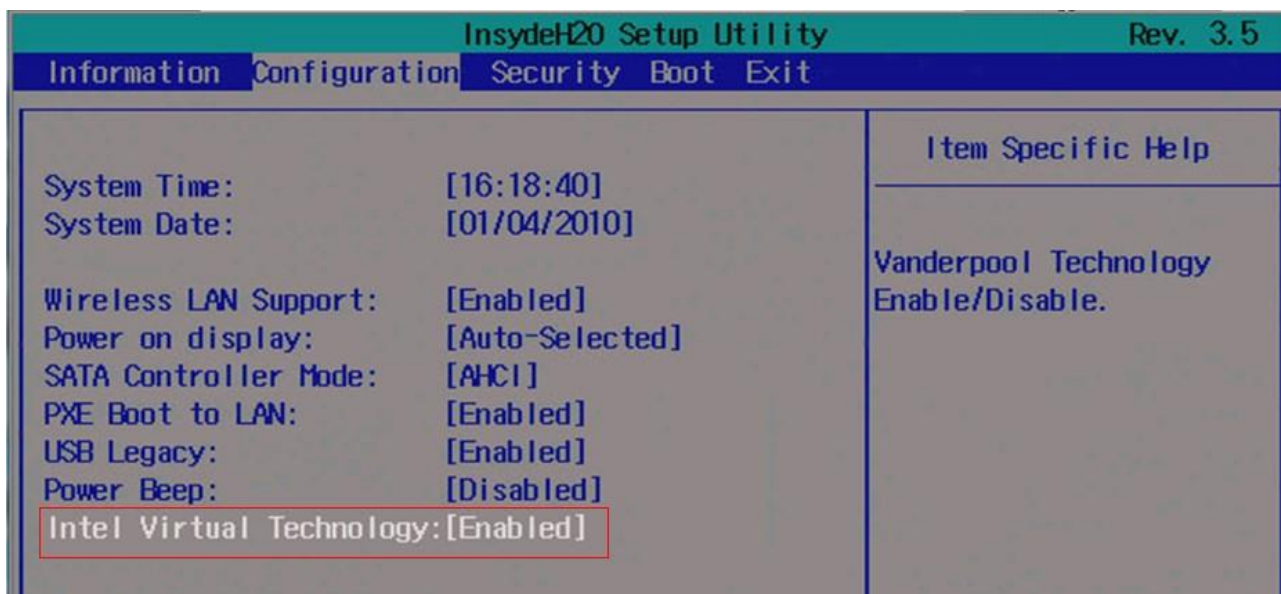
【检查主机 CPU 是否支持 Intel Virtualization Technology】

如下图所示，用户可以使用光盘目录下的 CPU-Z 软件检查自己的 CPU 是否支持虚拟机化技术。



【在 BIOS 中打开 Intel Virtualization Technology】

如果 CPU 支持虚拟化技术，请在 BIOS 设置里面打开（不同型号的主板 BIOS 界面可能不同，如果找不到可以自行搜索“机型+BIOS+VT”的关键字）。



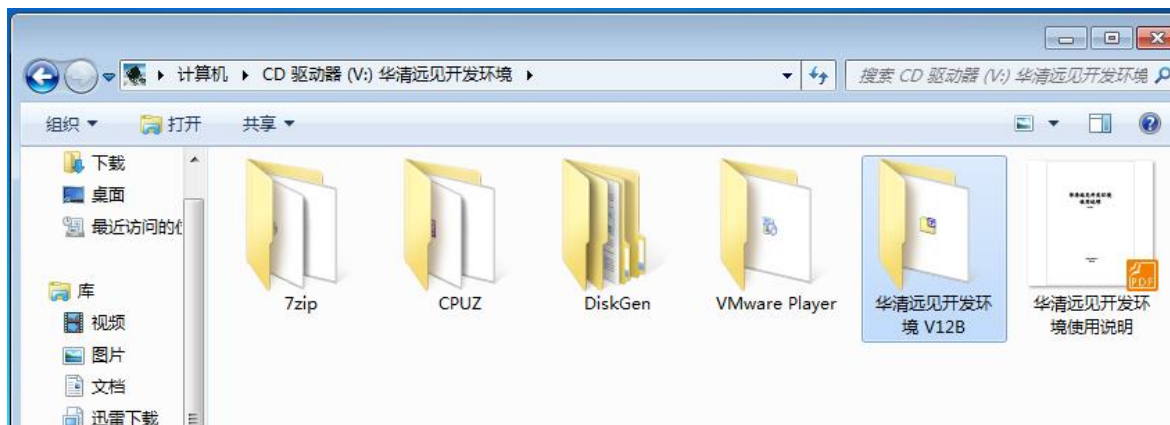
1.2 安装 VMware Player

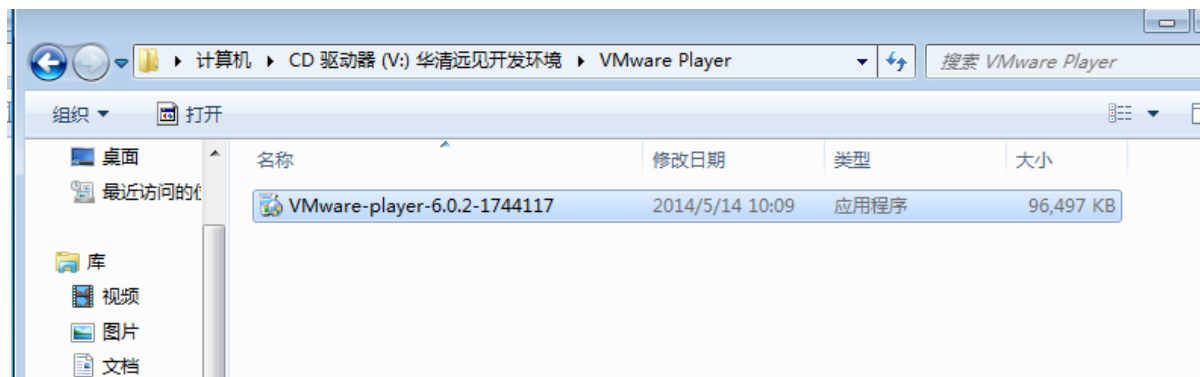
VMware Player 从 6.0 版本之后默认支持中文，所以华清远见开发环境 V12B 使用当前最新版的 VMware Player（版本号为 6.0.2 build-1744117），如要正常使用此开发环境，必须保证 VMware Player 版本号大于等于当前给出的版本号，否则可能会出现因为 VMware Tools 版本过高引起虚拟机无法正常启动的情况。

（如果用户使用 VMware Workstation，版本号应该大于等于（10.0.1-1379776），否则可能会出现因为 VMware Tools 版本过高引起虚拟机无法正常启动的情况。）

【打开 VMware Player 安装程序】

打开【华清远见开发环境光盘 VMware Player】目录下的 VMware Player 安装程序（VMware-player-6.0.2-1744117.exe）。





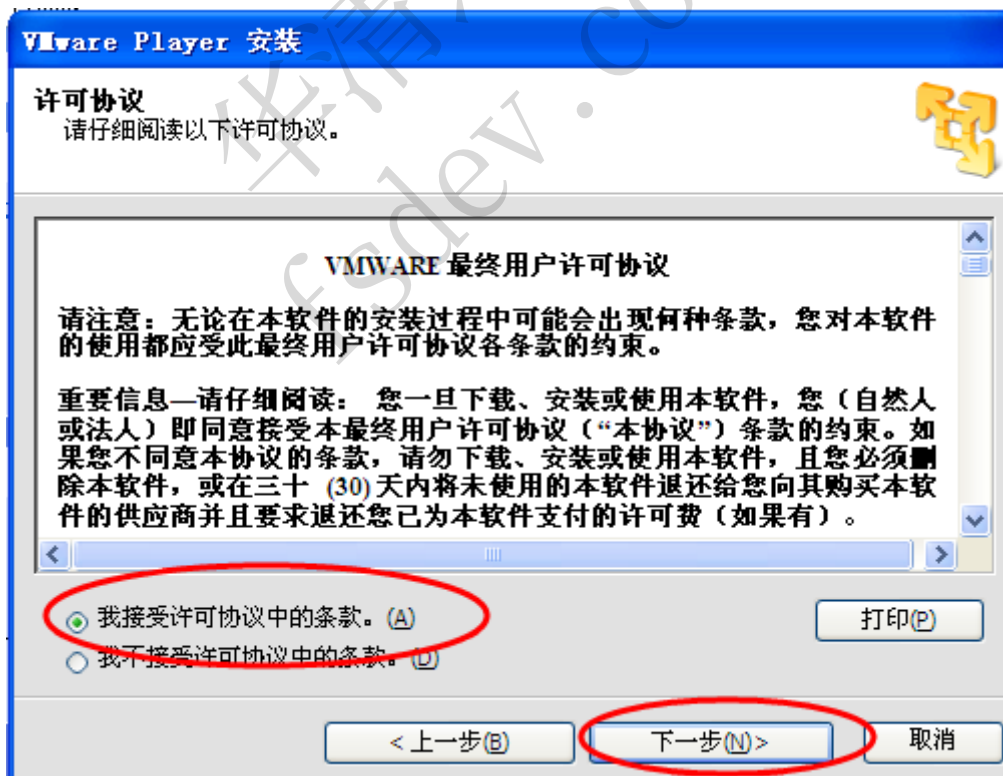
【VMware Player 安装程序初始化】



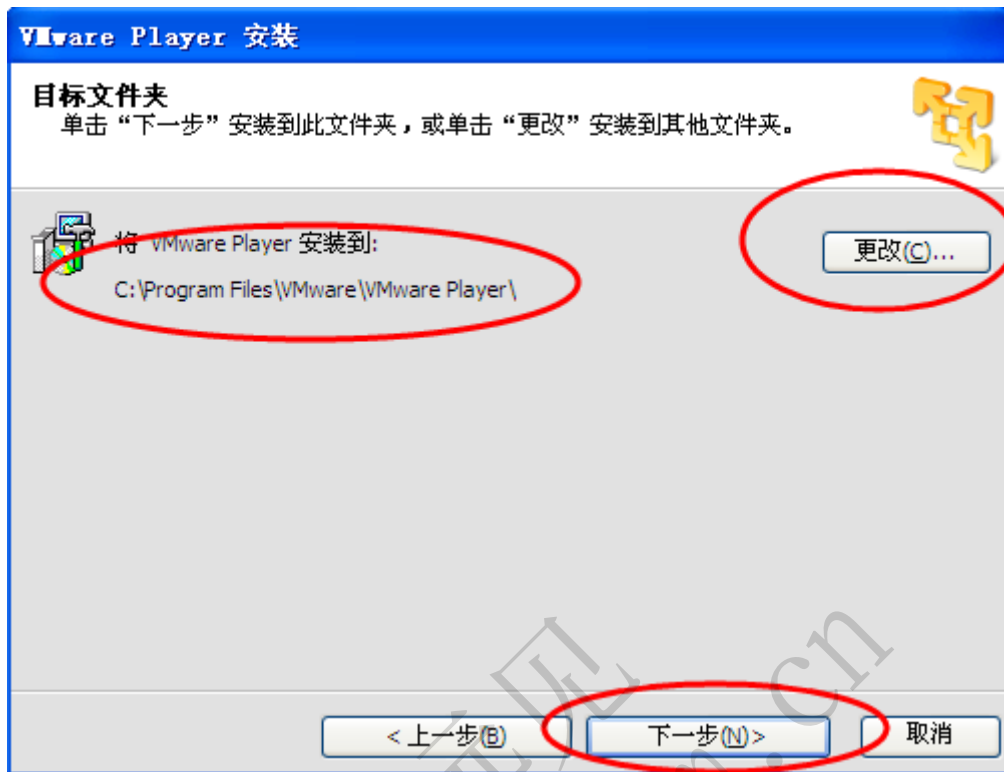
【VMware Player 安装向导】



【VMware Player 许可协议】

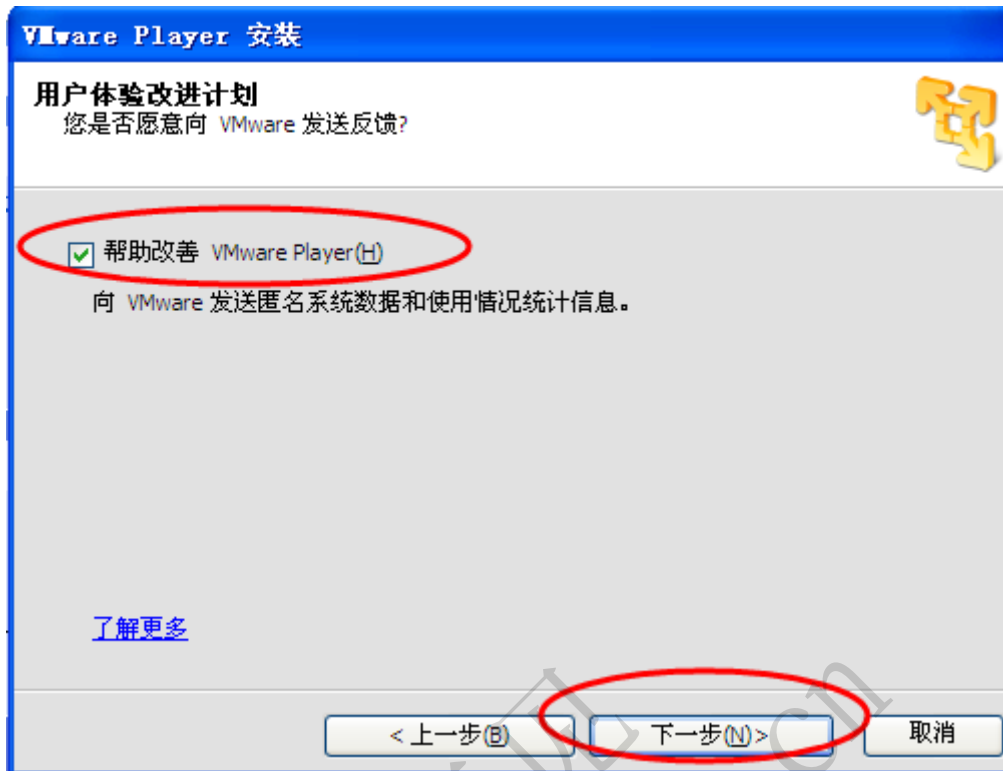


【更改安装路径】

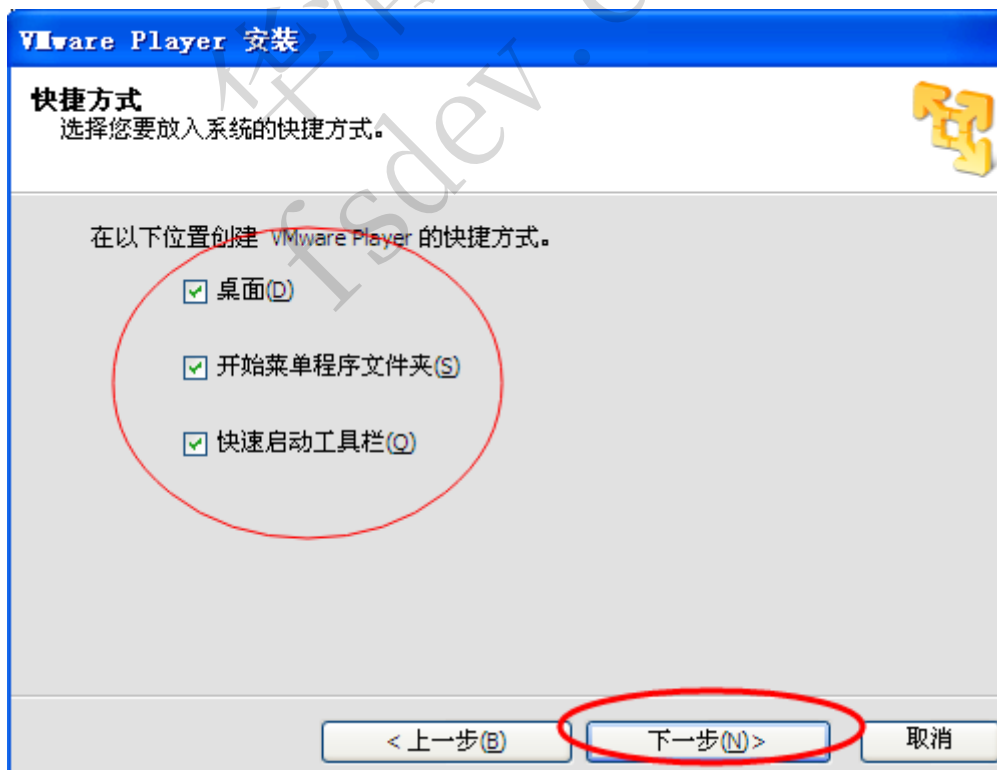


【选择是否检查更新，加入用户体验改进计划】



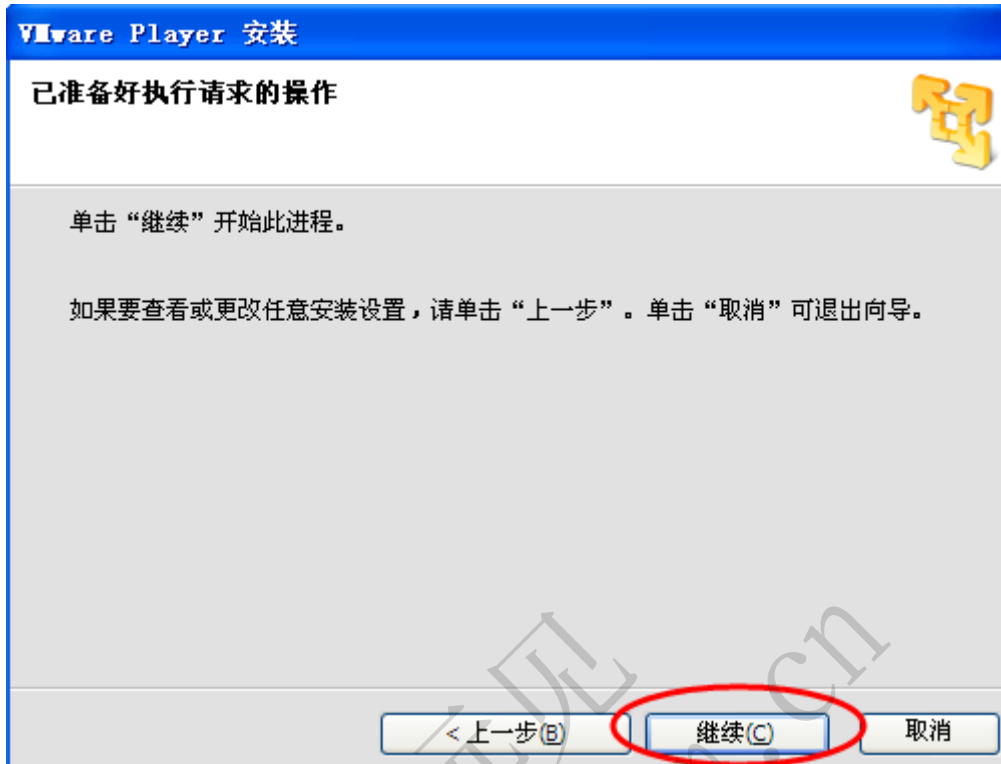


【创建快捷方式】

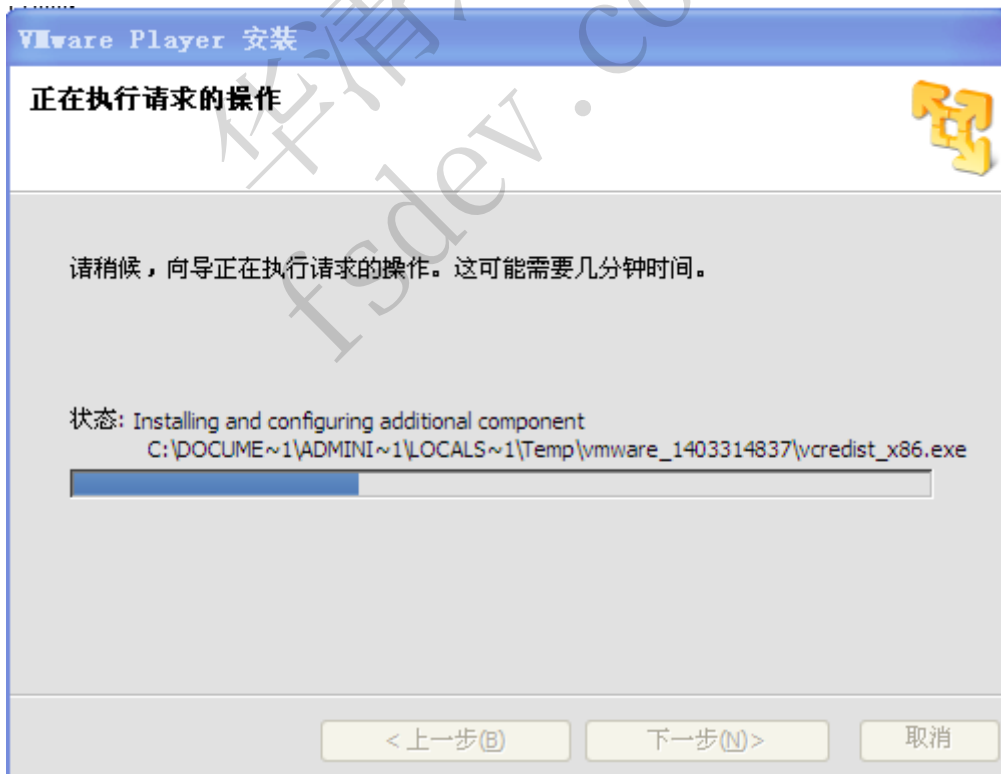




【确认安装】



【正在安装】



【安装完成】

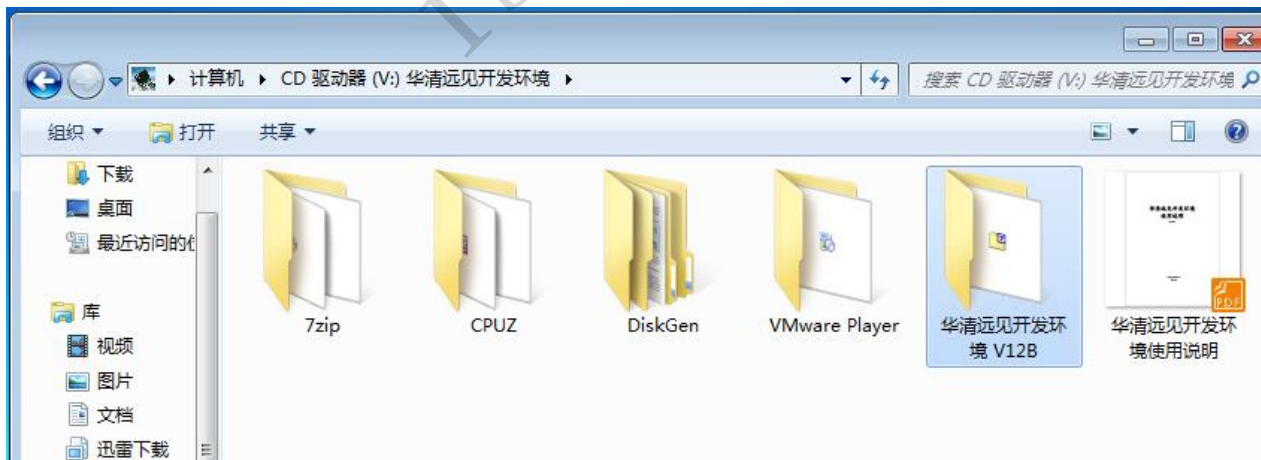


1.3 运行开发环境

1.3.1 解压虚拟机镜像

【打开华清远见开发环境】

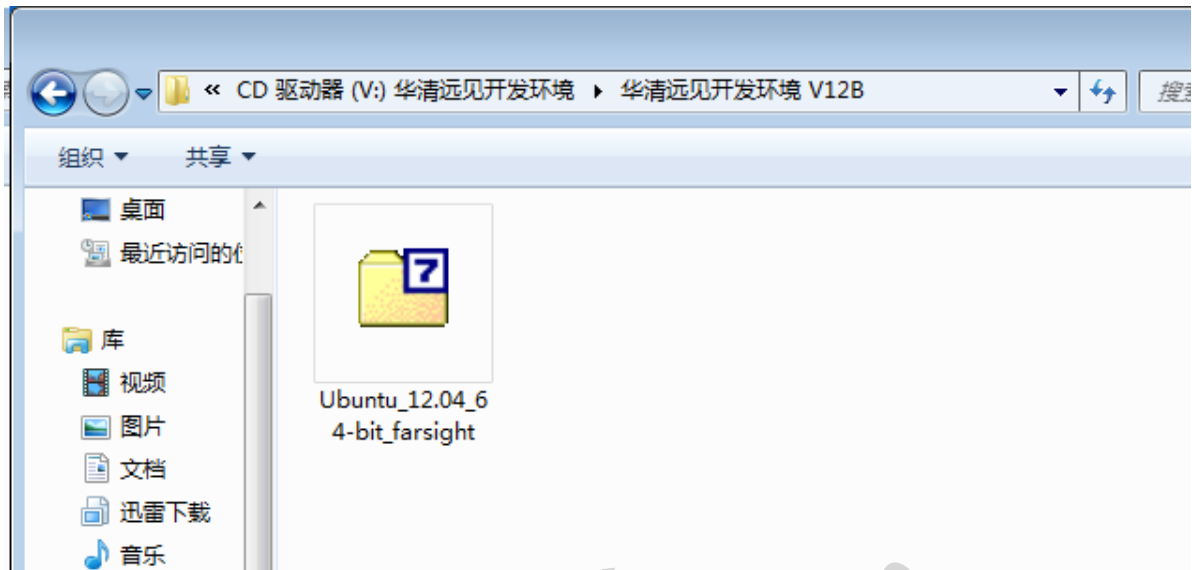
打开【打开华清远见开发环境\华清远见开发环境 V12B】目录。



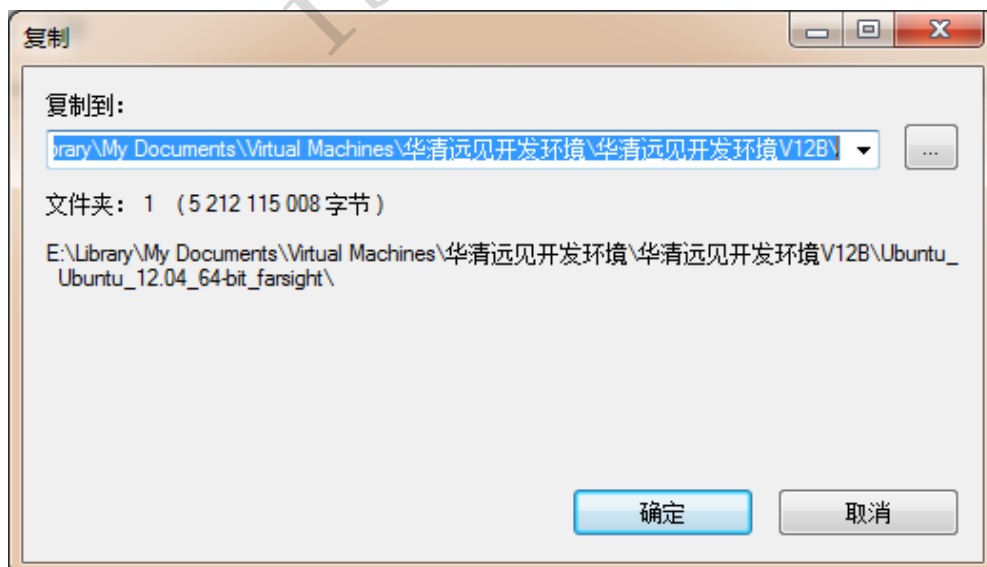
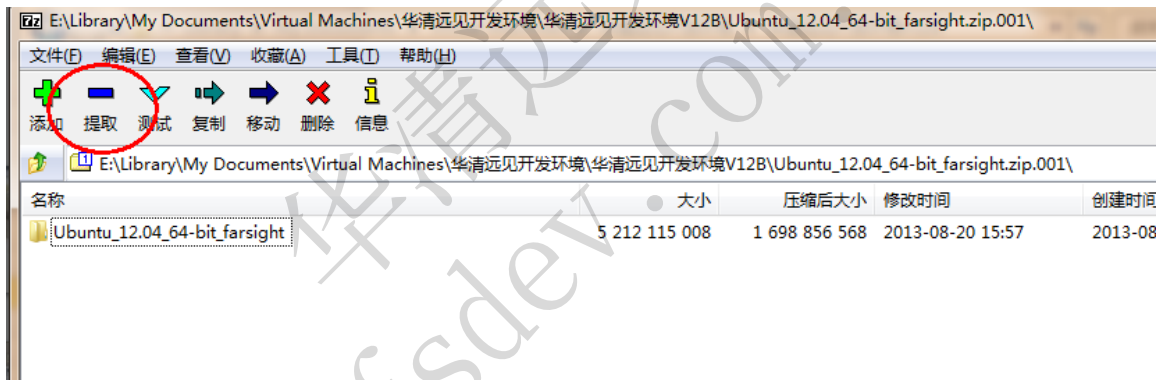
【打开华清远见开发环境压缩文件】

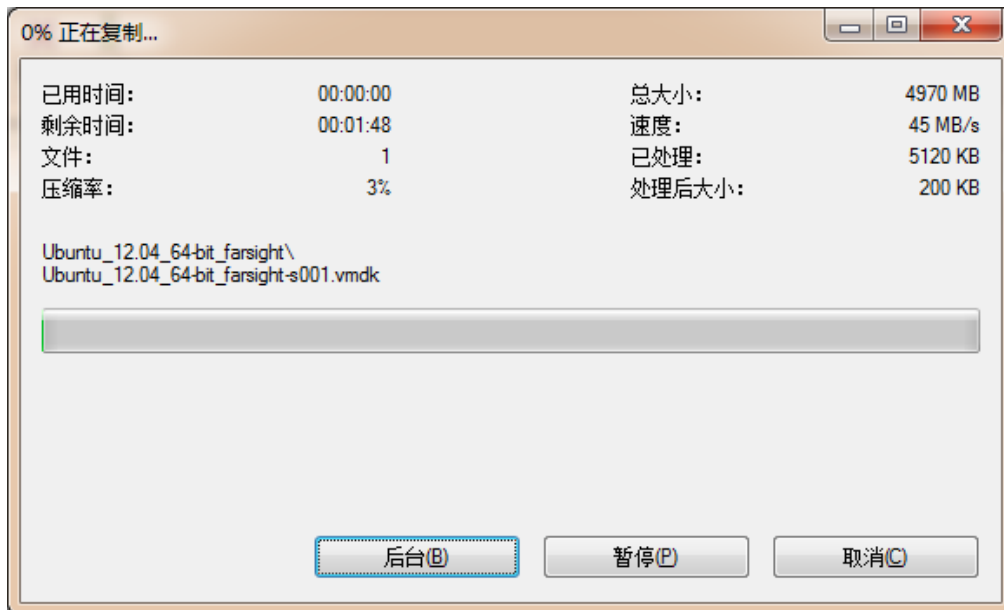


此处建议使用光盘下的 7zip 压缩软件解压。



【解压华清远见开发环境】





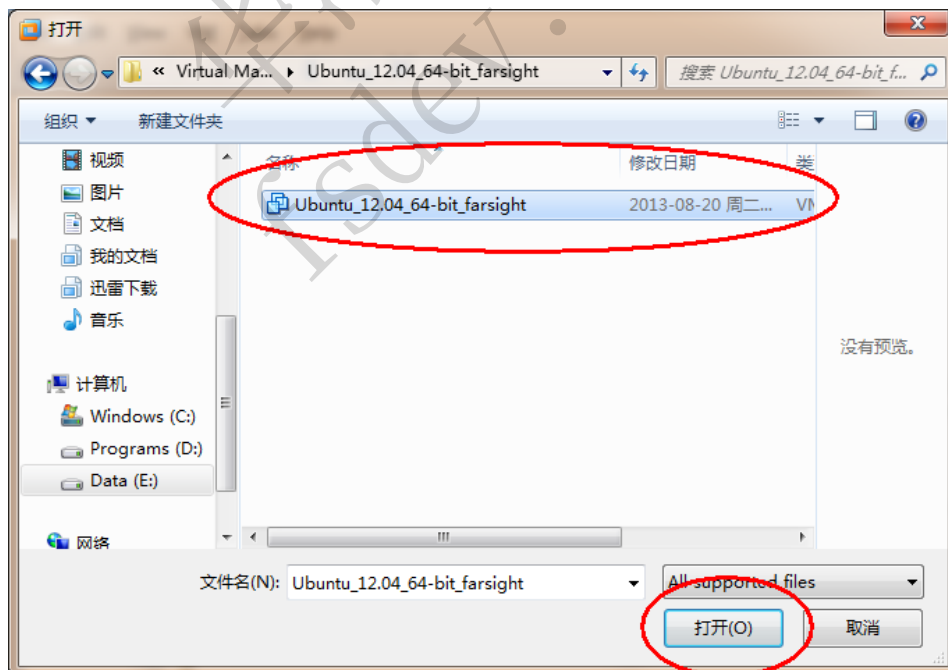
1.3.2 打开虚拟机

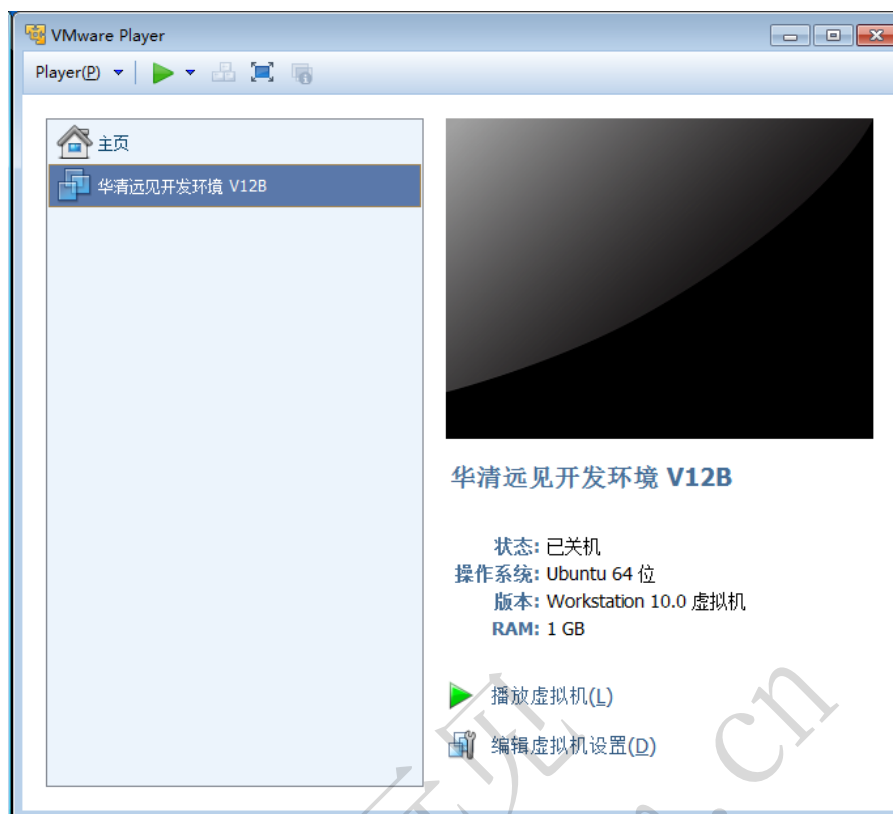
【打开 VMware Player】





【打开虚拟机镜像】





1.3.3 配置优化虚拟机





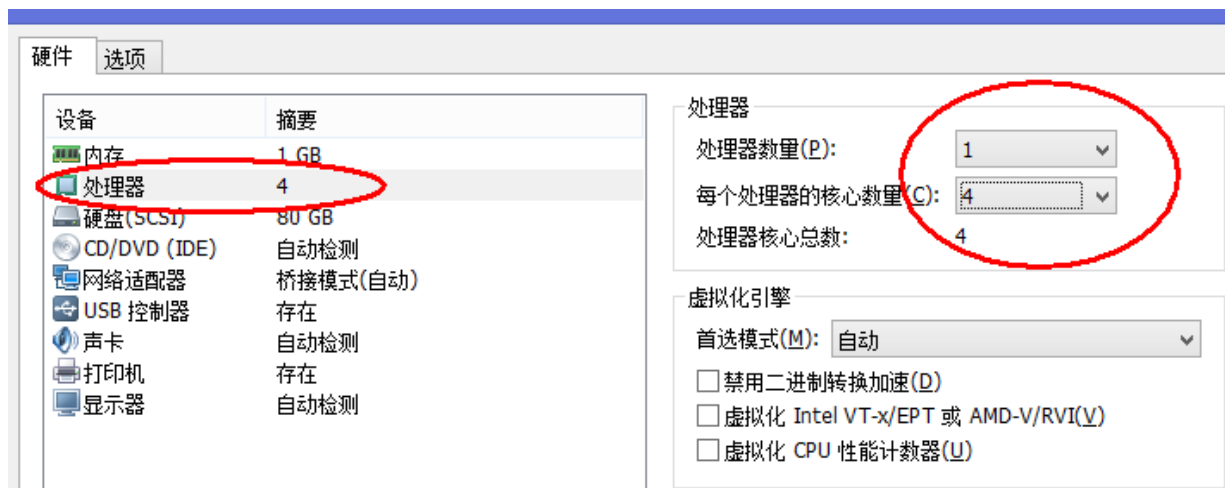
【增加内存大小】

根据主机配置修改虚拟机内存大小。例如主机内存 1G，那分配虚拟机的内存大小应该小于 512M，否则物理机操作系统运行会卡；如果主机内存大于 4G（足够大），那可以根据 VMware Player 的提示和自己的需求修改内存大小。**注意：如果需要编译 Android，那内存大小最好大于 1G。**



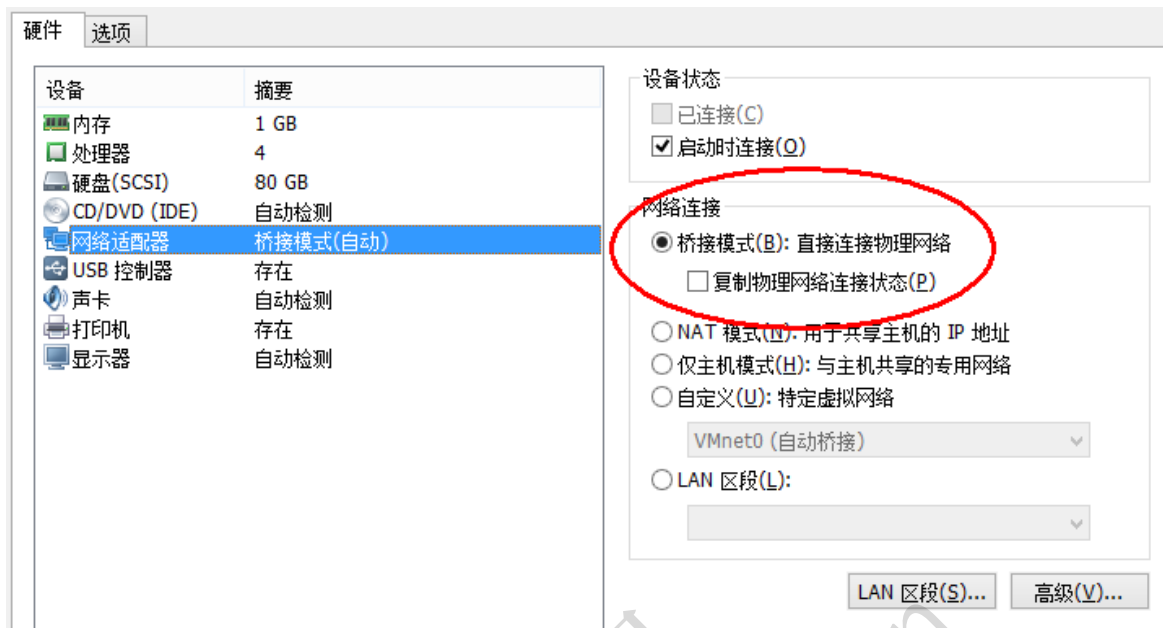
【修改 CPU 数量】

根据主机 CPU 配置修改虚拟机 CPU 数量。例如笔者 CPU 为 Intel Core-i3 M380（双核四线程），那处理器数量设置为 1,每个处理器的核心数量设置为 4。**注意：如果设置的总核心数不要超过 CPU 的核心数。**



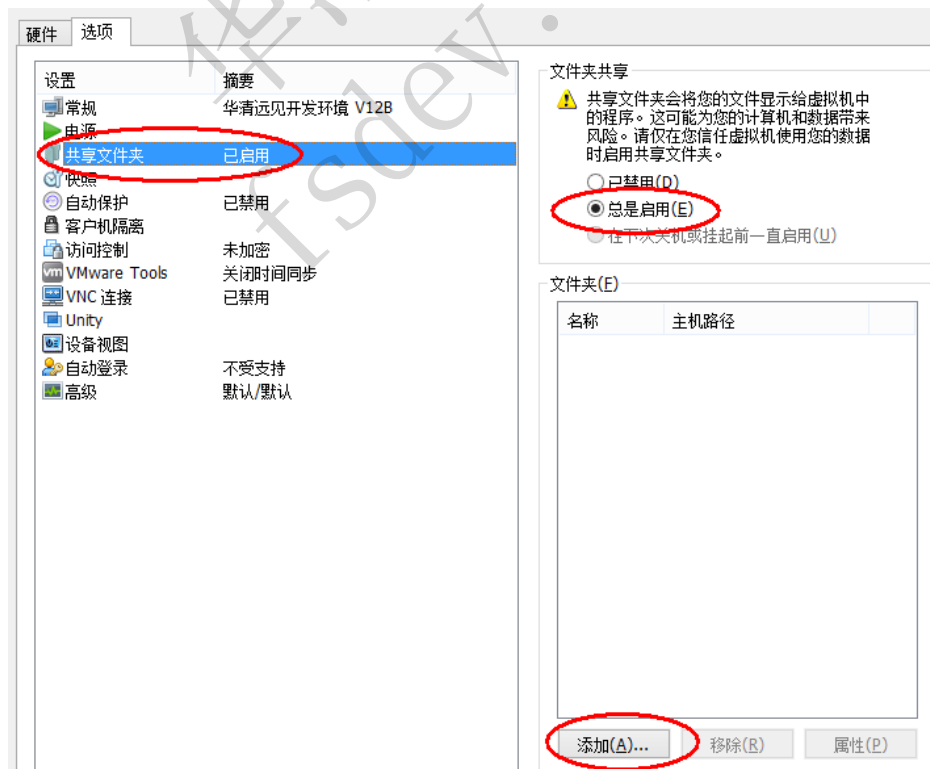


【确保网络连接为桥接模式】



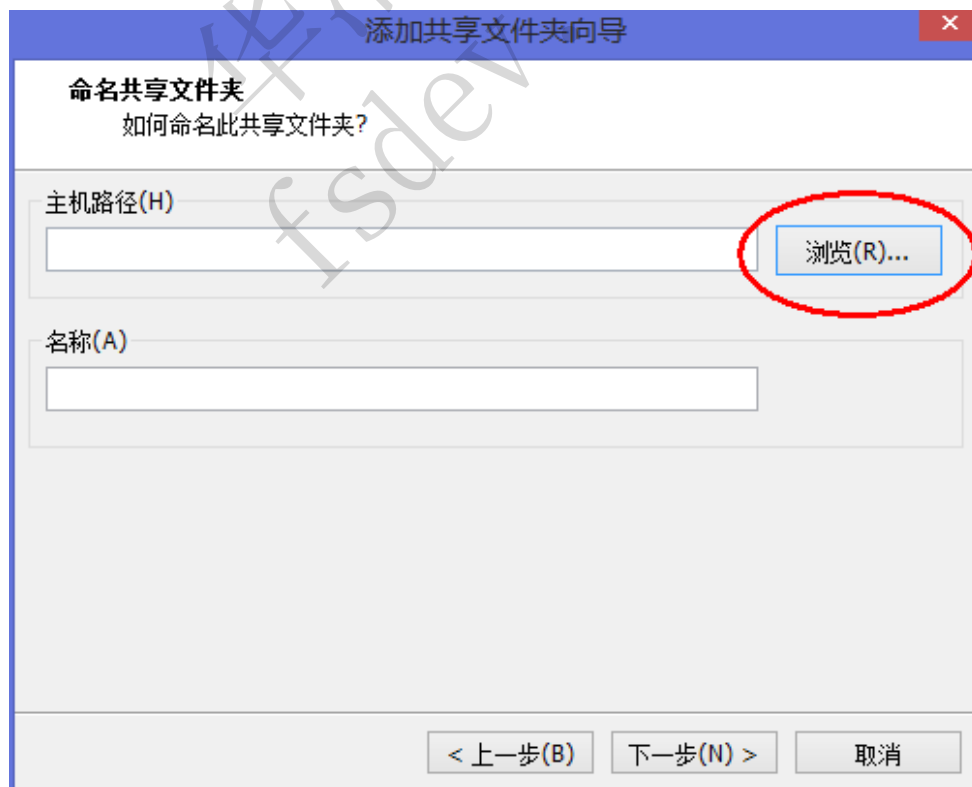
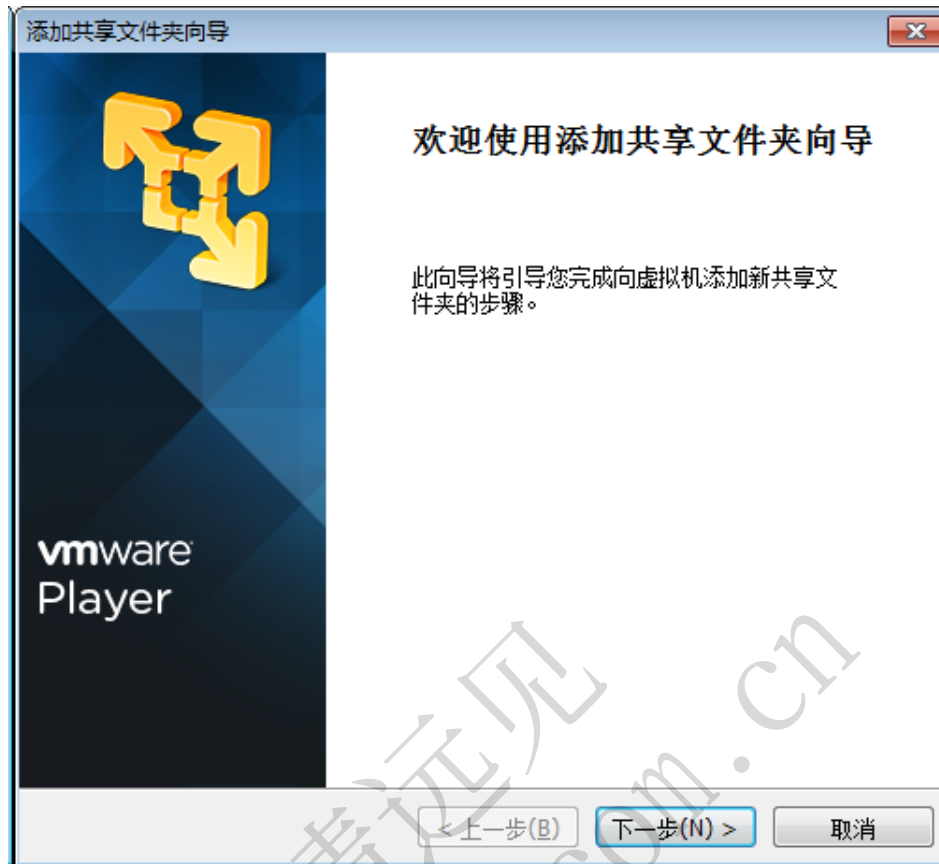
【增加共享目录】

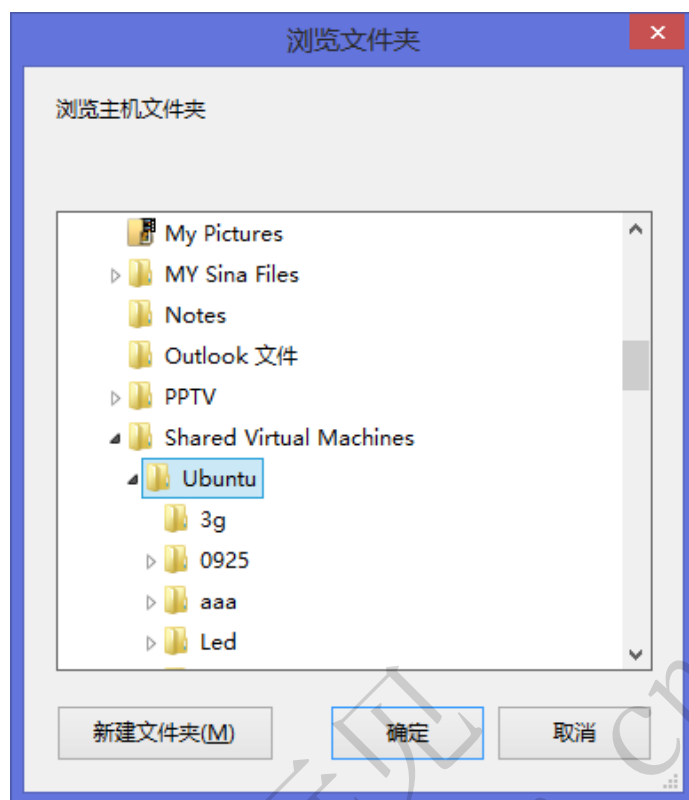
共享目录可以在虚拟机访问物理硬盘分区的内容，也可以将虚拟机里的文件拷贝至物理机，是虚拟机和物理机很好的交流桥梁。





【点击上图“添加”】



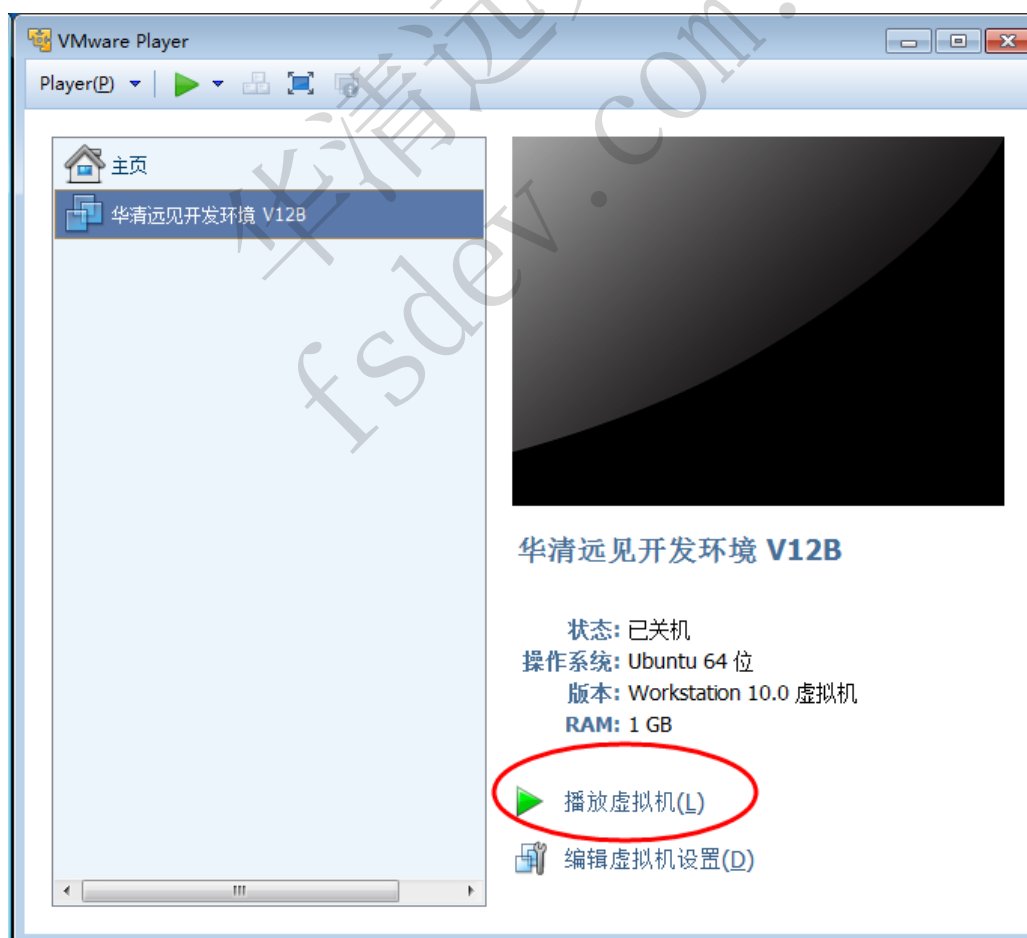


【修改在虚拟机内看到物理磁盘目录的名字】



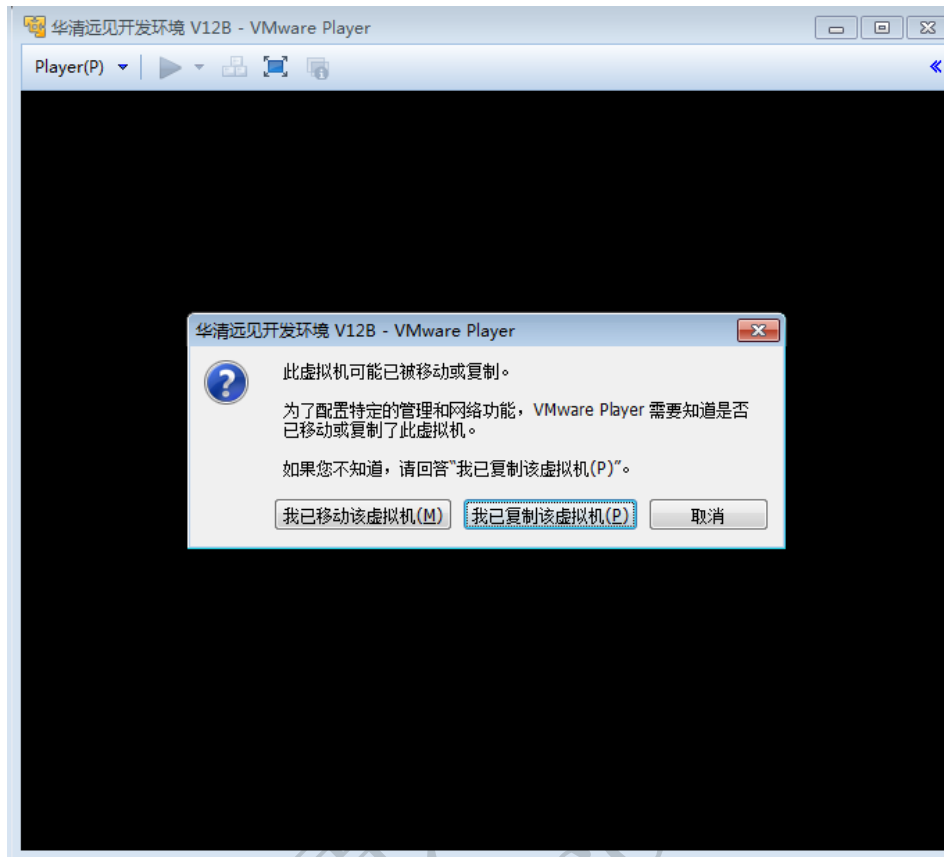


1.3.4 启动虚拟机

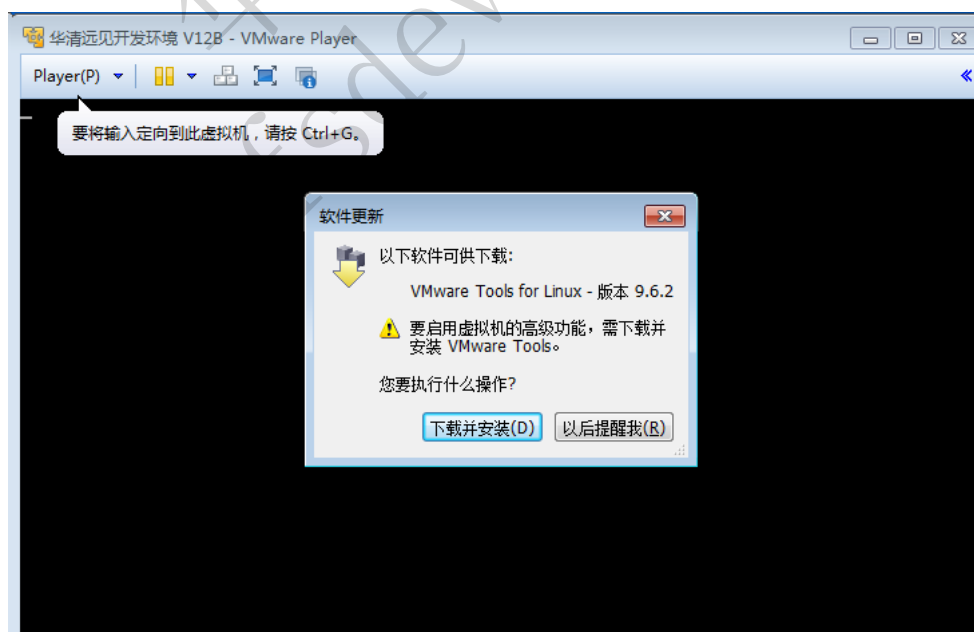


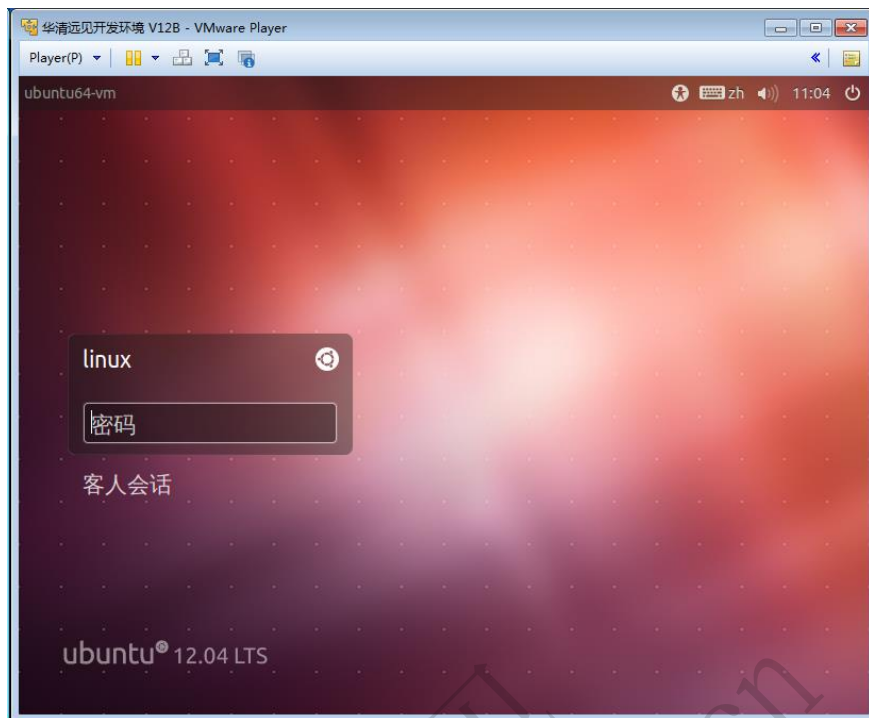


【启动虚拟机，选择“我已复制该虚拟机”】

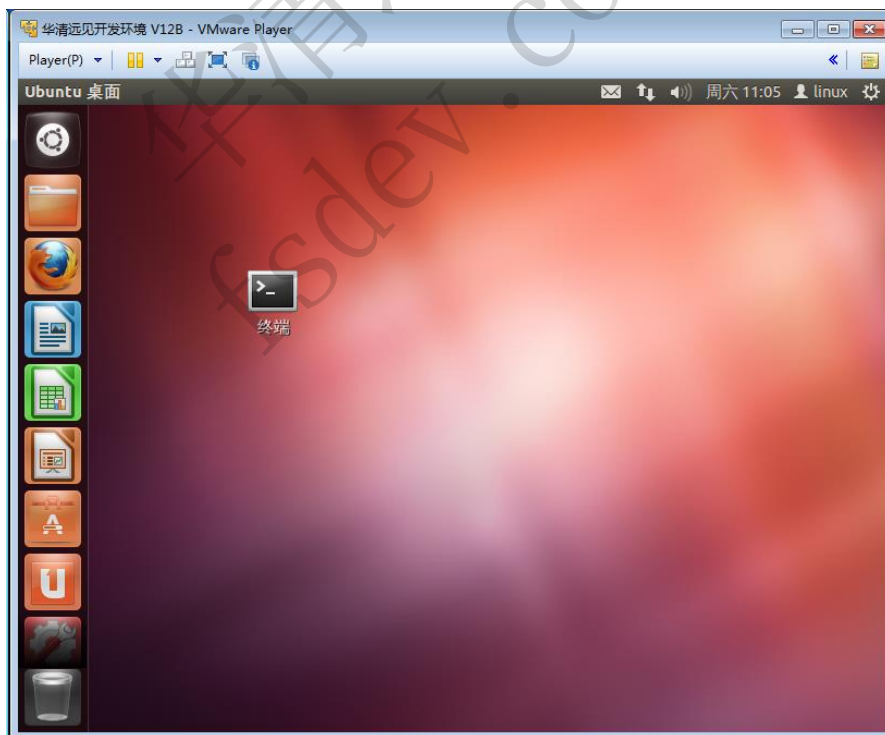


【在有网络的情况下，会提示有软件更新可以下载，可以选择下载，或者以后提醒】





【输入密码，密码为 1】



嵌入式 Linux 开发环境至此搭建完毕。



1.3.5 设置 ROOT 密码

华清远见开发环境默认没有设置 Linux Root 管理员密码，建议一般用户使用普通用户模式下操作，需要管理员操作时使用 `sudo` 命令获取临时管理员权限。

设置 Linux Root 管理员密码步骤如下：

- 使用 `passwd` 命令生成 Root 密码（Linux 用户默认密码为 1）

```
$ sudo passwd
```

输入两次新的 UNIX 密码，即为 ROOT 用户密码

```
linux@ubuntu64-vm: ~
linux@ubuntu64-vm:~$ ls
examples.desktop  README  workdir  模板  图片  下载  桌面
lx.leesheen@gmail.com  toolchain  公共的  视频  文档  音乐
linux@ubuntu64-vm:~$
linux@ubuntu64-vm:~$ sudo passwd
[sudo] password for linux:
输入新的 UNIX 密码：
重新输入新的 UNIX 密码：
passwd：已成功更新密码
linux@ubuntu64-vm:~$
```

- 使用 `su` 命令切换至 Root 用户模式下，键入使用 `passwd` 的密码

```
root@ubuntu64-vm: /home/linux
linux@ubuntu64-vm:~$ ls
examples.desktop  README  workdir  模板  图片  下载  桌面
lx.leesheen@gmail.com  toolchain  公共的  视频  文档  音乐
linux@ubuntu64-vm:~$
linux@ubuntu64-vm:~$ sudo passwd
[sudo] password for linux:
输入新的 UNIX 密码：
重新输入新的 UNIX 密码：
passwd：已成功更新密码
linux@ubuntu64-vm:~$
linux@ubuntu64-vm:~$ su
密码：
root@ubuntu64-vm: /home/linux#
```



第 2 章 嵌入式 Linux 主机调试环境搭建

NFS 方式是开发板通过 NFS 挂载放在主机（PC）上的根文件系统。此时在主机在文件系统中进行的操作同步反映在开发板上；反之，在开发板上进行的操作同步反映在主机中的根文件系统上。

实际工作中，我们经常使用 NFS 方式挂载文件系统，这种方式对于系统的调试非常方便。

2.1 配置主机网络环境

2.1.1 实验目的

在做应用开发实验或者驱动开发实验时，用户经过交叉编译开发的应用程序或者驱动文件需要放到开发板中进行测试，这里一般采用的方式都是通过 nfs 方式传输文件，那么这个时候我们需要首先配置主机的网络环境。

用户需要掌握虚拟机如何配置网络以及一般的 linux 操作系统如何配置网络环境。

2.1.2 实验平台

华清远见开发环境。

2.1.3 实验原理

Linux 系统下提供了配置文件以及图形界面两种配置网络的方式，用户通过这两种方式中的任何一种都可以轻易的对系统的网络环境进行配置（此处介绍通过修改配置文件的方式配置网络）。

2.1.4 实验步骤

虚拟机网络方式为桥接模式，此状态下虚拟机下的操作系统和主机操作系统为平级状态。为了调试方便，我们可以给虚拟机下的 Ubuntu 一个静态的 IP 地址。

假设我们使用的网络地址为 192.168.100.x 段的，那么我可以给 Ubuntu 分配一个 IP 为 192.168.100.192。

```
$ sudo vim /etc/network/interfaces
```

修改文件如下图所示。保存退出（环境默认是加【#】注释掉的内容，修改前删除 4-11 行每行的注释）。

```
1 auto lo
2 iface lo inet loopback
3
4 auto eth0
5 iface eth0 inet static
6 address 192.168.100.192
7 netmask 255.255.255.0
8 gateway 192.168.100.1
9 network 192.168.100.0
10 broadcast 192.168.100.255
11 dns-nameservers 192.168.100.1
12
```



重新启动网络配置。

```
$ sudo /etc/init.d/networking restart
```

```
[sudo] password for linux:
linux@ubuntu64-vm:~$ sudo /etc/init.d/networking restart
* Running /etc/init.d/networking restart is deprecated because it may not enable
e again some interfaces
* Reconfiguring network interfaces...
ssh stop/waiting
ssh start/running, process 3252
[ OK ]
linux@ubuntu64-vm:~$
```

如上图所示表明 IP 修改成功。使用【ifconfig】命令查看我们修改的结果。如果没有修改成功，重复上述步骤，或者重新启动虚拟机 Ubuntu 系统即可。

2.2 Linux 系统配置 NFS 实验

2.2.1 实验目的

通过实验熟悉 Linux NFS 文件系统的配置过程，为后续 Linux 底层开发实验做准备。

2.2.2 实验平台

华清远见开发环境。

2.2.3 实验原理

NFS 方式是开发板通过网络的方式挂载放在主机（PC）上的根文件系统。此时在主机在文件系统中进行的操作同步反映在开发板上；反之，在开发板上进行的操作同步反映在主机中的根文件系统中。实际工作中，我们经常使用 NFS 方式挂载系统，这种方式对于系统的调试非常方便。

2.2.4 实验步骤

打开虚拟机，运行 Ubuntu 12.04 系统，打开命令行终端。

配置/etc/exports **说明：（sudo 获取权限。输入密码，默认为 1；如不会使用 vim，命令行 vim 字段用**

gedit 代替，下面省略此说明。）

```
$ sudo vim /etc/exports
```

NFS 允许挂载的目录及权限在文件/etc/exports 中进行了定义。例如，我们要将/source/rootfs 目录共享出来，那么我们需要在/etc/exports 文件末尾添加如下行：

```
/source/rootfs *(rw,sync,no_root_squash,no_subtree_check)
```



```
linux@ubuntu64-vm: ~
1 # /etc/exports: the access control list for filesystems which may be exported
2 #   to NFS clients.  See exports(5).
3 #
4 # Example for NFSv2 and NFSv3:
5 # /srv/homes hostname1(rw,sync,no_subtree_check) hostname2(ro,sync,no_
6 #
7 # Example for NFSv4:
8 # /srv/nfs4 gss/krb5i(rw,sync,fsid=0,crossmnt,no_subtree_check)
9 # /srv/nfs4/homes gss/krb5i(rw,sync,no_subtree_check)
10 #
11
12 /source/rootfs *(rw,sync,no_root_squash,no_subtree_check)
13
```

其中：/source/rootfs 是要共享的目录，*代表允许所有的网络段访问，rw 是可读写权限，sync 是资料同步写入内存和硬盘，no_root_squash 是 NFS 客户端分享目录使用者的权限，如果客户端使用的是 root 用户，那么对于该共享目录而言，该客户端就具有 root 权限。

重启服务：

```
$ sudo /etc/init.d/nfs-kernel-server restart
```

重启服务成功如下显示。（如果设置的路径没有相应内容，会提示错误，可以先忽略这个问题）

```
linux@ubuntu64-vm:~$ sudo /etc/init.d/nfs-kernel-server restart
* Stopping NFS kernel daemon [ OK ]
* Unexporting directories for NFS kernel daemon... [ OK ]
* Exporting directories for NFS kernel daemon...
exportfs: Failed to stat /source/rootfs: No such file or directory [ OK ]
* Starting NFS kernel daemon [ OK ]
linux@ubuntu64-vm:~$
```

2.3 交叉编译环境配置实验

所谓交叉开发是指先在一台通用 PC 上进行软件的编辑、编译与连接，然后下载到嵌入式设备中运行调试的开发过程。通用 PC 成为宿主机，嵌入式设备成为目标机。

2.3.1 实验目的

掌握如何配置交叉编译工具链，如何使用交叉编译工具链。

2.3.2 实验平台

华清远见开发环境。

2.3.3 实验原理

交叉编译：是 A 机器上编译生成，运行在 B 机器上。两个机器有不同的机器指令。



工具链：一般由编译器、连接器、解释器和调试器组成。

交叉编译工具链就是为了编译、链接、处理和调试跨平台体系结构的程序代码。每次执行工具链软件时，通过带有不同的参数，可以实现编译、链接、处理或者调试等不同的功能。从工具链的组成上来说，它一般由多个程序构成，分别对应着各个功能。

用户需要了解如何获取相应的交叉编译工具链或者如何制作交叉编译工具链。

2.3.4 实验步骤

华清远见开发环境包含了 3 个版本的交叉工具链，路径在/usr/local/toolchain/下（旧版本在/home/linux/toolchain/下），不必再次解压，直接指向路径即可。下列步骤均按照新版本的路径介绍，旧版本的用户可以选择更新或者自行修改路径即可。

一般情况下，当我们输入一个 Linux 命令，比如【ls】可以直接执行功能，但我当前的目录并没有【ls】这个文件。【ls】这个命令文件在【/bin】这个目录中，我们可以执行这是因为这些命令所在的路径包含在了用户的环境变量中。我们为了使用方便也将我们经常使用的交叉工具链添加到环境变量中。

```
linux@ubuntu64-vm:~$ cd /usr/local/toolchain/
linux@ubuntu64-vm:/usr/local/toolchain$ ls
toolchain-4.3.2  toolchain-4.4.6  toolchain-4.5.1
linux@ubuntu64-vm:/usr/local/toolchain$
```

修改文件 ~/.bashrc，添加如下内容

```
$ sudo vim /etc/bash.bashrc // 注意 bashrc 前面有句点
```

添加下面一行代码到文件的末尾。

```
export PATH=$PATH:/usr/local/toolchain/toolchain-4.5.1/bin/
```

重启配置文件

```
$ source /etc/bash.bashrc
```

工具链的测试

```
$ arm-none-linux-gnueabi-gcc -v
```

获取到如下的信息即说明交叉编译工具链配置完成了。

```
linux@ubuntu64-vm:~/workdir/music_player$ arm-linux-gcc -v
Using built-in specs.
COLLECT_GCC=arm-linux-gcc
COLLECT_LTO_WRAPPER=/usr/local/toolchain/toolchain-4.5.1/bin/../libexec/gcc/arm-none-linux-gnueabi/4.5.1/lto-wrapper
Target: arm-none-linux-gnueabi
Configured with: /work/toolchain/build/src/gcc-4.5.1/configure --build=i686-build_pc-linux-gnu --host=i686-build_pc-linux-gnu --target=arm-none-linux-gnueabi --prefix=/opt/FriendlyARM/toolschain/4.5.1 --with-sysroot=/opt/FriendlyARM/toolschain/4.5.1/arm-none-linux-gnueabi/sys-root --enable-languages=c,c++ --disable-multilib --with-cpu=arm1176jzf-s --with-tune=arm1176jzf-s --with-fpu=vfp --with-float=softfp --with-pkgversion=ctng-1.8.1-FA --with-bugurl=http://www.arm9.net/ --disable-sjlj-exceptions --enable-__cxa_atexit --disable-libmudflap --with-host-libstdcxx=-static-libgcc -Wl,-Bstatic,-lstdc++,-Bdynamic -lm' --with-gmp=/work/toolchain/build/arm-none-linux-gnueabi/build/static --with-mpfr=/work/toolchain/build/arm-none-linux-gnueabi/build/static --with-ppc=/work/toolchain/build/arm-none-linux-gnueabi/build/static --with-cloog=/work/toolchain/build/arm-none-linux-gnueabi/build/static --with-mpc=/work/toolchain/build/arm-none-linux-gnueabi/build/static --with-libelf=/work/toolchain/build/arm-none-linux-gnueabi/build/static --enable-threads=posix --with-local-prefix=/opt/FriendlyARM/toolschain/4.5.1/arm-none-linux-gnueabi/sys-root --disable-nls --enable-symvers=gnu --enable-c99 --enable-long-long
Thread model: posix
gcc version 4.5.1 (ctng-1.8.1-FA)
```

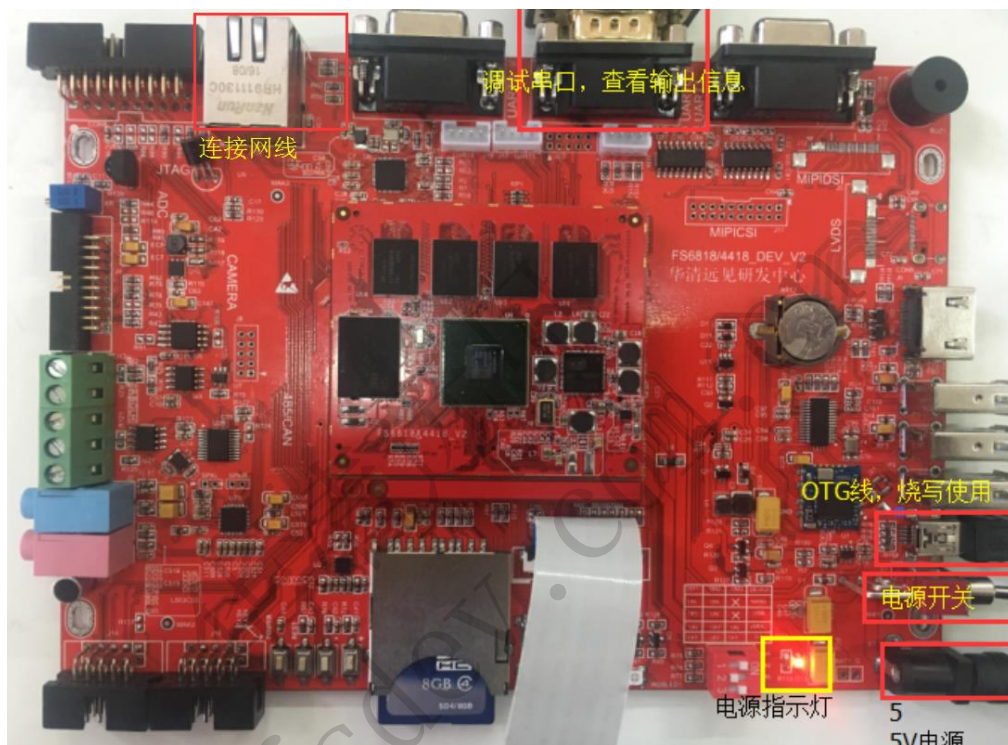


第 3 章 目标机调试环境搭建

所谓的目标机就是相应的开发平台，这里以 6818 平台为例，用户需要首先配置目标机的启动参数以及启动系统，为后续的实验做好准备。

3.1 连接开发平台

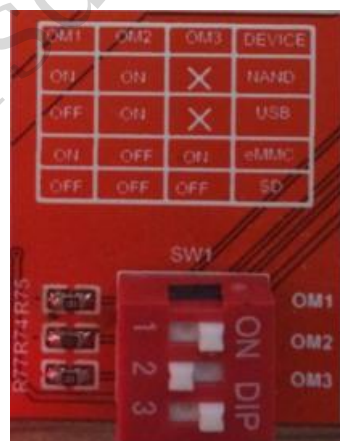
当单独使用开发板时，用户需要按照如下的方式连接好平台，保证串口、网线及电源准确无误的连接，如有烧写需要则连接 OTG 线。电源线用来给开发板供电，网线用来传输数据（nfs 方式必须连接网线），串口线用来接收系统启动信息，OTG 线用来进行镜像烧写。



当用户使用实验箱时，处电源和串口需按下图链接，其他接线位置不变。



在供电之前，请先检查拨码开关是否为 eMMC 启动方式，尽量使用 eMMC 启动方式。



3.2 设置串口调试工具

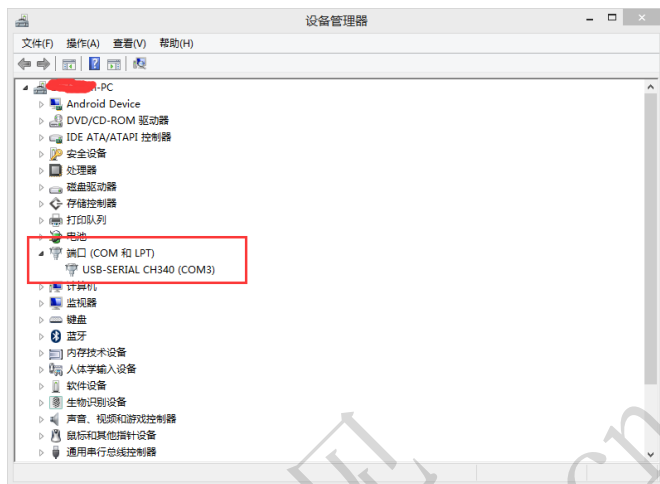
一般的开发平台都会留有系统调试的串口，目的是让用户可以通过串口和开发板进行通信，一般打开



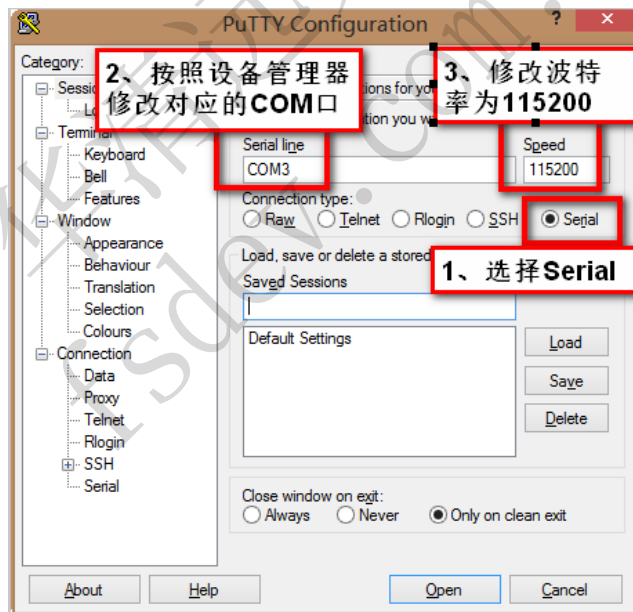
系统开关时，系统会向串口输出启动信息，此时我们接入相应的串口线（我们提供的串口线为 CH340USB 转串口线），让数据通过串口线传至 PC 机上，我们在 PC 上使用串口软件（putty）进行信息显示。Putty 在使用时需要首先配置一下，配置过程如下。

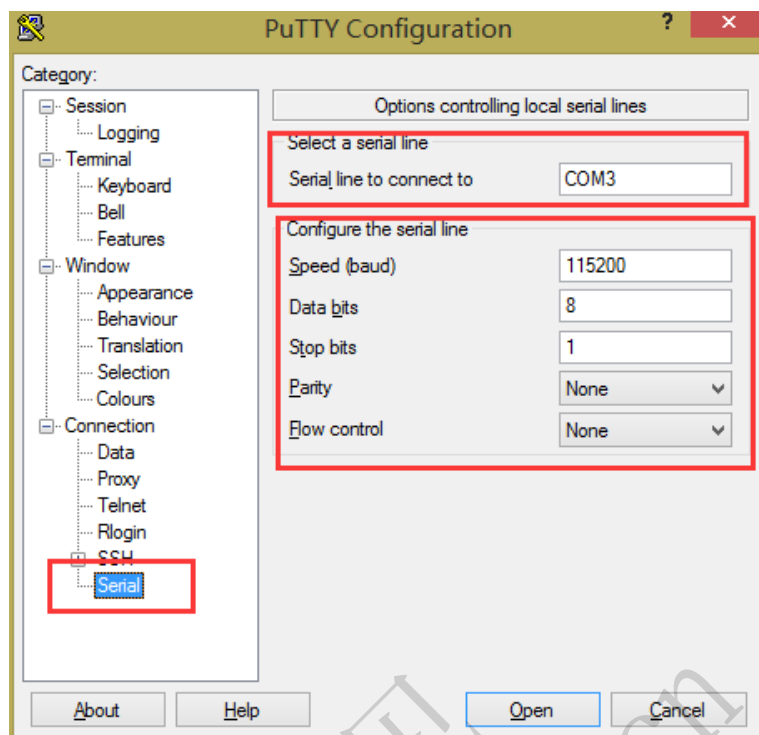
打开【华清远见-嵌入式 ARM 实验箱资料- I -FS6818: \工具软件\串口调试工具\putty.exe】文件。

打开设备管理器，确定当前 PC 机上的串口号。



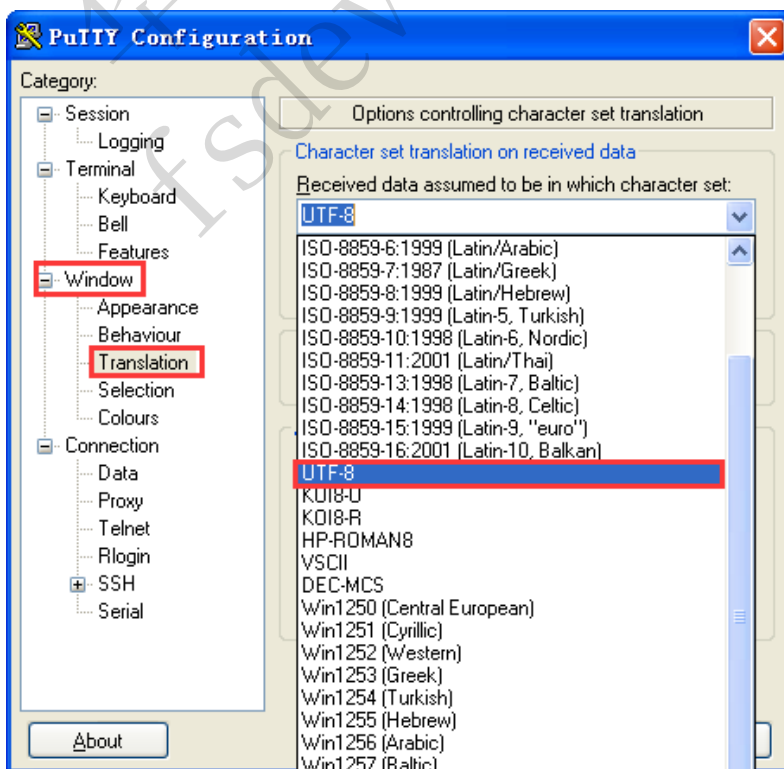
选择串口（Serial）连接方式：





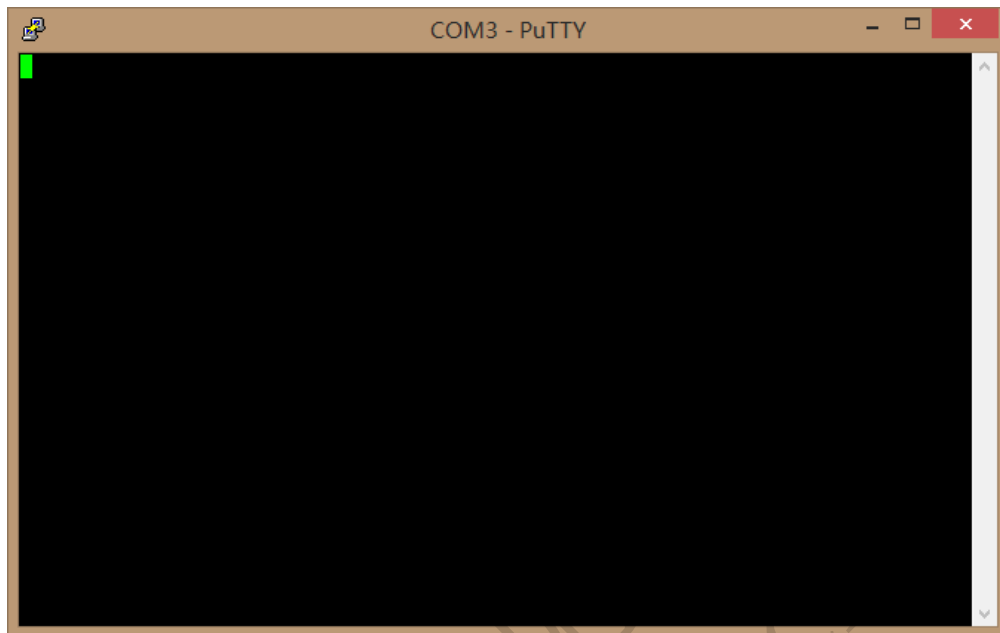
选中第一步方框内的 Serial,再点击第二步中的 Serial,进入串口设置的对话框（注意：这里如果使用台式机物理串口一般为 COM1；使用 USB 转串口线需要提前安装驱动程序，光盘中提供常用的 CH340 和 PL2303 两种驱动，路径为【华清远见-嵌入式 ARM 实验箱资料- I -FS6818: \工具软件\USB 转串口驱动】，如果使用其他 USB 转串口线请自行安装驱动程序）。

再选中 Window 中的 Translation,在选项栏中选择文本编码格式为 UTF-8，防止出现中文乱码：





点击 open 打开串口。



打开串口后，不提示任何错误信息说明串口号是正确的，开发平台供电后，能正常显示启动信息，无乱码说明串口设置是正常的。

开发板供电之后的现象应和下面一致。

```
-----
Second Boot by Nexell Co. : Ver0.3.6 - Built on Jun 12 2015 20:06:08
-----
NSIH : Ver0.0.xx

DDR3 POR Init Start 0
phy init
Lock value  = 193
GATE CYC    = 0x00000492
GATE CODE   = 0x00000000
Read DQ     = 0x08080808
Write DQ    = 0x08080808
```

有倒数计时的提示：



```
DSIM_STATUS : 0x10040f
MIPI clk: 420MHz
DSIM_ESCMODE 2 : 0x0
DSIM_STATUS : 0x10040f
## Skip BAT Animation.
## IRQ(0x48) : 0x00 0x00 0x00 0x00 0x00
## chg_type : ADP
## battery_vol : 0mV
## battery_cap : 100%
## Booting
Card did not respond to voltage select!
Hit any key to stop autoboot: 3
```

3.2.1 错误处理

前两节介绍了如何连接开发板和如何设置使用串口，据我们了解很多人在这两步会出现错误，连接串口后无任何输出信息，这里对常见的错误进行一些说明。

- 串口位置必须按照图中的位置接，其他串口并不是调试串口，没有任何打印信息。
- 串口号并非一成不变，每一次插拔都会造成串口号的改变，每次打开串口需通过设备管理器确认设备号。
- 波特率、校验位等参数必须和图中参数一致，否则会出现乱码现象。
- 如果以上都确认是正常的，请断电并断开串口线两端重新连接，连接完成后重启开发板。开发板供电时，确认电源指示灯是常亮状态，并确认当前的启动方式是否为 EMMC 启动。
- 供电及启动方式正常后，如再无启动信息，请确认核心板中是否有程序，如没有程序则可以参照《FS6818-M4 使用手册》制作 SD 卡启动盘章节制作 sd 卡启动盘并从 sd 卡启动。

3.3 目标机启动 Linux 系统

Android 平台是基于 Linux 内核搭建的，所以安卓系统也可以完成大部分 Linux 操作，但 Android 平台和一般的 Linux 系统存在一定的差异，所以建议使用 Linux 系统进行操作。

本手册中提供两种方式启动 Linux 系统：NFS 挂载方式启动和 eMMC 方式启动，这两种启动方式各有优点，在开发测试阶段通常选择 NFS 挂载方式启动，因为在这种方式下我们可以通过直接修改虚拟机本地文件系统从而达到测试的效果，当然也可以将文件系统制作成镜像直接烧写到目标机的 eMMC 上实现开发板脱机工作。

3.3.1 通过 eMMC 方式启动 Linux

本节通过 fastboot 工具烧写 Linux 系统，如果为第一次烧写需要首先在 PC 端拷贝 fastboot 工具及安装驱动，详细步骤请查阅《FS6818-M4 使用手册》环境配置章节

通过 usb 线连接目标机与 PC，开发板供电后，在开机倒计时结束前按任意键进入 Uboot 交互窗口并输入如下命令。

```
# fastboot
```

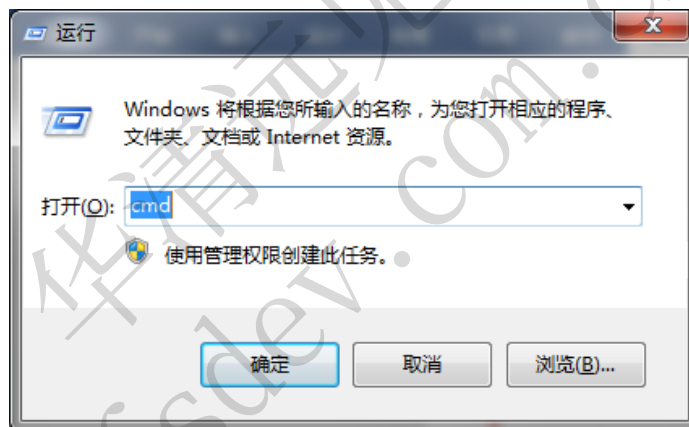


```
Card did not respond to voltage select!
Hit any key to stop autoboot: 0
FS6818# fastboot

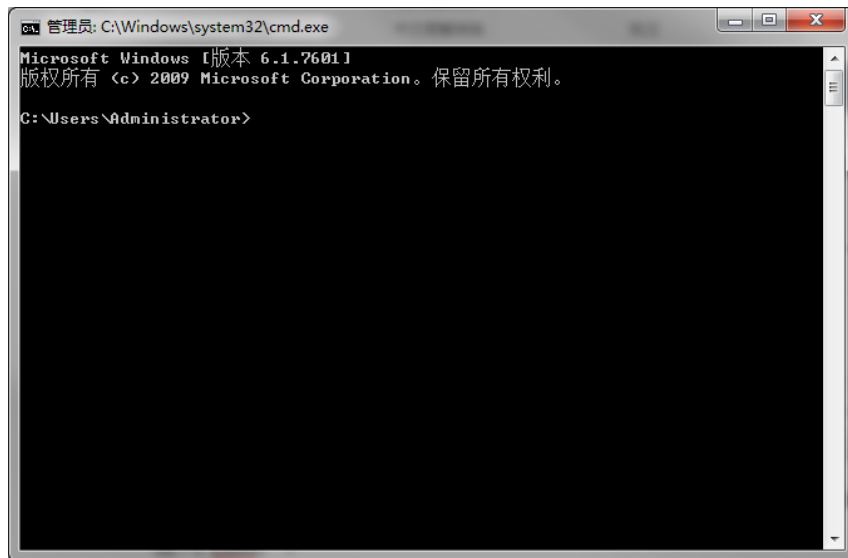
Fastboot Partitions:
mmc.2: ubootpak, img : 0x200, 0x78000
mmc.2: 2ndboot, img : 0x200, 0x4000
mmc.2: bootloader, img : 0x8000, 0x70000
mmc.2: boot, fs : 0x100000, 0x4000000
mmc.2: system, fs : 0x4100000, 0x2f200000
mmc.2: cache, fs : 0x33300000, 0x1ac00000
mmc.2: misc, fs : 0x4e000000, 0x800000
mmc.2: recovery, fs : 0x4e900000, 0x1600000
mmc.2: userdata, fs : 0x50000000, 0x0
Support fstype : 2nd boot factory raw fat ext4 emmc nand ubi ubifs
Reserved part : partmap mem env cmd
DONE: Logo bmp 1024 by 600 (3bpp), len=1843256
DRAW: 0x47000000 -> 0x46000000
Load USB Driver: android
Core usb device tie configuration done
OTG cable Connected!
-----
```

打开【华清远见-嵌入式 ARM 实验箱资料-I-FS6818: 烧写镜像\linux 镜像】目录，将该目录下所有文件拷贝到【D:\share\linux\】文件下（用户也可以通过本文“linux 系统编译”章节自定义系统镜像）。

在 window 系统下面进行烧写，点击 window+R:



输入 cmd:



进入烧写镜像的文件夹里面进行烧写，这里放在 D:\share\linux 文件夹下：

输入 D: +回车

输入 cd share\linux+回车

a) 烧写 uboot

> fastboot flash ubootpak ubootpak.bin

```
D:\share\linux>fastboot flash ubootpak ubootpak.bin
target reported max download size of 38000000 bytes
sending 'ubootpak' (296 KB)...
OKAY [ 0.328s]
writing 'ubootpak'...
OKAY [ 0.151s]
finished. total time: 0.483s
```

b) 烧写内核

> fastboot flash boot boot.img

```
D:\share\linux>fastboot flash boot boot.img
target reported max download size of 38000000 bytes
sending 'boot' (19548 KB)...
OKAY [ 3.059s]
writing 'boot'...
OKAY [ 2.768s]
finished. total time: 5.831s
```

c) 烧写文件系统

> fastboot flash system system_linux.img

```
D:\share\linux>fastboot flash system system_linux.img
target reported max download size of 38000000 bytes
sending 'system' (13745 KB)...
OKAY [ 2.200s]
writing 'system'...
OKAY [ 3.654s]
finished. total time: 5.860s
```

到这里为止系统所需要的镜像全部烧写完毕，重启开发板并在倒数计时结束前按任意键进入 Uboot 交互窗口，输入如下命令查看当前的环境变量。

print



我们需要设置 eMMC 启动的环境变量参数，命令如下：（该命令为一行命令，每一个单词数据之间均有空格）

```
# setenv bootcmd ext4load mmc 2:1 0x48000000 uImage\;bootm 0x48000000
# setenv bootargs lcd=vs070cxn root=/dev/mmcblk0p2 rw rootfstype=ext4
# saveenv
```

重新启动开发板，会进入到 Linux 系统，此时可以在串口（putty）操作 Linux。

3.3.2 通过 NFS 挂载方式启动 Linux

NFS 为网络文件系统(Network file system)的缩写，由 Sun 公司开发，通过网络，NFS 支持在不同的文件系统之间共享文件，因此在启动 Linux 前必须确保目标机与 PC 通过网线连接并处于相同网段。

通过 NFS 挂载方式启动 Linux 需确保目标机已经烧写 Uboot 及内核，读者可通过烧写 eMMC 方式进行烧写 Uboot 和内核（linux 镜像中 uboot 镜像与安卓镜像相同，Linux 内核镜像为未安装驱动镜像，为配合后续实验，请用户烧写 linux 镜像的内核）。

修改开发板环境变量（该命令为一行命令，每一个单词数据之间均有空格）：

```
# setenv serverip 192.168.100.192
# setenv ipaddr 192.168.100.191
# setenv gatewayip 192.168.100.1
# setenv bootcmd ext4load mmc 2:1 0x48000000 uImage\;bootm 0x48000000
# setenv bootargs root=/dev/nfs nfsroot=192.168.100.192:/source/rootfs console=ttySAC0,115200
init=/linuxrc ip=192.168.100.191:::eth0:off
# saveenv
```

打开【华清远见-嵌入式 ARM 实验箱资料-I-FS6818：程序源码\文件系统源码】目录，将该目录下的文件拷贝到【D:\share\】文件下（虚拟机共享目录、详细内容请参照本文【配置优化虚拟机】章节）。

进入虚拟机输入如下命令将文件系统拷贝到【/source】路径下并解压。

```
$ cp /mnt/hgfs/share/rootfs.tar.xz /source/
$ tar xvf rootfs.tar.xz
```

重启开发板，如下图所示表明文件系统挂载成功，输入如下命令查看文件系统。

```
# ls
```

```
[ 10.621000] PHY: stmmac-0:04 - Link is Up - 100/Full
[ 10.690000] VFS: Mounted root (nfs filesystem) on device 0:11.
[ 10.691000] devtmpfs: mounted
[ 10.694000] Freeing init memory: 240K
[ 10.697000] Write protecting the kernel text section c0008000 - c08de000
[ 10.705000] rodata_test: attempting to write to read-only section:
[ 10.710000] write to read-only section trapped, success
[root@farsight]#
[root@farsight]# ls
bin          fs6818M4_test  mnt          sbin         usr
dev          lib            proc         sys          var
etc          linuxrc       root         tmp
```



第 4 章 Linux 系统编译

4.1 准备源码

4.1.1 拷贝内核源码

本章节编译源码所使用的环境为上述的【华清远见开发环境】，强烈推荐用户使用此环境作为此开发板的开发环境。用户也可以使用非虚拟机的方式，或者使用其他发行版本的 Linux 操作系统，方法参见本章节，就不再详细叙述。

建议用户首先在虚拟机的工作目录下建立自己的工作目录，此处就以【/home/linux/workdir】为例。

```
$ mkdir /home/linux/workdir -p
```

首先将 FS6818 开发平台的 linux 内核源码拷贝到虚拟机的共享目录（D:\share\）下。（源码位置：【华清远见-嵌入式 ARM 实验箱资料- I -FS6818: \程序源码\linux 内核源码】）



此时，我们就可以在虚拟机中查看到拷贝的源码了。

拷贝源码到工作目录下

```
$ cd /home/linux/workdir
```

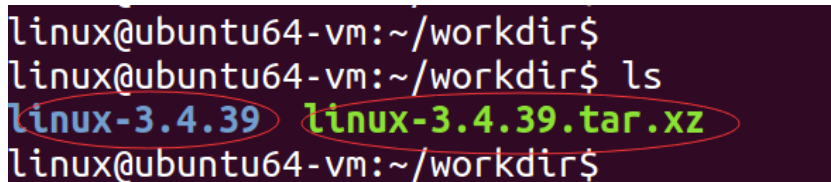
```
$ cp /mnt/hgfs/share/FS6818/linux-3.4.39.tar.xz .
```

4.1.2 解压内核源码

拷贝源码到虚拟机下后解压源码：

```
$ tar xvf linux-3.4.39.tar.xz
```

解压后显示如下：



4.1.3 拷贝 ext4 文件系统制作工具

拷贝【华清远见-嵌入式 ARM 实验箱资料- I -FS6818: \工具软件\make_ext4fs】拷贝到虚拟机“/usr/bin”目录下，为后文制作内核镜像提供工具。



4.2 编译源码

4.2.1 编译 Bootloader 源码

Bootloader 源码包含在安卓源码内，Linux 系统的 bootloader 镜像与安卓的 bootloader 镜像通用，如果用户希望自行编译 bootloader 源码，请查看《Android 底层及应用开发》手册“Android 系统编译”章节。

4.2.2 编译内核源码

进入已经准备好的内核源码目录，查看内核源码：

```
linux@ubuntu64-vm:~/workdir/linux-3.4.39$ ls
android  COPYING      drivers  ipc      MAINTAINERS  net      security  usr
arch     CREDITS      firmware Kbuild   Makefile     README   sound     virt
block    crypto       fs       Kconfig  mkboot.sh    REPORTING-BUGS  System.map  vmlinux
boot     diff.txt     include  kernel   mm           samples    tags       vmlinux.o
boot.img Documentation init     lib       Module.symvers  scripts    tools      vmlinux
```

【配置源码】

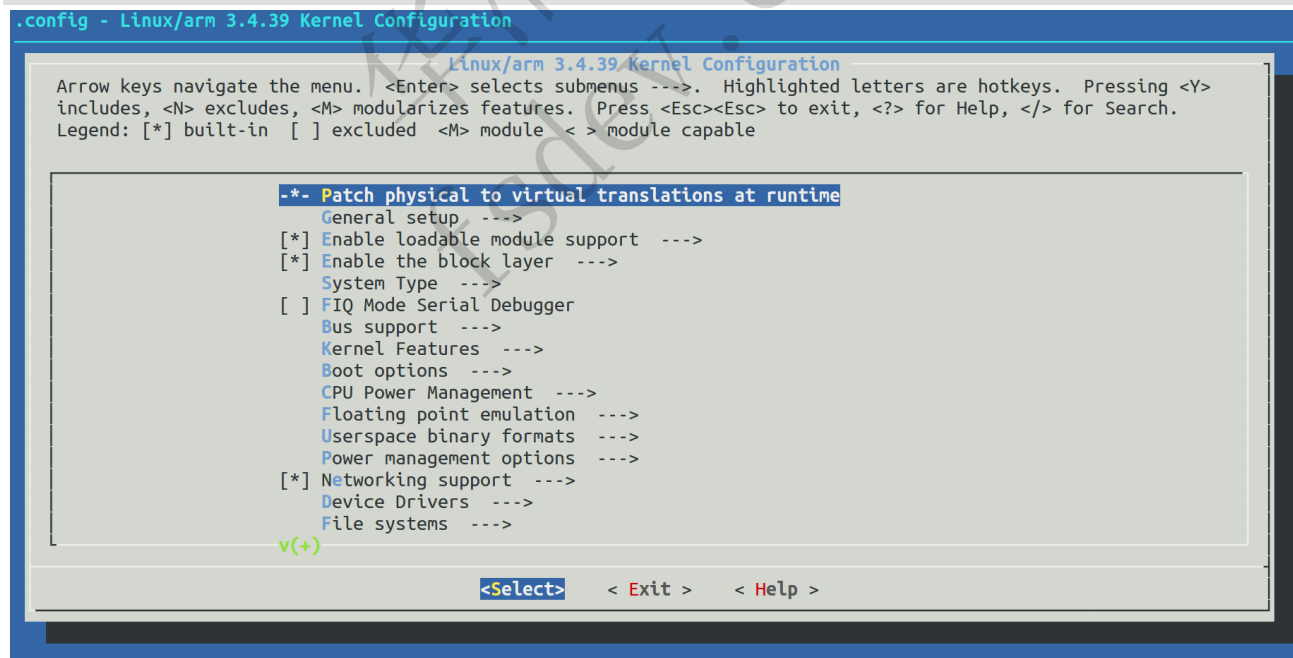
Linux 内核通常使用 menuconfig 图形界面配置内核编译选项，配置更改的内容会保存在内核源码目录下的【.config】文件中。首先拷贝相关开发平台的标准配置文件为【.config】。（此处以 Fs4412M4 为例）

```
$ cp arch/arm/configs/fs6818M4_defconfig_vX .config // X 为版本号，请拷贝最新版本的配置文件
```

```
linux@ubuntu64-vm:~/workdir/linux-3.4.39$ cp arch/arm/configs/fs6818M4_defconfig_v5 .
linux@ubuntu64-vm:~/workdir/linux-3.4.39$
```

在终端下输入如下列命令可以进入到 Linux 内核配置图形界面。

```
$ make menuconfig
```



Menuconfig 菜单使用【Enter】键进入下级菜单；使用【Space】键选中或者清除选项；使用【?】查看此菜单的帮助文件，使用连击【ESC】两次后退至上级菜单。

【编译源码】



编译内核源码前需要先查看源码配置的交叉编译链，进入内核源码，查看 Makefile 文件：

```
194 export KBUILD_BUILDHOST := $(SUBARCH)
195 ARCH           ?= arm
196 #CROSS_COMPILE ?= $(CONFIG_CROSS_COMPILE:""=)
197 CROSS_COMPILE ?= /usr/local/toolchain/toolchain-4.5.1/bin/arm-none-linux-gnueabi-
198 # Architecture as present in compile.h
```

搜索 CROSS_COMPILE 关键字，配置交叉编译工具路径，如使用华清远见开发环境可忽略此步骤。

进入内核源码首层目录，输入 make uImage 编译内核：

```
Kernel: arch/arm/boot/Image is ready
Kernel: arch/arm/boot/zImage is ready
UIIMAGE arch/arm/boot/uImage
Image Name:   Linux-3.4.39-farsight
Created:      Tue Dec 19 11:24:22 2017
Image Type:   ARM Linux Kernel Image (uncompressed)
Data Size:    4994192 Bytes = 4877.14 kB = 4.76 MB
Load Address: 0x40008000
Entry Point:  0x40008000
Image arch/arm/boot/uImage is ready
```

执行脚本打包内核镜像：

```
$ ./mkboot.sh
```

```
linux@ubuntu64-vm:~/workdir/linux-3.4.39$ ./mkboot.sh
Creating filesystem with parameters:
  Size: 67108864
  Block size: 4096
  Blocks per group: 32768
  Inodes per group: 4096
  Inode size: 256
  Journal blocks: 1024
  Label:
  Blocks: 16384
  Block groups: 1
  Reserved block group size: 7
Created filesystem with 14/4096 inodes and 3137/16384 blocks
```

6818 平台烧写内核镜像为 boot.img，改文件为 ext4 格式，其中包括开机 LOGO、ramdisk 文件系统及 uImage 内核镜像，所以在编译完内核后需要按照 6818 平台需求进行打包，如上图所示即为打包成功。

```
linux@ubuntu64-vm:~/workdir/linux-3.4.39$ ls
android  crypto          include  MAINTAINERS    net          System.map  vmlinux
arch     diff.txt        init     Makefile        README      tags
block   Documentation   ipc      mkboot.sh      REPORTING-BUGS  tools
boot    drivers        Kbuild  mm             samples      usr
boot.img  firmware      Kconfig modules.builtin  scripts      virt
COPYING  fs            kernel  modules.order  security     vmlinux
CREDITS  fs6818M4_defconfig_v5 lib      Module.symvers  sound        vmlinux.o
```



4.2.3 Linux 文件系统镜像制作

【准备文件系统源码】

首先将 FS6818 开发平台的源码拷贝到虚拟机的共享目录（D:\share\）下。按照【运行开发环境】章节中【配置优化虚拟机】中【增加共享目录】的步骤，将源码（源码位置：【华清远见-嵌入式 ARM 实验箱资料-I-FS6818：程序源码\文件系统源码】）拷贝到虚拟机共享文件夹下。

名称	修改日期	类型	大小
 rootfs.tar.xz	2017/3/27 12:31	WinRAR 压缩文件	5,001 KB

此时，我们就可以在虚拟机中查看到拷贝的源码了。

\$ ls -l /mnt/hgfs/share/rootfs.tar.xz

```
linux@ubuntu64-vm:~/workdir$ ls -l /mnt/hgfs/share/rootfs.tar.xz
-rwxrwxrwx 1 root root 5120004 3月 27 12:31 /mnt/hgfs/share/rootfs.tar.xz
```

拷贝源码到 Linux 下

\$ cd /source

\$ cp /mnt/hgfs/share/rootfs.tar.xz .

```
linux@ubuntu64-vm:/source$ cp /mnt/hgfs/share/rootfs.tar.xz .
linux@ubuntu64-vm:/source$ ls rootfs.tar.xz
rootfs.tar.xz
```

输入如下命令解压源码：

\$ tar -xvf rootfs.tar.xz

\$ ls rootfs

```
linux@ubuntu64-vm:/source$ ls rootfs
bin dev etc fs6818M4_test lib linuxrc mnt -p proc root sbin sys tmp usr var
```

输入如下命令将文件系统源码打包成 ext4 镜像：

\$ make_ext4fs -s system_linux.img -l 300M rootfs/

\$ ls system_linux.img

```
linux@ubuntu64-vm:/source$ ls system_linux.img
system_linux.img
```

至此 linux 系统所需镜像已经制作完成，用户可通过本文“通过 eMMC 方式启动 linux”章节验证镜像。



第 5 章 Linux 基础操作

Linux 是个高可靠、高性能的系统，而所有这些优越性只有在直接使用 Linux 命令行（Shell 环境）才能充分地体现出来。本章内容让大家掌握一些基本的 Linux 命令并能够独立定制 Linux 中的系统服务。

5.1 Linux 基本命令

在安装完 Linux 再次启动之后，就可以进入到与 Windows 类似的图形化界面了。这个界面就是 Linux 图形化界面 X 窗口系统（简称 X）的一部分。要注意的是，X 窗口系统仅仅是 Linux 上面的一个软件（或者也可称为服务），它不是 Linux 自身的一部分。虽然现在的 X 窗口系统已经与 Linux 整合地相当好了，但毕竟还不能保证绝对的可靠性。另外，X 窗口系统是一个相当耗费系统资源的软件，它会大大地降低 Linux 的系统性能。因此，若是希望更好地享受 Linux 所带来的高效及高稳定性，建议读者尽可能地使用 Linux 的命令行界面，也就是 Shell 环境。

当用户在命令行下工作时，不是直接同操作系统内核交互信息的，而是由命令解释器接受命令，分析后再传给相关的程序。Shell 是一种 Linux 中的命令行解释程序，就如同 Command.com 是 DOS 下的命令解释程序一样，为用户提供使用操作系统的接口。它们之间的关系如图 2.1 所示。用户在提示符下输入的命令都由 Shell 先解释然后传给 Linux 内核。

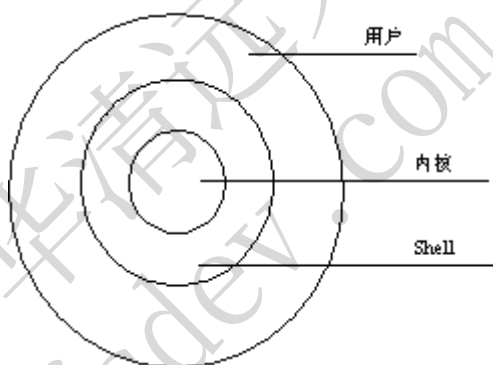


图 2.1 内核、Shell 和用户的关系

Linux 中运行 Shell 的环境是“系统工具”下的“终端”，读者可以单击“终端”以启动 Shell 环境。这时屏幕上显示类似“[linux@ubuntu64]\$”的信息，其中，linux 是指系统用户，而 home 是指当前所在的目录。

由于 Linux 中的命令非常多，要全部介绍几乎不可能。因此，在本书中按照命令的用途进行分类讲解，并且对每一类中最常用的命令详细讲解，同时列出同一类中的其他命令。由于同一类的命令都有很大的相似性，因此，读者通过学习本书中所列命令，可以很快地掌握其他命令。

命令格式说明。

- 格式中带[]的表明为可选项，其他为必选项。
- 选项可以多个连带写入。
- 本章后面选项参数列表表中加粗的含义是：该选项是非常常用的选项。



5.1.1 用户系统相关命令

Linux 是一个多用户的操作系统，每个用户又可以属于不同的用户组，下面，首先来熟悉一下 Linux 中的用户切换和用户管理的相关命令。

1. 用户切换 (su)

(1) 作用

变更为其它使用者的身份，主要用于将普通用户身份转变为超级用户，而且需输入相应用户密码。

(2) 格式

`su [选项] [使用者]`

其中的使用者为要变更的对应使用者。

(3) 常见参数

主要选项参数见表 2.1 所示。

表 2.1 su 命令常见参数列表

选 项	参 数 含 义
<code>-, -l, --login</code>	为该使用者重新登录，大部分环境变量（如 HOME、SHELL 和 USER 等）和工作目录都是以该使用者（USER）为主。若没有指定 USER，缺省情况是 root
<code>-m, -p</code>	执行 su 时不改变环境变量
<code>-c, --command</code>	变更账号为 USER 的使用者，并执行指令（command）后再变回原来使用者

(4) 使用示例

```
[linuxlinux@www linux]$ su - root
Password:
[root@www root]#
```

示例通过 su 命令将普通用户变更为 root 用户，并使用选项“-”携带 root 环境变量。

(5) 使用说明

- 在将普通用户变更为 root 用户时建议使用“-”选项，这样可以将 root 的环境变量和工作目录同时带入，否则在以后的使用中可能会由于环境变量的原因而出错。

- 在转变为 root 权限后，提示符变为#。

2. 用户管理 (useradd 和 passwd)

Linux 中常见用户管理命令如表 2.2 所示，本书仅以 useradd 和 passwd 为例进行详细讲解，其他命令



类似，请读者自行学习使用。

表 2.2 Linux 常见用户管理命令

命 令	命 令 含 义	格 式
useradd	添加用户账号	useradd [选项] 用户名
usermod	设置用户账号属性	usermod [选项] 属性值
userdel	删除对应用户账号	userdel [选项] 用户名
groupadd	添加组账号	groupadd [选项] 组账号
groupmod	设置组账号属性	groupmod [选项] 属性值
groupdel	删除对应组账号	groupdel [选项] 组账号
passwd	设置账号密码	passwd [对应账号]
id	显示用户 ID、组 ID 和用户所属的组列表	id [用户名]
groups	显示用户所属的组	groups [组账号]
who	显示登录到系统的所有用户	who

(1) 作用

- ① useradd: 添加用户账号。
- ② passwd: 更改对应用户账号密码。

(2) 格式

- ① useradd: useradd [选项] 用户名。
- ② passwd: passwd [选项] [用户名]。

其中的用户名为修改账号密码的用户，若不带用户名，缺省为更改当前使用者账号密码。

(3) 常用参数

- ① useradd 主要选项参数见表 2.3 所示。

表 2.3 useradd 命令常见参数列表

选 项	参 数 含 义
-g	指定用户所属的群组
-m	自动建立用户的登入目录
-n	取消建立以用户名称为名的群组

- ② passwd: 一般很少使用选项参数。

(4) 使用实例

```
[root@www root]# useradd ycw
```

```
[root@www root]# passwd ycw
```



```
New password:
Retype new password:
passwd: all authentication tokens updated successfully
[root@www root]# su - ycw
[ycwycw@www ycw]$
[ycw@www ycw]$ pwd (查看当前目录)
/home/ycw
```

实例中先添加了用户名为 ycw 的用户，接着又为该用户设置了账号密码。并从 su 的命令可以看出，该用户添加成功，其工作目录为“/home/ycw”。

(5) 使用说明

- 在使用添加用户时，这两个命令是一起使用的，其中，useradd 必须用 root 的权限。而且 useradd 指令所建立的账号，实际上是保存在“/etc/passwd”文本文件中，文件中每一行包含一个账号信息。
- 在缺省情况下，useradd 所做的初始化操作包括在“/home”目录下为对应账号建立一个名为同名的主目录，并且还为该用户单独建立一个与用户名同名的组。
- adduser 只是 useradd 的符号链接（关于符号链接的概念在本节后面会有介绍），两者是相同的。
- passwd 还可用于普通用户修改账号密码，Linux 并不采用类似 windows 的密码回显（显示为*号），所以输入的这些字符用户是看不见的。密码最好包括字母、数字和特殊符号，并且设成 6 位以上。

3. 系统管理命令（ps 和 kill）

Linux 中常见的系统管理命令如表 2.4 所示，本书以 ps 和 kill 为例进行讲解。

表 2.4 Linux 常见系统管理命令

命 令	命 令 含 义	格 式
ps	显示当前系统中由该用户运行的进程列表	ps [选项]
top	动态显示系统中运行的程序（一般为每隔 5s）	top
kill	输出特定的信号给指定 PID（进程号）的进程	kill [选项] 进程号 (PID)
uname	显示系统的信息（可加选项-a）	uname [选项]
setup	系统图形化界面配置	setup
crontab	循环执行例行性命令	crontab [选项]
shutdown	关闭或重启 Linux 系统	shutdown [选项] [时间]
uptime	显示系统已经运行了多长时间	uptime
clear	清除屏幕上的信息	clear

(1) 作用

- ① ps: 显示当前系统中由该用户运行的进程列表。



② **kill**: 输出特定的信号给指定 PID（进程号）的进程，并根据该信号而完成指定的行为。其中可能的信号有进程挂起、进程等待、进程终止等。

(2) 格式

① **ps**: **ps** [选项]。

② **kill**: **kill** [选项] 进程号 (PID)。

kill 命令中的进程号为信号输出的指定进程的进程号，当选项是缺省时为输出终止信号给该进程。

(3) 常见参数

① **ps** 主要选项参数见表 2.5 所示。

表 2.5 **ps** 命令常见参数列表

选 项	参 数 含 义
-ef	查看所有进程及其 PID（进程号）、系统时间、命令详细目录、执行者等
-aux	除可显示 -ef 所有内容外，还可显示 CPU 及内存占用率、进程状态
-w	显示加宽并且可以显示较多的信息

② **kill** 主要选项参数见表 2.6 所示。

表 2.6 **kill** 命令常见参数列表

选 项	参 数 含 义
-s	根据指定信号发送给进程
-p	打印出进程号 (PID)，但并不送出信号
-l	列出所有可用的信号名称

(4) 使用实例

```
[root@www root]# ps -ef
```

```

UID      PID  PPID  C  STIME TTY          TIME CMD
root      1    0    0   2005 ?        00:00:05 init
root      2    1    0   2005 ?        00:00:00 [keventd]
root      3    0    0   2005 ?        00:00:00 [ksoftirqd_CPU0]
root      4    0    0   2005 ?        00:00:00 [ksoftirqd_CPU1]
root    7421    1    0   2005 ?        00:00:00 /usr/local/bin/ntpd -c /etc/ntp.
root   21787 21739  0 17:16 pts/1    00:00:00 grep ntp
```

```
[root@www root]# kill 7421
```

```
[root@www root]# ps -ef|grep ntp
```



```
root      21789 21739  0 17:16 pts/1    00:00:00 grep ntp
```

该实例中首先查看所有进程，并终止进程号为 7421 的 ntp 进程，之后再次查看时已经没有该进程号的进程。

(5) 使用说明

- ps 在使用中通常可以与其他一些命令结合起来使用，主要作用是提高效率。
- ps 选项中的参数 w 可以写多次，通常最多写 3 次，它的含义表示加宽 3 次，这足以显示很长的命令行了。例如：ps -auxwww。

4. 磁盘相关命令 (fdisk)

Linux 中与磁盘相关的命令如表 2.7 所示，本书仅以 fdisk 为例进行讲解。

表 2.7 Linux 常见系统管理命令

选 项	参 数 含 义	格 式
free	查看当前系统内存的使用情况	free [选项]
df	查看文件系统的磁盘空间占用情况	df [选项]
du	统计目录（或文件）所占磁盘空间的大小	du [选项]
fdisk	查看硬盘分区情况及对硬盘进行分区管理	fdisk [-l]

(1) 作用

fdisk 可以查看硬盘分区情况，并可对硬盘进行分区管理，这里主要向读者介绍查看硬盘分区情况，另外，fdisk 也是一个非常好的硬盘分区工具，感兴趣的读者可以另外查找资料学习使用 fdisk 进行硬盘分区。

(2) 格式

fdisk [-l]

(3) 使用实例

```
[root@linux ~]# fdisk -l
```

```
Disk /dev/hda: 40.0 GB, 40007761920 bytes
```

```
240 heads, 63 sectors/track, 5168 cylinders
```

```
Units = cylinders of 15120 * 512 = 7741440 bytes
```

Device	Boot	Start	End	Blocks	Id	System
/dev/hda1	*	1	1084	8195008+	c	W95 FAT32 (LBA)
/dev/hda2		1085	5167	30867480	f	W95 Ext'd (LBA)
/dev/hda5		1085	2439	10243768+	b	W95 FAT32



/dev/hda6	2440	4064	12284968+	b	W95 FAT32
/dev/hda7	4065	5096	7799526	83	Linux
/dev/hda8	5096	5165	522081	82	Linux swap

可以看出，使用“fdisk -l”列出了文件系统的分区情况。

(4) 使用说明

- 使用 fdisk 必须拥有 root 权限。
- IDE 硬盘对应的设备名称分别为 hda、hdb、hdc 和 hdd，SCSI 硬盘对应的设备名称则为 sda、sdb、…此外，hda1 代表 hda 的第一个硬盘分区，hda2 代表 hda 的第二个分区，依此类推。
- 通过查看/var/log/messages 文件，可以找到 Linux 系统已辨认出来的设备代号。

5. 磁盘挂载命令 (mount)

(1) 作用

挂载文件系统，它的使用权限是超级用户或/etc/fstab 中允许的使用者。正如 1.2.1 节中所述，挂载是指把分区和目录对应的过程，而挂载点是指挂载在文件树中的位置。mount 命令就可以把文件系统挂载到相应的目录下，并且由于 Linux 中把设备都当作文件一样使用，因此，mount 命令也可以挂载不同的设备。

通常，在 Linux 下“/mnt”目录是专门用于挂载不同的文件系统的，它可以在该目录下新建不同的子目录来挂载不同的设备文件系统。

(2) 格式

mount [选项] [类型] 设备文件名 挂载点目录
其中的类型是指设备文件的类型。

(3) 常见参数

mount 常见参数如表 2.8 所示。

表 2.8 mount 命令选项常见参数列表

选 项	参 数 含 义
-a	依照/etc/fstab 的内容装载所有相关的硬盘
-l	列出当前已挂载的设备、文件系统名称和挂载点
-t 类型	将后面的设备以指定类型的文件格式装载到挂载点上。常见的类型有前面介绍过的几种：vfat、ext3、ext2、iso9660、nfs 等
-f	通常用于除错。它会使 mount 不执行实际挂上的动作，而是模拟整个挂上的过程，通常会和-v 一起使用

(4) 使用实例

使用 mount 命令主要通过以下几个步骤。



① 确认是否为 Linux 可以识别的文件系统，Linux 可识别的文件系统只要是以下几种。

- Windows95/98 常用的 FAT32 文件系统：vfat。
- WinNT/2000 的文件系统：ntfs。
- OS/2 用的文件系统：hpfs。
- Linux 用的文件系统：ext2、ext3、nfs。
- CD-ROM 光盘用的文件系统：iso9660。

② 确定设备的名称，确定设备名称可通过使用命令“fdisk -l”查看。

③ 查找挂接点。

必须确定挂载点已经存在，也就是在“/mnt”下的相应子目录已经存在，一般建议在“/mnt”下新建几个如“/mnt/windows”，“/mnt/usb”的子目录，现在有些新版本的 Linux 都可自动挂载文件系统，。

④ 挂载文件系统如下所示。

```
[root@linux mnt]# mount -t vfat /dev/hda1 /mnt/c
[root@linux mnt]# cd /mnt/c
24.s03e01.pdtv.xvid-sfm.rmvb  Documents and Settings  Program Files
24.s03e02.pdtv.xvid-sfm.rmvb  Downloads                Recycled
...
```

C 盘是原先笔者 Windows 系统的启动盘。可见，在挂载了 C 盘之后，可直接访问 Windows 下的 C 盘的内容。

⑤ 在使用完该设备文件后可使用命令 umount 将其卸载。

```
[root@linux mnt]# umount /mnt/c
[root@linux mnt]# cd /mnt/c
[root@linux c]# ls
```

可见，此时目录“/mnt/c”下为空。Windows 下的 C 盘成功卸载。

5.1.2 文件目录相关命令

由于 Linux 中有关文件目录的操作非常重要，也非常常用，因此在本节中，作者将基本所有的文件操作命令都进行了讲解。

1. cd

(1) 作用

改变工作目录。

(2) 格式



`cd [路径]`

其中的路径为要改变的工作目录，可为相对路径或绝对路径。

(3) 使用实例

```
[root@www uclinux]# cd /home/linux/
[root@www linux]# pwd
[root@www linux]# /home/linux/
```

该实例中变更工作目录为“/home/linux/”，在后面的pwd（显示当前目录）的结果中可以看出。

(4) 使用说明

- 该命令将当前目录改变至指定路径的目录。若没有指定路径，则回到用户的主目录。为了改变到指定目录，用户必须拥有对指定目录的执行和读权限。

- 该命令可以使用通配符。
- 可使用“`cd -`”可以回到前次工作目录。
- “`./`”代表当前目录，“`../`”代表上级目录。

2. ls

(1) 作用

列出目录的内容。

(2) 格式：`ls [选项] [文件]`

其中文件选项为指定查看指定文件的相关内容，若未指定文件，默认查看当前目录下的所有文件。

(3) 常见参数

ls 主要选项参数见表 2.9 所示

表 2.9

ls 命令常见参数列表

选 项	参 数 含 义
<code>-l, --format=single-column</code>	一行输出一个文件（单列输出）
<code>-a, -all</code>	列出目录中所有文件，包括以“ <code>.</code> ”开头的文件
<code>-d</code>	将目录名和其他文件一样列出，而不是列出目录的内容
<code>-l, --format=long, --format=verbose</code>	除每个文件名外，增加显示文件类型、权限、硬链接数、所有者名、组名、大小（Byte）及时间信息（如未指明是其他时间即指修改时间）
<code>-f</code>	不排序目录内容，按它们在磁盘上存储的顺序列出

(4) 使用实例

```
[ycwing@www /]$ ls -l
total 220
```



```
drwxr-xr-x  2 root  root    4096 Mar 31  2005 bin
drwxr-xr-x  3 root  root    4096 Apr  3  2005 boot
-rw-r--r--  1 root  root      0 Apr 24  2002 test.run
...
```

该实例查看当前目录下的所有文件，并通过选项“-l”显示出详细信息。

显示格式说明如下。

文件类型与权限 链接数 文件属主 文件属组 文件大小 修改的时间 名字

(5) 使用说明

- 在ls的常见参数中，-l（长文件名显示格式）的选项是最为常见的。可以详细显示出各种信息。
- 若想显示出所有“.”开头的文件，可以使用-a，这在嵌入式的开发中很常用。

3. mkdir

(1) 作用

创建一个目录。

(2) 格式

mkdir [选项] 路径

(3) 常见参数

mkdir 主要选项参数如表 2.10 所示

表 2.10

mkdir 命令常见参数列表

选 项	参 数 含 义
-m	对新建目录设置存取权限，也可以用 chmod 命令（在本节后期会有详细说明）设置
-p	可以是一个路径名称。此时若此路径中的某些目录尚不存在，在加上此选项后，系统将自动建立好那些尚不存在的目录，即一次可以建立多个目录

(4) 使用实例

```
[root@www linux]# mkdir -p ./hello/my
[root@www my]# pwd (查看当前目录命令)
/home/linux/hello/my
```

该实例使用选项“-p”一次创建了./hello/my 多级目录。

```
[root@www my]# mkdir -m 777 ./why
[root@www my]# ls -l
total 4
drwxrwxrwx  2 root  root    4096 Jan 14 09:24 why
```



该实例使用改选项“-m”创建了相应权限的目录。对于“777”的权限在本节后面会有详细的说明。

(5) 使用说明

该命令要求创建目录的用户在创建路径的上级目录中具有写权限，并且路径名不能是当前目录中已有的目录或文件名称。

4. cat

(1) 作用

连接并显示指定的一个和多个文件的有关信息。

(2) 格式

cat[选项]文件 1 文件 2...

其中的文件 1、文件 2 为要显示的多个文件。

(3) 常见参数

cat 命令的常见参数如表 2.11 所示。

表 2.11

cat 命令常见参数列表

选 项	参 数 含 义
-n	由第一行开始对所有输出的行数编号
-b	和-n 相似，只不过对于空白行不编号

(4) 使用实例

```
[ycw@www ycw]$ cat -n hello1.c hello2.c
1  #include <stdio.h>
2  void main()
3  {
4      printf("Hello!This is my home!\n");
5  }
6  #include <stdio.h>
7  void main()
8  {
9      printf("Hello!This is your home!\n");
10 }
```

在该实例中，指定对 hello1.c 和 hello2.c 进行输出，并指定行号。



5. cp、mv 和 rm

(1) 作用

- ① **cp**: 将给出的文件或目录复制到另一文件或目录中。
- ② **mv**: 为文件或目录改名或将文件由一个目录移入另一个目录中。
- ③ **rm**: 删除一个目录中的一个或多个文件或目录。

(2) 格式

- ① **cp**: **cp** [选项] 源文件或目录 目标文件或目录。
- ② **mv**: **mv** [选项] 源文件或目录 目标文件或目录。
- ③ **rm**: **rm** [选项] 文件或目录。

(3) 常见参数

- ① **cp** 主要选项参数见表 2.12 所示。

表 2.12

cp 命令常见参数列表

选 项	参 数 含 义
-a	保留链接、文件属性，并复制其子目录，其作用等于 dpr 选项的组合
-d	拷贝时保留链接
-f	删除已经存在的目标文件而不提示
-i	在覆盖目标文件之前将给出提示要求用户确认。回答 y 时目标文件将被覆盖，而且是交互式拷贝
-p	此时 cp 除复制源文件的内容外，还将把其修改时间和访问权限也复制到新文件中
-r	若给出的源文件是一目录文件，此时 cp 将递归复制该目录下所有的子目录和文件。此时目标文件必须为一个目录名

- ② **mv** 主要选项参数如表 2.13 所示。

表 2.13

mv 命令常见参数列表

选 项	参 数 含 义
-i	若 mv 操作将导致对已存在的目标文件的覆盖，此时系统询问是否重写，并要求用户回答 y 或 n，这样可以避免误覆盖文件
-f	禁止交互操作。在 mv 操作要覆盖某已有的目标文件时不给任何指示，在指定此选项后，i 选项将不再起作用

- ③ **rm** 主要选项参数如表 2.14 所示。

表 2.14

rm 命令常见参数列表



选 项	参 数 含 义
-i	进行交互式删除
-f	忽略不存在的文件，但从不给出提示
-r	指示 rm 将参数中列出的全部目录和子目录均递归地删除

(4) 使用实例

① cp

```
[root@www hello]# cp -a ./my/why/ ./
[root@www hello]# ls
my  why
```

该实例使用-a 选项将“/my/why”目录下的所有文件复制到当前目录下。而此时在原先目录下还有原有的文件。

② mv

```
[root@www hello]# mv -i ./my/why/ ./
[root@www hello]# ls
my  why
```

该实例中把“/my/why”目录下的所有文件移至当前目录，则原目录下文件被自动删除。

③ rm

```
[root@www hello]# rm -r -i ./why
rm: descend into directory './why'? y
rm: remove './why/my.c'? y
rm: remove directory './why'? y
```

该实例使用“-r”选项删除“./why”目录下所有内容，系统会进行确认是否删除。

(5) 使用说明

① cp: 该命令把指定的源文件复制到目标文件或把多个源文件复制到目标目录中。

② mv:

- 该命令根据命令中第二个参数类型的不同（是目标文件还是目标目录）来判断是重命名还是移动文件，当第二个参数类型是文件时，mv 命令完成文件重命名，此时，它将所给的源文件或目录重命名为给定的目标文件名；

- 当第二个参数是已存在的目录名称时，mv 命令将各参数指定的源文件均移至目标目录中；
- 在跨文件系统移动文件时，mv 先复制，再将原有文件删除，而链至该文件的链接也将丢失。



③ rm:

- 如果没有使用-r 选项，则 rm 不会删除目录；
- 使用该命令时一旦文件被删除，它是不能被恢复的，所以最好使用-i 参数。

6. chown 和 chgrp

(1) 作用

- ① chown: 修改文件所有者和组别。
- ② chgrp: 改变文件的组所有权。

(2) 格式

- ① chown: chown [选项]...文件所有者[所有者组名] 文件

其中的文件所有者为修改后的文件所有者。

- ② chgrp: chgrp [选项]... 文件所有组 文件

其中的文件所有组为改变后的文件组拥有者。

(3) 常见参数

chown 和 chgrp 的常见参数意义相同，其主要选项参数如表 2.15 所示。

表 2.15

chown 和 chgrp 命令常见参数列表

选 项	参 数 含 义
-c, -changes	详尽地描述每个 file 实际改变了哪些所有权
-f, --silent,--quiet	不打印文件所有权就不能修改的报错信息

(4) 使用实例

在笔者的系统中一个文件的所有者原先是这样的。

```
[root@www linux]# ls -l
-rwxr-xr-x  15 apectel  linux          4096  6月  4  2005 uClinux-dist.tar
```

可以看出，这是一个文件，它的文件拥有者是 apectel，具有可读写和执行的权限，它所属的用户组是 linux，具有可读和执行的权限，但没有可写的全权，同样，系统其他用户对其也只有可读和执行的权限。

首先使用 chown 将文件所有者改为 root。

```
[root@www linux]# chown root uClinux-dist.tar
[root@www linux]# ls -l
-rwxr-xr-x  15 root    linux          4096  6月  4  2005 uClinux-dist.tar
```

可以看出，此时，该文件拥有者变为了 root，它所属文件用户组不变。

接着使用 chgrp 将文件用户组变为 root。



```
[root@www linux]# chgrp root uClinux-dist.tar
[root@www linux]# ls -l
-rwxr-xr-x 15 root root 4096 6月 4 2005 uClinux-dist.tar
```

(5) 使用说明

- 使用 `chown` 和 `chgrp` 必须拥有 `root` 权限。

7. chmod

(1) 作用

改变文件的访问权限。

(2) 格式

`chmod` 可使用符号标记进行更改和八进制数指定更改两种方式，因此它的格式也有两种不同的形式。

① 符号标记：`chmod [选项]…符号权限[符号权限]…文件`

其中的符号权限可以指定为多个，也就是说，可以指定多个用户级别的权限，但它们中间要用逗号分开表示，若没有显示指出则表示不作更改。

② 八进制数：`chmod [选项] …八进制权限 文件…`

其中的八进制权限是指要更改后的文件权限。

(3) 选项参数

`chmod` 主要选项参数如表 2.16 所示。

表 2.16

`chmod` 命令常见参数列表

选 项	参 数 含 义
-c	若该文件权限确实已经更改，才显示其更改动作
-f	若该文件权限无法被更改也不要显示错误信息
-v	显示权限变更的详细资料

(4) 使用实例

`chmod` 涉及文件的访问权限，在此对相关的概念进行简单的回顾。

在 1.6.1 节中已经提到，文件的访问权限可表示成：**-rwx rwx rwx**。在此设有三种不同的访问权限：读（**r**）、写（**w**）和运行（**x**）。三个不同的用户级别：文件拥有者（**u**）、所属的用户组（**g**）和系统里的其他用户（**o**）。在此，可增加一个用户级别 **a**（**all**）来表示所有这三个不同的用户级别。

① 对于第一种符号连接方式的 `chmod` 命令中，用加号“+”代表增加权限，用减号“-”删除权限，等于号“=”设置权限。

例如原先笔者系统中有文件 `uClinux20031103.tgz`，其权限如下所示。

```
[root@www linux]# ls -l
```



```
-rw-r--r-- 1 root root 79708616 Mar 24 2005 uClinux20031103.tgz
[root@www linux]# chmod a+rx,u+w uClinux20031103.tgz
[root@www linux]# ls -l
-rwxr-xr-x 1 root root 79708616 Mar 24 2005 uClinux20031103.tgz
```

可见，在执行了 `chmod` 之后，文件拥有者除拥有所有用户都有的可读和执行的权限外，还有可写的权限。

② 对于第二种八进制数指定的方式，将文件权限字符代表的有效位设为“1”，即“rw-”、“rw-”和“r--”的八进制表示为“110”、“110”、“100”，把这个 2 进制串转换成对应的 8 进制数就是 6、6、4，也就是说该文件的权限为 664（三位八进制数）。这样对于转化后 8 进制数、2 进制及对应权限的关系如表 2.17 所示。

表 2.17 转化后 8 进制数、2 进制及对应权限的关系

转换后 8 进制数	2 进制	对 应 权 限	转换后 8 进制数	2 进制	对 应 权 限
0	000	没有任何权限	1	001	只能执行
2	010	只写	3	011	只写和执行
4	100	只读	5	101	只读和执行
6	110	读和写	7	111	读，写和执行

同上例，原先笔者系统中有文件 `genromfs-0.5.1.tar.gz`，其权限如下所示。

```
[root@www linux]# ls -l
-rw-rw-r-- 1 linux linux 20543 Dec 29 2004 genromfs-0.5.1.tar.gz
[root@www linux]# chmod 765 genromfs-0.5.1.tar.gz
[root@www linux]# ls -l
-rwxrw-r-x 1 linux linux 20543 Dec 29 2004 genromfs-0.5.1.tar.gz
```

可见，在执行了 `chmod 765` 之后，该文件的拥有者权限、文件组权限和其他用户权限都恰当地对应了。

(5) 使用说明

- 使用 `chmod` 必须具有 root 权限。

8. grep

(1) 作用

在指定文件中搜索特定的内容，并将含有这些内容的行标准输出。

(2) 格式



grep [选项] 格式 [文件及路径]

其中的格式是指要搜索的内容格式，若缺省“文件及路径”则默认表示在当前目录下搜索。

(3) 常见参数

grep 主要选项参数如表 2.18 所示。

表 2.18

grep 命令常见参数列表

选 项	参 数 含 义
-c	只输出匹配行的计数
-I	不区分大小写（只适用于单字符）
-h	查询多文件时不显示文件名
-l	查询多文件时只输出包含匹配字符的文件名
-n	显示匹配行及行号
-s	不显示不存在或无匹配文本的错误信息
-v	显示不包含匹配文本的所有行

(4) 使用实例

```
[root@www linux]# grep "hello" / -r
Binary file ./iscit2005/备份/iscit2004.sql matches
./ARM_TOOLS/uClinux-Samsung/linux-2.4.x/Documentation/s390/Debugging390.txt:hello
world$2 = 0
...
```

该本例中，“hello”是要搜索的内容，“/ -r”是指定文件，表示搜索根目录下的所有文件。

(5) 使用说明

- 在缺省情况下，“grep”只搜索当前目录。如果此目录下有许多子目录，“grep”会以如下形式列出：“grep:sound:Is a directory”这会使“grep”的输出难于阅读。但有两种解决的方法：
 - ① 明确要求搜索子目录：grep -r（正如上例中所示）；
 - ② 忽略子目录：grep -d skip。
- 当预料到有许多输出，可以通过管道将其转到“less”（分页器）上阅读：如 grep "h" ./ -r |less 分页阅读。
- grep 特殊用法：
 - grep pattern1|pattern2 files: 显示匹配 pattern1 或 pattern2 的行；
 - grep pattern1 files|grep pattern2: 显示既匹配 pattern1 又匹配 pattern2 的行；



9. find

(1) 作用

在指定目录中搜索文件，它的使用权限是所有用户。

(2) 格式

`find [路径][选项][描述]`

其中的路径为文件搜索路径，系统开始沿着此目录树向下查找文件。它是一个路径列表，相互用空格分离。若缺省路径，那么默认为当前目录。

其中的描述是匹配表达式，是 `find` 命令接受的表达式。

(3) 常见参数

[选项]主要参数如表 2.19 所示。

表 2.19

find 选项常见参数列表

选 项	参 数 含 义
<code>-depth</code>	使用深度级别的查找过程方式，在某层指定目录中优先查找文件内容
<code>-mount</code>	不在其他文件系统（如 Msdos、Vfat 等）的目录和文件中查找

[描述]主要参数如表 2.20 所示。

表 2.20

find 描述常见参数列表

选 项	参 数 含 义
<code>-name</code>	支持通配符*和?
<code>-user</code>	用户名：搜索文件属主为用户名（ID 或名称）的文件
<code>-print</code>	输出搜索结果，并且打印

(4) 使用实例

```
[root@www linux]# find ./ -name qiong*.c
./qiong1.c
./iscit2005/qiong.c
```

在该实例中使用了 `-name` 的选项支持通配符。

(5) 使用说明

- 若使用目录路径为“/”，通常需要查找较多的时间，可以指定更为确切的路径以减少查找时间。
- `find` 命令可以使用混合查找的方法，例如，想在 `/etc` 目录中查找大于 500000 字节，并且在 24 小时内修改的某个文件，则可以使用 `-and`（与）把两个查找参数链接起来组合成一个混合的查找方式，如“`find /etc -size +500000c -and -mtime +1`”。



10. locate

(1) 作用

用于查找文件。其方法是先建立一个包括系统内所有文件名称及路径的数据库，之后当寻找时就只需查询这个数据库，而不必实际深入档案系统之中了。因此其速度比 `find` 快很多。

(2) 格式

`locate [选项]`

(3) `locate` 主要选项参数如表 2.21 所示。

表 2.21

`locate` 命令常见参数列表

选 项	参 数 含 义
<code>-u</code>	从根目录开始建立数据库
<code>-U</code>	指定开始的位置建立数据库
<code>-f</code>	将特定的文件系统排除在数据库外，例如 <code>proc</code> 文件系统中的文件
<code>-r</code>	使用正则运算式做寻找的条件
<code>-o</code>	指定数据库的名称

(4) 使用实例

```
[root@www linux]# locate issue -U ./
[root@www linux]# updatedb
[root@www linux]# locate -r issue*
./ARM_TOOLS/uClinux-Samsung/lib/libpam/doc/modules/pam_issue.sgml
./ARM_TOOLS/uClinux-Samsung/lib/libpam/modules/pam_issue
./ARM_TOOLS/uClinux-Samsung/lib/libpam/modules/pam_issue/Makefile
./ARM_TOOLS/uClinux-Samsung/lib/libpam/modules/pam_issue/pam_issue.c
...
```

示例中首先在当前目录下建立了一个数据库，并且在更新了数据库之后进行正则匹配查找。通过运行可以发现 `locate` 的运行速度非常快。

(5) 使用说明

`locate` 命令所查询的数据库由 `updatedb` 程序来更新的，而 `updatedb` 是由 `cron daemon` 周期性建立的，但若所找到的档案是最近才建立或刚更名的，可能会找不到，因为 `updatedb` 默认每天运行一次，用户可以由修改 `crontab` (`etc/crontab`) 来更新周期值。

11. ln

(1) 作用

为某一个文件在另外一个位置建立一个符号链接。当需要在不同的目录用到相同的文件时，Linux 允



许用户不用在每一个需要的目录下都存放一个相同的文件，而只需将其他目录下文件用 `ln` 命令链接即可，这样就不必重复地占用磁盘空间。

(2) 格式

`ln[选项] 目标 目录`

(3) 常见参数

➤ `-s` 建立符号链接（这也是通常惟一使用的参数）。

(4) 使用实例

```
[root@www uclinux]# ln -s ../genromfs-0.5.1.tar.gz ./hello
[root@www uclinux]# ls -l
total 77948
lrwxrwxrwx  1 root  root   24 Jan 14 00:25 hello -> ../genromfs-0.5.1.tar.gz
```

该实例建立了当前目录的 `hello` 文件与上级目录之间的符号连接，可以看见，在 `hello` 的 `ls -l` 中的第一位为“l”，表示符号链接，同时还显示了链接的源文件。

(5) 使用说明

- `ln` 命令会保持每一处链接文件的同步性，也就是说，不论改动了哪一处，其他的文件都会发生相同的变化。

- `ln` 的链接又软链接和硬链接两种：

软链接就是上面所说的 `ln -s ** *`，它只会在用户选定的位置上生成一个文件的镜像，不会重复占用磁盘空间，平时使用较多的都是软链接；

硬链接是不带参数的 `ln ** *`，它会在用户选定的位置上生成一个和源文件大小相同的文件，无论是软链接还是硬链接，文件都保持同步变化。

5.1.3 压缩打包相关命令

Linux 中打包压缩的如表 2.22 所示，本书以 `gzip` 和 `tar` 为例进行讲解。

表 2.22

Linux 常见系统管理命令

命令	命令含义	格式
<code>bzip2</code>	.bz2 文件的压缩（或解压）程序	<code>bzip2[选项]</code> 压缩（解压缩）的文件名
<code>bunzip2</code>	.bz2 文件的解压缩程序	<code>bunzip2[选项]</code> .bz2 压缩文件
<code>bzip2recover</code>	用来修复损坏的.bz2 文件	<code>bzip2recover</code> .bz2 压缩文件
<code>gzip</code>	.gz 文件的压缩程序	<code>gzip [选项]</code> 压缩（解压缩）的文件名
<code>gunzip</code>	解压被 <code>gzip</code> 压缩过的文件	<code>gunzip [选项]</code> .gz 文件名
<code>unzip</code>	解压 <code>winzip</code> 压缩的.zip 文件	<code>unzip [选项]</code> .zip 压缩文件



compress	早期的压缩或解压程序（压缩后文件名为.Z）	compress [选项] 文件
tar	对文件目录进行打包或解包	tar [选项] [打包后文件名]文件目录列表

1. gzip

（1）作用

对文件进行压缩和解压缩，而且 **gzip** 根据文件类型可自动识别压缩或解压。

（2）格式

gzip [选项] 压缩（解压缩）的文件名。

（3）常见参数

gzip 主要选项参数如表 2.23 所示。

表 2.23

gzip 命令常见参数列表

选 项	参 数 含 义
-c	将输出信息写到标准输出上，并保留原有文件
-d	将压缩文件解压
-l	对每个压缩文件，显示下列字段：压缩文件的大小、未压缩文件的大小、压缩比、未压缩文件的名字
-r	查找指定目录并压缩或解压缩其中的所有文件
-t	测试，检查压缩文件是否完整
-v	对每一个压缩和解压的文件，显示文件名和压缩比

（4）使用实例

```
[root@www my]# gzip hello.c

[root@www my]# ls
hello.c.gz

[root@www my]# gzip -l hello.c

      compressed          uncompressed  ratio uncompressed_name
61              39.3% hello.c
```

该实例将目录下的“hello.c”文件进行压缩，选项“-l”列出了压缩比。

（5）使用说明

- 使用 **gzip** 压缩只能压缩单个文件，而不能压缩目录，其选项“-d”是将该目录下的所有文件逐个进行压缩，而不是压缩成一个文件。



2. tar

(1) 作用

对文件目录进行打包或解包。

在此需要对**打包**和**压缩**这两个概念进行区分。**打包**是指将一些文件或目录变成一个总的文件，而**压缩**则是将一个大的文件通过一些压缩算法变成一个小文件。为什么要区分这两个概念呢？这是由于在 Linux 中的很多压缩程序（如前面介绍的 **gzip**）只能针对一个文件进行压缩，这样当想要压缩较多文件时，就要借助它的工具将这些堆文件先打成一个包，然后再用原来的压缩程序进行压缩。

(2) 格式

tar [选项] [打包后文件名]文件目录列表。

tar 可自动根据文件名识别打包或解包动作，其中打包后文件名为用户自定义的打包后文件名称，文件目录列表可以是要进行打包备份的文件目录列表，也可以是进行解包的文件目录列表。

(3) 主要参数

tar 主要选项参数如表 2.24 所示。

表 2.24

tar 命令常见参数列表

选 项	参 数 含 义
-c	建立新的打包文件
-r	向打包文件末尾追加文件
-x	从打包文件中解出文件
-o	将文件解开到标准输出
-v	处理过程中输出相关信息
-f	对普通文件操作
-z	调用 gzip 来压缩打包文件，与-x 联用时调用 gzip 完成解压缩
-j	调用 bzip2 来压缩打包文件，与-x 联用时调用 bzip2 完成解压缩
-Z	调用 compress 来压缩打包文件，与-x 联用时调用 compress 完成解压缩

(4) 使用实例

```
[root@www home]# tar -cvf ycw.tar ./ycw
./ycw/
./ycw/.bash_logout
./ycw/.bash_profile
./ycw/.bashrc
./ycw/.bash_history
./ycw/my/
./ycw/my/1.c.gz
```



```
./ycw/my/my.c.gz
./ycw/my/hello.c.gz
./ycw/my/why.c.gz
[root@www home]# ls -l ycw.tar
-rw-r--r--  1 root    root      10240 Jan 14 15:01 ycw.tar
```

该实例将“./ycw”目录下的文件加以打包，其中选项“-v”在屏幕上输出了打包的具体过程。

```
[root@www linux]# tar -zxvf linux-2.6.11.tar.gz
linux-2.6.11/
linux-2.6.11/drivers/
linux-2.6.11/drivers/video/
linux-2.6.11/drivers/video/aty/
...
```

该实例用选项“-z”调用 gzip，并-x 联用时完成解压缩。

(5) 使用说明

tar 命令除了用于常规的打包之外，使用更为频繁的是用选项“-z”或“-j”调用 gzip 或 bzip2（Linux 中另一种解压工具）完成对各种不同的文件的解压。

表 2.25 对 Linux 中常见类型的文件解压命令做一总结。

表 2.25 Linux 常见类型的文件解压命令一览表

文件后缀	解压命令	示例
.a	tar xv	tar xv hello.a
.z	Uncompress	uncompress hello.Z
.gz	Gunzip	gunzip hello.gz
.tar.Z	tar xvZf	tar xvZf hello.tar.Z
.tar.gz/.tgz	tar xvzf	tar xvzf hello.tar.gz
tar.bz2	tar jxvf	tar jxvf hello.tar.bz2
.rpm	安装: rpm -i	安装: rpm -i hello.rpm
	解压: rpm2cpio	解压: rpm2cpio hello.rpm
.deb (Debian 中的文件格式)	安装: dpkg -i	安装: dpkg -i hello.deb
	解压: dpkg-deb --fsys-tarfile	解压: dpkg-deb --fsys-tarhello hello.deb
.zip	Unzip	unzip hello.zip



5.1.4 比较合并文件相关命令

1. diff

(1) 作用

比较两个不同的文件或不同目录下的两个同名文件功能，并生成补丁文件。

(2) 格式

`diff[选项] 文件 1 文件 2`

`diff` 比较文件 1 和文件 2 的不同之处，并按照选项所指定的格式加以输出。`diff` 的格式分为命令格式和上下文格式，其中上下文格式又包括了旧版上下文格式和新版上下文格式，命令格式分为标准命令格式、简单命令格式及混合命令格式，它们之间的区别会在使用实例中进行详细地讲解。当选项缺省时，`diff` 默认使用混合命令格式。

(3) 主要参数

`diff` 主要选项参数如表 2.26 所示。

表 2.26

`diff` 命令常见参数列表

选 项	参 数 含 义
<code>-r</code>	对目录进行递归处理
<code>-q</code>	只报告文件是否有不同，不输出结果
<code>-e, -ed</code>	命令格式
<code>-f</code>	RCS（修订控制系统）命令简单格式
<code>-c, --context</code>	旧版上下文格式
<code>-u, --unified</code>	新版上下文格式
<code>-Z</code>	调用 <code>compress</code> 来压缩归档文件，与 <code>-x</code> 联用时调用 <code>compress</code> 完成解压缩

(4) 使用实例

以下有两个文件 `hello1.c` 和 `hello2.c`。

```
//hello1.c

#include <stdio.h>

void main()

{

    printf("Hello!This is my home!\n");
```



```
}

//hello2.c

#include <stdio.h>

void main()

{

    printf("Hello!This is your home!\n");

}
```

以下实例主要讲解了各种不同格式的比较和补丁文件的创建方法。

① 主要格式比较

首先使用旧版上下文格式进行比较。

```
[root@www ycw]# diff -c hello1.c hello2.c

*** hello1.c    Sat Jan 14 16:24:51 2006
--- hello2.c    Sat Jan 14 16:54:41 2006
*****
*** 1,5 ****

    #include <stdio.h>

    void main()

    {

!       printf("Hello!This is my home!\n");

    }

--- 1,5 ----

    #include <stdio.h>

    void main()

    {
```



```
!    printf("Hello!This is your home!\n");

}
```

可以看出，用旧版上下文格式进行输出时，在显示每个有差别行的同时还显示该行的上下三行，区别的地方用“!”加以标出，由于示例程序较短，上下三行已经包含了全部代码。

接着使用新版的上下文格式进行比较。

```
[root@www ycw]# diff -u hello1.c hello2.c

--- hello1.c    Sat Jan 14 16:24:51 2006
+++ hello2.c    Sat Jan 14 16:54:41 2006
@@ -1,5 +1,5 @@

#include <stdio.h>

void main()

{

-    printf("Hello!This is my home!\n");
+    printf("Hello!This is your home!\n");

}
```

可以看出，在新版上下文格式输出时，仅把两个文件的不同之处分别列出，而相同之处没有重复列出，这样大大方便了用户的阅读。

接下来使用命令格式进行比较。

```
[root@www ycw]# diff -e hello1.c hello2.c

4c

    printf("Hello!This is your home!\n");
```

可以看出，命令符格式输出时仅输出了不同的行，其中命令符“4c”中的数字表示行数，字母的含义



为 **a**——添加，**b**——删除，**c**——更改。因此，**-e** 选项的命令符表示：若要把 `hello1.c` 变为 `hello2.c`，就需要把 `hello1.c` 的第四行改为显示出的 “`printf (“Hello!This is your home!\n”);`” 即可。

选项 “**-f**” 和选项 “**-e**” 显示的内容基本相同，就是数字和字母的顺序相交换了，从以下的输出结果可以看出。

```
[root@www ycw]# diff -f hello1.c hello2.c

c4

    printf("Hello!This is your home!\n");
```

在 `diff` 选项缺省的情况下，输出结果如下所示。

```
[root@www ycw]# diff hello1.c hello2.c

4c4

<     printf("Hello!This is my home!\n");
---
>     printf("Hello!This is your home!\n");
```

可以看出，`diff` 缺省情况下的输出格式充分显示了如何将 `hello1.c` 转化为 `hello2.c` 的方法，即通过 “4c4” 实现。

② 创建补丁文件（也就是差异文件）是 `diff` 的功能之一，不同的选项格式可以生成与之相对应的补丁文件。见下例。

```
[root@www ycw]# diff hello1.c hello2.c >hello.patch

[root@www ycw]# vi hello.patch

4c4

<     printf("Hello!This is my home!\n");
---
>     printf("Hello!This is your home!\n");
```



可以看出，使用缺省选项创建补丁文件的内容和前面使用缺省选项的输出内容是一样的。

2. patch

(1) 作用

命令跟 diff 配合使用，把生成的补丁文件应用到现有代码上。

(2) 格式

patch [选项] [待 patch 的文件[patch 文件]]。

常用的格式为：patch -pnum [patch 文件]，其中的-pnum 是选项参数，在后面会详细介绍。

(3) 常见参数

patch 主要选项参数如表 2.27 所示。

表 2.27

patch 命令常见参数列表

选 项	参 数 含 义
-b	生成备份文件
-d	把 dir 设置为解释补丁文件名的当前目录
-e	把输入的补丁文件看作是 ed 脚本
-pnum	剥离文件名中的前 NUM 个目录成分
-t	在执行过程中不要求任何输入
-v	显示 patch 的版本号

以下对-pnum 选项进行说明。

首先查看以下示例（对分别位于 xc.orig/config/cf/Makefile 和 xc.bsd/config/cf/Makefile 的文件使用 patch 命令）。

```
diff -ruNa xc.orig/config/cf/Makefile xc.bsd/config/cf/Makefile
```

以下是 patch 文件的头标记。

```
--- xc.orig/config/cf/Imake.cf Fri Jul 30 12:45:47 1999
+++ xc.new/config/cf/Imake.cf Fri Jan 21 13:48:44 2000
```



这个 patch 如果直接应用，那么它会去找“xc.orig/config/cf”目录下的 Makefile 文件，假如用户源码树的根目录是缺省的 xc 而不是 xc.orig，则除了可以把 xc.orig 移到 xc 处之外，还有什么简单的方法应用此 patch 吗？NUM 就是为此而设的：patch 会把目标路径名剥去 NUM 个“/”，也就是说，在此例中，-p1 的结果是 config/cf/Makefile，-p2 的结果是 cf/Makefile。因此，在此例中就可以用命令 `cd xc; patch _p1 < /pathname/xxx.patch` 完成操作。

(4) 使用实例

```
[root@www ycw]# diff hello1.c hello2.c >hello1.patch

[root@www ycw]# patch ./hello1.c < hello1.patch

patching file ./hello1.c

[root@www ycw] ]# vi hello1.c

#include <stdio.h>

void main()

{

    printf("Hello!This is your home!\n");

}
```

在该实例中，由于 patch 文件和源文件在同一目录下，因此直接给出了目标文件的目录，在应用了 patch 之后，hello1.c 的内容变为了 hello2.c 的内容。

(5) 使用说明

- 如果 patch 失败，patch 命令会把成功的 patch 行补上其差异，同时（无条件）生成备份文件和一个 .rej 文件。rej 文件里是没有成功提交的 patch 行，需要手工打上补丁。这种情况在原码升级的时候有可能会发生。
- 在多数情况下，patch 程序可以确定补丁文件的格式，当它不能识别时，可以使用 -c、-e、-n 或者 -u 选项来指定输入的补丁文件的格式。由于只有 GNU patch 可以创建和读取新版上下文格式的 patch 文件，因此，除非能够确定补丁所面向的只是那些使用 GNU 工具的用户，否则应该使用旧版上下文格式来生成补丁文件。
- 为了使 patch 程序能够正常工作，需要上下文的行数至少是 2 行（即至少是有一处差别的文件）。

5.1.5 网络相关命令

Linux 下网络相关的常见命令如下表 2.28 所示，本书仅以 ifconfig 和 ftp 为例进行说明。



表 2.28

Linux 下网络相关命令

选 项	参 数 含 义	常见选项格式
netstat	显示网络连接、路由表和网络接口信息	netstat [-an]
nslookup	查询一台机器的 IP 地址和其对应的域名	Nslookup [IP 地址/域名]
finger	查询用户的信息	finger [选项] [使用者] [用户@主机]
ping	用于查看网络上的主机是否在工作	ping [选项] 主机名/IP 地址
ifconfig	查看和配置网络接口的参数	ifconfig [选项] [网络接口]
ftp	利用 ftp 协议上传和下载文件	在本节中会详细讲述
telnet	利用 telnet 协议浏览信息	telnet [选项] [IP 地址/域名]
ssh	利用 ssh 登录对方主机	ssh [选项] [IP 地址]

1. ifconfig

(1) 作用

用于查看和配置网络接口的地址和参数，包括 IP 地址、网络掩码、广播地址，它的使用权限是超级用户。

(2) 格式

ifconfig 有两种使用格式，分别用于查看和更改网络接口。

- ① ifconfig [选项] [网络接口]：用来查看当前系统的网络配置情况。
- ② ifconfig 网络接口 [选项] 地址：用来配置指定接口（如 eth0，eth1）的 IP 地址、网络掩码、广播地址等。

(3) 常见参数

ifconfig 第二种格式常见选项参数如表 2.29 所示。

表 2.29

ftp 命令选项常见参数列表

选 项	参 数 含 义
-interface	指定的网络接口名，如 eth0 和 eth1
up	激活指定的网络接口卡
down	关闭指定的网络接口
broadcast address	设置接口的广播地址
poin to point	启用点对点方式
address	设置指定接口设备的 IP 地址
netmask address	设置接口的子网掩码



(4) 使用实例

首先，在本例中使用 `ifconfig` 的第一种格式来查看网口配置情况。

```
[root@linux workplace]# ifconfig

eth0      Link encap:Ethernet  HWaddr 00:08:02:E0:C1:8A

          inet addr:59.64.205.70  Bcast:59.64.207.255  Mask:255.255.252.0
          inet6 addr: fe80::208:2ff:fee0:c18a/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:26931 errors:0 dropped:0 overruns:0 frame:0
          TX packets:3209 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:6669382 (6.3 MiB)  TX bytes:321302 (313.7 KiB)
          Interrupt:11

lo        Link encap:Local Loopback

          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:2537 errors:0 dropped:0 overruns:0 frame:0
          TX packets:2537 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:2093403 (1.9 MiB)  TX bytes:2093403 (1.9 MiB)
```

可以看出，使用 `ifconfig` 的显示结果中详细列出了所有活跃接口的 IP 地址、硬件地址、广播地址、子网掩码、回环地址等。

```
[root@linux workplace]# ifconfig eth0

eth0      Link encap:Ethernet  HWaddr 00:08:02:E0:C1:8A

          inet addr:59.64.205.70  Bcast:59.64.207.255  Mask:255.255.252.0
          inet6 addr: fe80::208:2ff:fee0:c18a/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:27269 errors:0 dropped:0 overruns:0 frame:0
          TX packets:3212 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:6698832 (6.3 MiB)  TX bytes:322488 (314.9 KiB)
          Interrupt:11
```



在此例中，通过指定接口显示出对应接口的详细信息。另外，用户还可以通过指定参数“-a”来查看所有接口（包括非活跃接口）的信息。

接下来的示例指出了如何使用 `ifconfig` 的第二种格式来改变指定接口的网络参数配置。

```
[root@linux ~]# ifconfig eth0 down
[root@linux ~]# ifconfig

lo          Link encap:Local Loopback

            inet addr:127.0.0.1  Mask:255.0.0.0

            inet6 addr: ::1/128 Scope:Host

            UP LOOPBACK RUNNING  MTU:16436  Metric:1

            RX packets:1931 errors:0 dropped:0 overruns:0 frame:0

            TX packets:1931 errors:0 dropped:0 overruns:0 carrier:0

            collisions:0 txqueuelen:0

            RX bytes:2517080 (2.4 MiB)  TX bytes:2517080 (2.4 MiB)
```

在此例中，通过将指定接口的状态设置为 `DOWN`，暂时暂停该接口的工作。

```
[root@linux workplace]# ifconfig eth0 210.25.132.142 netmask 255.255.255.0
[root@linux workplace]# ifconfig

eth0       Link encap:Ethernet  HWaddr 00:08:02:E0:C1:8A

            inet addr:210.25.132.142  Bcast:210.25.132.255  Mask:255.255.255.0

            inet6 addr: fe80::208:2ff:fee0:c18a/64 Scope:Link

            UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1

            RX packets:1722 errors:0 dropped:0 overruns:0 frame:0

            TX packets:5 errors:0 dropped:0 overruns:0 carrier:0

            collisions:0 txqueuelen:1000

            RX bytes:147382 (143.9 KiB)  TX bytes:398 (398.0 b)

            Interrupt:11

...
```

从上例可以看出，`ifconfig` 改变了接口 `eth0` 的 IP 地址、子网掩码等，在之后的 `ifconfig` 查看中可以看出确实发生了变化。

（5）使用说明

用 `ifconfig` 命令配置的网络设备参数不需重启就可生效，但在机器重新启动以后将会失效。

5.2 Linux 编辑器 vi 的使用

Linux 系统提供了一个完整的编辑器家族系列，如 `ed`、`ex`、`vi` 和 `emacs` 等。按功能它们可以分为两大



类：行编辑器（`ed`、`ex`）和全屏幕编辑器（`vi`、`emacs`）。行编辑器每次只能对单行进行操作，使用起来很不方便。而全屏幕编辑器可以对整个屏幕进行编辑，用户编辑的文件直接显示在屏幕上，从而克服了行编辑的那种不直观的操作方式，便于用户学习和使用，具有强大的功能。

`vi` 是 Linux 系统的第一个全屏幕交互式编辑程序，它从诞生至今一直得到广大用户的青睐，历经数十年仍然是人们主要使用的文本编辑工具，足以及其生命力之强，而强大的生命力是其强大的功能带来的。由于大多数读者在此之前都已经用惯了 Windows 的 `word` 等编辑器，因此，在刚刚接触时总会或多或少不适应，但只要习惯之后，就能感受到它的方便与快捷。

5.2.1 vi 的模式

`vi` 有 3 种模式，分别为命令行模式、插入模式及底行模式各模式的功能，下面具体进行介绍。

（1）命令行模式

用户在用 `vi` 编辑文件时，最初进入的为一般模式。在该模式中可以通过上下移动光标进行“删除字符”或“整行删除”等操作，也可以进行“复制”、“粘贴”等操作，但无法编辑文字。

（2）插入模式

只有在该模式下，用户才能进行文字编辑输入，用户需按[ESC]键回到命令行模式。

（3）底行模式

在该模式下，光标位于屏幕的底行。用户可以进行文件保存或退出操作，也可以设置编辑环境，如寻找字符串、列出行号等。

5.2.2 vi 的基本流程

（1）进入 `vi`，即在命令行下键入 `vi hello`（文件名）。此时进入的是命令行模式，光标位于屏幕的上方，如图 3.1 所示。

（2）在命令行模式下键入 `i` 进入到插入模式，如图 3.2 所示。可以看出，在屏幕底部显示有“插入”表示插入模式，在该模式下可以输入文字信息。

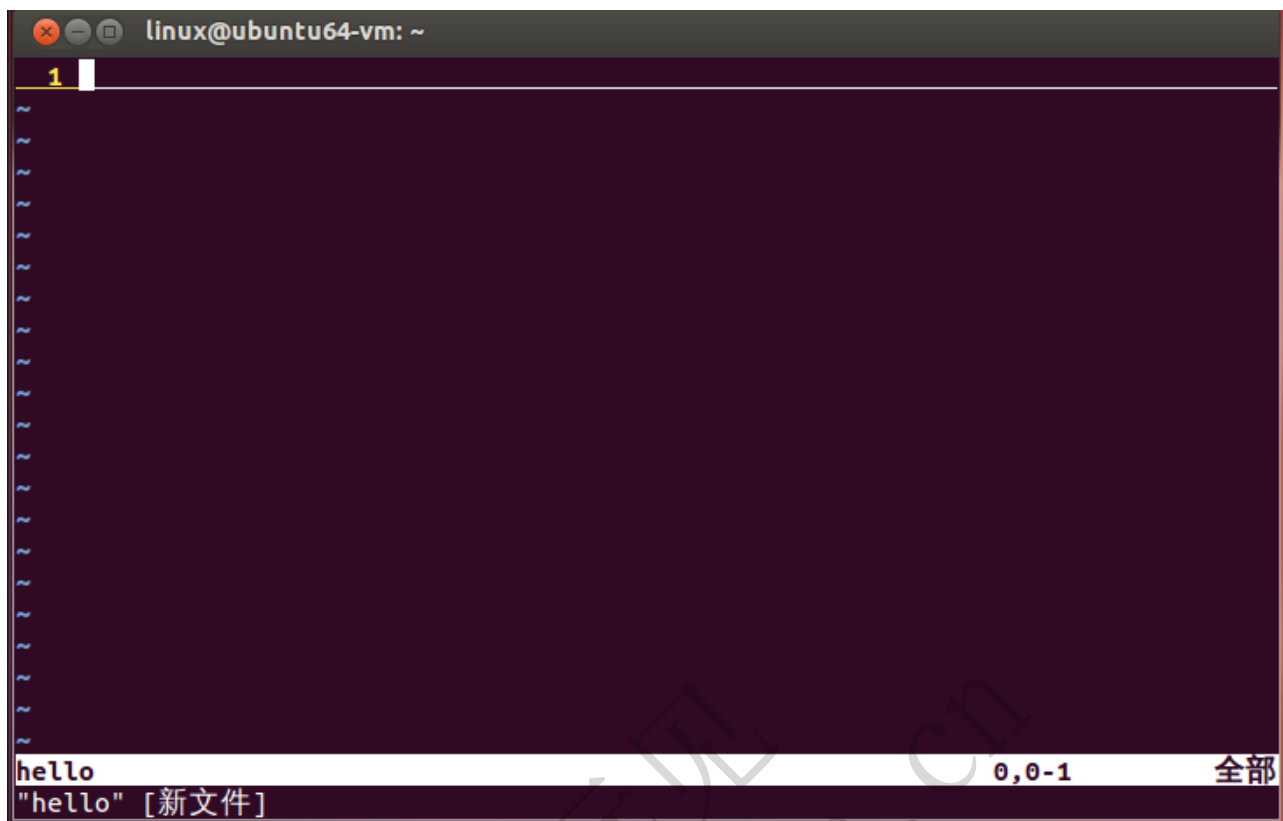


图 3.1 进入 vi 命令行模式

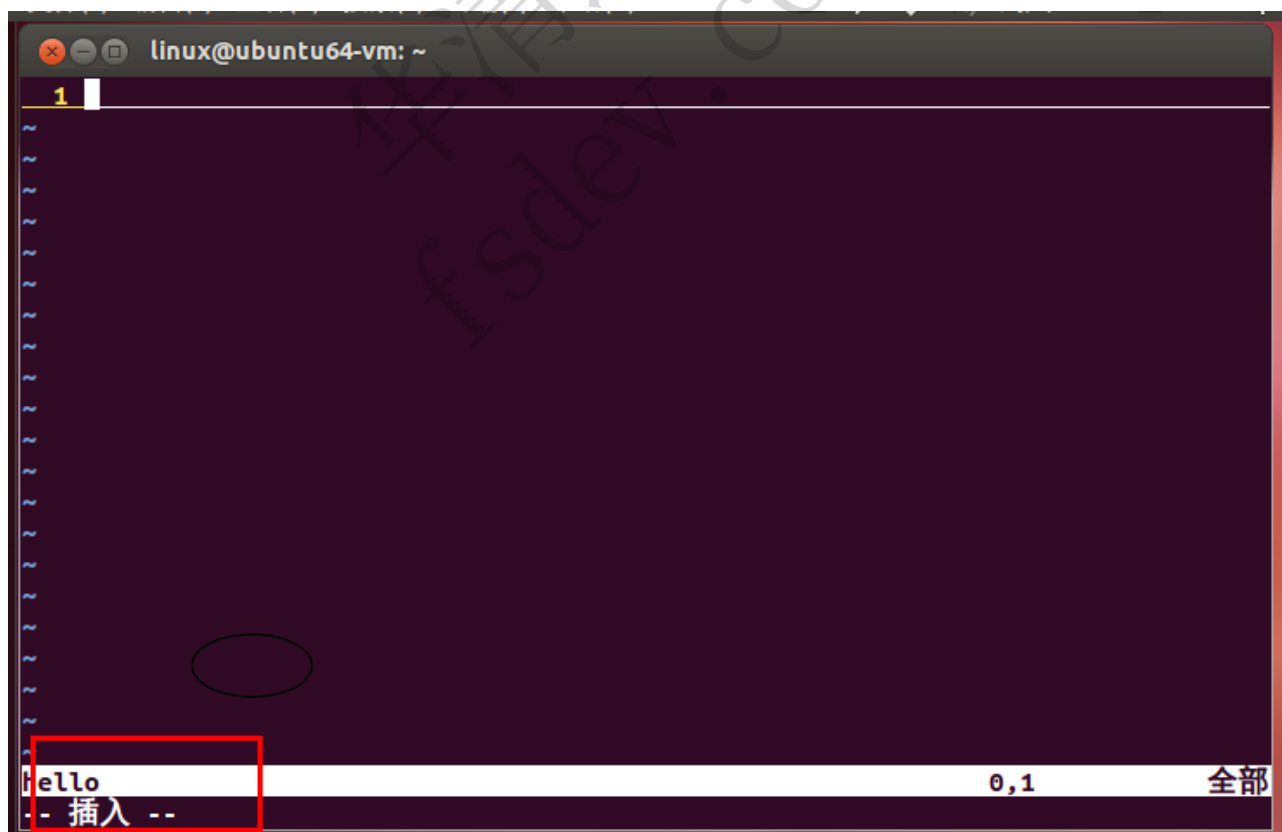




图 3.2 进入 vi 插入模式

(3) 最后，在插入模式中，输入“Esc”，则当前模式转入命令行模式，并在底行行中输入“:wq”（存盘退出）进入底行模式，如图 3.3 所示。

这样，就完成了简单的 vi 操作流程：命令行模式→插入模式→底行模式。由于 vi 在不同的模式下有不同的操作功能，因此，读者一定要时刻注意屏幕最下方的提示，分清所在的模式。

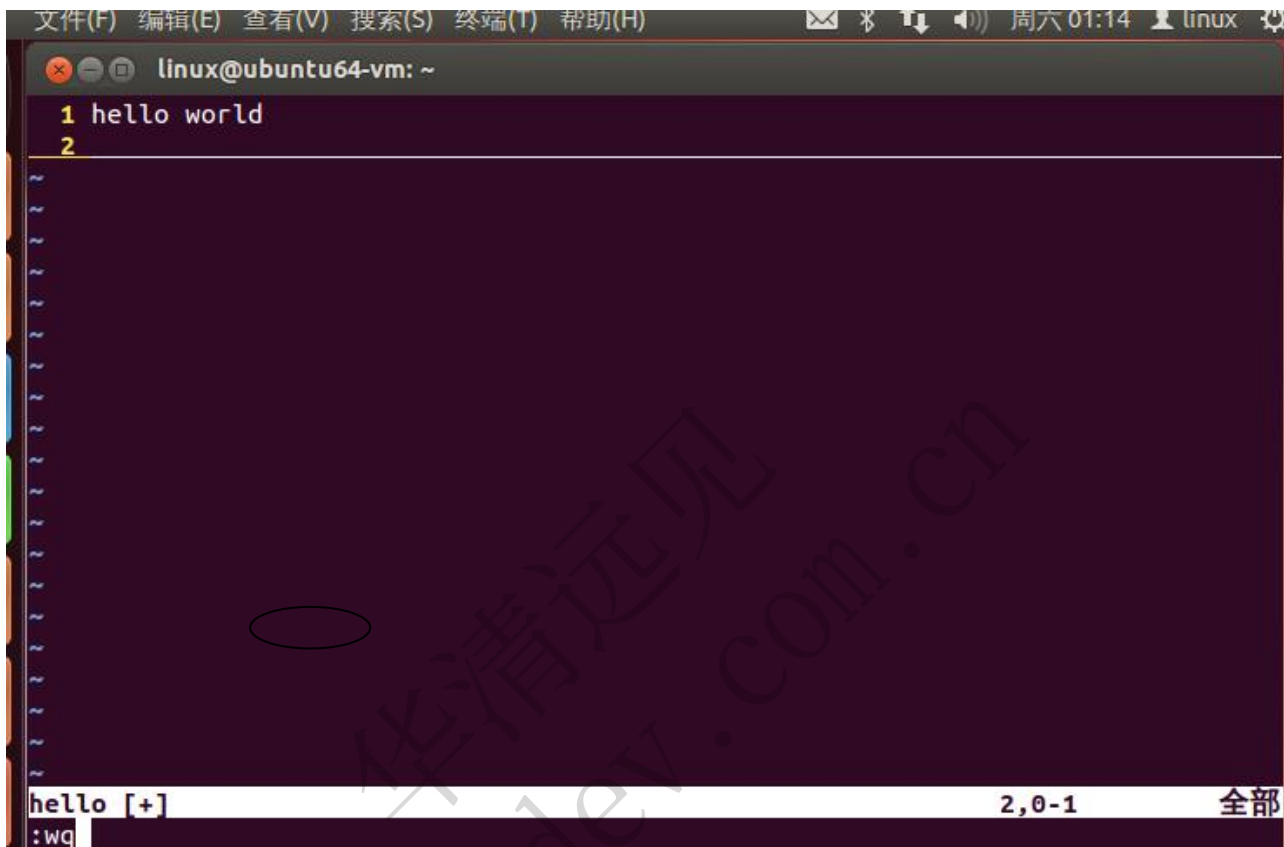


图 3.3 进入 Vi 底行模式

5.2.3 vi 的各模式功能键

(1) 命令行模式常见功能键如表 3.1 所示。

表 3.1 vi 命令行模式功能键

目 录	目 录 内 容
I	切换到插入模式，此时光标当于开始输入文件处
A	切换到插入模式，并从目前光标所在位置的下一个位置开始输入文字
O	切换到插入模式，且从行首开始插入新的一行
[ctrl]+[b]	屏幕往“后”翻动一页



[ctrl]+[f]	屏幕往“前”翻动一页
[ctrl]+[u]	屏幕往“后”翻动半页
[ctrl]+[d]	屏幕往“前”翻动半页
0（数字 0）	光标移到本行的开头
G	光标移动到文章的最后
nG	光标移动到第 n 行
\$	移动到光标所在行的“行尾”
n<Enter>	光标向下移动 n 行
/name	在光标之后查找一个名为 name 的字符串
?name	在光标之前查找一个名为 name 的字符串
x	删除光标所在位置的一个字符

续表

目 录	目 录 内 容
X	删除光标所在位置的“前面”一个字符
dd	删除光标所在行
ndd	从光标所在行开始向下删除 n 行
yy	复制光标所在行
nyy	复制光标所在行开始的向下 n 行
p	将缓冲区内的字符粘贴到光标所在位置（与 yy 搭配）
u	恢复前一个动作

（2）插入模式的功能键只有一个，也就是 Esc 退出到命令行模式。

（3）底行模式常见功能键如表 3.2 所示。

表 3.2 vi 底行模式功能键



目 录	目 录 内 容
:w	将编辑的文件保存到磁盘中
:q	退出 Vi（系统对做过修改的文件会给出提示）
:q!	强制退出 Vi（对修改过的文件不作保存）
:wq	存盘后退出
:w [filename]	另存一个名为 filename 的文件
:set nu	显示行号，设定之后，会在每一行的前面显示对应行号
:set nonu	取消行号显示

华清远见
fsdev.com.cn



第 6 章 Linux Shell 编程

6.1 实验原理

Shell 是系统的用户界面，提供了用户与内核进行交互操作的一种接口。它接收用户输入的命令并把它送入内核去执行。

实际上 Shell 是一个命令解释器，它解释由用户输入的命令并且把它们送到内核。不仅如此，Shell 有自己的编程语言用于对命令的编辑，它允许用户编写由 shell 命令组成的程序。Shell 编程语言具有普通编程语言的很多特点，比如它也有循环结构和分支控制结构等，用这种编程语言编写的 Shell 程序与其他应用程序具有同样的效果。

Linux 提供了像 Microsoft Windows 那样的可视的命令输入界面--X Window 的图形用户界面（GUI）。它提供了很多桌面环境系统，其操作就像 Windows 一样，有窗口、图标和菜单，所有的管理都是通过鼠标控制。

每个 Linux 系统的用户可以拥有他自己的用户界面或 Shell，用以满足他们自己专门的 Shell 需要。

同 Linux 本身一样，Shell 也有多种不同的版本。主要有下列版本的 Shell：

Bourne Shell：是贝尔实验室开发的。

BASH：是 GNU 的 Bourne Again Shell，是 GNU 操作系统上默认的 shell。

Korn Shell：是对 Bourne Shell 的发展，在大部分内容上与 Bourne Shell 兼容。

C Shell：是 SUN 公司 Shell 的 BSD 版本。

Z Shell：The last shell you'll ever need! Z 是最后一个字母，也就是终极 Shell。它集成了 bash、ksh 的重要特性，同时又增加了自己独有的特性。

不论是哪一种 Shell，它最主要的功用都是解译使用者在命令列提示符号下输入的指令。Shell 语法分析命令列，把它分解成以空白区分开的符号（token），在此空白包括了跳位键（tab）、空白和换行（New Line）。如果这些字包含了 metacharacter，shell 将会评估（evaluate）它们的正确用法。另外，shell 还管理档案输入输出及幕后处理（background processing）。在处理命令列之后，shell 会寻找命令并开始执行它们。

6.2 实验目的

熟悉 Linux shell 脚本的编写方法

6.3 实验平台

华清远见开发环境

6.4 实验步骤

6.4.1 拷贝代码

将【华清远见-嵌入式 ARM 实验箱资料- I -FS6818：程序源码\Linux 应用实验源码】目录拷贝到虚拟



机共享目录【D:\share\】下。

(注意：此步骤已经将下文中 Linux 应用部分实验代码都拷贝到共享目录下，下文则省略此部分说明)

明)

Android底层实验源码	2017/8/18 10:22	文件夹	
Android应用实验源码	2017/8/18 10:22	文件夹	
ARM裸机实验源码	2017/8/23 10:42	文件夹	
Linux驱动实验源码	2017/8/18 10:42	文件夹	
Linux应用实验源码	2017/8/18 10:43	文件夹	
文件系统源码	2017/8/18 10:43	文件夹	
Readme.txt	2017/8/18 10:20	文本文档	1 KB

将 Linux Shell 部分代码拷贝到虚拟机 Linux 下。

1、建立相关目录；

```
$ mkdir ~/workdir/linux/application/shell -p
$ cd ~/workdir/linux/application/shell
```

```
linux@ubuntu64-vm:~$ mkdir ~/workdir/linux/application/shell -p
linux@ubuntu64-vm:~$ cd ~/workdir/linux/application/shell
linux@ubuntu64-vm:~/workdir/linux/application/shell$
```

2、将代码从共享目录拷入虚拟机 Linux 操作系统下；（可使用【ctrl+空格】切换输入法）

```
$ cp /mnt/hgfs/share/Linux应用实验源码/02.\ Linux系统shell程序设计实验/实验代码/* . -a
$ ls
```

```
linux@ubuntu64-vm:~/workdir/linux/application/shell$ cp /mnt/hgfs/share/Linux应用实验源码/02.\ Linux系统shell程序设计实验/实验代码/* . -a
linux@ubuntu64-vm:~/workdir/linux/application/shell$ ls
case_and_if.sh file_or_dir.sh simple unload.sh
linux@ubuntu64-vm:~/workdir/linux/application/shell$
```

6.4.2 执行代码

```
$ chmod 777 file_or_dir.sh //给脚本添加可执行权限
$ ./file_or_dir.sh //执行脚本，脚本的作用是判断该目录下的文件时目录还是普通文件
```

6.4.3 实验结果

```
linux@ubuntu64-vm:~/workdir/linux/application/shell$ chmod 777 file_or_dir.sh
linux@ubuntu64-vm:~/workdir/linux/application/shell$ ./file_or_dir.sh
case_and_if.sh is a file
file_or_dir.sh is a file
simple is a directory
unload.sh is a file
```

6.5 相关代码

Java Code



```
1#!/bin/sh
2for i in *
3do
4    if [ -f $i ]
5    then
6        echo "$i is a file"
7    elif [ -d $i ]
8    then
9        echo "$i is a directory"
10    fi
11done
```

华清远见
fsdev.com.cn



第 7 章 GCC 编译

7.1 实验原理

GNU CC（简称为 Gcc）是 GNU 项目中符合 ANSI C 标准的编译系统，能够编译用 C、C++ 和 Object C 等语言编写的程序。gcc 不仅功能强大，而且可以编译如 C、C++、Object C、Java、Fortran、Pascal、Modula-3 和 Ada 等多种语言，而且 gcc 又是一个交叉平台编译器，它能够在当前 CPU 平台上为多种不同体系结构的硬件平台开发软件，因此尤其适合在嵌入式领域的开发编译。

下表 3.3 是 gcc 支持编译源文件的后缀及其解释。

表 3.3 Gcc 所支持后缀名解释

后 缀 名	所对应的语言	后 缀 名	所对应的语言
.c	C 原始程序	.s/.S	汇编语言原始程序
.C/.cc/.cxx	C++原始程序	.h	预处理文件（头文件）
.m	Objective-C 原始程序	.o	目标文件
.i	已经过预处理的 C 原始程序	.a/.so	编译后的库文件
.ii	已经过预处理的 C++原始程序		

如本章开头提到的，gcc 的编译流程分为了 4 个步骤，分别为：

- 预处理（Pre-Processing）；
- 编译（Compiling）；
- 汇编（Assembling）；
- 链接（Linking）。

gcc 有超过 100 个的可用选项，主要包括总体选项、告警和出错选项、优化选项和体系结构相关选项。以下对每一类中最常用的选项进行讲解。

（1）总体选项

gcc 的总结选项如表 3.4 所示，很多在前面的示例中已经有所涉及。

表 3.4 gcc 总体选项列表

后 缀 名	所对应的语言
-c	只是编译不链接，生成目标文件“.o”
-S	只是编译不汇编，生成汇编代码
-E	只进行预编译，不做其他处理
-g	在可执行程序中包含标准调试信息



-o file	把输出文件输出到 file 里
-v	打印出编译器内部编译各过程的命令行信息和编译器的版本
-I dir	在头文件的搜索路径列表中添加 dir 目录
-L dir	在库文件的搜索路径列表中添加 dir 目录
-static	链接静态库
-llibrary	连接名为 library 的库文件

对于“-c”、“-E”、“-o”、“-S”选项在上一小节中已经讲解了其使用方法，在此主要讲解另外两个非常常用的库依赖选项“-I dir”和“-L dir”。

7.2 实验目的

通过实验学习 gcc 编译器编译 c 程序的方法，熟悉 gcc 编译程序的各个阶段。

7.3 实验平台

华清远见开发环境，FS6818 开发板

7.4 实验步骤

7.4.1 拷贝代码

将【华清远见-嵌入式 ARM 实验箱资料-I-FS6818：程序源码\Linux 应用实验源码】目录拷贝到虚拟机共享目录【D:\share\】下。**（注意：此步骤已经在“Linux Shell 编程”章节中完成，如果没有完成请参照此章节。如已经完成可省略此步骤）**

将 gcc 部分代码拷贝到虚拟机 Linux 下。

1、建立相关目录；

```
$ mkdir ~/workdir/linux/application/gcc -p
$ cd ~/workdir/linux/application/gcc
```

```
linux@ubuntu64-vm:~$
linux@ubuntu64-vm:~$ mkdir ~/workdir/linux/application/gcc -p
linux@ubuntu64-vm:~$ cd ~/workdir/linux/application/gcc
linux@ubuntu64-vm:~/workdir/linux/application/gcc$
```

2、将代码从共享目录拷入虚拟机 Linux 操作系统下；（可使用【ctrl+空格】切换输入法）

```
$ cp /mnt/hgfs/share/Linux 应用实验源码/03.\ Linux 系统 GCC 编译器的使用实验/实验代码/* . -a
$ ls
```



```
linux@ubuntu64-vm:~/workdir/linux/application/gcc$ cp /mnt/hgfs/share/Linux应用实验源码/03.\ Linux系统GCC编译器的使用实验/实验代码/* . -a
linux@ubuntu64-vm:~/workdir/linux/application/gcc$ ls
hello helloworld.c
```

7.4.2 编译代码

```
$ arm-none-linux-gnueabi-gcc helloworld.c -o hello
$ mkdir /source/rootfs/app
$ cp hello /source/rootfs/app/
```

```
linux@ubuntu64-vm:~/workdir/linux/application/gcc$ arm-none-linux-gnueabi-gcc helloworld.c -o hello
linux@ubuntu64-vm:~/workdir/linux/application/gcc$ ls
hello helloworld.c
linux@ubuntu64-vm:~/workdir/linux/application/gcc$ mkdir /source/rootfs/app
linux@ubuntu64-vm:~/workdir/linux/application/gcc$ cp hello /source/rootfs/app
linux@ubuntu64-vm:~/workdir/linux/application/gcc$
```

7.4.3 执行代码

本实验通过交叉编译工具编译实验源码，此处我们通过 NFS 挂载的方式启动开发板（启动方式详细步骤请参照“目标机调试环境搭建”章节）。

开发板启动成功后，在开发板串口终端执行应用程序。

```
# cd /app
# ./hello
```

7.4.4 实验结果

```
[root@farsight]# ls
app      etc      linuxrc  root     tmp
bin      fs6818M4_test  mnt      sbin     usr
dev      lib      proc     sys      var

[root@farsight]# cd app
[root@farsight app]# ./hello
hello,world!
```

7.5 相关代码

C++ Code

```
1#include <stdio.h>
2
3int main (int argc, char **argv)
4{
5    printf("hello,world!\n");
6
7    return 0;
8}
9
```



第 8 章 Linux 系统 Makefile 编写实验

8.1 实验原理

到此为止，读者已经了解了如何在 Linux 下使用编辑器编写代码，如何使用 gcc 把代码编译成可执行文件，那么，所有的工作看似已经完成了，为什么还需要 Make 这个工程管理器呢？

所谓工程管理器，顾名思义，是指管理较多的文件的。读者可以试想一下，有一个上百个文件的代码构成的项目，如果其中只有一个或少数几个文件进行了修改，按照之前所学的 gcc 编译工具，就不得不把这所有的文件重新编译一遍，因为编译器并不知道哪些文件是最近更新的，而只知道需要包含这些文件才能把源代码编译成可执行文件，于是，程序员就不能不再重新输入数目如此庞大的文件名以完成最后的编译工作。

人们就希望有一个工程管理器能够自动识别更新了的文件代码，同时又不需要重复输入冗长的命令行，这样，Make 工程管理器也就应运而生了。

实际上，Make 工程管理器也就是个“自动编译管理器”，这里的“自动”是指它能够根据文件时间戳自动发现更新过的文件而减少编译的工作量，同时，它通过读入 Makefile 文件的内容来执行大量的编译工作。用户只需编写一次简单的编译语句就可以了。它大大提高了实际项目的工作效率，而且几乎所有 Linux 下的项目编程均会涉及它，希望读者能够认真学习本节内容。

8.1.1 Makefile 基本结构

Makefile 是 Make 读入的惟一配置文件，因此本节的内容实际就是讲述 Makefile 的编写规则。在一个 Makefile 中通常包含如下内容：

- 需要由 make 工具创建的目标体（target），通常是目标文件或可执行文件；
- 要创建的目标体所依赖的文件（dependency_file）；
- 创建每个目标体时需要运行的命令（command）。

它的格式为：

```
target: dependency_files
      command
```

例如，有两个文件分别为 hello.c 和 hello.h，创建的目标体为 hello.o，执行的命令为 gcc 编译指令：gcc -c hello.c，那么，对应的 Makefile 就可以写为：

```
#The simplest example
hello.o: hello.c hello.h
      gcc -c hello.c -o hello.o
```

接着就可以使用 make 了。使用 make 的格式为：make target，这样 make 就会自动读入 Makefile（也可以是首字母小写 makefile）并执行对应 target 的 command 语句，并会找到相应的依赖文件。如下所示：

```
[root@localhost makefile]# make hello.o
gcc -c hello.c -o hello.o
[root@localhost makefile]# ls
hello.c hello.h hello.o Makefile
```



可以看到，Makefile 执行了“hello.o”对应的命令语句，并生成了“hello.o”目标体。

8.1.2 Makefile 变量

上面示例的 Makefile 在实际中是几乎不存在的，因为它过于简单，仅包含两个文件和一个命令，在这种情况下完全不必要编写 Makefile 而只需在 Shell 中直接输入即可，在实际中使用的 Makefile 往往是包含很多的文件和命令的，这也是 Makefile 产生的原因。下面就可给出稍微复杂一些的 Makefile 进行讲解：

```

sunq:kang.o yul.o
Gcc kang.o bar.o -o myprog
kang.o : kang.c kang.h head.h
Gcc -Wall -O -g -c kang.c -o kang.o
yul.o : bar.c head.h
Gcc -Wall -O -g -c yul.c -o yul.o
    
```

在这个 Makefile 中有 3 个目标体 (target)，分别为 sunq、kang.o 和 yul.o，其中第一个目标体的依赖文件就是后两个目标体。如果用户使用命令“make sunq”，则 make 管理器就是找到 sunq 目标体开始执行。

这时，make 会自动检查相关文件的时间戳。首先，在检查“kang.o”、“yul.o”和“sunq”3 个文件的时间戳之前，它会向下查找那些把“kang.o”或“yul.o”作为目标文件的时间戳。比如，“kang.o”的依赖文件为“kang.c”、“kang.h”、“head.h”。如果这些文件中任何一个的时间戳比“kang.o”新，则命令“gcc -Wall -O -g -c kang.c -o kang.o”将会执行，从而更新文件“kang.o”。在更新完“kang.o”或“yul.o”之后，make 会检查最初的“kang.o”、“yul.o”和“sunq”3 个文件，只要文件“kang.o”或“yul.o”中的任比文件时间戳比“sunq”新，则第二行命令就会被执行。这样，make 就完成了自动检查时间戳的工作，开始执行编译工作。这也就是 Make 工作的基本流程。

接下来，为了进一步简化编辑和维护 Makefile，make 允许在 Makefile 中创建和使用变量。变量是在 Makefile 中定义的名字，用来代替一个文本字符串，该文本字符串称为该变量的值。在具体要求下，这些值可以代替目标体、依赖文件、命令以及 makefile 文件中其他部分。在 Makefile 中的变量定义有两种方式：一种是递归展开方式，另一种是简单方式。

递归展开方式定义的变量是在引用在该变量时进行替换的，即如果该变量包含了对其他变量的应用，则在引用该变量时一次性将内嵌的变量全部展开，虽然这种类型的变量能够很好地完成用户的指令，但是它也有严重的缺点，如不能在变量后追加内容（因为语句：CFLAGS=\$(CFLAGS) -O 在变量扩展过程中可能导致无穷循环）。

为了避免上述问题，简单扩展型变量的值在定义处展开，并且只展开一次，因此它不包含任何对其他变量的引用，从而消除变量的嵌套引用。

递归展开方式的定义格式为：VAR=var。

简单扩展方式的定义格式为：VAR:=var。

Make 中的变量使用均使用格式为：\$(VAR)。

下面给出了上例中用变量替换修改后的 Makefile，这里用 OBJS 代替 kang.o 和 yul.o，用 CC 代替 gcc，用 CFLAGS 代替“-Wall -O -g”。这样在以后修改时，就可以只修改变量定义，而不需要修改下面的定义实体，从而大大简化了 Makefile 维护的工作量。

经变量替换后的 Makefile 如下所示：

```

OBJS = kang.o yul.o
CC = gcc
CFLAGS = -Wall -O -g
    
```



```

sunq : $(OBJS)
      $(CC) $(OBJS) -o sunq
kang.o : kang.c kang.h
      $(CC) $(CFLAGS) -c kang.c -o kang.o
yul.o : yul.c yul.h
      $(CC) $(CFLAGS) -c yul.c -o yul.o
    
```

可以看到，此处变量是以递归展开方式定义的。

Makefile 中的变量分为用户自定义变量、预定义变量、自动变量及环境变量。如上例中的 **OBJS** 就是用户自定义变量，自定义变量的值由用户自行设定，而预定义变量和自动变量为通常在 **Makefile** 都会出现的变量，其中部分有默认值，也就是常见的设定值，当然用户可以对其进行修改。

预定义变量包含了常见编译器、汇编器的名称及其编译选项。表 3.11 列出了 **Makefile** 中常见预定义变量及其部分默认值。

表 3.11 Makefile 中常见预定义变量

命令格式	含义
AR	库文件维护程序的名称，默认值为 ar
AS	汇编程序的名称，默认值为 as
CC	C 编译器的名称，默认值为 cc
CPP	C 预编译器的名称，默认值为 \$(CC) -E
CXX	C++编译器的名称，默认值为 g++
FC	FORTTRAN 编译器的名称，默认值为 f77
RM	文件删除程序的名称，默认值为 rm -f
ARFLAGS	库文件维护程序的选项，无默认值
ASFLAGS	汇编程序的选项，无默认值
CFLAGS	C 编译器的选项，无默认值
CPPFLAGS	C 预编译的选项，无默认值
CXXFLAGS	C++编译器的选项，无默认值
FFLAGS	FORTTRAN 编译器的选项，无默认值



可以看出，上例中的 CC 和 CFLAGS 是预定义变量，其中由于 CC 没有采用默认值，因此，需要把“CC=gcc”明确列出来。

由于常见的 gcc 编译语句中通常包含了目标文件和依赖文件，而这些文件在 Makefile 文件中目标体的一行已经有所体现，因此，为了进一步简化 Makefile 的编写，就引入了自动变量。自动变量通常可以代表编译语句中出现目标文件和依赖文件等，并且具有本地含义（即下一语句中出现的相同变量代表的是下一语句的目标文件和依赖文件）。表 3.12 列出了 Makefile 中常见自动变量。

表 3.12 Makefile 中常见自动变量

命令格式	含 义
\$*	不包含扩展名的目标文件名称
\$+	所有的依赖文件，以空格分开，并以出现的先后为序，可能包含重复的依赖文件
\$<	第一个依赖文件的名称
\$?	所有时间戳比目标文件晚的依赖文件，并以空格分开

续表

命令格式	含 义
\$@	目标文件的完整名称
\$^	所有不重复的依赖文件，以空格分开
%	如果目标是归档成员，则该变量表示目标的归档成员名称

自动变量的书写比较难记，但是在熟练了之后会非常的方便，请读者结合下例中的自动变量改写的 Makefile 进行记忆。

```
OBJS = kang.o yul.o
CC = gcc
CFLAGS = -Wall -O -g
sung : $(OBJS)
    $(CC) $^ -o $@
kang.o : kang.c kang.h
    $(CC) $(CFLAGS) -c $< -o $@
yul.o : yul.c yul.h
    $(CC) $(CFLAGS) -c $< -o $@
```

另外，在 Makefile 中还可以使用环境变量。使用环境变量的方法相对比较简单，make 在启动时会自动读取系统当前已经定义了的环境变量，并且会创建与之具有相同名称和数值的变量。但是，如果用户在 Makefile 中定义了相同名称的变量，那么用户自定义变量将会覆盖同名的环境变量。



8.1.3 Makefile 规则

Makefile 的规则是 Make 进行处理的依据，它包括了目标体、依赖文件及其之间的命令语句。一般的，Makefile 中的一条语句就是一个规则。在上面的例子中，都显示地指出了 Makefile 中的规则关系，如“\$(CC) \$(CFLAGS) -c \$< -o \$@"，但为了简化 Makefile 的编写，make 还定义了隐式规则和模式规则，下面就分别对其进行讲解。

1. 隐式规则

隐含规则能够告诉 make 怎样使用传统的技术完成任务，这样，当用户使用它们时就不必详细指定编译的具体细节，而只需把目标文件列出即可。Make 会自动搜索隐式规则目录来确定如何生成目标文件。如上例就可以写成：

```
OBJS = kang.o yul.o
CC = gcc
CFLAGS = -Wall -O -g
suno : $(OBJS)
        $(CC) $^ -o $@
```

为什么可以省略后两句呢？因为 Make 的隐式规则指出：所有“.o”文件都可自动由“.c”文件使用命令“\$(CC) \$(CPPFLAGS) \$(CFLAGS) -c file.c -o file.o”生成。这样“kang.o”和“yul.o”就会分别调用“\$(CC) \$(CFLAGS) -c kang.c -o kang.o”和“\$(CC) \$(CFLAGS) -c yul.c -o yul.o”生成。

表 3.13 给出了常见的隐式规则目录：

表 3.13 Makefile 中常见隐式规则目录

对应语言后缀名	规 则
C 编译：.c 变为.o	\$(CC) -c \$(CPPFLAGS) \$(CFLAGS)
C++编译：.cc 或.C 变为.o	\$(CXX) -c \$(CPPFLAGS) \$(CXXFLAGS)
Pascal 编译：.p 变为.o	\$(PC) -c \$(PFLAGS)
Fortran 编译：.f 变为.o	\$(FC) -c \$(FFLAGS)

2. 模式规则

模式规则是用来定义相同处理规则的多个文件的。它不同于隐式规则，隐式规则仅仅能够用 make 默认的变量来进行操作，而模式规则还能引入用户自定义变量，为多个文件建立相同的规则，从而简化 Makefile 的编写。

模式规则的格式类似于普通规则，这个规则中的相关文件前必须用“%”标明。使用模式规则修改后的 Makefile 的编写如下：

```
OBJS = kang.o yul.o
CC = gcc
CFLAGS = -Wall -O -g
```



```

sunq : $(OBJS)
      $(CC) $^ -o $@

%.o : %.c
      $(CC) $(CFLAGS) -c $< -o $@
    
```

8.1.4 Make 管理器的使用

使用 Make 管理器非常简单，只需在 make 命令的后面键入目标名即可建立指定的目标，如果直接运行 make，则建立 Makefile 中的第一个目标。

此外 make 还有丰富的命令行选项，可以完成各种不同的功能。下表 3.17 列出了常用的 make 命令行选项。

表 3.14 make 的命令行选项

命令格式	含义
-C dir	读入指定目录下的 Makefile
-f file	读入当前目录下的 file 文件作为 Makefile
续表	
命令格式	含义
-i	忽略所有的命令执行错误
-I dir	指定被包含的 Makefile 所在目录
-n	只打印要执行的命令，但不执行这些命令
-p	显示 make 变量数据库和隐含规则
-s	在执行命令时不显示命令
-w	如果 make 在执行过程中改变目录，则打印当前目录名

8.2 实验目的

通过对包含多文件的 Makefile 的编写，熟悉各种形式的 Makefile，并且进一步加深对 Makefile 中用户自定义变量、自动变量及预定义变量的理解。

8.3 实验平台

华清远见开发环境，FS6818 开发板



8.4 实验步骤

8.4.1 拷贝代码

将【华清远见-嵌入式 ARM 实验箱资料- I -FS6818: 程序源码\Linux 应用实验源码】目录拷贝到虚拟机共享目录【D:\share\】下。(注意：此步骤已经在“Linux Shell 编程”章节中完成，如果没有完成请参照此章节。如已经完成可省略此步骤)

将 Makefile 部分代码拷贝到虚拟机 Linux 下。

1、建立相关目录；

```
$ mkdir ~/workdir/linux/application/Makefile -p
$ cd ~/workdir/linux/application/Makefile
```

```
linux@ubuntu64-vm:~$
linux@ubuntu64-vm:~$ mkdir ~/workdir/linux/application/Makefile -p
linux@ubuntu64-vm:~$ cd ~/workdir/linux/application/Makefile
linux@ubuntu64-vm:~/workdir/linux/application/Makefile$
```

2、将代码从共享目录拷入虚拟机 Linux 操作系统下；（可使用【ctrl+空格】切换输入法）

```
$ cp /mnt/hgfs/share/Linux 应用实验源码/05.\ Linux 系统 Makefile 编写实验/实验代码/* . -a
$ cd 5.2/makefileTest/
$ ls
```

```
linux@ubuntu64-vm:~/workdir/linux/application/Makefile$ cp /mnt/hgfs/share/Linux应用实验源码/05.\
Linux系统Makefile编写实验/实验代码/* . -a
linux@ubuntu64-vm:~/workdir/linux/application/Makefile$ cd 5.2/makefileTest/
linux@ubuntu64-vm:~/workdir/linux/application/Makefile/5.2/makefileTest$ ls
ls  cd  cp  include  mv  Makefile  ps
linux@ubuntu64-vm:~/workdir/linux/application/Makefile/5.2/makefileTest$
```

8.4.2 执行代码

```
$ make CLEAN
$ make
```

会出现如下信息：



```
linux@ubuntu64-vm:~/workdir/linux/application/Makefile/5.2/makefileTest$ make
mkdir -p bin
f1 f2 main obj
begin compile
make -C f1
make[1]: 正在进入目录 `/home/linux/workdir/linux/application/Makefile/5.2/makefileTest/f1'
make[1]: “../obj/f1.o”是最新的。
make[1]:正在离开目录 `/home/linux/workdir/linux/application/Makefile/5.2/makefileTest/f1'
make -C f2
make[1]: 正在进入目录 `/home/linux/workdir/linux/application/Makefile/5.2/makefileTest/f2'
make[1]: “../obj/f2.o”是最新的。
make[1]:正在离开目录 `/home/linux/workdir/linux/application/Makefile/5.2/makefileTest/f2'
make -C main
make[1]: 正在进入目录 `/home/linux/workdir/linux/application/Makefile/5.2/makefileTest/main'
make[1]: “../obj/main.o”是最新的。
make[1]:正在离开目录 `/home/linux/workdir/linux/application/Makefile/5.2/makefileTest/main'
make -C obj
make[1]: 正在进入目录 `/home/linux/workdir/linux/application/Makefile/5.2/makefileTest/obj'
make[1]: “../bin/myapp”是最新的。
make[1]:正在离开目录 `/home/linux/workdir/linux/application/Makefile/5.2/makefileTest/obj'
cp bin/myapp /source/rootfs/app
```

目录树结构如下：

```

├── bin
│   └── myapp
├── f1
│   ├── f1.c
│   └── Makefile
├── f2
│   ├── f2.c
│   └── Makefile
├── include
│   └── myinclude.h
├── main
│   ├── main.c
│   └── Makefile
├── Makefile
├── obj
│   ├── f1.o
│   ├── f2.o
│   ├── main.o
│   └── Makefile
└── tree.txt
```

6 directories, 14 files

我们看到在 bin 目录下生成了我们的目标文件 myapp，并且拷贝到了/source/rootfs/app 下，在 obj 目录下生成了.o 的中间文件。让我们在开发板运行下 myapp 看下结果吧。

本实验通过交叉编译工具编译实验源码，此处我们通过 N F S 挂载的方式启动开发板（启动方式详细步骤请参照前文），在开发板串口终端执行应用程序。

```
# cd app
```



```
# ./myapp
```

```
[root@farsight ~]#  
[root@farsight ~]# cd app  
[root@farsight app]# ./myapp  
Message from f1.c...  
Message from f2.c...  
[root@farsight app]#
```

我们也可以用如下命令清除中间文件和目标文件，恢复 make 之前的状态，在虚拟机下输入：

```
$ make CLEAN
```

执行清除命令后目录结构如下：

```
.  
├── f1  
│   ├── f1.c  
│   └── Makefile  
├── f2  
│   ├── f2.c  
│   └── Makefile  
├── include  
│   └── myinclude.h  
├── main  
│   ├── main.c  
│   └── Makefile  
├── Makefile  
├── obj  
│   └── Makefile  
└── tree.txt
```

5 directories, 10 files

我们可以看到已经变为 make 之前的目录状态了。

8.5 相关代码

Java Code

```
1 CC = arm-none-linux-gnueabi-gcc  
2 SUBDIRS = f1 \  
3     f2 \  
4     main \  
5     obj  
6  
7 OBJS = f1.o f2.o main.o  
8 BIN = myapp  
9 OBJS_DIR = obj  
10 BIN_DIR = bin  
11 export CC OBJS BIN OBJS_DIR BIN_DIR  
12  
13 all : CHECK_DIR $(SUBDIRS)
```



```
14     cp bin/myapp /source/rootfs/app
15 CHECK_DIR :
16     mkdir -p $(BIN_DIR)
17 $(SUBDIRS) : ECHO
18     make -C $@
19 ECHO:
20     @echo $(SUBDIRS)
21     @echo begin compile
22 CLEAN :
23     @$(RM) $(OBS_DIR)/*.o
24     @rm -rf $(BIN_DIR)
```

华清远见
fsdev.com.cn



第 9 章 嵌入式文件 I/O 编程实验

在 Linux 系统中，大部分机制都会抽象成一个文件，这样对它们的操作就像对文件的操作一样。在嵌入式应用开发中，文件 I/O 编程是最常用的，也最基本的内容，希望读者好好掌握。

9.1 实验原理

9.1.1 系统调用

所谓系统调用是指操作系统提供给用户程序调用的一组“特殊”接口，用户程序可以通过这组“特殊”接口获得操作系统内核提供的服务。例如用户可以通过进程控制相关的系统调用来创建进程、实现进程之间的通信等。

在这里，为什么用户程序不能直接访问系统内核提供的服务呢？这是由于在 Linux 中，为了更好地保护内核空间，将程序的运行空间分为内核空间和用户空间（也就是常称的内核态和用户态），它们分别运行在不同的级别上，逻辑上是相互隔离的。因此，用户进程在通常情况下不允许访问内核数据，也无法使用内核函数，它们只能在用户空间操作用户数据，调用用户空间的函数。

但是，在有些情况下，用户空间的进程需要获得一定的系统服务（调用内核空间程序），这时操作系统就必须利用系统提供给用户的“特殊接口”——系统调用规定用户进程进入内核空间的具体位置。进行系统调用时，程序运行空间需要从用户空间进入内核空间，处理完后再返回到用户空间。

Linux 系统调用非常精简（只有 250 个左右），它继承了 Unix 系统调用中最基本和最有用的部分。这些系统调用按照功能逻辑大致可分为进程控制、进程间通信、文件系统控制、存储管理、网络管理、套接字控制、用户管理等几类。

9.1.2 用户编程接口（API）

前面讲到的系统调用并不直接与程序员进行交互，它仅仅是一个通过软中断机制向内核提交请求以获取内核服务的接口。实际使用中程序员调用的通常是用户编程接口——API。

例如创建进程的 API 函数 `fork()` 对应于内核空间的 `sys_fork()` 系统调用。但并不是所有的函数都对应一个系统调用，有时，一个 API 函数会需要几个系统调用来共同完成函数的功能，甚至还有一些 API 函数不需要调用相应的系统调用（因此它所完成的不是内核提供的服务）。

在 Linux 中，用户编程接口（API）遵循了在 Unix 中最流行的应用编程界面标准——POSIX 标准。POSIX 标准是由 IEEE 和 ISO/IEC 共同开发的标准系统。该标准基于当时现有的 Unix 实践和经验，描述了操作系统的系统调用编程接口（实际上就是 API），用于保证应用程序可以在源代码一级上在多种操作系统上移植运行。这些系统调用编程接口主要是通过 C 库（libc）实现的。

9.1.3 系统命令

系统命令相对 API 更高了一层，它实际上一个可执行程序，它的内部引用了用户编程接口（API）来实现相应的功能。它们之间的关系如下图 4.1 所示。

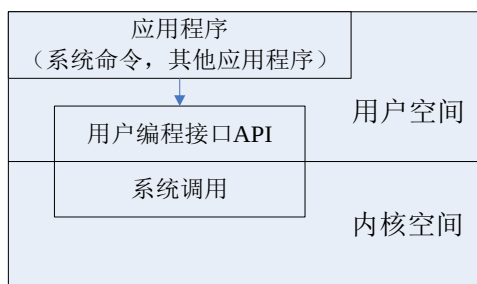


图 4.1 系统调用、API 及系统命令之间的关系

9.1.4 虚拟文件系统 (VFS)

Linux 系统成功的关键因素之一就是具有与其他操作系统和谐共存的能力。Linux 的文件系统由两层结构构建。第一层是虚拟文件系统 (VFS)，第二层是各种不同的具体的文件系统。

VFS 就是把各种具体的文件系统的公共部分抽取出来，形成一个抽象层，是系统内核的一部分。它位于用户程序和具体的文件系统之间。它对用户程序提供了标准的文件系统调用接口，对具体的文件系统（比如：Ext2，FAT32 等），它通过一系列的对不同文件系统公用的函数指针来实际调用具体的文件系统函数，完成实际的各有差异的操作。任何使用文件系统的程序必须经过这层接口来使用它。通过这样的方式，VFS 就对用户屏蔽了底层文件系统的实现细节和差异。

VFS 不仅可以对具体文件系统的数据结构进行抽象，以一种统一的数据结构进行管理，并且还可以接受用户层的系统调用，例如：open()、read()、write()、stat()、link()等。此外，它还支持多种具体文件系统之间的相互访问，接受内核其他子系统的操作请求，例如内存管理和进程调度。VFS 在 Linux 系统中的位置如图 4.2 所示。

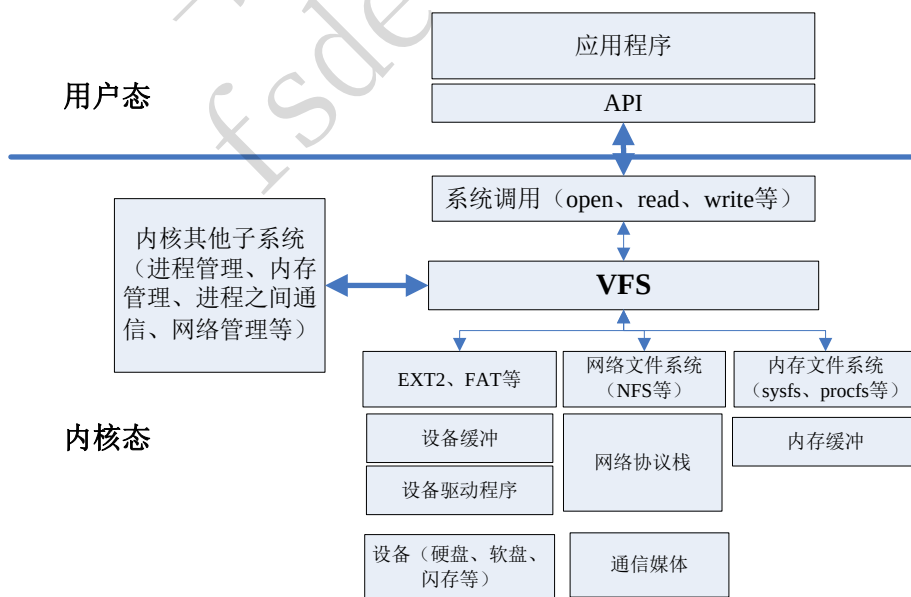


图 4.2 vfs 在 Linux 系统中的位置

通过以下命令可以查看系统中支持哪些文件系统：



```
$ cat /proc/filesystems
```

```
nodev    sysfs
nodev    rootfs
.....
nodev    tmpfs
nodev    pipefs
.....
        ext2
nodev    ramfs
nodev    hugetlbfs
        iso9660
nodev    mqueue
nodev    selinuxfs
        ext3
nodev    rpc_pipefs
.....
```

9.1.5 Linux 中文件及文件描述符

Linux 操作系统都是基于文件概念的。文件是以字符序列而构成的信息载体。根据这一点，可以把 I/O 设备当作文件来处理。因此，与磁盘上的普通文件进行交互所用的同一系统调用可以直接用于 I/O 设备。这样大大简化了系统对不同设备的处理，提高了效率。Linux 中的文件主要分为 4 种：普通文件、目录文件、链接文件和设备文件。

那么，内核如何区分和引用特定的文件呢？这里用到了一个重要的概念——文件描述符。对于 Linux 而言，所有对设备和文件的操作都是使用文件描述符来进行的。文件描述符是一个非负的整数，它是一个索引值，并指向在内核中每个进程打开文件的记录表。当打开一个现存文件或创建一个新文件时，内核就向进程返回一个文件描述符；当需要读写文件时，也需要把文件描述符作为参数传递给相应的函数。

通常，一个进程启动时，都会打开 3 个文件：标准输入、标准输出和标准出错处理。这 3 个文件分别对应文件描述符为 0、1 和 2（也就是宏替换 STDIN_FILENO、STDOUT_FILENO 和 STDERR_FILENO，鼓励读者使用这些宏替换）。

基于文件描述符的 I/O 操作虽然不能直接移植到类 Linux 以外的系统上去（如 Windows），但它往往是实现某些 I/O 操作的唯一途径，如 Linux 中低层文件操作函数、多路 I/O、TCP/IP 套接字编程接口等。同时，它们也很好地兼容 POSIX 标准，因此，可以很方便地移植到任何 POSIX 平台上。基于文件描述符的 I/O 操作是 Linux 中最常用的操作之一，希望读者能够很好地掌握。



9.1.6 标准 I/O 编程

本章前面几节所述的文件及 I/O 读写都是基于文件描述符的。这些都是基本的 I/O 控制，是不带缓存的。而本节所要讨论的 I/O 操作都是基于流缓冲的，它是符合 ANSI C 的标准 I/O 处理，这里有很多函数读者已经非常熟悉了（如 `printf()`、`scanf()` 函数等），因此本节中仅简要介绍最主要的函数。

前面讲述的系统调用是操作系统直接提供的函数接口。因为运行系统调用时，Linux 必须从用户态切换到内核态，执行相应的请求，然后再返回到用户态，所以应该尽量减少系统调用的次数，从而提高程序的效率。

标准 I/O 提供流缓冲的目的是尽可能减少使用 `read()` 和 `write()` 等系统调用的数量。标准 I/O 提供了 3 种类型的缓冲存储。

- 全缓冲：在这种情况下，当填满标准 I/O 缓存后才进行实际 I/O 操作。对于存放在磁盘上的文件通常是由标准 I/O 库实施全缓冲的。标准 I/O 尽量多读写文件到缓冲区，当缓冲区已满或手动 `flush` 时才会进行磁盘操作。
- 行缓冲：在这种情况下，当在输入和输出中遇到行结束符时，标准 I/O 库执行 I/O 操作。这允许我们一次输出一个字符（如 `fputc()` 函数），但只有写了一行之后才进行实际 I/O 操作。标准输入和标准输出就是使用行缓冲的典型例子。
- 不带缓冲：标准 I/O 库不对字符进行缓冲。如果用标准 I/O 函数写若干字符到不带缓冲的流中，则相当于用系统调用 `write()` 函数将这些字符全写到被打开的文件上。标准出错 `stderr` 通常是不带缓存的，这就使得出错信息可以尽快显示出来，而不管它们是否含有一个行结束符。

在下面讨论具体函数时，请读者注意区分以上的三种不同情况。

1. 打开文件

打开文件有三个标准函数，分别为：`fopen()`、`fdopen()` 和 `freopen()`。它们可以以不同的模式打开，但都返回一个指向 `FILE` 的指针，该指针指向对应的 I/O 流。此后，对文件的读写都是通过这个 `FILE` 指针来进行。其中 `fopen()` 可以指定打开文件的路径和模式，`fdopen()` 可以指定打开的文件描述符和模式，而 `freopen()` 除可指定打开的文件、模式外，还可指定特定的 I/O 流。

`fopen()` 函数格式如表 4.16 所示。

表 4.16 `fopen()` 函数语法要点

所需头文件	<code>#include <stdio.h></code>
函数原型	<code>FILE * fopen(const char * path, const char * mode)</code>
函数传入值	<code>path</code> : 包含要打开的文件路径及文件名
	<code>mode</code> : 文件打开状态，详细信息参考表 2.17
函数返回值	成功: 指向 <code>FILE</code> 的指针
	失败: <code>NULL</code>

其中，`mode` 类似于 `open()` 函数中的 `flag`，可以定义打开文件的访问权限等，表 4.17 说明了 `fopen()` 中



mode 的各种取值。

表 4.17 mode 取值说明

r 或 rb	打开只读文件，该文件必须存在
r+ 或 r+b	打开可读写的文件，该文件必须存在
w 或 wb	打开只写文件，若文件存在则文件长度清为 0，即会擦写文件以前的内容。若文件不存在则建立该文件
w+ 或 w+b	打开可读写文件，若文件存在则文件长度清为 0，即会擦写文件以前的内容。若文件不存在则建立该文件
a 或 ab	以附加的方式打开只写文件。若文件不存在，则会建立该文件；如果文件存在，写入的数据会被加到文件尾，即文件原先的内容会被保留
a+ 或 a+b	以附加方式打开可读写的文件。若文件不存在，则会建立该文件；如果文件存在，写入的数据会被加到文件尾后，即文件原先的内容会被保留

注意在每个选项中加入 b 字符用来告诉函数库打开的文件为二进制文件，而非纯文本文件。不过在 Linux 系统中会自动识别不同类型的文件而将此符号可以忽略。

fdopen()函数格式如表 4.18 所示。

表 4.18 fdopen()函数语法要点

所需头文件	#include <stdio.h>
函数原型	FILE * fdopen(int fd, const char * mode)
函数传入值	fd: 要打开的文件描述符
	mode: 文件打开状态, , 详细信息参考表 2.17
函数返回值	成功: 指向 FILE 的指针 失败: NULL

fopen()函数格式如表 4.19 所示。

表 4.19 fopen()函数语法要点

所需头文件	#include <stdio.h>
函数原型	FILE * fopen(const char *path, const char * mode, FILE * stream)
函数传入值	path: 包含要打开的文件路径及文件名
	mode: 文件打开状态, , 详细信息参考表 2.17
	stream: 已打开的文件指针
函数返回值	成功: 指向 FILE 的指针 失败: NULL



2. 关闭文件

关闭标准流文件的函数为 `fclose()`，该函数将缓冲区内的数据全部写入到文件中，并释放系统所提供的文件资源。

`fclose()`函数格式如表 4.20 所示。

表 4.20 `fclose()`函数语法要点

所需头文件	<code>#include <stdio.h></code>
函数原型	<code>int fclose(FILE * stream)</code>
函数传入值	stream : 已打开的文件指针
函数返回值	成功: 0 失败: EOF

3. 读文件

在文件流被打开之后，可对文件流进行读写等操作，其中读操作的函数为 `fread()`。

`fread()`函数格式如表 2.21 所示。

表 4.21 `fread()`函数语法要点

所需头文件	<code>#include <stdio.h></code>
函数原型	<code>size_t fread(void * ptr, size_t size, size_t nmemb, FILE * stream)</code>
函数传入值	ptr : 存放读入记录的缓冲区
	size : 读取的记录大小
	nmemb : 读取的记录数
	stream : 要读取的文件流
函数返回值	成功: 返回实际读取到的 nmemb 数目 失败: EOF

4. 写文件

`fwrite()`函数是用于对指定的文件流进行写操作。`fwrite()`函数格式如表 4.22 所示。

表 4.22 `fwrite()`函数语法要点

所需头文件	<code>#include <stdio.h></code>
函数原型	<code>size_t fwrite(const void * ptr, size_t size, size_t nmemb, FILE * stream)</code>
函数传入值	ptr : 存放写入记录的缓冲区
	size : 写入的记录大小
	nmemb : 写入的记录数
	stream : 要写入的文件流
函数返回值	成功: 返回实际写入到的 nmemb 数目 失败: EOF

文件打开之后，根据一次读写文件中字符的数目可分为字符输入输出、行输入输出和格式化输入输



出，下面分别对这 3 种不同的方式进行讨论。

1. 字符输入输出

字符输入输出函数一次仅读写一个字符。其中字符输入输出函数如表 4.23 和表 4.24 所示。

表 4.23 字符输出函数语法要点

所需头文件	#include <stdio.h>
函数原型	int getc(FILE * stream) int fgetc(FILE * stream) int getchar(void)
函数传入值	stream: 要输入的文件流
函数返回值	成功: 下一个字符 失败: EOF

表 4.24 字符输入函数语法要点

所需头文件	#include <stdio.h>
函数原型	int putc(int c, FILE * stream) int fputc(int c, FILE * stream) int putchar(int c)
函数返回值	成功: 字符 c 失败: EOF

这几个函数功能类似，其区别仅在于 getc()和 putc()通常被实现为宏，而 fgetc()和 fputc()不能实现为宏，因此，函数的实现时间会有所差别。

下面这个实例结合 fputc()和 fgetc()，将标准输入复制到标准输出中去。

```
/*fput.c*/
#include<stdio.h>
main()
{
    int c;
    /*把 fgetc()的结果作为 fputc()的输入*/
    fputc(fgetc(stdin), stdout);
}
```

运行结果如下所示：

```
$ ./fput
w (用户输入)
w (屏幕输出)
```

2. 行输入输出



行输入输出函数一次操作一行。其中行输入输出函数如表 4.25 和表 4.26 所示。

表 4.25 行输出函数语法要点

所需头文件	#include <stdio.h>
函数原型	char * gets(char *s) char fgets(char * s, int size, FILE * stream)
函数传入值	s: 要输入的字符串 size: 输入的字符串长度 stream: 对应的文件流
函数返回值	成功: s 失败: NULL

表 4.26 行输入函数语法要点

所需头文件	#include <stdio.h>
函数原型	int puts(const char *s) int fputs(const char * s, FILE * stream)
函数传入值	s: 要输出的字符串 stream: 对应的文件流
函数返回值	成功: s 失败: NULL

这里以 gets()和 puts()为例进行说明，本实例将标准输入复制到标准输出，如下所示：

```
/*gets.c*/
#include<stdio.h>
main()
{
    char s[80];
    fputs(fgets(s, 80, stdin), stdout);
}
```

运行该程序，结果如下所示：

```
$ ./gets
This is stdin（用户输入）
This is stdin（屏幕输出）
```

3. 格式化输入输出



格式化输入输出函数可以指定输入输出的具体格式，这里有读者已经非常熟悉的 printf()、scanf() 等函数，这里就简要介绍一下它们的格式。如下表 4.25～表 4.27 所示。

表 4.25 格式化输出函数 1

所需头文件	#include <stdio.h>
函数原型	int printf(const char *format,...) int fprintf(FILE *fp, const char *format,...) int sprintf(char *buf, const char *format,...)
函数传入值	format: 记录输出格式 fp: 文件描述符 buf: 记录输出缓冲区
函数返回值	成功: 输出字符数 (sprintf 返回存入数组中的字符数) 失败: NULL

表 4.26 格式化输出函数 2

所需头文件	#include <stdarg.h> #include <stdio.h>
函数原型	int vprintf(const char *format, va_list arg) int vfprintf(FILE *fp, const char *format, va_list arg) int vsprintf(char *buf, const char *format, va_list arg)
函数传入值	format: 记录输出格式 fp: 文件描述符 arg: 相关命令参数
函数返回值	成功: 存入数组的字符数 失败: NULL

表 4.27 格式化输入函数

所需头文件	#include <stdio.h>
函数原型	int scanf(const char *format,...) int fscanf(FILE *fp, const char *format,...) int sscanf(char *buf, const char *format,...)
函数传入值	format: 记录输出格式 fp: 文件描述符 buf: 记录输入缓冲区
函数返回值	成功: 输出字符数 (sprintf 返回存入数组中的字符数) 失败: NULL



9.2 标准 IO 实验

9.2.1 实验目的

通过实验学习标准 IO 的编程方法。

9.2.2 实验平台

华清远见开发环境

9.2.3 实验步骤

本实验通过一个简单的程序计算默认缓冲区的大小，理解标准 I/O 提供的三种类型的缓存，这三种缓存类型分别是：

全缓冲：当填满 I/O 缓存后才进行实际的 I/O 操作；

行缓冲：当在输入和输出中遇到新换行符(‘\n’)时，进行 I/O 操作；

不带缓冲：标准 I/O 库不对字符进行缓冲，例如 stderr；

本实验验证全缓存类型。

将【华清远见-嵌入式 ARM 实验箱资料- I -FS6818：程序源码\Linux 应用实验源码】目录拷贝到虚拟机共享目录【D:\share\】下。**(注意：此步骤已经在“Linux Shell 编程”章节中完成，如果没有完成请参照此章节。如已经完成可省略此步骤)**

将 io 部分代码拷贝到虚拟机 Linux 下。

1、建立相关目录；

```
$ mkdir ~/workdir/linux/application/stdio -p
$ cd ~/workdir/linux/application/stdio
```

```
linux@ubuntu64-vm:~$
linux@ubuntu64-vm:~$ mkdir ~/workdir/linux/application/stdio -p
linux@ubuntu64-vm:~$ cd ~/workdir/linux/application/stdio
linux@ubuntu64-vm:~/workdir/linux/application/stdio$
```

2、将代码从共享目录拷入虚拟机 Linux 操作系统下；（可使用【ctrl+空格】切换输入法）

```
$ cp /mnt/hgfs/share/Linux 应用实验源码/06.\ Linux 系统标准 IO 实验/实验代码/* . -a
$ ls
```

```
linux@ubuntu64-vm:~/workdir/linux/application/stdio$ cp /mnt/hgfs/share/Linux应用实验源码/06.\ Linux系统标准IO实验/实验代码/* . -a
linux@ubuntu64-vm:~/workdir/linux/application/stdio$ ls
61.c
```

3、执行代码：

```
$ gcc 61.c -o stdio_test
$ ./stdio_test
```



9.2.4 实验现象

可以看到终端屏幕共输出了 1024 字节大小的内容，多余的内容没有输出，这就验证了全缓存的特点：当填满 I/O 缓存后才进行实际的 I/O 操作。

```
linux@ubuntu64-vm:~/workdir/linux/application/stdio$ ./stdio_test
000000100200300400500600700800901001101201301401501601701801902002102202302402502602702802903003103203303403
50360370380390400410420430440450460470480490500510520530540550560570580590600610620630640650660670680690700
71072073074075076077078079080081082083084085086087088089090091092093094095096097098099100101102103104105106
10710810911011111211311411511611711811912012112212312412512612712812913013113213313413513613713813914014114
21431441451461471481491501511521531541551561571581591601611621631641651661671681691701711721731741751761771
78179180181182183184185186187188189190191192193194195196197198199200201202203204205206207208209210211212213
21421521621721821922022122222322422522622722822923023123223323423523623723823924024124224324424524624724824
92502512522532542552562572582592602612622632642652662672682692702712722732742752762772782792802812822832842
85286287288289290291292293294295296297298299300301302303304305306307308309310311312313314315316317318319320
3213223233243253263273283293303313323333343353363373383393403
```

9.2.5 实验代码

Java Code

```
1#include <stdio.h>
2int main()
3{
4    int i=0;
5    for(i=0;i<379;i++)//每次向缓冲区内写三个字符
6    {
7        if(i>=100)
8            fprintf(stdout,"%d",i);
9        else if (i>=10)
10            fprintf(stdout,"%d",i);
11        else if (i>=0)
12            fprintf(stdout,"%0d",i);
13    }
14    while(1);//强制执行，如果取消，程序结束时将会输出所有字符，看不到效果了。
15}
```

9.3 Linux 系统文件目录操作编程实验

9.3.1 实验目的

通过实验学习用 Linux C 操作文件目录的方法。

9.3.2 实验平台

华清远见开发环境

9.3.3 实验步骤

将【华清远见-嵌入式 ARM 实验箱资料- I -FS6818：程序源码\Linux 应用实验源码】目录拷贝到虚拟



机共享目录【D:\share\】下。(注意：此步骤已经在“Linux Shell 编程”章节中完成，如果没有完成请参照此章节。如已经完成可省略此步骤)

将 io 部分代码拷贝到虚拟机 Linux 下。

1、建立相关目录：

```
$ mkdir ~/workdir/linux/application/io_file -p
$ cd ~/workdir/linux/application/io_file
```

```
linux@ubuntu64-vm:~$
linux@ubuntu64-vm:~$ mkdir ~/workdir/linux/application/io_file -p
linux@ubuntu64-vm:~$ cd ~/workdir/linux/application/io_file/
linux@ubuntu64-vm:~/workdir/linux/application/io_file$
```

2、将代码从共享目录拷入虚拟机 Linux 操作系统下；（可使用【ctrl+空格】切换输入法）

```
$ cp /mnt/hgfs/share/Linux 应用实验源码/07.\ Linux 系统文件目录操作编程实验/实验代码/7.2/* . -a
$ ls
```

```
linux@ubuntu64-vm:~/workdir/linux/application/io_file$ cp /mnt/hgfs/share/Linux应用实验源码/07.\
Linux系统文件目录操作编程实验/实验代码/7.2/* . -a
linux@ubuntu64-vm:~/workdir/linux/application/io_file$ ls
filecopy.c getopt_test.c myls.c
```

3、执行代码：

```
$ mkdir example
$ touch example/1.c example/2.c //在 io_file 目录下创建两个文件，后面程序会读这两个文件
$ gcc myls.c -o myls
$ ./mys example
```

9.3.4 实验现象

程序执行后会列出 example 目录下的两个文件。

```
linux@ubuntu64-vm:~/workdir/linux/application/io_file$ ls
example filecopy.c getopt_test.c myls myls.c
linux@ubuntu64-vm:~/workdir/linux/application/io_file$ ./mys example
2.c 1.c
```

9.3.5 实验代码

mys.c

```
1#include <stdio.h>
2#include <stdlib.h>
3#include <sys/types.h>
4#include <sys/stat.h>
5#include <errno.h>
6#include <unistd.h>
7#include <string.h>
```



```

8#include <time.h>
9#include <pwd.h>
10#include <grp.h>
11#include <dirent.h>
12
13#define N 128
14
15int aflag = 0, lflag = 0;
16
17void displayfile(char *name, char *showname);
18void displaydir(const char *name);
19
20int main(int argc, char *argv[])
21{
22    struct stat buf;
23    int ch, i;
24
25    if (argc < 2)
26    {
27        fprintf(stdout, "usage: %s filename\n", argv[0]);
28        exit(-1);
29    }
30
31    while ((ch = getopt(argc, argv, "al")) != -1)
32    {
33        switch(ch)
34        {
35            case 'a':
36                aflag = 1;
37                break;
38            case 'l':
39                lflag = 1;
40                break;
41            default:
42                printf("wrong option %c\n", optopt);
43        }
44    }
45
46    // printf("aflag=%d lflag=%d\n", aflag, lflag);
47
48    for (i = optind; i < argc; i++)
49    {
50        if (stat(argv[i], &buf) == -1)

```



```

51     {
52         perror("stat");
53         exit(-1);
54     }
55
56     if (!S_ISDIR(buf.st_mode))
57         displayfile(argv[i], argv[i]);
58     else
59         displaydir(argv[i]);
60 }
61 if (lflag == 0)
62     printf("\n");
63 return 0;
64 }
65
66 void displayfile(char *name, char *showname)
67 {
68     struct stat buf;
69     struct tm *p;
70     int i = 8;
71
72     if (lflag == 0)
73     {
74         printf("%s ", showname);
75         return;
76     }
77
78     if (stat(name, &buf) == -1)
79     {
80         perror("stat");
81         exit(-1);
82     }
83
84     switch (buf.st_mode & S_IFMT)
85     {
86     case S_IFSOCK:
87         printf("s");
88         break;
89     case S_IFLNK:
90         printf("l");
91         break;
92     case S_IFREG:
93         printf("-");

```



```

94         break;
95     case S_IFBLK:
96         printf("b");
97         break;
98     case S_IFDIR:
99         printf("d");
100        break;
101    case S_IFCHR:
102        printf("c");
103        break;
104    case S_IFIFO:
105        printf("p");
106        break;
107    default:
108        printf("?");
109    }
110    while (i >= 0)
111    {
112        if ((buf.st_mode) >> i & 1)
113        {
114            switch (i % 3)
115            {
116                case 2:
117                    printf("r");
118                    break;
119                case 1:
120                    printf("w");
121                    break;
122                case 0:
123                    printf("x");
124                    break;
125            }
126        }
127        else
128        {
129            printf("-");
130        }
131        i--;
132    }
133
134    printf(" ");
135
136    /* hard links */

```



```

137     printf("%d", buf.st_nlink);
138
139     /* uid gid */
140     printf(" %s %s ", getpwuid(buf.st_uid)->pw_name, getgrgid(buf.st_gid)->gr_name);
141
142     printf("%ld ", buf.st_size);
143
144     p = localtime(&buf.st_mtime);
145     printf("%d-%d-%d %d:%d ", p->tm_year + 1900, p->tm_mon + 1,
146           p->tm_mday, p->tm_hour, p->tm_min);
147     printf("%s\n", showname);
148 }
149 void displaydir(const char *name)
150 {
151     DIR *dir;
152     struct dirent *diritem;
153     char str[N];
154
155     dir = opendir(name);
156     while ((diritem = readdir(dir)) != NULL)
157     {
158         if (strncmp(".", diritem->d_name, 1) == 0 && aflag == 0)
159             continue;
160         sprintf(str, "./%s", diritem->d_name);
161         displayfile(str, diritem->d_name);
162     }
163 }

```



第 10 章 嵌入式 Linux 多任务编程实验

10.1 使用 ps 命令查看进程信息

10.1.1 实验内容

ps 是基本的 Linux 命令，通过本实验，不仅要熟悉 ps 命令方法，更重要的是可以了解 Linux 进程的组成。

10.1.2 实验平台

华清远见开发环境

10.1.3 实验原理

ps: 查看系统中的进程，Linux 中可以使用 ps -aux 查看所有进程。其中 PID 代表进程 ID，TTY 是该进程是由哪个控制台启动的，CMD 则是命令。

如果想列出更详细的信息，则可使用命令：“ps -auxw”。参数 w 表示加宽显示的命令行，参数 w 可以写多次，通常最多写 3 次，表示加宽 3 次，这足以显示很长的命令行了。

10.1.4 实验步骤

在 shell 提示符下输入如下命令，并解释输出的结果：

```
$ ps
```

终端显示：

```
PID TTY TIME CMD
```

```
16767 pts/1 0:00 ps
```

```
18029 pts/1 0:00 bash
```

在 shell 提示符下输入如下命令，并解释输出的结果：

```
$ ps aux
```

终端显示：

```
PID TTY TIME CMD
```

10.2 使用 proc 文件系统查看进程信息

10.2.1 实验目的

本实验将指导学员了解 proc 文件系统，通过 proc 文件系统查询进程信息，可以扩展到修改系统参数。

10.2.2 实验平台

华清远见开发环境



10.2.3 实验原理

/proc 文件系统是一个虚拟文件系统，通过文件系统接口实现对内核的访问，输出系统运行状态。它以文件系统的形式，为操作系统本身和应用进程之间的通信提供了一个界面，使应用程序能够安全、方便的获得系统当前的运行状况和内核的内部数据信息，并且可以修改某些系统的配置信息。

10.2.4 实验内容

认识 proc 文件系统的文件和目录：

```
$ cd /proc
```

```
$ ls
```

通过 proc 文件系统查看系统当前进行状态：

```
$ cat /proc/self/status
```

查询文件句柄的当前使用情况：

```
$ cat /proc/sys/fs/file-nr
```

```
6304    0    200189
```

file-nr 文件显示了三个参数：分配的文件句柄总数、当前使用的文件句柄数以及可以分配的最大文件句柄数。如果需要增大/proc/sys/fs/file-max 中的值，请确保正确设置 ulimit。对于 2.4.20，通常将其设置为 unlimited。使用 ulimit 命令来验证 ulimit 设置：

```
$ ulimit
```

```
unlimited
```

通过 proc 文件系统修改内核中预定的一些变量。

修改整个系统中文件句柄的最大数量：

```
$ cat /proc/sys/fs/file-max
```

```
52458
```

```
$ echo 65536 >/proc/sys/fs/file-max
```

```
$ cat /proc/sys/fs/file-max
```

```
65536
```

修改网络 TTL：

```
$ cat /proc/sys/net/ipv4/ip_default_ttl
```

```
64
```

```
$ echo 128 >/proc/sys/net/ipv4/ip_default_ttl
```

```
$ cat /proc/sys/net/ipv4/ip_default_ttl
```

```
128
```

修改系统中最大进程数量：

```
$ cat /proc/sys/kernel/pid_max
```

```
32768
```



```
$ echo 65536 >/proc/sys/kernel/pid_max
$ cat /proc/sys/kernel/pid_max
65536
```

修改普通用户的最大 RTC 频率：

```
$ cat /proc/sys/dev/rtc/max-user-freq
64
$ echo 128 >/proc/sys/dev/rtc/max-user-freq
$ cat /proc/sys/dev/rtc/max-user-freq
128
$ cat /proc/cpuinfo      //查看 CPU 信息
$ cat /proc/interrupts  //查看中断信息
$ cat /proc/ioports     //设备 IO 端口
$ cat /proc/meminfo     //内存信息
$ cat /proc/partitions  //所有设备的所有分区
$ cat /proc/pci         //PCI 设备的信息
$ cat /proc/swaps       //所有 Swap 分区的信息
$ cat /proc/version     //Linux 的版本号
```

10.3 使用 fork、exit 和 exec 系统调用编写多进程程序

10.3.1 实验目的

本实验将通过编写 fork 等系统调用的程序，加深对系统进程及其控制的了解。

10.3.2 实验平台

华清远见开发环境

10.3.3 实验原理

fork 后父子进程会同步运行，但父子进程的返回顺序是不确定的。设两个变量 global 和 test 来检测父子进程共享资源的情况。

10.3.4 实验步骤

将【华清远见-嵌入式 ARM 实验箱资料- I -FS6818：程序源码\Linux 应用实验源码】目录拷贝到虚拟机共享目录【D:\share\】下。（注意：此步骤已经在“Linux Shell 编程”章节中完成，如果没有完成请参照此章节。如已经完成可省略此步骤）

将进程实验部分代码拷贝到虚拟机 Linux 下。

1、建立相关目录；



```
$ mkdir ~/workdir/linux/application/fork_test -p
$ cd ~/workdir/linux/application/fork_test
```

```
linux@ubuntu64-vm:~$
linux@ubuntu64-vm:~$ mkdir ~/workdir/linux/application/fork_test -p
linux@ubuntu64-vm:~$ cd ~/workdir/linux/application/fork_test
linux@ubuntu64-vm:~/workdir/linux/application/fork_test$ ls
```

2、将代码从共享目录拷入虚拟机 Linux 操作系统下；（可使用【ctrl+空格】切换输入法）

```
$ cp /mnt/hgfs/share/Linux 应用实验源码/09.\ Linux 系统 fork 等系统调用实验/实验代码/93.c .
$ ls
```

```
linux@ubuntu64-vm:~/workdir/linux/application/fork_test$ cp /mnt/hgfs/share/Linux应用实验源码/09.
\ Linux系统fork等系统调用实验/实验代码/93.c .
linux@ubuntu64-vm:~/workdir/linux/application/fork_test$ ls
93.c
```

3、执行代码：

```
$ gcc 93.c -o fork
$ ./fork //执行观察结果
```

10.3.5 实验现象

可以看出父子进程打印出了各自的进程号和对应变量的值，显然 global 和 test 在父子进程间是独立的，

其各自的操作不会对对方的值有影响。

```
93.c
linux@ubuntu64-vm:~/workdir/linux/application/9-process/fork$ gcc 93.c -o fork
linux@ubuntu64-vm:~/workdir/linux/application/9-process/fork$ ./fork
the test content!
fork test!
global=24 test=2 Parent,my PID is 32845
global=23 test=1 Child,my PID is 32846
linux@ubuntu64-vm:~/workdir/linux/application/9-process/fork$
```

10.3.6 实验代码

93.c

```
1#include <stdio.h>
2#include <sys/types.h>
3#include <sys/wait.h>
4#include <unistd.h>
5#include <stdlib.h>
6#include <fcntl.h>
7
8int global = 22;
9char buf[] = "the test content!\n";
10
11int main(void)
```



```

12{
13    int test = 0, stat;
14    pid_t pid;
15    if(write(STDOUT_FILENO, buf, sizeof(buf)) != sizeof(buf))
16    {
17        perror("write error!");
18    }
19    printf(" fork test!\n");
20    /* fork */
21    pid = fork();    /*we should check the error*/
22    if (pid == -1)
23    {
24        perror("fork");
25        exit(0);
26    }
27    else if (pid == 0)
28    {
29        global++;
30        test++;
31        printf("global=%d test=%d Child,my PID is %d\n", global, test, getpid());
32        exit(0);
33    }
34    /*else be the parent*/
35    global += 2;
36    test += 2;
37    printf("global=%d test=%d Parent,my PID is %d\n", global, test, getpid());
38    exit(0);
39    //printf("global=%d test=%d Parent,my PID is %d",global,test,getpid());
40    //_exit(0);}

```

10.4 Linux 系统守护进程实验

10.4.1 实验目的

通过实验熟悉守护进程的编写过程。

10.4.2 实验平台

华清远见开发环境

10.4.3 实验原理

守护进程编写的主要步骤如下：

将程序进入后台执行。由于守护进程最终脱离控制终端，到后台去运行。方法是在进程中调用 fork 使



父进程终止，让 Daemon 在子进程中后台执行。这就是常说的“脱壳”。子进程继续函数 fork() 的定义如下：

```
pid_t fork(void);
```

脱离控制终端、登录会话和进程组。开发人员如果要摆脱它们，不受它们的影响，一般使用 setsid() 设置新会话的领头进程，并与原来的登录会话和进程组脱离。

禁止进程重新打开控制终端。

重设文件权限掩码

关闭打开的文件描述符，并重定向标准输入、标准输出和标准错误输出的文件描述符。进程从创建它的父进程那里继承了打开的文件描述符。如果不关闭，将会浪费系统资源，引起无法预料的错误。关闭三者的代码如下：

```
for (fd = 0, fdtablesize = getdtablesize(); fd < fdtablesize; fd++)
```

```
close(fd);
```

改变工作目录到根目录或特定目录进程活动时，其工作目录所在的文件系统不能卸下。

10.4.4 实验步骤

将【华清远见-嵌入式 ARM 实验箱资料-I-FS6818：程序源码\Linux 应用实验源码】目录拷贝到虚拟机共享目录【D:\share\】下。（注意：此步骤已经在“Linux Shell 编程”章节中完成，如果没有完成请参照此章节。如已经完成可省略此步骤）

将守护进程实验部分代码拷贝到虚拟机 Linux 下。

1、建立相关目录；

```
$ mkdir ~/workdir/linux/application/daemon -p
```

```
$ cd ~/workdir/linux/application/daemon
```

```
linux@ubuntu64-vm:~$  
linux@ubuntu64-vm:~$ mkdir ~/workdir/linux/application/daemon -p  
linux@ubuntu64-vm:~$ cd ~/workdir/linux/application/daemon  
linux@ubuntu64-vm:~/workdir/linux/application/daemon$
```

2、将代码从共享目录拷入虚拟机 Linux 操作系统下；（可使用【ctrl+空格】切换输入法）

```
$ cp /mnt/hgfs/share/Linux 应用实验源码/11.\ Linux 系统守护进程实验/实验代码/*.c .
```

```
$ ls
```

```
linux@ubuntu64-vm:~/workdir/linux/application/daemon$ cp /mnt/hgfs/share/Linux应用实验源码/11.\ L  
inux系统守护进程实验/实验代码/*.c .  
linux@ubuntu64-vm:~/workdir/linux/application/daemon$ ls  
init.c test.c
```

3、执行代码：

```
$ gcc init.c test.c -o test
```

```
$ ./test
```



10.4.5 实验现象

\$ ps -ef //查看进程

```

linux@ubuntu64-vm: ~/workdir/linux/application/9-process/deamon
linux@ubuntu64-vm: ~/workdir/linux/application
linux 2685 1 0 04:47 ? 00:00:00 /usr/bin/python /usr/lib/unity-scope-video-remote/unity-scope-vi
linux 2710 2685 0 04:47 ? 00:00:00 sh -c /usr/lib/x86_64-linux-gnu/libproxy/0.4.7/pxgsettings org.g
linux 2711 2710 0 04:47 ? 00:00:00 /usr/lib/x86_64-linux-gnu/libproxy/0.4.7/pxgsettings org.gnome.s
linux 2725 2231 0 04:48 ? 00:00:01 /usr/lib/gnome-disk-utility/gdu-notification-daemon
linux 2735 2231 0 04:48 ? 00:00:01 telepathy-indicator
linux 2743 1 0 04:48 ? 00:00:00 /usr/lib/telepathy/mission-control-5
linux 2748 1 0 04:48 ? 00:00:00 /usr/lib/gnome-online-accounts/goa-daemon
linux 2756 2231 0 04:48 ? 00:00:01 gnome-screensaver
linux 2846 2231 0 04:48 ? 00:00:02 update-notifier
root 2863 1 0 04:48 ? 00:00:00 /usr/bin/python /usr/lib/system-service/system-service-d
root 3011 1 0 04:49 ? 00:00:00 /usr/sbin/sshd -D
linux 8249 2231 0 04:49 ? 00:00:00 /usr/lib/deja-dup/deja-dup-monitor
root 9092 399 0 04:53 ? 00:00:00 /sbin/udev --daemon
root 9093 2 0 04:53 ? 00:00:00 [kdmflush]
root 9094 399 0 04:53 ? 00:00:00 /sbin/udev --daemon
root 9103 2 0 04:53 ? 00:00:09 [kworker/2:0]
root 9165 2 0 04:54 ? 00:00:06 [jbd2/dm-0-8]
root 9166 2 0 04:54 ? 00:00:00 [ext4-dio-unwrit]
linux 29607 2533 0 10:15 pts/3 00:00:01 bash
root 32406 2 0 11:13 ? 00:00:00 [kworker/0:1]
root 32432 2 0 11:16 ? 00:00:00 [flush-8:0]
root 32636 2 0 11:23 ? 00:00:00 [kworker/0:0]
linux 32928 1 0 11:35 ? 00:00:00 ./test
linux 32935 2551 0 11:36 pts/1 00:00:00 ps -ef
root 61721 2 0 07:19 ? 00:00:02 [kworker/0:2]
linux@ubuntu64-vm: ~/workdir/linux/application/9-process/deamon$

```

查看/tmp/test.log 文件，从输出可以发现 test 守护进程的各种特性满足上面的要求。

\$ cat /tmp/test.log

```

root 61721 2 0 07:19 ? 00:00:02 [kworker/0:2]
linux@ubuntu64-vm: ~/workdir/linux/application/9-process/deamon$ cat /tmp/test.log
I'm here at Sat Aug 2 11:35:42 2014
nI'm here at Sat Aug 2 11:35:44 2014
nI'm here at Sat Aug 2 11:35:46 2014
nI'm here at Sat Aug 2 11:35:48 2014
nI'm here at Sat Aug 2 11:35:50 2014
nI'm here at Sat Aug 2 11:35:52 2014
nI'm here at Sat Aug 2 11:35:54 2014
nI'm here at Sat Aug 2 11:35:56 2014
nI'm here at Sat Aug 2 11:35:58 2014
nI'm here at Sat Aug 2 11:36:00 2014
nI'm here at Sat Aug 2 11:36:02 2014
nI'm here at Sat Aug 2 11:36:04 2014
nI'm here at Sat Aug 2 11:36:06 2014
nI'm here at Sat Aug 2 11:36:08 2014
nI'm here at Sat Aug 2 11:36:10 2014
nI'm here at Sat Aug 2 11:36:12 2014
nI'm here at Sat Aug 2 11:36:14 2014
nI'm here at Sat Aug 2 11:36:16 2014
nI'm here at Sat Aug 2 11:36:18 2014
nI'm here at Sat Aug 2 11:36:20 2014

```



10.4.6 实验代码

init.c

```
1#include <unistd.h>
2#include <sys/types.h>
3#include <sys/stat.h>
4#include <stdlib.h>
5void init_daemon(void)
6{
7    int pid;
8    int i;
9
10   if (pid = fork())
11   {
12       exit(0);
13   } //是父进程, 结束父进程
14
15   else if (pid < 0)
16   {
17       exit( -1 );
18   } //fork 失败, 退出
19
20   setsid(); //第一子进程成为新的会话组长和进程组长 并与控制终端分离
21
22   if (pid = fork())
23   {
24       exit(0);
25   } //是第一子进程, 结束第一子进程
26   else if (pid < 0)
27   {
28       exit(1);
29   } //fork 失败, 退出
30
31   //第二子进程不再是会话组长
32   for(i = 0; i < getdtablesize(); ++i)
33       //关闭打开的文件描述符
34       close(i);
35   chdir("/tmp"); //改变工作目录到 /tmp
36   umask(0); //重设文件创建掩模
37   return;
38 }
```

test.c



```

1#include <stdio.h>
2#include <time.h>
3void init_daemon(void); //守护进程初始化函数
4
5int main()
6{
7    FILE *fp;
8    time_t t;
9    init_daemon(); //初始化为 Daemon
10
11    while(1) //每隔 2 秒钟向 test.log 报告运行状态
12    {
13        sleep(2); //睡眠 2 秒钟
14        if((fp = fopen("test.log", "a")) != NULL)
15        {
16            t = time(0);
17            fprintf(fp, "I'm here at %sn", asctime(localtime(&t)) );
18            fclose(fp);
19        }
20    }
21    return 0;
22}

```



第 11 章 Linux 系统进程间通信实验

11.1 实验原理

通过前面的学习，读者已经知道了进程是一个程序的一次执行，系统资源分配的最小单元。这里所说的进程一般是指运行在用户态的进程，而由于处于用户态的不同进程之间是彼此隔离的，就像处于不同城市的人们，它们必须通过某种方式来进行通信，例如人们现在广泛使用的手机等方式。本章就是讲述如何建立这些不同的通话方式，就像人们有多种通信方式一样。

Linux 下的进程通信手段基本上是从 Unix 平台上的进程通信手段继承而来的。而对 Unix 发展做出重大贡献的两大主力 AT&T 的贝尔实验室及 BSD（加州大学伯克利分校的伯克利软件发布中心）在进程间的通信方面的侧重点有所不同。前者是对 Unix 早期的进程间通信手段进行了系统的改进和扩充，形成了“system V IPC”，其通信进程主要局限在单个计算机内；后者则跳过了该限制，形成了基于套接口（socket）的进程间通信机制。而 Linux 则把两者的优势都继承了下来，如图 4.1 所示。

- Unix 进程间通信（IPC）方式包括管道、FIFO 以及信号。
- System V 进程间通信（IPC）包括 System V 消息队列、System V 信号量以及 System V 共享内存区。
- Posix 进程间通信（IPC）包括 Posix 消息队列、Posix 信号量以及 Posix 共享内存区。

现在在 Linux 中使用较多的进程间通信方式主要有以下几种。

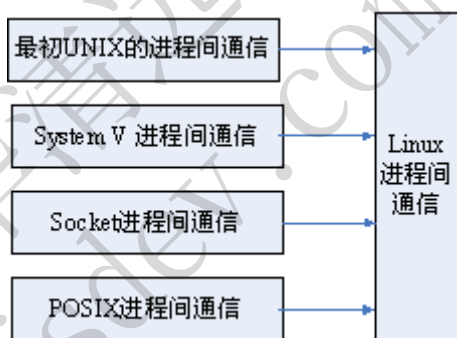


图 6.1 进程间通信发展历程

（1）管道（Pipe）及有名管道（named pipe）：管道可用于具有亲缘关系进程间的通信，有名管道，除具有管道所具有的功能外，它还允许无亲缘关系进程间的通信。

（2）信号（Signal）：信号是在软件层次上对中断机制的一种模拟，它是比较复杂的通信方式，用于通知进程有某事件发生，一个进程收到一个信号与处理器收到一个中断请求效果上可以说是一样的。

（3）消息队列（Message Queue）：消息队列是消息的链接表，包括 Posix 消息队列 SystemV 消息队列。它克服了前两种通信方式中信息量有限的缺点，具有写权限的进程可以按照一定的规则向消息队列中添加新消息；对消息队列有读权限的进程则可以从消息队列中读取消息。

（4）共享内存（Shared memory）：可以说这是最有效的进程间通信方式。它使得多个进程可以访问同一块内存空间，不同进程可以及时看到对方进程中对共享内存中数据的更新。这种通信方式需要依靠某种



同步机制，如互斥锁和信号量等。

(5) 信号量 (Semaphore): 主要作为进程之间以及同一进程的不同线程之间的同步和互斥手段。

(6) 套接字 (Socket): 这是一种更为一般的进程间通信机制，它可用于网络中不同机器之间的进程间通信，应用非常广泛。

11.1.1 管道通信

管道是 Linux 中进程间通信的一种方式，它把一个程序的输出直接连接到另一个程序的输入。Linux 的管道主要包括两种：无名管道和有名管道。

(1) 无名管道

无名管道是 Linux 中管道通信的一种原始方法，如图 6.2 (左图) 所示，它具有如下特点。

- 它只能用于具有亲缘关系的进程之间的通信 (也就是父子进程或者兄弟进程之间)。
- 它是一个半双工的通信模式，具有固定的读端和写端。
- 管道也可以看成是一种特殊的文件，对于它的读写也可以使用普通的 `read()`、`write()` 等函数。但是它不是普通的文件，并不属于其他任何文件系统，并且只存在于内存中。

(2) 有名管道 (FIFO)

有名管道是对无名管道的一种改进，如图 6.2 (右图) 所示，它具有如下特点：

- 它可以使互不相关的两个进程实现彼此通信。
- 该管道可以通过路径名来指出，并且在文件系统中是可见的。在建立了管道之后，两个进程就可以把它当作普通文件一样进行读写操作，使用非常方便。
- FIFO 严格地遵循先进先出规则，对管道及 FIFO 的读总是从开始处返回数据，对它们的写则把数据添加到末尾，它们不支持如 `lseek()` 等文件定位操作。

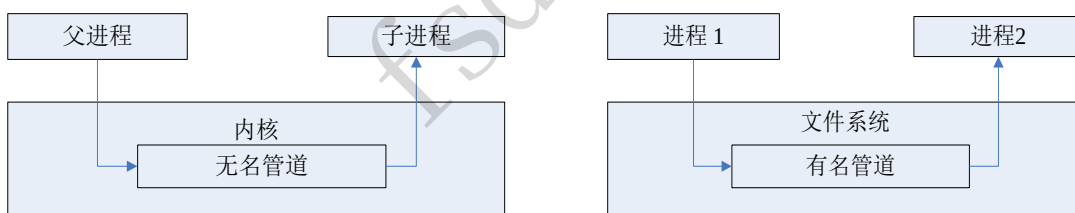


图 6.2 无名管道 (左) 和有名管道 (右)

11.1.2 无名管道系统调用

(1) 管道创建与关闭说明

管道是基于文件描述符的通信方式，当一个管道建立时，它会创建两个文件描述符 `fds[0]` 和 `fds[1]`，其中 `fds[0]` 固定用于读管道，而 `fd[1]` 固定用于写管道，如图 6.3 所示，这样就构成了一个半双工的通道。

管道关闭时只需将这两个文件描述符关闭即可，可使用普通的 `close()` 函数逐个关闭各个文件描述符。



图 6.3 无名管道的读写机制

(2) 管道创建函数

创建管道可以通过调用 `pipe()` 来实现，下表 6.1 列出了 `pipe()` 函数的语法要点。

表 6.1 `pipe()` 函数语法要点

所需头文件	#include <unistd.h>
函数原型	int pipe(int fd[2])
函数传入值	fd[2]: 管道的两个文件描述符，之后就可以直接操作这两个文件描述符
函数返回值	成功: 0
	出错: -1

(3) 管道读写说明

用 `pipe()` 函数创建的管道两端处于一个进程中，由于管道是主要用于在不同进程间通信的，因此这在实际应用中没有太大意义。实际上，通常先是创建一个管道，再调用 `fork()` 函数创建一子进程，该子进程会继承父进程所创建的管道，这时，父子进程管道的文件描述符对应关系如图 6.4 所示。

此时的关系看似非常复杂，实际上却已经给不同进程之间的读写创造了很好的条件。父子进程分别拥有自己的读写通道，为了实现父子进程之间的读写，只需把无关的读端或写端的文件描述符关闭即可。例如在图 6.5 中将父进程的写端 `fd[1]` 和子进程的读端 `fd[0]` 关闭。此时，父子进程之间就建立起了一条“子进程写入父进程读取”的通道。

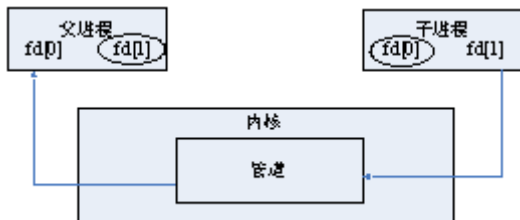


图 6.4 父子进程管道的文件描述符对应关系

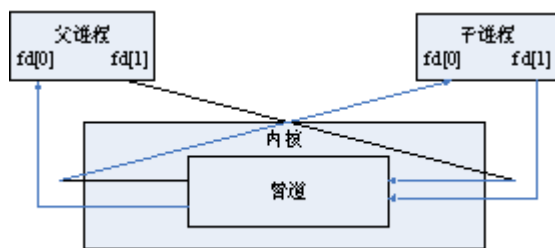


图 6.5 关闭父进程 fd[1]和子进程 fd[0]

同样，也可以关闭父进程的 fd[0]和子进程的 fd[1]，这样就可以建立一条“父进程写入子进程读取”的通道。另外，父进程还可以创建多个子进程，各个子进程都继承了相应的 fd[0]和 fd[1]，这时，只需要关闭相应端口就可以建立其各子进程之间的通道。

(4) 管道读写注意点

- 只有在管道的读端存在时，向管道写入数据才有意义。否则，向管道写入数据的进程将收到内核传来的 SIGPIPE 信号（通常为 Broken pipe 错误）。
- 向管道写入数据时，Linux 将不保证写入的原子性，管道缓冲区一有空闲区域，写进程就会试图向管道写入数据。如果读进程不读取管道缓冲区中的数据，那么写操作将会一直阻塞。
- 父子进程在运行时，它们的先后次序并不能保证，因此，在为了保证父子进程已经关闭了相应的文件描述符，可在两个进程中调用 sleep()函数，当然这种调用不是很好的解决方法，在后面学到进程之间的同步与互斥机制之后，请读者自行修改本小节的实例程序。

11.1.3 标准流管道

(1) 标准流管道函数说明

与 Linux 的文件操作中有基于文件流的标准 I/O 操作一样，管道的操作也支持基于文件流的模式。这种基于文件流的管道主要是用来创建一个连接到另一个进程的管道，这里的“另一个进程”也就是一个可以进行一定操作的可执行文件，例如，用户执行“ls -l”或者自己编写的程序“./pipe”等。由于这一类操作很常用，因此标准流管道就将一系列的创建过程合并到一个函数 popen()中完成。它所完成的工作有以下几步。

- 创建一个管道。
- fork()一个子进程。
- 在父子进程中关闭不需要的文件描述符。
- 执行 exec 函数族调用。
- 执行函数中所指定的命令。

这个函数的使用可以大大减少代码的编写量，但同时也有一些不利之处，例如，它不如前面管道创建的函数那样灵活多样，并且用 popen()创建的管道必须使用标准 I/O 函数进行操作，但不能使用前面的 read()、write()一类不带缓冲的 I/O 函数。

与之相对应，关闭用 popen()创建的流管道必须使用函数 pclose()来关闭该管道流。该函数关闭标准



I/O 流，并等待命令执行结束。

(2) 函数格式

`popen()`和 `pclose()`函数格式如表 6.2 和表 6.3 所示。

表 6.2 `popen()`函数语法要点

所需头文件	#include <stdio.h>	
函数原型	FILE *popen(const char *command, const char *type)	
函数传入值	command: 指向的是一个以 null 结束符结尾的字符串，这个字符串包含一个 shell 命令，并被送到/bin/sh 以-c 参数执行，即由 shell 来执行	
	type:	“r”: 文件指针连接到 command 的标准输出，即该命令的结果产生输出 “w”: 文件指针连接到 command 的标准输入，即该命令的结果产生输入
函数返回值	成功: 文件流指针	
	出错: -1	

表 6.3 `pclose()`函数语法要点

所需头文件	#include <stdio.h>	
函数原型	int pclose(FILE *stream)	
函数传入值	stream: 要关闭的文件流	
函数返回值	成功: 返回由 popen()所执行的进程的退出码	
	出错: -1	

11.1.4 有名管道 (FIFO)

有名管道的创建可以使用函数 `mkfifo()`，该函数类似文件中的 `open()`操作，可以指定管道的路径和打开的模式。(用户还可以在命令行使用“`mknod 管道名 p`”来创建有名管道。)

在创建管道成功之后，就可以使用 `open()`、`read()`和 `write()`这些函数了。与普通文件的开发设置一样，对于为读而打开的管道可在 `open()`中设置 `O_RDONLY`，对于为写而打开的管道可在 `open()`中设置 `O_WRONLY`，在这里与普通文件不同的是阻塞问题。由于普通文件的读写时不会出现阻塞问题，而在管道的读写中却有阻塞的可能，这里的非阻塞标志可以在 `open()`函数中设定为 `O_NONBLOCK`。下面分别对阻塞打开和非阻塞打开的读写进行讨论。

对于读进程

- 若该管道是阻塞打开，且当前 FIFO 内没有数据，则对读进程而言将一直阻塞到有数据写入。
- 若该管道是非阻塞打开，则不论 FIFO 内是否有数据，读进程都会立即执行读操作。即如果 FIFO 内没有数据，则读函数将立刻返回 0。

对于写进程



- 若该管道是阻塞打开，则写操作将一直阻塞到数据可以被写入。
- 若该管道是非阻塞打开而不能写入全部数据，则读操作进行部分写入或者调用失败。

表 4.4 列出了 mkfifo()函数的语法要点。

表 6.4 mkfifo()函数语法要点

所需头文件	#include <sys/types.h> #include <sys/stat.h>	
函数原型	int mkfifo(const char *filename, mode_t mode)	
函数传入值	filename: 要创建的管道	
函数传入值	mode:	O_RDONLY: 读管道
		O_WRONLY: 写管道
		O_RDWR: 读写管道
		O_NONBLOCK: 非阻塞
		O_CREAT: 如果该文件不存在，那么就创建一个新的文件，并用第三个参数为其设置权限
		O_EXCL: 如果使用 O_CREAT 时文件存在，那么可返回错误消息。这一参数可测试文件是否存在
函数返回值	成功: 0	
	出错: -1	

表 4.5 再对 FIFO 相关的出错信息做一归纳，以方便用户查错。

表 6.5 FIFO 相关的出错信息

EACCESS	参数 filename 所指定的目录路径无可执行的权限
EEXIST	参数 filename 所指定的文件已存在
ENAMETOOLONG	参数 filename 的路径名称太长
ENOENT	参数 filename 包含的目录不存在
ENOSPC	文件系统的剩余空间不足
ENOTDIR	参数 filename 路径中的目录存在但却非真正的目录
EROFS	参数 filename 指定的文件存在于只读文件系统内

11.1.5 信号通信

信号是在软件层次上对中断机制的一种模拟。在原理上，一个进程收到一个信号与处理器收到一个中断请求可以说是一样的。信号是异步的，一个进程不必通过任何操作来等待信号的到达，事实上，进程也不知道信号到底什么时候到达。信号可以直接进行用户空间进程和内核进程之间的交互，内核进程也可以利用它来通知用户空间进程发生了哪些系统事件。它可以在任何时候发给某一进程，而无需知道该进程的



状态。如果该进程当前并未处于执行态，则该信号就由内核保存起来，直到该进程恢复执行再传递给它为止；如果一个信号被进程设置为阻塞，则该信号的传递被延迟，直到其阻塞被取消时才被传递给进程。

信号是进程间通信机制中惟一的异步通信机制，可以看作是异步通知，通知接收信号的进程有哪些事情发生了。信号机制经过 POSIX 实时扩展后，功能更加强大，除了基本通知功能外，还可以传递附加信息。

信号事件的发生有两个来源：硬件来源（比如我们按下了键盘或者其他硬件故障）；软件来源，最常用发送信号的系统函数有 `kill()`、`raise()`、`alarm()`、`setitimer()` 和 `sigqueue()` 等，软件来源还包括一些非法运算等操作。

进程可以通过 3 种方式来响应一个信号。

（1）忽略信号

即对信号不做任何处理，其中，有两个信号不能忽略：`SIGKILL` 及 `SIGSTOP`。

（2）捕捉信号

定义信号处理函数，当信号发生时，执行相应的处理函数。

（3）执行缺省操作

Linux 对每种信号都规定了默认操作，如表 6.6 所示。

表 6.6 常见信号的含义及其默认操作

信 号 名	含 义	默 认 操 作
<code>SIGHUP</code>	该信号在用户终端连接（正常或非正常）结束时发出，通常是在终端的控制进程结束时，通知同一会话内的各个进程与控制终端不再关联	终止
<code>SIGINT</code>	该信号在用户键入 <code>INTR</code> 字符（通常是 <code>Ctrl-C</code> ）时发出，终端驱动程序发送此信号并送到前台进程中的每一个进程	终止
<code>SIGQUIT</code>	该信号和 <code>SIGINT</code> 类似，但由 <code>QUIT</code> 字符（通常是 <code>Ctrl-\</code> ）来控制	终止
<code>SIGILL</code>	该信号在一个进程企图执行一条非法指令时（可执行文件本身出现错误，或者试图执行数据段、堆栈溢出时）发出	终止
<code>SIGFPE</code>	该信号在发生致命的算术运算错误时发出。这里不仅包括浮点运算错误，还包括溢出及除数为 0 等其他所有的算术的错误	终止
<code>SIGKILL</code>	该信号用来立即结束程序的运行，并且不能被阻塞、处理和忽略	终止
<code>SIGALRM</code>	该信号当一个定时器到时的时候发出	终止



SIGSTOP	该信号用于暂停一个进程，且不能被阻塞、处理或忽略	暂停进程
SIGTSTP	该信号用于交互停止进程，用户可键入 SUSP 字符时（通常是 Ctrl+Z）发出这个信号	停止进程
SIGCHLD	子进程改变状态时，父进程会收到这个信号	忽略

一个完整的信号生命周期可以分为 3 个重要阶段，这 3 个阶段由 4 个重要事件来刻画的：信号产生、信号在进程中注册、信号在进程中注销、执行信号处理函数。这里信号的产生、注册、注销等是指信号的内部实现机制，而不是信号的函数实现。因此，信号注册与否与本节后面讲到的发送信号函数（如 kill（）等）以及信号安装函数（如 signal（）等）无关，只与信号值有关。

相邻两个事件的时间间隔构成信号生命周期的一个阶段。要注意这里的信号处理有多种方式，一般是由内核完成的，当然也可以由用户进程来完成，故在此没有明确画出。

信号的处理包括信号的发送、捕获以及信号的处理，它们各自相对应的常见函数有

发送信号的函数：kill()、raise()。

捕获信号的函数：alarm()、pause()。

处理信号的函数：signal()、sigaction()。

(1) 信号发送：kill()和 raise()

kill()函数同读者熟知的 kill 系统命令一样，可以发送信号给进程或进程组（实际上，kill 系统命令只是 kill()函数的一个用户接口）。这里需要注意的是，它不仅中止进程（实际上发出 SIGKILL 信号），也可以向进程发送其他信号。

与 kill()函数所不同的是，raise()函数允许进程向自身发送信号。

表 6.7 列出了 kill()函数的语法要点。

表 6.7 kill()函数语法要点

所需头文件	#include <signal.h> #include <sys/types.h>	
函数原型	int kill(pid_t pid, int sig)	
函数传入值	pid	正数：要发送信号的进程号
		0：信号被发送到所有和当前进程在同一个进程组的进程
		-1：信号发给所有的进程表中的进程（除了进程号最大的进程外）
		<-1：信号发送给进程组号为-pid 的每一个进程
	sig：信号	
函数返回值	成功：0	
	出错：-1	



表 6.8 列出了 `raise()` 函数的语法要点。

表 6.8 `raise()` 函数语法要点

所需头文件	<code>#include <signal.h></code> <code>#include <sys/types.h></code>
函数原型	<code>int raise(int sig)</code>
函数传入值	sig: 信号
函数返回值	成功: 0
	出错: -1

(2) 信号捕捉: `alarm()`、`pause()`

`alarm()` 也称为闹钟函数，它可以在进程中设置一个定时器，当定时器指定的时间到时，它就向进程发送 `SIGALARM` 信号。要注意的是，一个进程只能有一个闹钟时间，如果在调用 `alarm()` 之前已设置过闹钟时间，则任何以前的闹钟时间都被新值所代替。

`pause()` 函数是用于将调用进程挂起直至捕捉到信号为止。这个函数很常用，通常可以用于判断信号是否已到。

表 6.9 列出了 `alarm()` 函数的语法要点。

表 6.9 `alarm()` 函数语法要点

所需头文件	<code>#include <unistd.h></code>
函数原型	<code>unsigned int alarm(unsigned int seconds)</code>
函数传入值	seconds: 指定秒数，系统经过 <code>seconds</code> 秒之后向该进程发送 <code>SIGALRM</code> 信号
函数返回值	成功: 如果调用此 <code>alarm()</code> 前，进程中已经设置了闹钟时间，则返回上一个闹钟时间的剩余时间，否则返回 0
	出错: -1

表 6.10 列出了 `pause()` 函数的语法要点。

表 6.10 `pause()` 函数语法要点

所需头文件	<code>#include <unistd.h></code>
函数原型	<code>int pause(void)</code>
函数返回值	-1，并且把 <code>error</code> 值设为 <code>EINTR</code>

(3) 信号的处理

信号处理的主要方法有两种，一种是使用简单的 `signal()` 函数，另一种是使用信号集函数组。下面分别介绍这两种处理方式。

● 信号处理函数



使用 `signal()` 函数处理时，只需要指出要处理的信号和处理函数即可。它主要是用于前 32 种非实时信号的处理，不支持信号传递信息，但是由于使用简单、易于理解，因此也受到很多程序员的欢迎。Linux 还支持一个更健壮更新的信号处理函数 `sigaction()`，推荐使用该函数。

`signal()` 函数的语法要点如表 6.11 所示。

表 6.11 `signal()` 函数语法要点

所需头文件	#include <signal.h>	
函数原型	typedef void (*sighandler_t)(int); sighandler_t signal(int signum, sighandler_t handler);	
函数传入值	signum: handl er:	SIG_IGN: 忽略该信号
		SIG_DFL: 采用系统默认方式处理信号
		自定义的信号处理函数指针
函数返回值	成功: 以前的信号处理配置	
	出错: -1	

这里需要对这个函数原型进行说明。这个函数原型有点复杂。首先该函数原型整体指向一个无返回值并且带一个整型参数的函数的指针，也就是信号的原始配置函数。接着该原型又带有两个参数，其中的第二个参数可以是用户自定义的信号处理函数的函数指针。

表 6.12 列举了 `sigaction()` 的语法要点。

表 6.12 `sigaction()` 函数语法要点

所需头文件	#include <signal.h>
函数原型	int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact)
函数传入值	signum: 信号代码，可以为除 SIGKILL 及 SIGSTOP 外的任何一个特定有效的信号
	act: 指向结构 sigaction 的一个实例的指针，指定对特定信号的处理
	oldact: 保存原来对相应信号的处理
函数返回值	成功: 0
	出错: -1

这里要说明的是 `sigaction()` 函数中第 2 个和第 3 个参数用到的 `sigaction` 结构。这是一个看似非常复杂的结构，希望读者能够慢慢阅读此段内容。

首先给出了 `sigaction` 的定义，如下所示：

```
struct sigaction
{
    void (*sa_handler)(int signo);
```



```
sigset_t sa_mask;
int sa_flags;
void (*sa_restore)(void);
}
```

`sa_handler` 是一个函数指针，指定信号处理函数，这里除可以是用户自定义的处理函数外，还可以为 `SIG_DFL`（采用缺省的处理方式）或 `SIG_IGN`（忽略信号）。它的处理函数只有一个参数，即信号值。

`sa_mask` 是一个信号集，它可以指定在信号处理程序执行过程中哪些信号应当被屏蔽，在调用信号捕获函数之前，该信号集要加入到信号的信号屏蔽字中。

`sa_flags` 中包含了许多标志位，是对信号进行处理的各个选择项。它的常见可选值如下表 6.13 所示。

表 6.13 常见信号的含义及其默认操作

选 项	含 义
<code>SA_NODEFER / SA_NOMASK</code>	当捕捉到此信号时，在执行其信号捕捉函数时，系统不会自动屏蔽此信号
<code>SA_NOCLDSTOP</code>	进程忽略子进程产生的任何 <code>SIGSTOP</code> 、 <code>SIGTSTP</code> 、 <code>SIGTTIN</code> 和 <code>SIGTTOU</code> 信号
<code>SA_RESTART</code>	令重启的系统调用起作用
<code>SA_ONESHOT / SA_RESETHAND</code>	自定义信号只执行一次，在执行完毕后恢复信号的系统默认动作

● 信号集函数组

使用信号集函数组处理信号时涉及一系列的函数，这些函数按照调用的先后次序可分为以下几大功能模块：创建信号集合、注册信号处理函数以及检测信号。

其中，创建信号集合主要用于处理用户感兴趣的一些信号，其函数包括以下几个。

- `sigemptyset()`：将信号集合初始化为空。
- `sigfillset()`：将信号集合初始化为包含所有已定义的信号的集合。
- `sigaddset()`：将指定信号加入到信号集合中去。
- `sigdelset()`：将指定信号从信号集合中删去。
- `sigismember()`：查询指定信号是否在信号集合之中。

注册信号处理函数主要用于决定进程如何处理信号。这里要注意的是，信号集里的信号并不是真正可以处理的信号，只有当信号的状态处于非阻塞状态时才会真正起作用。因此，首先使用 `sigprocmask()` 函数检测并更改信号屏蔽字（信号屏蔽字是用来指定当前被阻塞的一组信号，它们不会被进程接收），然后使用 `sigaction()` 函数来定义进程接收到特定信号之后的行为。检测信号是信号处理的后续步骤，因为被阻塞的信号不会传递给进程，所以这些信号就处于“未处理”状态（也就是进程不清楚它的存在）。`sigpending()` 函数允许进程检测“未处理”信号，并进一步决定对它们作何处理。

首先介绍创建信号集合的函数格式，表 6.14 列举了这一组函数的语法要点。



表 6.14 创建信号集合函数语法要点

所需头文件	#include <signal.h>
函数原型	int sigemptyset(sigset_t *set)
	int sigfillset(sigset_t *set)
	int sigaddset(sigset_t *set, int signum)
	int sigdelset(sigset_t *set, int signum)
	int sigismember(sigset_t *set, int signum)
函数传入值	set: 信号集
	signum: 指定信号代码
函数返回值	成功: 0 (sigismember 成功返回 1, 失败返回 0)
	出错: -1

表 6.15 列举了 sigprocmask 的语法要点。

表 6.15 sigprocmask 函数语法要点

所需头文件	#include <signal.h>	
函数原型	int sigprocmask(int how, const sigset_t *set, sigset_t *oset)	
函数传入值	how: 决定函数的 操作方式	SIG_BLOCK: 增加一个信号集合到当前进程的阻塞集合之中
		SIG_UNBLOCK: 从当前的阻塞集合之中删除一个信号集合
		SIG_SETMASK: 将当前的信号集合设置为信号阻塞集合
	set: 指定信号集	
	oset: 信号屏蔽字	
函数返回值	成功: 0	
	出错: -1	

此处, 若 set 是一个非空指针, 则参数 how 表示函数的操作方式; 若 how 为空, 则表示忽略此操作。

最后, 表 6.16 列举了 sigpending 函数的语法要点。

表 4.16 sigpending 函数语法要点

所需头文件	#include <signal.h>
函数原型	int sigpending(sigset_t *set)
函数传入值	set: 要检测的信号集
函数返回值	成功: 0



出错：-1

总之，在处理信号时，一般遵循如图 6.6 所示的操作流程。

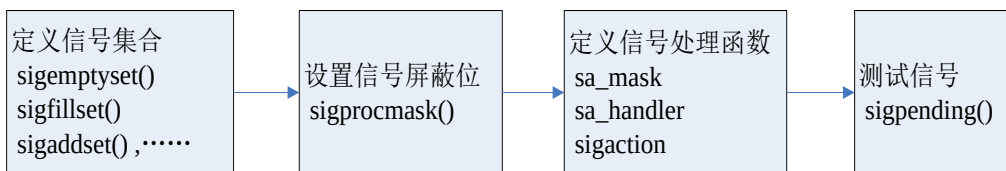


图 6.6 一般的信号操作处理流程

11.1.6 信号量

在多任务操作系统环境下，多个进程会同时运行，并且一些进程之间可能存在一定的关联。多个进程可能为了完成同一个任务会相互协作，这样形成进程之间的同步关系。而且在不同进程之间，为了争夺有限的系统资源（硬件或软件资源）会进入竞争状态，这就是进程之间的互斥关系。

进程之间的互斥与同步关系存在的根源在于临界资源。临界资源是在同一个时刻只允许有限个（通常只有一个）进程可以访问（读）或修改（写）的资源，通常包括硬件资源（处理器、内存、存储器以及其他外围设备等）和软件资源（共享代码段，共享结构和变量等）。访问临界资源的代码叫做临界区，临界区本身也会成为临界资源。

信号量是用来解决进程之间的同步与互斥问题的一种进程之间通信机制，包括一个称为信号量的变量和在该信号量下等待资源的进程等待队列，以及对信号量进行的两个原子操作（PV 操作）。其中信号量对应于某一种资源，取一个非负的整型值。信号量值指的是当前可用的该资源的数量，若它等于 0 则意味着目前没有可用的资源。

PV 原子操作的具体定义为：

P 操作：如果有可用的资源（信号量值>0），则占用一个资源（给信号量值减去一，进入临界区代码）；如果没有可用的资源（信号量值等于 0），则被阻塞到，直到系统将资源分配给该进程（进入等待队列，一直等到资源轮到该进程）。

V 操作：如果在该信号量的等待队列中有进程在等待资源，则唤醒一个阻塞进程。如果没有进程等待它，则释放一个资源（给信号量值加一）。

常见的使用信号量访问临界区的伪代码所下所示：

```

{
    /* 设 R 为某种资源，S 为资源 R 的信号量*/
    INIT_VAL(S);          /* 对信号量 S 进行初始化 */
    非临界区;
    P(S);                  /* 进行 P 操作 */
    临界区（使用资源 R）;  /* 只有有限个（通常只有一个）进程被允许进入该区*/
    V(S);                  /* 进行 V 操作 */
    非临界区;
}
    
```



```
}
```

最简单的信号量是只能取 0 和 1 两种值，这种信号量被叫做二维信号量。在本节中，主要讨论二维信号量。二维信号量的应用比较容易地扩展到使用多维信号量的情况。

1. 函数说明

在 Linux 系统中，使用信号量通常分为以下几个步骤。

第一步：创建信号量或获得在系统已存在的信号量，此时需要调用 `semget()` 函数。不同进程通过使用同一个信号量键值来获得同一个信号量。

第二步：初始化信号量，此时使用 `semctl()` 函数的 `SETVAL` 操作。当使用二维信号量时，通常将信号量初始化为 1。

第三步：进行信号量的 PV 操作，此时调用 `semop()` 函数。这一步是实现进程之间的同步和互斥的核心工作部分。

第四步：如果不需要信号量，则从系统中删除它，此时使用 `semctl()` 函数的 `IPC_RMID` 操作。此时需要注意，在程序中不应该出现对已经被删除的信号量的操作。

2. 函数格式

表 6.17 列举了 `semget()` 函数的语法要点。

表 6.17 `semget()` 函数语法要点

所需头文件	<pre>#include <sys/types.h> #include <sys/ipc.h> #include <sys/sem.h></pre>
函数原型	<pre>int semget(key_t key, int nsems, int semflg)</pre>
函数传入值	key: 信号量的键值，多个进程可以通过它访问同一个信号量，其中有个特殊值 <code>IPC_PRIVATE</code>。它用于创建当前进程的私有信号量。
	nsems: 需要创建的信号量数目，通常取值为 1。
	semflg: 同 <code>open()</code> 函数的权限位，也可以用八进制表示法，其中使用 <code>IPC_CREAT</code> 标志创建新的信号量，即使该信号量已经存在（具有同一个键值的信号量已在系统中存在），也不会出错。如果同时使用 <code>IPC_EXCL</code> 标志可以创建一个新的唯一的信号量，此时如果该信号量已经存在，该函数会返回出错。
函数返回值	成功：信号量标识符，在信号量的其他函数中都会使用该值。



出错：-1

表 6.18 列举了 semctl()函数的语法要点。

表 6.18 semctl()函数语法要点

所需头文件	<pre>#include <sys/types.h> #include <sys/ipc.h> #include <sys/sem.h></pre>
函数原型	<pre>int semctl(int semid, int semnum, int cmd, union semun arg)</pre>
函数传入值	semid: semget()函数返回的信号量标识符。
	semnum: 信号量编号, 当使用信号量集时才会被用到。通常取值为 0, 就是使用单个信号量 (也是第一个信号量)。
	cmd: 指定对信号量的各种操作, 当使用单个信号量 (而不是信号量集) 时, 常用的操作有以下几种: IPC_STAT: 获得该信号量 (或者信号量集合) 的 semid_ds 结构, 并存放在由第四个参数 arg 结构变量的 buf 域指向的 semid_ds 结构中。semid_ds 是在系统中描述信号量的数据结构。 IPC_SETVAL: 将信号量值设置为 arg 的 val 值。 IPC_GETVAL: 返回信号量的当前值。 IPC_RMID: 从系统中, 删除信号量 (或者信号量集)
	arg: 是 union semnn 结构, 可能在某些系统中不给出该结构的定义, 此时必须由程序员自己定义。 <pre>union semun { int val; struct semid_ds *buf; unsigned short *array; }</pre>
函数返回值	成功: 根据 cmd 值的不同而返回不同的值。 IPC_STAT、IPC_SETVAL、IPC_RMID: 返回 0 IPC_GETVAL: 返回信号量的当前值
	出错: -1

表 6.19 列举了 semop()函数的语法要点。



表 6.19 semop()函数语法要点

所需头文件	<pre>#include <sys/types.h> #include <sys/ipc.h> #include <sys/sem.h></pre>
函数原型	<pre>int semop(int semid, struct sembuf *sops, size_t nsops)</pre>
函数传入值	semid: semget()函数返回的信号量标识符。 sops: 指向信号量操作数组，一个数组包括以下成员： <pre>struct sembuf { short sem_num; /* 信号量编号，使用单个信号量时，通常取值为 0 */ short sem_op; /* 信号量操作：取值为-1 则表示 P 操作，取值为+1 则表示 V 操作*/ short sem_flg; /* 通常设置为 SEM_UNDO。这样在进程没释放信号量而退出时，系统自动释放该进程中未释放的信号量 */ }</pre> nsops: 操作数组 sops 中的操作个数（元素数目），通常取值为 1（一个操作）
函数返回值	成功：信号量标识符，在信号量的其他函数中都会使用该值。 出错：-1

因为信号量相关的函数调用接口比较复杂，我们可以将它们封装成二维单个信号量的几个基本函数。它们分别为信号量初始化函数（或者信号量赋值函数）init_sem()、P 操作函数 sem_p()、V 操作函数 sem_v()以及删除信号量的函数 del_sem()等，具体实现如下所示：

```
/* sem_com.c */
#include "sem_com.h"
/* 信号量初始化（赋值）函数*/
int init_sem(int sem_id, int init_value)
{
    union semun sem_union;
    sem_union.val = init_value; /* init_value 为初始值 */
```



```

    if (semctl(sem_id, 0, SETVAL, sem_union) == -1)
    {
        perror("Initialize semaphore");
        return -1;
    }
    return 0;
}

/* 从系统中删除信号量的函数 */
int del_sem(int sem_id)
{
    union semun sem_union;
    if (semctl(sem_id, 0, IPC_RMID, sem_union) == -1)
    {
        perror("Delete semaphore");
        return -1;
    }
}

/* P 操作函数 */
int sem_p(int sem_id)
{
    struct sembuf sem_b;
    sem_b.sem_num = 0; /* 单个信号量的编号应该为 0 */
    sem_b.sem_op = -1; /* 表示 P 操作 */
    sem_b.sem_flg = SEM_UNDO; /* 系统自动释放将会在系统中残留的信号量 */
    if (semop(sem_id, &sem_b, 1) == -1)
    {
        perror("P operation");
        return -1;
    }
    return 0;
}

/* V 操作函数 */
int sem_v(int sem_id)
{
    struct sembuf sem_b;

```



```
sem_b.sem_num = 0; /* 单个信号量的编号应该为 0 */
sem_b.sem_op = 1; /* 表示 V 操作 */
sem_b.sem_flg = SEM_UNDO; /* 系统自动释放将会在系统中残留的信号量*/
if (semop(sem_id, &sem_b, 1) == -1)
{
    perror("V operation");
    return -1;
}
return 0;
}
```

11.1.7 共享内存

可以说，共享内存是一种最为高效的进程间通信方式。因为进程可以直接读写内存，不需要任何数据的拷贝。为了在多个进程间交换信息，内核专门留出了一块内存区。这段内存区可以由需要访问的进程将其映射到自己的私有地址空间。因此，进程就可以直接读写这一内存区而不需要进行数据的拷贝，从而大大提高了效率。当然，由于多个进程共享一段内存，因此也需要依靠某种同步机制，如互斥锁和信号量等（请参考本章的共享内存实验）。其原理示意图如图 6.7 所示。

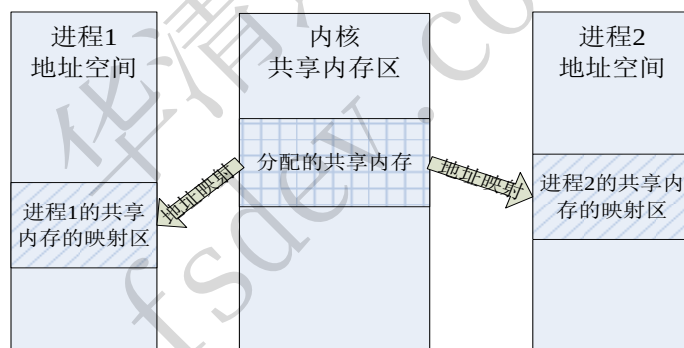


图 4.7 共享内存原理示意图

共享内存的实现分为两个步骤，第一步是创建共享内存，这里用到的函数是 `shmget()`，也就是从内存中获得一段共享内存区域。第二步映射共享内存，也就是把这段创建的共享内存映射到具体的进程空间中，这里使用的函数是 `shmat()`。到这里，就可以使用这段共享内存了，也就是可以使用不带缓冲的 I/O 读写命令对其进行操作。除此之外，当然还有撤销映射的操作，其函数为 `shmdt()`。这里就主要介绍这 3 个函数。

表 6.20 列举了 `shmget()` 函数的语法要点。

表 6.20 `shmget()` 函数语法要点

所需头文件	<pre>#include <sys/types.h> #include <sys/ipc.h></pre>
-------	--



	<code>#include <sys/shm.h></code>
函数原型	<code>int shmget(key_t key, int size, int shmflg)</code>
函数传入值	key: 共享内存的键值，多个进程可以通过它访问同一个共享内存，其中有个特殊值 <code>IPC_PRIVATE</code> 。它用于创建当前进程的私有共享内存。
	size: 共享内存区大小
	shmflg: 同 <code>open()</code> 函数的权限位，也可以用八进制表示法
函数返回值	成功：共享内存段标识符
	出错：-1

表 6.21 列举了 `shmget()` 函数的语法要点。

表 6.21 `shmget()` 函数语法要点

所需头文件	<code>#include <sys/types.h></code> <code>#include <sys/ipc.h></code> <code>#include <sys/shm.h></code>
函数原型	<code>char *shmat(int shmid, const void *shmaddr, int shmflg)</code>
函数传入值	shmid: 要映射的共享内存区标识符
	shmaddr: 将共享内存映射到指定地址（若为 0 则表示系统自动分配地址并把该段共享内存映射到调用进程的地址空间）
	shmflg SHM_RDONLY: 共享内存只读 默认 0: 共享内存可读写
函数返回值	成功：被映射的段地址
	出错：-1

表 6.22 列举了 `shmdt()` 函数的语法要点。

表 6.22 `shmdt()` 函数语法要点

所需头文件	<code>#include <sys/types.h></code> <code>#include <sys/ipc.h></code> <code>#include <sys/shm.h></code>
函数原型	<code>int shmdt(const void *shmaddr)</code>
函数传入值	shmaddr: 被映射的共享内存段地址
函数返回值	成功：0
	出错：-1



11.1.8 消息队列

顾名思义，消息队列就是一些消息的列表。用户可以在消息队列中添加消息和读取消息等。从这点上看，消息队列具有一定的 FIFO 特性，但是它可以实现消息的随机查询，比 FIFO 具有更大的优势。同时，这些消息又是存在于内核中的，由“队列 ID”来标识。

消息队列的实现包括创建或打开消息队列、添加消息、读取消息和控制消息队列这四种操作。其中创建或打开消息队列使用的函数是 `msgget()`，这里创建的消息队列的数量会受到系统消息队列数量的限制；添加消息使用的函数是 `msgsnd()` 函数，它把消息添加到已打开的消息队列末尾；读取消息使用的函数是 `msgrcv()`，它把消息从消息队列中取走，与 FIFO 不同的是，这里可以取走指定的某一条消息；最后控制消息队列使用的函数是 `msgctl()`，它可以完成多项功能。

表 6.23 列举了 `msgget()` 函数的语法要点。

表 6.23 `msgget()` 函数语法要点

所需头文件	<pre>#include <sys/types.h> #include <sys/ipc.h> #include <sys/shm.h></pre>
函数原型	<code>int msgget(key_t key, int msgflg)</code>
函数传入值	key: 消息队列的键值，多个进程可以通过它访问同一个消息队列，其中有个特殊值 <code>IPC_PRIVATE</code>。它用于创建当前进程的私有消息队列。 msgflg: 权限标志位
函数返回值	成功：消息队列 ID 出错：-1

表 6.24 列举了 `msgsnd()` 函数的语法要点。

表 6.24 `msgsnd()` 函数语法要点

所需头文件	<pre>#include <sys/types.h> #include <sys/ipc.h> #include <sys/shm.h></pre>
函数原型	<code>int msgsnd(int msqid, const void *msgp, size_t msgsz, int msgflg)</code>
函数传入值	msqid: 消息队列的队列 ID msgp: 指向消息结构的指针。该消息结构 <code>msgbuf</code> 通常为： <pre>struct msgbuf { long mtype; /* 消息类型，该结构必须从这个域开始 */ char mtext[1]; /* 消息正文 */ }</pre>



	msgsz: 消息正文的字节数（不包括消息类型指针变量）	
	msgflg:	IPC_NOWAIT 若消息无法立即发送（比如：当前消息队列已满），函数会立即返回。
		0: msgsnd 调用阻塞直到发送成功为止。
函数返回值	成功: 0	
	出错: -1	

表 6.25 列举了 msgrcv()函数的语法要点。

表 6.25 msgrcv()函数语法要点

所需头文件	#include <sys/types.h> #include <sys/ipc.h> #include <sys/shm.h>	
函数原型	int msgrcv(int msgid, void *msgp, size_t msgsz, long int msgtyp, int msgflg)	
函数传入值	msgid:	消息队列的队列 ID
	msgp:	消息缓冲区, 同于 msgsnd()函数的 msgp
	msgsz:	消息正文的字节数 (不包括消息类型指针变量)
	msgtyp:	0: 接收消息队列中第一个消息
		大于 0: 接收消息队列中第一个类型为 msgtyp 的消息
		小于 0: 接收消息队列中第一个类型值不小于 msgtyp 绝对值且类型值又最小的消息
	msgflg:	MSG_NOERROR: 若返回的消息比 msgsz 字节多, 则消息就会截短到 msgsz 字节, 且不通知消息发送进程
		IPC_NOWAIT 若在消息队列中并没有相应类型的消息可以接收, 则函数立即返回
0: msgsnd()调用阻塞直到接收一条相应类型的消息为止		
函数返回值	成功: 0	
	出错: -1	

表 6.26 列举了 msgctl()函数的语法要点。

表 6.26 msgctl()函数语法要点

所需头文件	#include <sys/types.h> #include <sys/ipc.h>
-------	--



	#include <sys/shm.h>	
函数原型	int msgctl (int msgqid, int cmd, struct msqid_ds *buf)	
函数传入值	msqid: 消息队列的队列 ID	
	cmd: 命令 参数	IPC_STAT: 读取消息队列的数据结构 msqid_ds, 并将其存储在 buf 指定的地址中
		IPC_SET: 设置消息队列的数据结构 msqid_ds 中的 ipc_perm 域 (IPC 操作权限描述结构) 值。这个值取自 buf 参数
		IPC_RMID: 从系统内核中删除消息队列
	buf: 描述消息队列的 msqid_ds 结构类型变量	
函数返回值	成功: 0	
	出错: -1	

11.2 Linux 系统无名管道通信实验

11.2.1 实验目的

熟悉 UNIX/LINUX 支持的无名管道通信方式。

11.2.2 实验平台

华清远见开发环境

11.2.3 实验步骤

将【华清远见-嵌入式 ARM 实验箱资料-I-FS6818: 程序源码\Linux 应用实验源码】目录拷贝到虚拟机共享目录【D:\share\】下。(注意: 此步骤已经在“Linux Shell 编程”章节中完成, 如果没有完成请参照此章节。如已经完成可省略此步骤)

将无名管道实验部分代码拷贝到虚拟机 Linux 下。

1、建立相关目录;

```
$ mkdir ~/workdir/linux/application/unnamed_pipe -p
$ cd ~/workdir/linux/application/unnamed_pipe
```

```
linux@ubuntu64-vm:~$ mkdir ~/workdir/linux/application/unnamed_pipe -p
linux@ubuntu64-vm:~$ cd ~/workdir/linux/application/unnamed_pipe
linux@ubuntu64-vm:~/workdir/linux/application/unnamed_pipe$
```

2、将代码从共享目录拷入虚拟机 Linux 操作系统下; (可使用【ctrl+空格】切换输入法)

```
$ cp /mnt/hgfs/share/Linux 应用实验源码/11.2.\ Linux 系统无名管道通信实验/实验代码/pipe.c .
$ ls
```



```
linux@ubuntu64-vm:~/workdir/linux/application/unnamed_pipe$ cp /mnt/hgfs/share/Linux应用实验源码/112.\ Linux系统无名管道通信实验/实验代码/pipe.c
linux@ubuntu64-vm:~/workdir/linux/application/unnamed_pipe$ ls
pipe.c
```

3、执行代码:

```
$ gcc pipe.c -o pipe
```

```
$ ./pipe
```

11.2.4 实验现象

运行结果, 延迟 5 秒后显示:

child 1 process is sending message!

再显示:

child 2 process is sending message!

```
linux@ubuntu64-vm:~/workdir/linux/application/unnamed_pipe$ gcc pipe.c -o pipe
linux@ubuntu64-vm:~/workdir/linux/application/unnamed_pipe$ ./pipe
child 2 process is sending message!
child 1 process is sending message!
linux@ubuntu64-vm:~/workdir/linux/application/unnamed_pipe$
```

11.2.5 实验代码

pipe.c

```
1#include <unistd.h>
2#include <stdio.h>
3#include <stdlib.h>
4
5int pid1, pid2;
6
7main( )
8{
9    int fd[2];
10   char outpipe[100], inpipe[100];
11   pipe(fd); /*创建一个管道*/
12   while ((pid1 = fork( )) == -1);
13   if (pid1 == 0)
14   {
15       lockf(fd[1], 1, 0);
16       sprintf(outpipe, "child 1 process is sending message!");
17       /*把串放入数组 outpipe 中*/
18       write(fd[1], outpipe, 50); /*向管道写长为 50 字节的串*/
19       sleep(5); /*自我阻塞 5 秒*/
20       lockf(fd[1], 0, 0);
21       exit(0);
```



```

22     }
23     else
24     {
25         while((pid2 = fork( )) == -1);
26         if (pid2 == 0)
27         {
28             lockf(fd[1], 1, 0);          /*互斥*/
29             sprintf(outpipe, "child 2 process is sending message!");
30             write(fd[1], outpipe, 50);
31             sleep(5);
32             lockf(fd[1], 0, 0);
33             exit(0);
34         }
35         else
36         {
37             wait(NULL);                  /*同步*/
38             read(fd[0], inpipe, 50);     /*从管道中读长为 50 字节的串*/
39             printf("%s\n", inpipe);
40             wait(NULL);
41             read(fd[0], inpipe, 50);
42             printf("%s\n", inpipe);
43             exit(0);
44         }
45     }
46 }

```

11.3 Linux 系统有名管道通信实验

11.3.1 实验目的

熟悉 UNIX/LINUX 支持的有名管道通信方式。

11.3.2 实验平台

华清远见开发环境

11.3.3 实验内容

创建有名管道；

本进程执行循环等待数据被写入到管道中并读有名管道；

打开有名管道并写数据到名管道；



11.3.4 实验步骤

将【华清远见-嵌入式 ARM 实验箱资料- I -FS6818: 程序源码\Linux 应用实验源码】目录拷贝到虚拟机共享目录【D:\share\】下。

（注意：此步骤已经在“Linux Shell 编程”章节中完成，如果没有完成请参照此章节。如已经完成可省略此步骤）

将有名管道实验部分代码拷贝到虚拟机 Linux 下。

1、建立相关目录：

```
$ mkdir ~/workdir/linux/application/named_pipe -p
$ cd ~/workdir/linux/application/named_pipe
```

```
linux@ubuntu64-vm:~$
linux@ubuntu64-vm:~$ mkdir ~/workdir/linux/application/named_pipe -p
linux@ubuntu64-vm:~$ cd ~/workdir/linux/application/named_pipe
linux@ubuntu64-vm:~/workdir/linux/application/named_pipe$
```

2、将代码从共享目录拷入虚拟机 Linux 操作系统下；（可使用【ctrl+空格】切换输入法）

```
$ cp /mnt/hgfs/share/Linux 应用实验源码/13.\ Linux 系统有名管道通信实验/实验代码/*.c .
$ ls
```

```
linux@ubuntu64-vm:~/workdir/linux/application/named_pipe$ cp /mnt/hgfs/share/Linux应用实验源码/13.\ Linux系统有名管道通信实验/实验代码/*.c .
linux@ubuntu64-vm:~/workdir/linux/application/named_pipe$ ls
create_fifo.c read_fifo.c write_fifo.c
```

3、编译代码：

```
$ gcc create_fifo.c -o create_fifo
$ gcc read_fifo.c -o read_fifo
$ gcc write_fifo.c -o write_fifo
```

运行程序

```
$ ./create_fifo test //在当前目录下，创建了管道文件“test”。
```

```
$ ./read_fifo test //执行读管道文件的程序
```

```
linux@ubuntu64-vm:~/workdir/linux/application/named_pipe$ gcc create_fifo.c -o create_fifo
linux@ubuntu64-vm:~/workdir/linux/application/named_pipe$ gcc read_fifo.c -o read_fifo
read_fifo.c: 在函数‘main’中:
read_fifo.c:56: 警告：格式‘%d’需要类型‘int’，但实参 3 的类型为‘ssize_t’
linux@ubuntu64-vm:~/workdir/linux/application/named_pipe$ gcc write_fifo.c -o write_fifo
linux@ubuntu64-vm:~/workdir/linux/application/named_pipe$ ./create_fifo test
linux@ubuntu64-vm:~/workdir/linux/application/named_pipe$ ./read_fifo test
```

打开另一个终端“Ctrl+Alt+t”，并进入该目：

```
cd ~/workdir/linux/application/named_pipe

$ ./write_fifo test //执行写管道文件的程序
$ hello //输入 hello
```



```
linux@ubuntu64-vm:~/workdir/linux/application/named_pipe$
linux@ubuntu64-vm:~/workdir/linux/application/named_pipe$ ./write_fifo test
open fifo ./write_fifo for writing succeeded.
hello
```

11.3.5 实验现象

在读取管道的终端会显示:

```
open fifo ./read_fifo for reading succeeded.
read 6 bytes from fifo: hello
```

```
linux@ubuntu64-vm:~/workdir/linux/application/10-comm/fifo$
linux@ubuntu64-vm:~/workdir/linux/application/10-comm/fifo$ ./read_fifo test
open fifo ./read_fifo for reading succeeded.
read 6 bytes from fifo: hello
```

11.3.6 实验代码

create_fifo.c

```
1#include <stdio.h>
2#include <stdlib.h>
3#include <sys/stat.h>
4#include <string.h>
5#include <errno.h>
6
7int main(int argc, char **argv)
8{
9    if (argc < 2)
10    {
11        fprintf(stdout, "Usage: %s <filename>\n", argv[0]);
12        exit(1);
13    }
14
15    //int mkfifo(const char *path, mode_t mode);
16    if (mkfifo(argv[1], 0644) < 0)
17    {
18        fprintf(stderr, "mkfifo() failed: %s\n", strerror(errno));
19        exit(1);
20    }
21
22    return 0;
23}
```

read_fifo.c



```

1#include <stdio.h>
2#include <stdlib.h>
3#include <unistd.h>
4#include <sys/stat.h>
5#include <fcntl.h>
6#include <string.h>
7#include <errno.h>
8
9#define BUFFER_SIZE 1024
10
11int main(int argc, char **argv)
12{
13    int fd;
14
15    if (argc < 2)
16    {
17        fprintf(stdout, "Usage: %s <filename>\n", argv[0]);
18        exit(1);
19    }
20
21    //int open(const char *path, int oflag, ...);
22    if ((fd = open(argv[1], O_RDONLY)) < 0)
23    {
24        fprintf(stderr, "open fifo %s for reading failed: %s\n", argv[1], strerror(errno));
25        exit(1);
26    }
27
28    fprintf(stdout, "open fifo %s for reading succeeded.\n", argv[0]);
29    char buffer[BUFFER_SIZE];
30    ssize_t n;
31
32    while (1)
33    {
34again:
35        //ssize_t read(int fd, void *buf, size_t count);
36        if ((n = read(fd, buffer, BUFFER_SIZE)) < 0)
37        {
38            if (errno == EINTR)
39            {
40                goto again;
41            }
42            else
43            {

```



```

44         fprintf(stderr, "read failed on %s: %s\n", argv[1], strerror(errno));
45         exit(1);
46     }
47 }
48 else if (n == 0)
49 {
50     fprintf(stderr, "peer closed fifo.\n");
51     break;
52 }
53 else
54 {
55     buffer[n] = '\0';
56     fprintf(stdout, "read %d bytes from fifo: %s\n", n, buffer);
57 }
58 }
59 return 0;
}

```

write_fifo.c

```

1#include <stdio.h>
2#include <stdlib.h>
3#include <unistd.h>
4#include <sys/stat.h>
5#include <fcntl.h>
6#include <signal.h>
7#include <string.h>
8#include <errno.h>
9
10#define BUFFER_SIZE 1024
11
12void signal_handler(int s);
13
14int main(int argc, char **argv)
15{
16    int fd;
17
18    if (argc < 2)
19    {
20        fprintf(stdout, "Usage: %s <filename>\n", argv[0]);
21        exit(1);
22    }
23
24    signal(SIGPIPE, signal_handler);
25

```



```

26 //int open(const char *path, int oflag, ...);
27 if ((fd = open(argv[1], O_WRONLY)) < 0)
28 {
29     fprintf(stderr, "open fifo %s for writting failed: %s\n", argv[1], strerror(errno));
30     exit(1);
31 }
32
33 fprintf(stdout, "open fifo %s for writting succeeded.\n", argv[0]);
34
35 char buffer[BUFFER_SIZE];
36 ssize_t n;
37
38 //char *fgets(char *s, int size, FILE * stream);
39 while (fgets(buffer, BUFFER_SIZE, stdin))
40 {
41 again:
42     //ssize_t write(int fd, const void *buf, size_t count);
43     if ((n = write(fd, buffer, strlen(buffer))) < 0)
44     {
45         if (errno == EINTR)
46         {
47             goto again;
48         }
49         else
50         {
51             fprintf(stderr, "write() failed on fifo: %s\n", strerror(errno));
52             break;
53         }
54     } // end if
55 } // end while
56
57 return 0;
58 }
59
60 void signal_handler(int signo)
61 {
62     fprintf(stdout, "Caught signal %d\n", signo);
63 }

```



11.4 Linux 系统信号机制实验

11.4.1 实验目的

熟悉 LINUX 系统中进程之间信号通信的基本原理。

11.4.2 实验平台

华清远见开发环境

11.4.3 实验内容

编写一段程序，使用系统调用 `fork()` 创建两个子进程，再用系统调用 `signal()` 让父进程捕捉键盘上来的中断信号（即按 `ctrl+c` 键），当捕捉到中断信号后，父进程用系统调用 `kill()` 向两个子进程发出信号，子进程捕捉到父进程发来的信号后，分别输出下列信息后终止：

Child process 1 is killed by parent!

Child process 2 is killed by parent!

父进程等待两个子进程终止后，输出以下信息后终止：

Parent process exit!

11.4.4 实验步骤

将【华清远见-嵌入式 ARM 实验箱资料- I -FS6818：程序源码\Linux 应用实验源码】目录拷贝到虚拟机共享目录【D:\share\】下。**（注意：此步骤已经在“Linux Shell 编程”章节中完成，如果没有完成请**

参照此章节。如已经完成可省略此步骤）

将信号实验部分代码拷贝到虚拟机 Linux 下。

1、建立相关目录；

```
$ mkdir ~/workdir/linux/application/signal -p
$ cd ~/workdir/linux/application/signal
```

```
linux@ubuntu64-vm:~$
linux@ubuntu64-vm:~$ mkdir ~/workdir/linux/application/signal -p
linux@ubuntu64-vm:~$ cd ~/workdir/linux/application/signal
linux@ubuntu64-vm:~/workdir/linux/application/signal$
```

2、将代码从共享目录拷入虚拟机 Linux 操作系统下；（可使用【`ctrl+空格`】切换输入法）

```
$ cp /mnt/hgfs/share/Linux 应用实验源码/14.\ Linux 系统信号机制实验/实验代码/signal.c .
$ ls
```

```
linux@ubuntu64-vm:~/workdir/linux/application/signal$ cp /mnt/hgfs/share/Linux应用实验源码/14.\ L
linux系统信号机制实验/实验代码/signal.c .
linux@ubuntu64-vm:~/workdir/linux/application/signal$ ls
signal.c
```

3、执行代码：

```
$ gcc signal.c -o signal
```



```
$ ./signal
```

11.4.5 实现现象

按“Ctrl+c”键，可以看到终端输出结果：

```
^Cchild process 1 is killed by parent!
```

```
child process 2 is killed by parent!
```

```
parent process exit!
```

```
linux@ubuntu64-vm:~/workdir/linux/application/signal$ gcc signal.c -o signal
linux@ubuntu64-vm:~/workdir/linux/application/signal$ ./signal
^Cchild process 2 is killed by parent!
child process 1 is killed by parent!
parent process exit!
linux@ubuntu64-vm:~/workdir/linux/application/signal$
```

11.4.6 实验代码

signal.c

```
1#include<stdio.h>
2#include<signal.h>
3#include<unistd.h>
4#include<stdlib.h>
5
6int wait_mark;
7void waiting(), stop();
8
9int main(int argc, char *argv[])
10{
11    int p1, p2;
12    signal(SIGINT, stop);
13
14    while((p1 = fork()) == -1);
15
16    if (p1 > 0) /*在父进程中*/
17    {
18        while((p2 = fork()) == -1);
19
20        if (p2 > 0) /*在父进程中*/
21        {
22            wait_mark = 1;
23            waiting();
24            kill(p1, 10);
25            kill(p2, 12);
```



```

26     wait(NULL);
27     wait(NULL);
28     printf("parent process exit!\n");
29     exit(0);
30 }
31 else                                     /*在子进程 2 中*/
32 {
33     wait_mark = 1;
34     signal(12, stop);
35     waiting();
36     lockf(1, 1, 0);
37     printf("child process 2 is killed by parent!\n");
38     lockf(1, 0, 0);
39     exit(0);
40 }
41 }
42 else                                     /*在子进程 1 中*/
43 {
44     wait_mark = 1;
45     signal(10, stop);
46     waiting();
47     lockf(1, 1, 0);
48     printf("child process 1 is killed by parent!\n");
49     lockf(1, 0, 0);
50     exit(0);
51 }
52 }
53
54 void waiting()
55 {
56     while(wait_mark != 0);
57 }
58
59 void stop()
60 {
61     wait_mark = 0;
62 }

```

11.5 Linux 系统共享内存通信实验

11.5.1 实验目的

熟悉 UNIX/LINUX 支持的共享内存通信方式。



11.5.2 实验平台

华清远见开发环境

11.5.3 实验内容

reader 和 writer 两个进程通过共享内存交换数据。writer 从标准输入读入字符串写入共享内存，reader 把共享内存里的字符串打印到标准输出。

reader 和 writer 通过信号实现同步。

reader 和 writer 通过信号通信必须获取对方的进程号，可利用共享内存保存双方的进程号。

reader 和 writer 运行的顺序不确定，可约定先运行的进程创建共享内存并初始化。

11.5.4 实验步骤

将【华清远见-嵌入式 ARM 实验箱资料- I -FS6818: 程序源码\Linux 应用实验源码】目录拷贝到虚拟机共享目录【D:\share\】下。(注意：此步骤已经在“Linux Shell 编程”章节中完成，如果没有完成请参照此章节。如已经完成可省略此步骤)

将共享内存实验部分代码拷贝到虚拟机 Linux 下。

1、建立相关目录；

```
$ mkdir ~/workdir/linux/application/shared_memory -p
$ cd ~/workdir/linux/application/shared_memory
```

```
linux@ubuntu64-vm:~$
linux@ubuntu64-vm:~$ mkdir ~/workdir/linux/application/shared_memory -p
linux@ubuntu64-vm:~$ cd ~/workdir/linux/application/shared_memory
linux@ubuntu64-vm:~/workdir/linux/application/shared_memory$
```

2、将代码从共享目录拷入虚拟机 Linux 操作系统下；（可使用【ctrl+空格】切换输入法）

```
$ cp /mnt/hgfs/share/Linux 应用实验源码/16.\ Linux 系统共享内存通信实验/实验代码/*.c .
$ ls
```

```
linux@ubuntu64-vm:~/workdir/linux/application/shared_memory$ cp /mnt/hgfs/share/Linux应用实验源码/16.\ Linux系统共享内存通信实验/实验代码/*.c .
linux@ubuntu64-vm:~/workdir/linux/application/shared_memory$ ls
reader.c  writer.c
```

3、编译代码：

```
$ gcc reader.c -o reader
$ gcc writer.c -o writer
```

4、执行代码

```
$ ./reader //执行 read 程序
```

```
linux@ubuntu64-vm:~/workdir/linux/application/shared_memory$ gcc reader.c -o reader
linux@ubuntu64-vm:~/workdir/linux/application/shared_memory$ gcc writer.c -o writer
linux@ubuntu64-vm:~/workdir/linux/application/shared_memory$ ./reader
```



打开另一个终端“Ctrl+Alt+t”，并进入该目：

```
$ cd ~/workdir/linux/application/shared_memory
$ ./writer //执行 write 程序
please input : hello //输入 “hello”
```

```
linux@ubuntu64-vm:~$ cd ~/workdir/linux/application/shared_memory
linux@ubuntu64-vm:~/workdir/linux/application/shared_memory$ ./writer
please input : hello
please input :
```

11.5.5 实验现象

在执行 read 程序的终端，会出现：

message from shm : hello

这样就成功的实现了对共享内存的读和写

```
linux@ubuntu64-vm:~/workdir/linux/application/shared_memory$ ./reader
message from shm : hello
```

11.5.6 实验代码

reader.c

```
1#include <stdio.h>
2#include <stdlib.h>
3#include <sys/types.h>
4#include <sys/shm.h>
5#include <sys/ipc.h>
6#include <unistd.h>
7#include <signal.h>
8#include <errno.h>
9
10#define N 64
11
12typedef struct
13{
14    int pid;
15    char buf[N];
16} shmbuf;
17
18void handler(int signo)
19{
20    return;
21}
22
```



```

23 int main(int argc, char *argv[])
24 {
25     int shmid;
26     key_t key;
27     pid_t pid;
28     shmbuf *shmaddr;
29
30     signal(SIGUSR1, handler);
31     if ((key = ftok(".", 'a')) < 0)
32     {
33         perror("fail to ftok");
34         exit(-1);
35     }
36     if((shmid = shmget(key, sizeof(shmbuf), IPC_CREAT | IPC_EXCL | 0666)) < 0)
37     {
38         if (errno == EEXIST)
39         {
40             shmid = shmget(key, sizeof(shmbuf), 0666);
41             shmaddr = (shmbuf *)shmat(shmid, NULL, 0);
42             pid = shmaddr->pid;
43             shmaddr->pid = getpid();
44             kill(pid, SIGUSR1);
45         }
46         else
47         {
48             perror("fail to shmget");
49             exit(-1);
50         }
51     }
52     else
53     {
54         shmaddr = (shmbuf *)shmat(shmid, NULL, 0);
55         shmaddr->pid = getpid();
56         pause();
57         pid = shmaddr->pid;
58     }
59
60     while ( 1 )
61     {
62         pause();
63         if ( strcmp(shmaddr->buf, "quit", 4) == 0)
64         {
65             break;

```



```

66     };
67     printf("message from shm : %s", shmaddr->buf);
68     usleep(100000);
69     kill(pid, SIGUSR1);
70 }
71 shmdt(shmaddr);
72
73     return 0;
74 }

```

writer.c

```

1#include <stdio.h>
2#include <stdlib.h>
3#include <sys/types.h>
4#include <sys/shm.h>
5#include <sys/ipc.h>
6#include <unistd.h>
7#include <signal.h>
8#include <errno.h>
9
10#define N 64
11
12typedef struct
13{
14     int pid;
15     char buf[N];
16} shmbuf;
17
18void handler(int signo)
19{
20     return;
21}
22
23int main(int argc, char *argv[])
24{
25     int shmid;
26     key_t key;
27     pid_t pid;
28     shmbuf *shmaddr;
29
30     signal(SIGUSR1, handler);
31     if ((key = ftok(".", 'a')) < 0)
32     {
33         perror("fail to ftok");

```



```

34     exit(-1);
35 }
36 if((shmid = shmget(key, sizeof(shmbuf), IPC_CREAT | IPC_EXCL | 0666)) < 0)
37 {
38     if (errno == EEXIST)
39     {
40         shmid = shmget(key, sizeof(shmbuf), 0666);
41         shmaddr = (shmbuf *)shmat(shmid, NULL, 0);
42         pid = shmaddr->pid;
43         shmaddr->pid = getpid();
44         kill(pid, SIGUSR1);
45     }
46     else
47     {
48         perror("fail to shmget");
49         exit(-1);
50     }
51 }
52 else
53 {
54     shmaddr = (shmbuf *)shmat(shmid, NULL, 0);
55     shmaddr->pid = getpid();
56     pause();
57     pid = shmaddr->pid;
58 }
59
60 while ( 1 )
61 {
62     printf("please input : ");
63     fgets(shmaddr->buf, N, stdin);
64     kill(pid, SIGUSR1);
65     if (strncmp(shmaddr->buf, "quit", 4) == 0)
66     {
67         break;
68     }
69     pause();
70 }
71 sleep(1);
72 shmdt(shmaddr);
73 shmctl(shmid, IPC_RMID, NULL);
74
75 return 0;
}

```



11.6 Linux 系统消息队列实验

11.6.1 实验目的

熟悉 UNIX/LINUX 支持的消息队列通信方式。

11.6.2 实验平台

华清远见开发环境

11.6.3 实验内容

本实验需要用消息队列设计一个简易的双人聊天程序（一个服务器，两个客户端）。消息队列重点在于消息类型的匹配，客户端和服务端的“通信协议”的设计。设计思想如下：

服务器端：接受客户端发来的任何信息，并根据其消息类型，转发给对应的客户端。同时，检测是否有退出标志，有则给所有的客户端发送退出标志，等待 1s 后，确定客户端都退出后，删除消息队列，释放空间，并退出。

客户端：A 和 B。A 给 B 发送信息，先发给服务器，由服务器根据自定义协议转发该消息给 B。同 B 可以也通过服务器给 A 发消息。

11.6.4 实验步骤

将【华清远见-嵌入式 ARM 实验箱资料-I-FS6818：程序源码\Linux 应用实验源码】目录拷贝到虚拟机共享目录【D:\share\】下。**（注意：此步骤已经在“Linux Shell 编程”章节中完成，如果没有完成请**

参照此章节。如已经完成可省略此步骤）

将消息队列实验部分代码拷贝到虚拟机 Linux 下。

1、建立相关目录；

```
$ mkdir ~/workdir/linux/application/msg_queue -p
$ cd ~/workdir/linux/application/msg_queue
```

```
linux@ubuntu64-vm:~$ mkdir ~/workdir/linux/application/msg_queue -p
linux@ubuntu64-vm:~$ cd ~/workdir/linux/application/msg_queue
linux@ubuntu64-vm:~/workdir/linux/application/msg_queue$
```

2、将代码从共享目录拷入虚拟机 Linux 操作系统下；（可使用【ctrl+空格】切换输入法）

```
$ cp /mnt/hgfs/share/Linux 应用实验源码/20.\ Linux 系统消息队列实验/实验代码/*.c .
$ ls
```

```
linux@ubuntu64-vm:~/workdir/linux/application/msg_queue$ cp /mnt/hgfs/share/Linux应用实验源码/20.
\ Linux系统消息队列实验/实验代码/*.c .
linux@ubuntu64-vm:~/workdir/linux/application/msg_queue$ ls
client1.c client2.c server.c
```

3、编译代码：



```
$ gcc client1.c -o client1
```

```
$ gcc client2.c -o client2
```

```
$ gcc server.c -o server
```

4、执行代码

```
$ ./server //执行 server 程序
```

```
linux@ubuntu64-vm:~/workdir/linux/application/msg_queue$ gcc client1.c -o client1
client1.c: 在函数‘main’中:
client1.c:25: 警告: 不建议使用‘gets’(声明于 /usr/include/stdio.h:638)
linux@ubuntu64-vm:~/workdir/linux/application/msg_queue$ gcc client2.c -o client2
client2.c: 在函数‘main’中:
client2.c:33: 警告: 不建议使用‘gets’(声明于 /usr/include/stdio.h:638)
linux@ubuntu64-vm:~/workdir/linux/application/msg_queue$ gcc server.c -o server
linux@ubuntu64-vm:~/workdir/linux/application/msg_queue$ ./server
```

打开另一个终端“Ctrl+Alt+t”，并进入该目运行客户端 A:

```
$ cd ~/workdir/linux/application/msg_queue
```

```
$ ./client1
```

```
linux@ubuntu64-vm:~$ cd ~/workdir/linux/application/msg_queue
linux@ubuntu64-vm:~/workdir/linux/application/msg_queue$ ./client1
input the msg to client2: 
```

打开另一个终端“Ctrl+Alt+t”，并进入该目运行客户端 B:

```
$ cd ~/workdir/linux/application/msg_queue
```

```
$ ./client2
```

```
linux@ubuntu64-vm:~$ 
linux@ubuntu64-vm:~$ cd ~/workdir/linux/application/msg_queue
linux@ubuntu64-vm:~/workdir/linux/application/msg_queue$ ./client2
```

11.6.5 实验现象

在客户端 A 下输入 hello client1，如下图所示。

```
linux@ubuntu64-vm:~/workdir/linux/application/msg_queue$ ./client1
input the msg to client2: hello client1
```

客户端 B 如下图所示

```
linux@ubuntu64-vm:~/workdir/linux/application/msg_queue$ ./client2
Receive from client1, message: hello client1
input the msg to client1: ^A
```

服务器端如下图所示



```
linux@ubuntu64-vm:~/workdir/linux/application/msg_queue$ gcc server.c
linux@ubuntu64-vm:~/workdir/linux/application/msg_queue$ ./server
Receive client1 message: hello client1
```

11.6.6 实验代码

server.c

```
1#include <stdio.h>
2#include <stdlib.h>
3#include <sys/types.h>
4#include <sys/ipc.h>
5#include <sys/msg.h>
6#include <string.h>
7
8#define KEY_MSG 0x101 //使用共有的 IPC key
9#define MSGSIZE 64
10
11typedef struct //定义消息结构体：消息类型和消息数据
12{
13    long mtype;
14    char mtext[MSGSIZE];
15} msgbuf;
16
17#define LEN sizeof(msgbuf) - sizeof(long)
18
19int main()
20{
21    int msgid;
22    msgbuf buf1, buf2;
23    msgid = msgget( KEY_MSG, IPC_CREAT | 0666 );
24    while(1)
25    {
26        msgrcv(msgid, &buf1, LEN, 1L, 0);
27        //接受客户端 1 的消息
28        printf( "Receive client1 message: %s\n", buf1.mtext ); //打印收到的消息
29        if ( buf1.mtext[0] == 'x' || buf1.mtext[0] == 'X' )
30            //若是退出标志，则给 2 个客户端都发退出信息
31            {
32                strcpy( buf1.mtext, "x" );
33                buf1.mtype = 3L;
34                msgsnd( msgid, &buf1, LEN, 0 );
35                buf1.mtype = 4L;
36                msgsnd( msgid, &buf1, LEN, 0 );
37                break;
```



```

38     }
39     buf1.mtype = 4L;
40     msgsnd( msgid, &buf1, LEN, 0 );    //将客户端 1 的消息转发给客户端 2
41     msgrcv( msgid, &buf2, LEN, 2L, 0 ); //接受客户端 2 的消息
42     printf( "Receive client2 message: %s\n", buf2.mtext ); //打印收到的消息
43     if ( buf2.mtext[0] == 'x' || buf2.mtext[0] == 'X' )
44         //若是退出标志, 则给 2 个客户端发退出信息
45     {
46         strcpy( buf2.mtext, "x" );
47         buf2.mtype = 3L;
48         msgsnd( msgid, &buf2, LEN, 0 );
49         buf2.mtype = 4L;
50         msgsnd( msgid, &buf2, LEN, 0 );
51         break;
52     }
53     buf2.mtype = 3L;
54     msgsnd( msgid, &buf2, LEN, 0 );    //将客户端 2 的消息转发给客户端 1
55 }
56 sleep(1);    //若退出, 则先等待, 以确保客户端程序退出
57 msgctl( msgid, IPC_RMID, NULL ); //删除消息队列, 释放空间
58 exit(0);
59 }

```

client1.c

```

1#include <stdio.h>
2#include <sys/types.h>
3#include <sys/ipc.h>
4#include <sys/msg.h>
5#include <string.h>
6
7#define KEY_MSG 0x101
8#define MSGSIZE 64
9
10typedef struct
11{
12     long mtype;
13     char mtext[MSGSIZE];
14} msgbuf;
15
16#define LEN sizeof(msgbuf) - sizeof(long)
17
18int main()
19{
20     int msgid;

```



```

21     msgbuf buf1, buf2;
22     msgid = msgget( KEY_MSG, 0666 );
23     while( 1 )
24     {
25         printf( "input the msg to client2:" );
26         gets( buf1.mtext );
27         buf1.mtype = 1L;
28         msgsnd( msgid, &buf1, LEN, 0 ); //客户端 1 获取消息并发送服务器
29
30         msgrcv( msgid, &buf2, LEN, 3L, 0 ); //准备从客户端 2 获取消息
31         if ( buf2.mtext[0] == 'x' || buf2.mtext[0] == 'X' )
32         {
33             printf( "client1 will quit!\n" );
34             break;
35         }
36         printf( "Receive from client2, message: %s\n", buf2.mtext );
37     }
38     return 0;
}

```

client2.c

```

1#include <stdio.h>
2#include <sys/types.h>
3#include <sys/ipc.h>
4#include <sys/msg.h>
5#include <string.h>
6
7#define KEY_MSG 0x101
8#define MSGSIZE 64
9
10typedef struct
11{
12     long mtype;
13     char mtext[MSGSIZE];
14} msgbuf;
15
16#define LEN sizeof(msgbuf) - sizeof(long)
17
18int main()
19{
20     int msgid;
21     msgbuf buf1, buf2;
22     msgid = msgget( KEY_MSG, 0666 );
23     while( 1 )

```



```

24 {
25     msgrcv( msgid, &buf2, LEN, 4L, 0 ); //等待客户端 1 发消息
26     if( buf2.mtext[0] == 'x' || buf2.mtext[0] == 'X' )
27     {
28         printf( "client2 will quit!\n" );
29         break;
30     }
31     else printf( "Receive from client1, message: %s\n", buf2.mtext );
32
33     sleep(1); //等待一秒，以确保客户端 1 已经收到了回执
34     printf( "input the msg to client1:" );
35     gets( buf1.mtext );
36     buf1.mtype = 2L;
37     msgsnd( msgid, &buf1, LEN, 0 ); //给客户端 1 发送回执消息
38 }
39 }
```

华清远见
fsdev.com.cn



第 12 章 嵌入式 Linux 多线程编程实验

12.1 实验目的

通过实验学习 Linux 下多线程编程。

12.2 实验平台

华清远见开发环境。

12.3 实验原理

12.3.1 线程基本编程

这里要讲的线程相关操作都是用户空间中的线程的操作。在 Linux 中，一般 pthread 线程库是一套通用的线程库，是由 POSIX 提出的，因此具有很好的可移植性。

创建线程实际上就是确定调用该线程函数的入口点，这里通常使用的函数是 pthread_create()。在线程创建之后，就开始运行相关的线程函数，在该函数运行完之后，该线程也就退出了，这也是线程退出的一种方法。另一种退出线程的方法是使用函数 pthread_exit()，这是线程的主动行为。这里要注意的是，在使用线程函数时，不能随意使用 exit() 退出函数进行出错处理，由于 exit() 的作用是使调用进程终止，往往一个进程包含多个线程，因此，在使用 exit() 之后，该进程中的所有线程都终止了。因此，在线程中就可以使用 pthread_exit() 来代替进程中的 exit()。

由于一个进程中的多个线程是共享数据段的，因此通常在线程退出之后，退出线程所占用的资源并不会随着线程的终止而得到释放。正如进程之间可以用 wait() 系统调用来同步终止并释放资源一样，线程之间也有类似机制，那就是 pthread_join() 函数。pthread_join() 可以用于将当前线程挂起来等待线程的结束。这个函数是一个线程阻塞的函数，调用它的函数将一直等待到被等待的线程结束为止，当函数返回时，被等待线程的资源就被收回。

前面已提到线程调用 pthread_exit() 函数主动终止自身线程。但是在很多线程应用中，经常会遇到在别的线程中要终止另一个线程的问题。此时调用 pthread_cancel() 函数实现这种功能，但在被取消的线程的内部需要调用 pthread_setcancel() 函数和 pthread_setcanceltype() 函数设置自己的取消状态，例如被取消的线程接收到另一个线程的取消请求之后，是接受还是忽略这个请求；如果是接受，则再判断立刻采取终止操作还是等待某个函数的调用等。

表 7.1 列出了 pthread_create() 函数的语法要点。

表 7.1 pthread_create() 函数语法要点

所需头文件	#include <pthread.h>
函数原型	int pthread_create ((pthread_t *thread, pthread_attr_t *attr, void *(*start_routine)(void *), void *arg))
函数传入值	thread: 线程标识符



	attr: 线程属性设置（其具体设置参见 6.4.3 小节），通常取为 NULL
	start_routine: 线程函数的起始地址，是一个以指向 void 的指针作为参数和返回值的函数指针
	arg: 传递给 start_routine 的参数
函数返回值	成功: 0
	出错: 返回错误码

表 7.2 列出了 pthread_exit() 函数的语法要点。

表 7.2 pthread_exit() 函数语法要点

所需头文件	#include <pthread.h>
函数原型	void pthread_exit(void *retval)
函数传入值	retval: 线程结束时的返回值，可由其他函数如 pthread_join() 来获取

表 7.3 列出了 pthread_join() 函数的语法要点。

表 7.3 pthread_join() 函数语法要点

所需头文件	#include <pthread.h>
函数原型	int pthread_join(pthread_t th, void **thread_return)
函数传入值	th: 等待线程的标识符
	thread_return: 用户定义的指针，用来存储被等待线程结束时的返回值（不为 NULL 时）
函数返回值	成功: 0
	出错: 返回错误码

表 7.4 列出了 pthread_cancel() 函数的语法要点。

表 7.4 pthread_cancel() 函数语法要点

所需头文件	#include <pthread.h>
函数原型	int pthread_cancel(pthread_t th)
函数传入值	th: 要取消的线程的标识符
函数返回值	成功: 0
	出错: 返回错误码



12.3.2 线程之间的同步与互斥

由于线程共享进程的资源和地址空间，因此在对这些资源进行操作时，必须考虑到线程间资源访问的同步与互斥问题。这里主要介绍 POSIX 中两种线程同步机制，分别为互斥锁和信号量。这两个同步机制可以互相通过调用对方来实现，但互斥锁更适合用于同时可用的资源是唯一的情况；信号量更适合用于同时可用的资源为多个的情况。

1. 互斥锁线程控制

互斥锁是用一种简单的加锁方法来控制对共享资源的原子操作。这个互斥锁只有两种状态，也就是上锁和解锁，可以把互斥锁看作某种意义上的全局变量。在同一时刻只能有一个线程掌握某个互斥锁，拥有上锁状态的线程能够对共享资源进行操作。若其他线程希望上锁一个已经被上锁的互斥锁，则该线程就会挂起，直到上锁的线程释放掉互斥锁为止。可以说，这把互斥锁保证让每个线程对共享资源按顺序进行原子操作。

互斥锁机制主要包括下面的基本函数。

- 互斥锁初始化：pthread_mutex_init()
- 互斥锁上锁：pthread_mutex_lock()
- 互斥锁判断上锁：pthread_mutex_trylock()
- 互斥锁解锁：pthread_mutex_unlock()
- 消除互斥锁：pthread_mutex_destroy()

其中，互斥锁可以分为快速互斥锁、递归互斥锁和检错互斥锁。这三种锁的区别主要在于其他未占有互斥锁的线程在希望得到互斥锁时是否需要阻塞等待。快速锁是指调用线程会阻塞直至拥有互斥锁的线程解锁为止。递归互斥锁能够成功地返回，并且增加调用线程在互斥上加锁的次数，而检错互斥锁则为快速互斥锁的非阻塞版本，它会立即返回并返回一个错误信息。默认属性为快速互斥锁。

表 7.5 列出了 pthread_mutex_init() 函数的语法要点。

表 7.5 pthread_mutex_init() 函数语法要点

所需头文件	#include <pthread.h>	
函数原型	int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *mutexattr)	
函数传入值	mutex: 互斥锁	
函数传入值	Mutexattr	PTHREAD_MUTEX_INITIALIZER: 创建快速互斥锁
		PTHREAD_RECURSIVE_MUTEX_INITIALIZER_NP: 创建递归互斥锁
		PTHREAD_ERRORCHECK_MUTEX_INITIALIZER_NP: 创建检错互斥锁
函数返回值	成功: 0	
	出错: 返回错误码	



表 7.6 列出了 pthread_mutex_lock() 等函数的语法要点。

表 7.6 pthread_mutex_lock() 等函数语法要点

所需头文件	#include <pthread.h>
函数原型	<pre>int pthread_mutex_lock(pthread_mutex_t *mutex,) int pthread_mutex_trylock(pthread_mutex_t *mutex,) int pthread_mutex_unlock(pthread_mutex_t *mutex,) int pthread_mutex_destroy(pthread_mutex_t *mutex,)</pre>
函数传入值	mutex: 互斥锁
函数返回值	成功: 0
	出错: -1

2. 信号量线程控制

在前面已经讲到，信号量也就是操作系统中所用到的 PV 原子操作，它广泛用于进程或线程间的同步与互斥。信号量本质上是一个非负的整数计数器，它被用来控制对公共资源的访问。这里先来简单复习一下 PV 原子操作的工作原理。

PV 原子操作是对整数计数器信号量 sem 的操作。一次 P 操作使 sem 减一，而一次 V 操作使 sem 加一。进程（或线程）根据信号量的值来判断是否对公共资源具有访问权限。当信号量 sem 的值大于等于零时，该进程（或线程）具有公共资源的访问权限；相反，当信号量 sem 的值小于零时，该进程（或线程）就将阻塞直到信号量 sem 的值大于等于 0 为止。

PV 原子操作主要用于进程或线程间的同步和互斥这两种典型情况。若用于互斥，几个进程（或线程）往往只设置一个信号量 sem，它们的操作流程如图 5.1 所示。

当信号量用于同步操作时，往往会设置多个信号量，并安排不同的初始值来实现它们之间的顺序执行，它们的操作流程如图 5.2 所示。

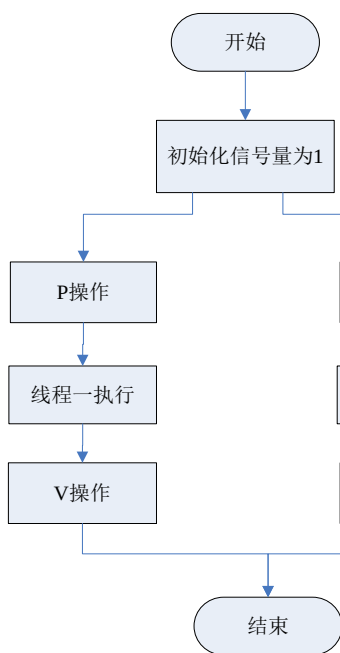


图 7.1 信号量互斥操作

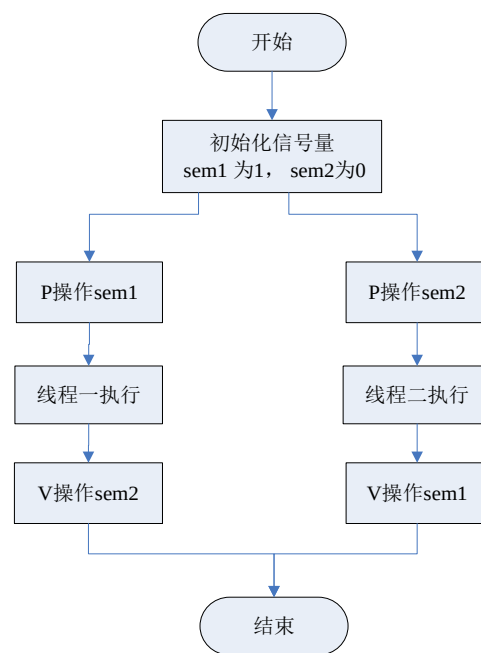


图 7.2 信号量同步操作

Linux 实现了 POSIX 的无名信号量，主要用于线程间的互斥与同步。这里主要介绍几个常见函数。

- `sem_init()`用于创建一个信号量，并初始化它的值。
- `sem_wait()`和 `sem_trywait()`都相当于 P 操作，在信号量大于零时它们都能将信号量的值减一，两者的区别在于若信号量小于零时，`sem_wait()`将会阻塞进程，而 `sem_trywait()`则会立即返回。
- `sem_post()`相当于 V 操作，它将信号量的值加一同时发出信号来唤醒等待的进程。
- `sem_getvalue()`用于得到信号量的值。
- `sem_destroy()`用于删除信号量。

表 7.7 列出了 `sem_init()`函数的语法要点。

表 7.7 `sem_init()`函数语法要点

所需头文件	#include <semaphore.h>
函数原型	int sem_init(sem_t *sem,int pshared,unsigned int value)
函数传入值	sem: 信号量指针
	pshared: 决定信号量能否在几个进程间共享。由于目前 Linux 还没有实现进程间共享信号量，所以这个值只能取 0，就表示这个信号量是当前进程的局部信号量
	value: 信号量初始化值
函数返回值	成功: 0
	出错: -1

表 7.8 列出了 `sem_wait()`等函数的语法要点。



表 7.8 sem_wait()等函数语法要点

所需头文件	#include <pthread.h>
函数原型	<pre>int sem_wait(sem_t *sem) int sem_trywait(sem_t *sem) int sem_post(sem_t *sem) int sem_getvalue(sem_t *sem) int sem_destroy(sem_t *sem)</pre>
函数传入值	sem: 信号量指针
函数返回值	成功: 0 出错: -1

12.3.3 线程属性

读者是否还记得 pthread_create()函数的第二个参数 (pthread_attr_t *attr) 表示线程的属性。在上一个实例中, 将该值设为 NULL, 也就是采用默认属性, 线程的多项属性都是可以更改的。这些属性主要包括绑定属性、分离属性、堆栈地址、堆栈大小以及优先级。其中系统默认的属性为非绑定、非分离、缺省 1M 的堆栈以及与父进程同样级别的优先级。下面首先对绑定属性和分离属性的基本概念进行讲解。

- 绑定属性

前面已经提到, Linux 中采用“一对一”的线程机制, 也就是一个用户线程对应一个内核线程。绑定属性就是指一个用户线程固定地分配给一个内核线程, 因为 CPU 时间片的调度是面向内核线程 (也就是轻量级进程) 的, 因此具有绑定属性的线程可以保证在需要的时候总有一个内核线程与之对应。而与之对应的非绑定属性就是指用户线程和内核线程的关系不是始终固定的, 而是由系统来控制分配的。

- 分离属性

分离属性是用来决定一个线程以什么样的方式来终止自己。在非分离情况下, 当一个线程结束时, 它所占用的系统资源并没有被释放, 也就是没有真正的终止。只有当 pthread_join()函数返回时, 创建的线程才能释放自己占用的系统资源。而在分离属性情况下, 一个线程结束时立即释放它所占有的系统资源。这里要注意的一点是, 如果设置一个线程的分离属性, 而这个线程运行又非常快, 那么它很可能在 pthread_create()函数返回之前就终止了, 它终止以后就可能将线程号和系统资源移交给其他的线程使用, 这时调用 pthread_create()的线程就得到了错误的线程号。

这些属性的设置都是通过特定的函数来完成的, 通常首先调用 pthread_attr_init()函数进行初始化, 之后再调用相应的属性设置函数, 最后调用 pthread_attr_destroy()函数对分配的属性结构指针进行清理和回收。设置绑定属性的函数为 pthread_attr_setscope(), 设置线程分离属性的函数为 pthread_attr_setdetachstate(), 设置线程优先级的相关函数为 pthread_attr_getschedparam() (获取线程优先级) 和 pthread_attr_setschedparam() (设置线程优先级)。在设置完这些属性后, 就可以调用 pthread_create()函数来创建线程了。

表 7.9 列出了 pthread_attr_init()函数的语法要点。



表 7.9 pthread_attr_init()函数语法要点

所需头文件	#include <pthread.h>
函数原型	int pthread_attr_init(pthread_attr_t *attr)
函数传入值	attr: 线程属性结构指针
函数返回值	成功: 0
	出错: 返回错误码

表 7.10 列出了 pthread_attr_setscope()函数的语法要点。

表 7.10 pthread_attr_setscope()函数语法要点

所需头文件	#include <pthread.h>	
函数原型	int pthread_attr_setscope(pthread_attr_t *attr, int scope)	
函数传入值	attr: 线程属性结构指针	
	scope	PTHREAD_SCOPE_SYSTEM: 绑定
		PTHREAD_SCOPE_PROCESS: 非绑定
函数返回值	成功: 0	
	出错: -1	

表 7.11 列出了 pthread_attr_setdetachstate()函数的语法要点。

表 7.11 pthread_attr_setdetachstate()函数语法要点

所需头文件	#include <pthread.h>	
函数原型	int pthread_attr_setscope(pthread_attr_t *attr, int detachstate)	
函数传入值	attr: 线程属性	
	detachstate	PTHREAD_CREATE_DETACHED: 分离
		PTHREAD_CREATE_JOINABLE: 非分离
函数返回值	成功: 0	
	出错: 返回错误码	

表 5.12 列出了 pthread_attr_getschedparam()函数的语法要点。

表 5.12 pthread_attr_getschedparam()函数语法要点

所需头文件	#include <pthread.h>
函数原型	int pthread_attr_getschedparam(pthread_attr_t *attr, struct sched_param *param)
函数传入值	attr: 线程属性结构指针



	param: 线程优先级
函数返回值	成功: 0
	出错: 返回错误码

表 7.13 列出了 pthread_attr_setschedparam()函数的语法要点。

表 7.13 pthread_attr_setschedparam()函数语法要点

所需头文件	#include <pthread.h>
函数原型	int pthread_attr_setschedparam (pthread_attr_t *attr, struct sched_param *param)
函数传入值	attr: 线程属性结构指针
	param: 线程优先级
函数返回值	成功: 0
	出错: 返回错误码

12.4 实验步骤

将【华清远见-嵌入式 ARM 实验箱资料- I -FS6818: 程序源码\Linux 应用实验源码】目录拷贝到虚拟机共享目录【D:\share\】下。(注意: 此步骤已经在“Linux Shell 编程”章节中完成, 如果没有完成请参照此章节。如已经完成可省略此步骤)

将线程实验部分代码拷贝到虚拟机 Linux 下。

1、建立相关目录;

```
$ mkdir ~/workdir/linux/application/thread -p
$ cd ~/workdir/linux/application/thread
```

```
linux@ubuntu64-vm:~$
linux@ubuntu64-vm:~$ mkdir ~/workdir/linux/application/thread -p
linux@ubuntu64-vm:~$ cd ~/workdir/linux/application/thread
linux@ubuntu64-vm:~/workdir/linux/application/thread$
```

2、将代码从共享目录拷入虚拟机 Linux 操作系统下; (可使用【ctrl+空格】切换输入法)

```
$ cp /mnt/hgfs/share/Linux 应用实验源码/21.\ Linux 系统多线程实验/实验代码/thread.c .
$ ls
```

```
linux@ubuntu64-vm:~/workdir/linux/application/thread$ cp /mnt/hgfs/share/Linux应用实验源码/21.\ L
inux系统多线程实验/实验代码/thread.c .
linux@ubuntu64-vm:~/workdir/linux/application/thread$ ls
thread.c
```

3、编译代码:

```
$ gcc thread.c -o thread -lpthread
```

4、执行代码



\$./thread

12.5 实验现象

执行./thread 之后按下回车会出现如下现象。

```
linux@ubuntu64-vm:~/workdir/linux/application/thread$ ./thread
dadf

welcome
***
result:welcome
```

12.6 实验代码

thread.c

```
1#include <stdio.h>
2#include <time.h>
3#include <stdlib.h>
4#include <errno.h>
5#include <unistd.h>
6#include <pthread.h>
7
8//char buf[] = "welcome";
9
10void *handler(void *p) //void *p = buf;
11{
12    getchar();
13    printf("%s\n", (char *)p);
14
15    //return p;
16    pthread_exit(p);
17}
18
19void f(char **p)//char **p = &s;
20{
21    char *q = "dadf";
22    *p = q;
23}
24
25int main(int argc, char *argv[])
26{
27    char *s = NULL;
28    f(&s);
```



```

29     printf("%s\n", s);
30
31 #if 1
32     pthread_t thread;
33     char buf[] = "welcome";
34     void *result;
35
36     if (pthread_create(&thread, NULL, handler, (void *)buf))
37         exit(-1);
38
39     pthread_join(thread, &result); //exit(0); waitpid(pid, &status, 0);
40
41     printf("***\n");
42     printf("result:%s\n", (char *)result);
43
44     while (1);
45 #endif
46
47     exit(0);
48 }

```

华清远见
fsdev.com.cn



第 13 章 嵌入式 Linux 网络编程实验

13.1 实验原理

13.1.1 TCP/IP 的分层模型

读者一定都听说过著名的 OSI 协议参考模型，它是基于国际标准化组织（ISO）的建议发展起来的，它分为 7 个层次：应用层、表示层、会话层、传输层、网络层、数据链路层及物理层。这个 7 层的协议模型虽然规定得非常细致和完善，但在实际中却得不到广泛的应用，其重要的原因之一就在于它过于复杂。但它仍是此后很多协议模型的基础。与此相区别的 TCP/IP 协议模型将 OSI 的 7 层协议模型简化为 4 层，从而更有利于实现和使用。TCP/IP 的协议参考模型和 OSI 协议参考模型的对应关系如下图 8.1 所示。

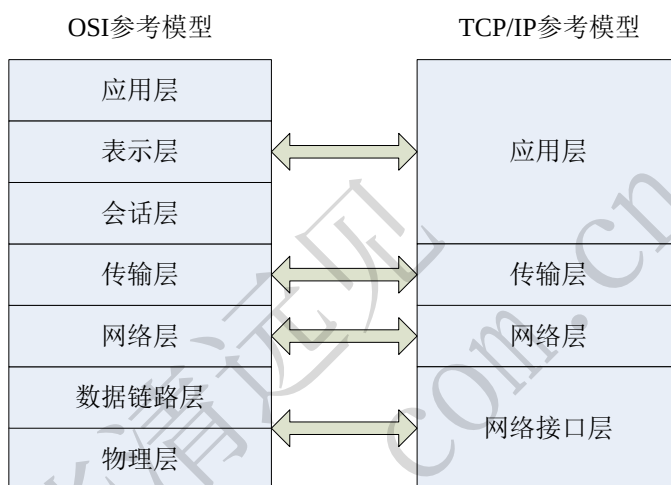


图 8.1 OSI 模型和 TCP/IP 参考模型对应关系

TCP/IP 协议是一个复制的协议，是由一组专业化协议组成的。这些协议包括 IP、TCP、UDP、ARP、ICMP 以及其他的一些被称为子协议的协议。TCP/IP 协议的前身是由美国国防部在 20 世纪 60 年代末期为其远景研究规划署网络（ARPANET）而开发的。由于低成本以及在多个不同平台通信的可靠性，TCP/IP 迅速发展并开始流行。它实际上是一个关于因特网的标准，迅速成为局域网的首选协议。下面具体讲解各层在 TCP/IP 整体架构中的作用。

1. 网络接口层（Network Interface Layer）

网络接口层是 TCP/IP 协议软件的最底层，负责将二进制流转换为数据帧，并进行数据帧的发送和接收。数据帧是网络传输的基本单元。

2. 网络层（Internet Layer）

网络层负责在主机之间的通信中选择数据报的传输路径，即路由。当网络层接收到传输层的请求后，传输某个具有目的地址信息的分组。该层把分组封装在 IP 数据报中，填入数据报的首部，使用路由算法来



确定是直接交付数据报，还是把它传递给路由器，然后把数据报交给适当的网络接口进行传输。

网络层还要负责处理传入的数据报，检验其有效性，使用路由算法来决定应该对数据报进行本地处理还是应该转发。

如果数据报的目的机处于本机所在的网络，该层软件就会除去数据报的首部，再选择适当的运输层协议来处理这个分组。最后，网络层还要根据需要发出和接收 ICMP（Internet 控制报文协议）差错和控制报文。

3. 传输层（Transport Layer）

传输层负责提供应用程序之间的通信服务。这种通信又称为端到端通信。传输层要系统地管理信息的流动，还要提供可靠的传输服务，以确保数据到达无差错、无乱序。为了达到这个目的，传输层协议软件要进行协商，让接收方回送确认信息及让发送方重发丢失的分组。传输层协议软件把要传输的数据流划分为分组，把每个分组连同目的地址交给网络层去发送。

4. 应用层（Application Layer）

应用层是分层模型的最高层，在这个最高层中，用户调用应用程序通过 TCP/IP 互联网来访问可行的服务。与各个传输层协议交互的应用程序负责接收和发送数据。每个应用程序选择适当的传输服务类型，把数据按照传输层的格式要求封装好向下层传输。

综上所述，TCP/IP 分层模型每一层负责不同的通信功能，整体联动合作，就可以完成互联网的大部分传输要求。

13.1.2 TCP/IP 分层模型特点

TCP/IP 是目前 Internet 上最成功、使用最频繁的互联协议。虽然现在已有很多协议都适用于互联网，但 TCP/IP 的使用最广泛。下面讲解一下 TCP/IP 的特点。

1. TCP/IP 模型边界特性

TCP/IP 分层模型中有两大边界特性：一个是地址边界特性，它将 IP 逻辑地址与底层网络的硬件地址分开；一个是操作系统边界特性，它将网络应用与协议软件分开，如图 8.2 所示。

TCP/IP 分层模型边界特性是指在模型中存在一个地址上的边界，它将底层网络的物理地址与网络层的 IP 地址分开。该边界出现在网络层与网络接口层之间。



应用层	操作系统外部
传输层	操作系统内部
网络层	IP地址
网络接口层	物理地址

图 6.2 TCP/IP 分层模型边界特性

网络层和其上的各层均使用 IP 地址，网络接口层则使用物理地址，即底层网络设备的硬件地址。TCP/IP 提供在两种地址之间进行映射的功能。划分地址边界的目的是为了屏蔽底层物理网络的地址细节，以便使互联网软件地址上易于实现和理解。

TCP/IP 软件在操作系统内具体的位置和 TCP/IP 的实现有关，但大部分实现都类似于图 6.2 所示情况。影响操作系统边界划分的最重要因素是协议的效率问题，在操作系统内部实现的协议软件，其数据传递的效率明显要高。

2. IP 层特性

IP 层作为通信子网的最高层，提供无连接的数据报传输机制，但 IP 协议并不能保证 IP 报文传递的可靠性，IP 的机制是点到点的。用 IP 进行通信的主机或路由器位于同一物理网络，对等机器之间拥有直接的物理连接。

TCP/IP 设计原则之一是为包容各种物理网络技术，包容性主要体现在 IP 层中。各种物理网络技术在帧或报文格式、地址格式等方面差别很大，TCP/IP 的重要思想之一就是通过 IP 将各种底层网络技术统一起来，达到屏蔽底层细节，提供统一虚拟网的目的。

IP 向上层提供统一的 IP 报文，使得各种网络帧或报文格式的差异性对高层协议不复存在。IP 层是 TCP/IP 实现异构网互联最关键的一层。

3. TCP/IP 的可靠性特性

在 TCP/IP 网络中，IP 采用无连接的数据报机制，对数据进行“尽力而为”的传递机制，即只管将报文尽力传送到目的主机，无论传输正确与否，不做验证，不发确认，也不保证报文的顺序。TCP/IP 的可靠性体现在传输层协议之一的 TCP 协议。TCP 协议提供面向连接的服务，因为传输层是端到端的，所以 TCP/IP 的可靠性被称为端到端可靠性。

综上所述，TCP/IP 的特点就是将不同的底层物理网络、拓扑结构隐藏起来，向用户和应用程序提供通用、统一的网络服务。这样，从用户的角度看，整个 TCP/IP 互联网就是一个统一的整体，它独立于具体的各种物理网络技术，能够向用户提供一个通用的网络服务。

TCP/IP 网络完全撇开了底层物理网络的特性，是一个高度抽象的概念，正是由于这个原因，其为



TCP/IP 网络赋予了巨大的灵活性和通用性。

13.1.3 TCP/IP 核心协议

在 TCP/IP 协议族中，有很多种协议，如图 8.3 所示。

TCP/IP 协议群中的核心协议被设计运行在网络层和传输层，它们为网络中的各主机提供通信服务，也为模型的最高层——应用层中的协议提供服务。

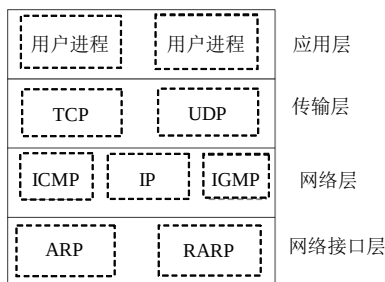


图 8.3 TCP/IP 协议族不同分层中的协议

在此主要介绍在网络编程中涉及的传输层 TCP 和 UDP 协议。

1. TCP

(1) 概述

TCP 的上一层是应用层，TCP 向应用层提供可靠的面向对象的数据流传输服务，TCP 数据传输实现了从一个应用程序到另一个应用程序的数据传递。它能提供高可靠性通信(即数据无误、数据无丢失、数据无失序、数据无重复到达的通信。)，应用程序通过向 TCP 层提交数据接发送/收端的地址和端口号而实现应用层的数据通信。

通过 IP 的源/目的可以惟一地区分网络中两个设备的连接，通过 socket 的源/目的可以惟一地区分网络中两个应用程序的连接。

(2) 3 次握手协议

TCP 是面向连接的，所谓面向连接，就是当计算机双方通信时必需先建立连接，然后进行数据通信，最后拆除连接三个过程。TCP 在建立连接时又分三步走：

第一步 (A->B)：主机 A 向主机 B 发送一个包含 SYN 即同步 (Synchronize) 标志的 TCP 报文，SYN 同步报文会指明客户端使用的端口以及 TCP 连接的初始序号；

第二步 (B->A)：主机 B 在收到客户端的 SYN 报文后，将返回一个 SYN+ACK 的报文，表示主机 B 的请求被接受，同时 TCP 序号被加一，ACK 即确认 (Acknowledgement)。

第三步 (A->B)：主机 A 也返回一个确认报文 ACK 给服务器端，同样 TCP 序列号被加一，到此一个 TCP 连接完成。图 8.4 就是这个流程的简单示意图。

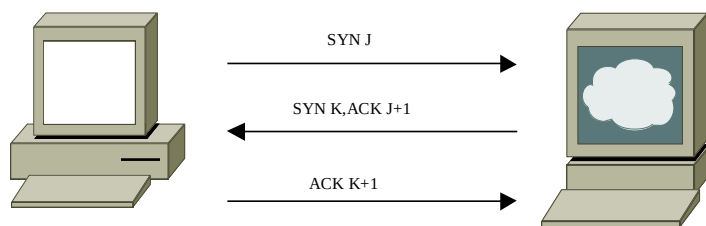


图 8.4 TCP3 次握手协议

TCP 实体所采用的基本协议是滑动窗口协议。当发送方传送一个数据报时，它将启动计时器。当该数据报到达目的地后，接收方的 TCP 实体向回发送一个数据报，其中包含有一个确认序号，它意思是希望收到的下一个数据报的序号。如果发送方的定时器在确认信息到达之前超时，那么发送方会重发该数据报。

(3) TCP 数据报头

图 8.5 为 TCP 数据报头的格式。



图 8.5 TCP 数据报头的格式

- 源端口、目的端口：16 位长，标识出远端和本地的端口号。
- 序号：32 位长，标识发送的数据报的顺序。
- 确认号：32 位长，希望收到的下一个数据报的序列号。
- TCP 头长：4 位长，表明 TCP 头中包含多少个 32 位字。
- 6 位未用。
- ACK：ACK 位置 1 表明确认号是合法的；如果 ACK 为 0，那么数据报不包含确认信息，确认字段被省略。
- PSH：表示是带有 PUSH 标志的数据。因此请求数据报一到接收方便可送往应用程序而不必等到缓冲区装满时才传送。
- RST：用于复位由于主机崩溃或其他原因而出现的错误的连接，还可以用于拒绝非法的数据报或拒绝连接请求。
- SYN：用于建立连接。
- FIN：用于释放连接。
- 窗口大小：16 位长，窗口大小字段表示在确认了字节之后还可以发送多少个字节。



- 校验和：16 位长，是为了确保高可靠性而设置的，它校验头部、数据和伪 TCP 头部之和。
- 可选项：0 个或多个 32 位字，包括最大 TCP 载荷、窗口比例、选择重发数据报等选项。

2. UDP

(1) 概述

UDP 即用户数据报协议，是一种面向无连接的不可靠传输协议，不需要通过 3 次握手来建立一个连接，同时，一个 UDP 应用可同时作为应用的客户或服务器方。

由于 UDP 协议并不需要建立一个明确的连接，因此建立 UDP 应用要比建立 TCP 应用简单得多。UDP 比 TCP 协议更为高效，也能更好地解决实时性的问题，如今，包括网络视频会议系统在内的众多的客户/服务器模式的网络应用都使用 UDP 协议。

(2) UDP 数据包头

UDP 数据包头如图 8.6 所示。

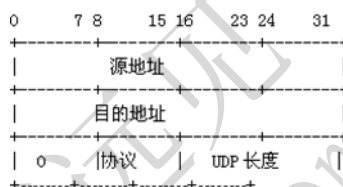


图 8.6 UDP 数据包头

- 源地址、目的地址：16 位长，标识出远端和本地的端口号。
- 数据报的长度是指包括报头和数据部分在内的总的字节数。因为报头的长度是固定的，所以该域主要用来计算可变长度的数据部分（又称为数据负载）。

3. 协议的选择

协议的选择应该考虑到数据可靠性、应用的实时性和网络的可靠性。

- 对数据可靠性要求高的应用需选择 TCP 协议，而对数据的可靠性要求不那么高的应用可选择 UDP 传送。
- TCP 协议中的 3 次握手、重传确认等手段可以保证数据传输的可靠性，但使用 TCP 协议会有较大的时延，因此不适合对实时性要求较高的应用；而 UDP 协议则有很好的实时性。

网络状况不是很好的情况下需选用 TCP 协议（如在广域网等情况），网络状况很好的情况下选择 UDP 协议可以减少网络负荷。



13.1.4 套接字 (socket) 概述

1. 套接字定义

在 Linux 中的网络编程是通过 socket 接口来进行的。套接字 (socket) 是一种特殊的 I/O 接口, 它也是一种文件描述符。socket 是一种常用的进程之间通信机制, 通过它不仅能实现本地机器上的进程之间的通信, 而且通过网络能够在不同机器上的进程之间进行通信。

每一个 socket 都用一个半相关描述 {协议、本地地址、本地端口} 来表示; 一个完整的套接字则用一个相关描述 {协议、本地地址、本地端口、远程地址、远程端口} 来表示。socket 也有一个类似于打开文件的函数调用, 该函数返回一个整型的 socket 描述符, 随后的连接建立、数据传输等操作都是通过 socket 来实现的。

2. 套接字类型

常见的 socket 有 3 种类型如下。

(1) 流式套接字 (SOCK_STREAM)

流式套接字提供可靠的、面向连接的通信流; 它使用 TCP 协议, 从而保证了数据传输的可靠性和顺序性。

(2) 数据报套接字 (SOCK_DGRAM)

数据报套接字定义了一种无可靠、面向无连接的服务, 数据通过相互独立的报文进行传输, 是无序的, 并且不保证是可靠、无差错的。它使用数据报协议 UDP。

(3) 原始套接字 (SOCK_RAW)

原始套接字允许对底层协议如 IP 或 ICMP 进行直接访问, 它功能强大但使用较为不便, 主要用于一些协议的开发。

13.1.5 地址及顺序处理

1. 地址结构相关处理

(1) 数据结构介绍

下面首先介绍两个重要的数据类型: sockaddr 和 sockaddr_in, 这两个结构类型都是用来保存 socket 信息的, 如下所示:

```
struct sockaddr
{
    unsigned short sa_family; /*地址族*/
    char sa_data[14]; /*14 字节的协议地址, 包含该 socket 的 IP 地址和端口号。*/
}
```



```
};

struct sockaddr_in
{
    short int sa_family; /*地址族*/
    unsigned short int sin_port; /*端口号*/
    struct in_addr sin_addr; /*IP 地址*/
    unsigned char sin_zero[8]; /*填充 0 以保持与 struct sockaddr 同样大小*/
};
```

这两个数据类型是等效的，可以相互转化，通常 `sockaddr_in` 数据类型使用更为方便。在建立 `socketadd` 或 `sockaddr_in` 后，就可以对该 `socket` 进行适当的操作了。

(2) 结构字段

表 8.1 列出了该结构 `sa_family` 字段可选的常见值。

表 8.1 `sa_family` 字段值

结构定义头文件	<code>#include <netinet/in.h></code>
sa_family	AF_INET: IPv4 协议
	AF_INET6: IPv6 协议
	AF_LOCAL: UNIX 域协议
	AF_LINK: 链路地址协议
	AF_KEY: 密钥套接字

`sockaddr_in` 其他字段的含义非常清楚，具体的设置涉及到其他函数，在后面会有详细的讲解。

2. 数据存储优先顺序

(1) 函数说明

计算机数据存储有两种字节优先顺序：高位字节优先（称为大端模式）和低位字节优先（称为小端模式，PC 机通常采用小端模式）。Internet 上数据以高位字节优先顺序在网络上传输，因此在有些情况下，需要对这两个字节存储优先顺序进行相互转化。这里用到了四个函数：`htons()`、`ntohs()`、`htonl()`和 `ntohl()`。这四个地址分别实现网络字节序和主机字节序的转化，这里的 `h` 代表 `host`，`n` 代表 `network`，`s` 代表 `short`，`l` 代表 `long`。通常 16 位的 IP 端口号用 `s` 代表，而 IP 地址用 `l` 来代表。调用这些函数只是使其得到相应的字节序，用户不需清楚该系统的主机字节序和网络字节序是否真正相等。如果是相同不需要转换的话，该系统的这些函数会定义成空宏。

(2) 函数格式

表 8.2 列出了这 4 个函数的语法格式。



表 8.2 htons 等函数语法要点

所需头文件	#include <netinet/in.h>
函数原型	uint16_t htons(uint16_t host16bit) uint32_t htonl(uint32_t host32bit) uint16_t ntohs(uint16_t net16bit) uint32_t ntohs(uint32_t net32bit)
函数传入值	host16bit: 主机字节序的 16bit 数据
	host32bit: 主机字节序的 32bit 数据
	net16bit: 网络字节序的 16bit 数据
	net32bit: 网络字节序的 32bit 数据
函数返回值	成功: 返回要转换的字节序
	出错: -1

3. 地址格式转化

(1) 函数说明

用户在表达地址时通常采用点分十进制表示的数值字符串（或者是以冒号分开的十进制 IPv6 地址），而在通常使用的 socket 编程中所使用的则是二进制值（例如，用 in_addr 结构和 in6_addr 结构分别表示 IPv4 和 IPv6 中的网络地址），这就需要将这两个数值进行转换。

这里在 IPv4 中用到的函数有 inet_aton()、inet_addr()和 inet_ntoa()，而 IPv4 和 IPv6 兼容的函数有 inet_pton()和 inet_ntop()。由于 IPv6 是下一代互联网的标准协议，因此，本书讲解的函数都能够同时兼容 IPv4 和 IPv6，但在具体举例时仍以 IPv4 为例。inet_pton()函数是将点分十进制地址字符串转换为二进制地址（例如：将 IPv4 的地址字符串“192.168.1.123”转换为 4 个字节的数据（从低字节起依次为 192、168、1、123）），而 inet_ntop()是 inet_pton()的反操作，将二进制地址转换为点分十进制地址字符串。

(2) 函数格式

表 8.3 列出了 inet_pton()函数的语法要点。

表 8.3 inet_pton()函数语法要点

所需头文件	#include <arpa/inet.h>	
函数原型	int inet_pton(int family, const char *strptr, void *addrptr)	
函数传入值	family	AF_INET: IPv4 协议
		AF_INET6: IPv6 协议
	strptr: 要转化的值（十进制地址字符串）	
	addrptr: 转化后的地址	
函数返回值	成功: 0	
	出错: -1	



表 8.4 列出了 `inet_ntop()` 函数的语法要点。

表 8.4 `inet_ntop()` 函数语法要点

所需头文件	#include <arpa/inet.h>	
函数原型	int inet_ntop(int family, void *addrptr, char *strptr, size_t len)	
函数传入值	family	AF_INET: IPv4 协议
		AF_INET6: IPv6 协议
	addrptr: 要转化的地址	
	strptr: 转化后的十进制地址字符串	
函数返回值	len: 转化后值的大小	
	成功: 0	
	出错: -1	

4. 名字地址转化

(1) 函数说明

在 Linux 中有一些函数可以实现主机名和地址的转化，如 `gethostbyname()`、`gethostbyaddr()` 和 `getaddrinfo()` 等，它们都可以实现 IPv4 和 IPv6 的地址和主机名之间的转化。

其中 `gethostbyname()` 是将主机名转化为 IP 地址，`gethostbyaddr()` 则是逆操作，是将 IP 地址转化为主机名，另外 `getaddrinfo()` 还能实现自动识别 IPv4 地址和 IPv6 地址。

`gethostbyname()` 和 `gethostbyaddr()` 都涉及到一个 `hostent` 的结构体，如下所示：

```
struct hostent
{
    char *h_name;           /* 正式主机名 */
    char **h_aliases;       /* 主机别名 */
    int h_addrtype;         /* 地址类型 */
    int h_length;           /* 地址字节长度 */
    char **h_addr_list;     /* 指向 IPv4 或 IPv6 的地址指针数组 */
}
```

调用 `gethostbyname()` 函数或 `gethostbyaddr()` 函数后就能返回 `hostent` 结构体的相关信息。`getaddrinfo()` 函数涉及到一个 `addrinfo` 的结构体，如下所示：

```
struct addrinfo
```



```
{
    int ai_flags;           /*AI_PASSIVE, AI_CANONNAME;*/
    int ai_family;         /*地址族*/
    int ai_socktype;       /*socket 类型*/
    int ai_protocol;       /*协议类型*/
    size_t ai_addrlen;     /*地址字节长度*/
    char *ai_canonname;    /*主机名*/
    struct sockaddr *ai_addr; /*socket 结构体*/
    struct addrinfo *ai_next; /*下一个指针链表*/
}
```

hostent 结构体而言，addrinfo 结构体包含更多的信息。

(2) 函数格式

表 8.5 列出了 gethostbyname()函数的语法要点。

表 8.5 gethostbyname 函数语法要点

所需头文件	#include <netdb.h>
函数原型	struct hostent *gethostbyname(const char *hostname)
函数传入值	hostname: 主机名
函数返回值	成功: hostent 类型指针
	出错: -1

调用该函数时可以首先对 hostent 结构体中的 h_addrtype 和 h_length 进行设置，若为 IPv4 可设置为 AF_INET 和 4；若为 IPv6 可设置为 AF_INET6 和 16；若不设置则默认为 IPv4 地址类型。

表 8.6 列出了 getaddrinfo()函数的语法要点。

表 8.6 getaddrinfo()函数语法要点

所需头文件	#include <netdb.h>
函数原型	int getaddrinfo(const char *node, const char *service, const struct addrinfo *hints, struct addrinfo **result)
函数传入值	node: 网络地址或者网络主机名
	service: 服务名或十进制的端口号字符串
	hints: 服务线索
	result: 返回结果
函数返回值	成功: 0
	出错: -1



在调用之前，首先要对 hints 服务线索进行设置。它是一个 `addrinfo` 结构体，表 8.7 列举了该结构体常见的选项值。

表 8.7 `addrinfo` 结构体常见选项值

结构体头文件	<code>#include <netdb.h></code>
<code>ai_flags</code>	<code>AI_PASSIVE</code> : 该套接口是用作被动地打开
	<code>AI_CANONNAME</code> : 通知 <code>getaddrinfo</code> 函数返回主机的名字
<code>ai_family</code>	<code>AF_INET</code> : IPv4 协议
	<code>AF_INET6</code> : IPv6 协议
	<code>AF_UNSPEC</code> : IPv4 或 IPv6 均可
<code>ai_socktype</code>	<code>SOCK_STREAM</code> : 字节流套接字 <code>socket</code> (TCP)
	<code>SOCK_DGRAM</code> : 数据报套接字 <code>socket</code> (UDP)
<code>ai_protocol</code>	<code>IPPROTO_IP</code> : IP 协议
	<code>IPPROTO_IPV4</code> : IPv4 协议
	<code>IPPROTO_IPV6</code> : IPv6 协议
	<code>IPPROTO_UDP</code> : UDP
	<code>IPPROTO_TCP</code> : TCP

注意：

- 通常服务器端在调用 `getaddrinfo()` 之前，`ai_flags` 设置 `AI_PASSIVE`，用于 `bind()` 函数（用于端口和地址的绑定，后面会讲到），主机名 `nodename` 通常会设置为 `NULL`。
- 客户端调用 `getaddrinfo()` 时，`ai_flags` 一般不设置 `AI_PASSIVE`，但是主机名 `nodename` 和服务名 `servname`（端口）则应该不为空。

即使不设置 `ai_flags` 为 `AI_PASSIVE`，取出的地址也可以被绑定，很多程序中 `ai_flags` 直接设置为 0，即 3 个标志位都不设置，这种情况下只要 `hostname` 和 `servname` 设置的没有问题就可以正确绑定。

13.1.6 套接字编程

(1) 函数说明

`socket` 编程的基本函数有 `socket()`、`bind()`、`listen()`、`accept()`、`send()`、`sendto()`、`recv()` 以及 `recvfrom()` 等，其中根据客户端还是服务端，或者根据使用 TCP 协议还是 UDP 协议，这些函数的调用流程都有所区别，这里先对每个函数进行说明，再给出各种情况下使用的流程图。

- `socket()`: 该函数用于建立一个套接字，一条通信路线的端点。在建立了 `socket` 之后，可对 `sockaddr` 或 `sockaddr_in` 结构进行初始化，以保存所建立的 `socket` 地址信息。
- `bind()`: 该函数是用于将 `sockaddr` 结构的地址信息与套接字进行绑定，它主要用于 TCP 的连接，而在 UDP 的连接中则无必要（但可以使用）。
- `listen()`: 在服务端程序成功建立套接字和与地址进行绑定之后，还需要准备在该套接字上接收新的



连接请求。此时调用 `listen()` 函数来创建一个等待队列，在其中存放未处理的客户端连接请求。

- `accept()`: 服务端程序调用 `listen()` 函数创建等待队列之后，调用 `accept()` 函数等待并接收客户端的连接请求。它通常从由 `listen()` 所创建的等待队列中取出第一个未处理的连接请求。

- `connect()`: 客户端通过一个未命名套接字（未使用 `bind()` 函数）和服务器监听套接字之间建立连接的方法来连接到服务器。这个工作客户端通过使用 `connect()` 函数来实现。

- `send()` 和 `recv()`: 这两个函数分别用于发送和接收数据，可以用在 TCP 中，也可以用在 UDP 中。当用在 UDP 时，可以在 `connect()` 函数建立连接之后再使用。

- `sendto()` 和 `recvfrom()`: 这两个函数的作用与 `send()` 和 `recv()` 函数类似，也可以用在 TCP 和 UDP 中。当用在 TCP 时，后面的几个与地址有关参数不起作用，函数作用等同于 `send()` 和 `recv()`；当用在 UDP 时，可以用在之前没有使用 `connect()` 的情况下，这两个函数可以自动寻找指定地址并进行连接。

服务器端和客户端使用 TCP 协议的流程如图 8.7 所示。

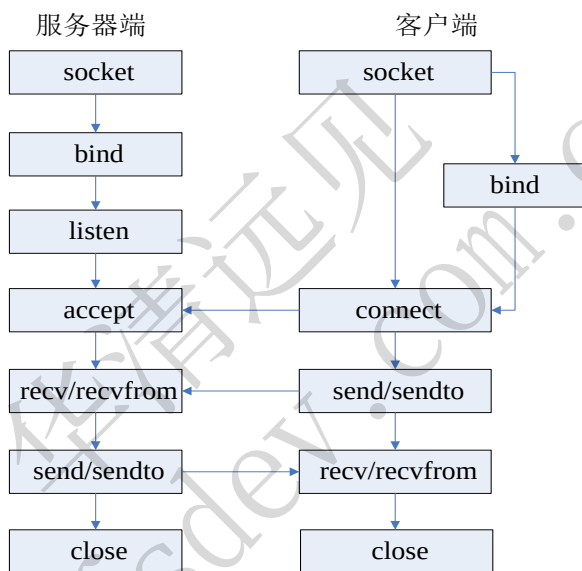


图 8.7 使用 TCP 协议 socket 编程流程图

服务器端和客户端使用 UDP 协议的流程图如图 8.8 所示。

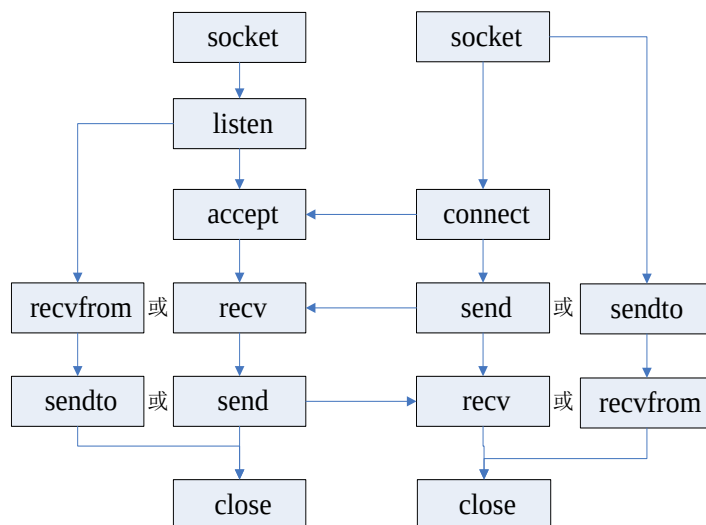


图 8.8 使用 UDP 协议 socket 编程流程图

(2) 函数格式

表 8.8 列出了 socket()函数的语法要点。

表 8.8 socket()函数语法要点

所需头文件	#include <sys/socket.h>
函数原型	int socket(int family, int type, int protocol)
函数传入值	family
	AF_INET: IPv4 协议
	AF_INET6: IPv6 协议
	AF_LOCAL: UNIX 域协议
	AF_ROUTE: 路由套接字 (socket)
套接字类型	type:
	SOCK_STREAM: 字节流套接字 socket
	SOCK_DGRAM: 数据报套接字 socket
函数返回值	SOCK_RAW: 原始套接字 socket
	protocol: 0 (原始套接字除外)
函数返回值	成功: 非负套接字描述符
	出错: -1

表 8.9 列出了 bind()函数的语法要点。

表 8.9 bind()函数语法要点

所需头文件	#include <sys/socket.h>
函数原型	int bind(int sockfd, struct sockaddr *my_addr, int addrlen)
函数传入值	sockfd: 套接字描述符
	my_addr: 本地地址



	addrlen: 地址长度
函数返回值	成功: 0
	出错: -1

端口号和地址在 `my_addr` 中给出了, 若不指定地址, 则内核随意分配一个临时端口给该应用程序。IP 地址可以直接指定 (比如: `inet_addr("192.168.1.112")`), 或者使用宏 `INADDR_ANY`, 允许将套接字与服务器的任一网络接口 (比如: `eth0`、`eth0:1`、`eth1` 等) 进行绑定。

表 8.10 列出了 `listen()` 函数的语法要点。

表 8.10 `listen()` 函数语法要点

所需头文件	<code>#include <sys/socket.h></code>
函数原型	<code>int listen(int sockfd, int backlog)</code>
函数传入值	sockfd: 套接字描述符
	backlog: 请求队列中允许的最大请求数, 大多数系统缺省值为 5
函数返回值	成功: 0
	出错: -1

表 8.11 列出了 `accept()` 函数的语法要点。

表 8.11 `accept()` 函数语法要点

所需头文件	<code>#include <sys/socket.h></code>
函数原型	<code>int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen)</code>
函数传入值	sockfd: 套接字描述符
	addr: 客户端地址
	addrlen: 地址长度
函数返回值	成功: 接收到的非负的套接字
	出错: -1

表 8.12 列出了 `connect()` 函数的语法要点。

表 8.12 `connect()` 函数语法要点

所需头文件	<code>#include <sys/socket.h></code>
函数原型	<code>int connect(int sockfd, struct sockaddr *serv_addr, int addrlen)</code>
函数传入值	sockfd: 套接字描述符
	serv_addr: 服务器端地址
	addrlen: 地址长度
函数返回值	成功: 0
	出错: -1



表 8.13 列出了 send()函数的语法要点。

表 8.13 send()函数语法要点

所需头文件	#include <sys/socket.h>
函数原型	int send(int sockfd, const void *msg, int len, int flags)
函数传入值	sockfd: 套接字描述符
	msg: 指向要发送数据的指针
	len: 数据长度
	flags: 一般为 0
函数返回值	成功: 实际发送的字节数
	出错: -1

表 8.14 列出了 recv()函数的语法要点。

表 8.14 recv()函数语法要点

所需头文件	#include <sys/socket.h>
函数原型	int recv(int sockfd, void *buf, int len, unsigned int flags)
函数传入值	sockfd: 套接字描述符
	buf: 存放接收数据的缓冲区
	len: 数据长度
	flags: 一般为 0
函数返回值	成功: 实际接收到的字节数
	出错: -1

表 8.15 列出了 sendto()函数的语法要点。

表 8.15 sendto()函数语法要点

所需头文件	#include <sys/socket.h>
函数原型	int sendto(int sockfd, const void *msg, int len, unsigned int flags, const struct sockaddr *to, int tolen)
函数传入值	sockfd: 套接字描述符
	msg: 指向要发送数据的指针
	len: 数据长度
	flags: 一般为 0
	to: 目的地机的 IP 地址和端口号信息
	tolen: 地址长度



函数返回值	成功：实际发送的字节数
	出错：-1

表 8.16 列出了 `recvfrom()` 函数的语法要点。

表 8.16 `recvfrom()` 函数语法要点

所需头文件	<code>#include <sys/socket.h></code>
函数原型	<code>int recvfrom(int sockfd, void *buf, int len, unsigned int flags, struct sockaddr *from, int *fromlen)</code>
函数传入值	<code>sockfd</code> : 套接字描述符
	<code>buf</code> : 存放接收数据的缓冲区
	<code>len</code> : 数据长度
	<code>flags</code> : 一般为 0
	<code>from</code> : 源主机的 IP 地址和端口号信息
	<code>fromlen</code> : 地址长度
函数返回值	成功：实际接收到的字节数
	出错：-1

!

在实际情况中，人们往往遇到多个客户端连接服务器端的情况。在之前介绍的例子中我们使用阻塞函数，因此如果资源没有准备好，则调用该函数的进程将进入睡眠状态，这样就无法处理其他请求的情况了。本节给出了三种解决 I/O 多路复用的解决方法，分别为非阻塞和异步式处理（使用 `fcntl()` 函数）以及多路复用处理（使用 `select()` 或 `poll()` 函数）。此外有多进程和多线程编程是在网络编程中常用的事务处理方法，下一节的例子中会用到多进程网络编程。读者可以尝试使用多线程机制修改书上的所有例子，会发现多线程编程在网络编程中非常有效的方法之一。

1. 非阻塞和异步 I/O

在 `socket` 编程中可以使用函数 `fcntl(int fd, int cmd, int arg)` 的如下的编程特性。

- 获得文件状态标志：将 `cmd` 设置为 `F_GETFL`，会返回由 `fd` 指向的文件的状态标志。
- 非阻塞 I/O：将 `cmd` 设置为 `F_SETFL`，将 `arg` 设置为 `O_NONBLOCK`。
- 异步 I/O：将 `cmd` 设置为 `F_SETFL`，将 `arg` 设置为 `O_ASYNC`，后面有说明。

尽管在很多情况下，使用阻塞式和非阻塞式以及多路复用等机制可以有效地进行网络通信，但效率最高的方法是使用异步通知机制。这种方法在设备 I/O 编程中常见的。

内核通过使用异步 I/O，在某一个进程需要处理的事件发生（例如，接收到新的连接请求）时，向该进程发送一个 `SIGIO` 信号。这样，应用程序不需要不停地等待着某些事件的发生，而可以往下运行，以完



成其它的工作。只有收到从内核发来的 SIGIO 信号时，去处理它（例如，读取数据）就可以。

我们使用 `fcntl()` 函数就可以实现高效率的异步 I/O 的方法。首先必须使用 `fcntl` 的 `F_SETOWN` 命令，使套接字归属于当前进程，以内核能够判断应该向哪个进程发送信号。接下来，使用 `fcntl` 的 `F_SETFL` 命令将套接字的状态标志位设置为异步通知方式（使用 `O_ASYNC` 参数）。

2. 使用多路复用

使用 `fcntl()` 函数虽然可以实现非阻塞，但在实际使用时往往会对资源是否准备完毕进行循环测试，这样就大大增加了不必要的 CPU 资源的占用。在这里可以使用 `select()` 函数（或者使用 `poll()` 函数）来解决这个问题，同时，使用 `select()` 函数还可以设置等待的时间，可以说功能更加强大。

13.2 Linux 系统 tcp 网络协议编程实验

13.2.1 实验目的

熟悉 socket 网络编程的基本方法

13.2.2 实验平台

华清远见开发环境

13.2.3 实验内容

本实验通过一个简单的 tcp 服务器端，接收客户端的连接请求，并接受客户端发来的信息。

13.2.4 实验步骤

将【华清远见-嵌入式 ARM 实验箱资料-I-FS6818：程序源码\Linux 应用实验源码】目录拷贝到虚拟机共享目录【D:\share\】下。

（注意：此步骤已经在“Linux Shell 编程”章节中完成，如果没有完成请参照此章节。如已经完成可省略此步骤）

将 tcp 部分代码拷贝到虚拟机 Linux 下。

1、建立相关目录；

```
$ mkdir ~/workdir/linux/application/tcp -p
$ cd ~/workdir/linux/application/tcp
```

```
linux@ubuntu64-vm:~$ mkdir ~/workdir/linux/application/tcp -p
linux@ubuntu64-vm:~$ cd ~/workdir/linux/application/tcp
linux@ubuntu64-vm:~/workdir/linux/application/tcp$
```

2、将代码从共享目录拷入虚拟机 Linux 操作系统下；（可使用【ctrl+空格】切换输入法）

```
$ cp /mnt/hgfs/share/Linux 应用实验源码/17.\ Linux 系统 tcp 网络协议编程实验/实验代码/*.c .
$ ls
```



```
linux@ubuntu64-vm:~/workdir/linux/application/tcp$ cp /mnt/hgfs/share/Linux应用实验源码/17.\ Linux
系统tcp网络协议编程实验/实验代码/*.c .
linux@ubuntu64-vm:~/workdir/linux/application/tcp$ ls
client.c  server.c
```

3、编译代码:

```
$ gcc client.c -o client
```

```
$ gcc server.c -o server
```

4、执行代码

首先运行服务器程序:

```
$ ./server 192.168.100.192 8888 //其中 ip 地址是 ubuntu 系统实际的 ip 地址(如用户未进行更改, 此处
默认为 192.168.100.192)
```

另外打开一个终端 (Ctrl+Alt+t), 进入该目录, 运行客户端程序

```
$ cd ~/workdir/linux/application/tcp
```

```
$ ./client 192.168.100.192 8888
```

```
>hello //向服务器端发送消息
```

```
Hello
```

```
linux@ubuntu64-vm:~$ cd ~/workdir/linux/application/tcp
linux@ubuntu64-vm:~/workdir/linux/application/tcp$ ./client 192.168.100.192 8888
>hello
hello
```

13.2.5 实验现象

服务器所在终端会显示:

```
from 192.168.100.192:xxxxx
```

```
n=6 hello
```

```
linux@ubuntu64-vm:~/workdir/linux/application/tcp$ ./server 192.168.100.192 8888
from 192.168.100.192:37562
n=6 hello
```

13.2.6 实验代码

client.c

```
1#include <stdio.h>
2#include <stdlib.h>
3#include <unistd.h>
4#include <sys/types.h>
5#include <sys/socket.h>
6#include <errno.h>
7#include <string.h>
8#include <arpa/inet.h>
9#include <netinet/in.h>
```



```

10
11 #define N 64
12
13 int main(int argc, char *argv[])//./server ip port
14 {
15     int sockfd;
16     struct sockaddr_in servaddr, myaddr;
17     char buf[N] = {0};
18
19     if (argc < 3)
20     {
21         printf("usage:%s ip port\n", argv[0]);
22         return 0;
23     }
24
25     if ((sockfd = socket(AF_INET, SOCK_STREAM, 0)) == -1)
26     {
27         perror("socket");
28         exit(-1);
29     }
30
31     #if 0
32     memset(&myaddr, 0, sizeof(myaddr));
33     myaddr.sin_family = AF_INET;
34     myaddr.sin_port = htons(8000);//"6000"--6000 htons(6000);
35     myaddr.sin_addr.s_addr = inet_addr(argv[1]);
36
37     if (bind(sockfd, (struct sockaddr *)&myaddr, sizeof(myaddr)) == -1)
38     {
39         perror("bind");
40         exit(-1);
41     }
42 #endif
43
44     memset(&servaddr, 0, sizeof(servaddr));
45     servaddr.sin_family = AF_INET;
46     servaddr.sin_port = htons(atoi(argv[2]));//"6000"--6000 htons(6000);
47     servaddr.sin_addr.s_addr = inet_addr(argv[1]);
48
49     if (connect(sockfd, (struct sockaddr *)&servaddr, sizeof(servaddr)) == -1)
50     {
51         perror("connect");
52         exit(-1);

```



```

53     }
54
55     printf(">");
56     while (fgets(buf, N, stdin) != NULL)
57     {
58         //abc\n    --- a b c \n \0----
59         send(sockfd, buf, strlen(buf), 0);
60
61         memset(buf, 0, sizeof(buf));
62         recv(sockfd, buf, N, 0);
63         printf("%s\n", buf);
64
65         printf(">");
66     }
67     close(sockfd);
68
69     return 0;
70 }

```

server.c

```

1#include <stdio.h>
2#include <stdlib.h>
3#include <unistd.h>
4#include <sys/types.h>
5#include <sys/socket.h>
6#include <errno.h>
7#include <string.h>
8#include <arpa/inet.h>
9#include <netinet/in.h>
10
11#define N 64
12
13int main(int argc, char *argv[])//./server ip port
14{
15     int listenfd, connfd;
16     struct sockaddr_in myaddr, peeraddr;
17     socklen_t len;
18     char buf[N] = {0};
19     ssize_t n;
20
21     if (argc < 3)
22     {
23         printf("usage:%s ip port\n", argv[0]);
24         return 0;

```



```

25     }
26
27     if ((listenfd = socket(AF_INET, SOCK_STREAM, 0)) == -1)
28     {
29         perror("socket");
30         exit(-1);
31     }
32
33     memset(&myaddr, 0, sizeof(myaddr));
34     myaddr.sin_family = AF_INET;
35     myaddr.sin_port = htons(atoi(argv[2])); // "6000" -- 6000 htons(6000);
36     myaddr.sin_addr.s_addr = inet_addr(argv[1]);
37
38     if (bind(listenfd, (struct sockaddr *)&myaddr, sizeof(myaddr)) == -1)
39     {
40         perror("bind");
41         exit(-1);
42     }
43
44     if (-1 == listen(listenfd, 5))
45     {
46         perror("listen");
47         exit(-1);
48     }
49
50     memset(&peeraddr, 0, sizeof(peeraddr));
51     len = sizeof(peeraddr);
52
53     while (1)
54     {
55         if ((connfd = accept(listenfd, (struct sockaddr *)&peeraddr, &len)) == -1)
56         {
57             perror("accept");
58             exit(-1);
59         }
60         printf("from %s:%d\n", inet_ntoa(peeraddr.sin_addr), ntohs(peeraddr.sin_port));
61
62         while (1)
63         {
64             memset(buf, 0, sizeof(buf));
65             n = recv(connfd, buf, N, 0); // a b c \n n=4
66             if (n == 0)
67                 break;

```



```

68         buf[n] = '\0';
69         printf("n=%d %s", n, buf);
70
71         send(connfd, buf, n, 0);
72     }
73
74     close(connfd);
75 }
76
77 return 0;
78 }
    
```

13.3 Linux 系统 udp 网络协议编程实验

13.3.1 实验目的

熟悉 socket 网络编程的基本方法。

13.3.2 实验平台

华清远见开发环境

13.3.3 实验内容

本实验通过一个简单的 udp 服务器端，接收客户端的连接请求，并接受客户端发来的信息。

13.3.4 实验步骤

将【华清远见-嵌入式 ARM 实验箱资料- I -FS6818: 程序源码\Linux 应用实验源码】目录拷贝到虚拟机共享目录【D:\share\】下。

（注意：此步骤已经在“Linux Shell 编程”章节中完成，如果没有完成请参照此章节。如已经完成可省略此步骤）

将 udp 部分代码拷贝到虚拟机 Linux 下。

1、建立相关目录；

```

$ mkdir ~/workdir/linux/application/udp -p
$ cd ~/workdir/linux/application/udp
    
```

```

linux@ubuntu64-vm:~$ mkdir ~/workdir/linux/application/udp -p
linux@ubuntu64-vm:~$ cd ~/workdir/linux/application/udp
linux@ubuntu64-vm:~/workdir/linux/application/udp$
    
```

2、将代码从共享目录拷入虚拟机 Linux 操作系统下；（可使用【ctrl+空格】切换输入法）

```

$ cp /mnt/hgfs/share/Linux 应用实验源码/18.\ Linux 系统 udp 网络协议编程实验/实验代码/*.c .
$ ls
    
```



```
linux@ubuntu64-vm:~/workdir/linux/application/udp$ cp /mnt/hgfs/share/Linux应用实验源码/18.\ Linux
x系统udp网络协议编程实验/实验代码/*.c
linux@ubuntu64-vm:~/workdir/linux/application/udp$ ls
client.c  server.c
linux@ubuntu64-vm:~/workdir/linux/application/udp$
```

3、编译代码:

```
$ gcc client.c -o client
```

```
$ gcc server.c -o server
```

4、执行代码

首先运行服务器程序:

```
$ ./server 192.168.100.192 8888 //其中 ip 地址是 ubuntu 系统实际的 ip 地址(如用户未进行更改, 此处
默认为 192.168.100.192)
```

另外打开一个终端 (Ctrl+Alt+t), 进入该目录, 运行客户端程序

```
$ cd ~/workdir/linux/application/udp
```

```
$ ./client 192.168.100.192 8888
```

```
>hello //向服务器端发送消息
```

```
Hello
```

```
linux@ubuntu64-vm:~$ cd ~/workdir/linux/application/udp
linux@ubuntu64-vm:~/workdir/linux/application/udp$ ./client 192.168.100.192 8888
>hello
hello
```

13.3.5 实验现象

服务器所在终端会显示:

```
from 192.168.100.192:xxxxx hello
```

```
linux@ubuntu64-vm:~/workdir/linux/application/udp$ ./server 192.168.100.192 8888
from 192.168.100.192:41273 hello
```

13.3.6 实验代码

client.c

```
1#include <stdio.h>
2#include <stdlib.h>
3#include <unistd.h>
4#include <sys/types.h>
5#include <sys/socket.h>
6#include <errno.h>
7#include <string.h>
8#include <arpa/inet.h>
9#include <netinet/in.h>
```



```

1
0 #define N 64
1
1 int main(int argc, char *argv[]) // ./server ip port
1 {
2     int sockfd;
1     struct sockaddr_in servaddr;
3     char buf[N] = {0};
1
4     if (argc < 3)
1     {
5         printf("usage:%s ip port\n", argv[0]);
1         return 0;
6     }
1
7     if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) == -1)
1     {
8         perror("socket");
1         exit(-1);
9     }
2
0
2     memset(&servaddr, 0, sizeof(servaddr));
1     servaddr.sin_family = AF_INET;
2     servaddr.sin_port = htons(atoi(argv[2])); // "6000" -- 6000 htons(6000);
2     servaddr.sin_addr.s_addr = inet_addr(argv[1]);
2
3     while (1)
2     {
4         printf(">");
2         fgets(buf, N, stdin);
5         sendto(sockfd, buf, strlen(buf) + 1, 0, (struct sockaddr *) &servaddr, sizeof(servaddr))
2;
6
2         memset(buf, 0, sizeof(buf));
7         recvfrom(sockfd, buf, N, 0, NULL, NULL);
2         printf("%s\n", buf);
8     }
2     close(sockfd);
9
    return 0;
}

```

server.c



```

1#include <stdio.h>
2#include <stdlib.h>
3#include <unistd.h>
4#include <sys/types.h>
5#include <sys/socket.h>
6#include <errno.h>
7#include <string.h>
8#include <arpa/inet.h>
9#include <netinet/in.h>
10
11#define N 64
12
13int main(int argc, char *argv[])//./server ip port
14{
15    int sockfd;
16    struct sockaddr_in myaddr, peeraddr;
17    char buf[N] = {0};
18    socklen_t len;
19    ssize_t n;
20
21    if (argc < 3)
22    {
23        printf("usage:%s ip port\n", argv[0]);
24        return 0;
25    }
26
27    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) == -1)
28    {
29        perror("socket");
30        exit(-1);
31    }
32
33
34    memset(&myaddr, 0, sizeof(myaddr));
35    myaddr.sin_family = AF_INET;
36    myaddr.sin_port = htons(atoi(argv[2]));//"6000"--6000 htons(6000);
37    myaddr.sin_addr.s_addr = inet_addr(argv[1]);
38
39    if (bind(sockfd, (struct sockaddr *)&myaddr, sizeof(myaddr)) == -1)
40    {
41        perror("bind");
42        exit(-1);
43    }

```



```

44
45     memset(&peeraddr, 0, sizeof(peeraddr));
46     len = sizeof(peeraddr);
47
48     while (1)
49     {
50         memset(buf, 0, sizeof(buf));
51         n = recvfrom(sockfd, buf, N, 0, (struct sockaddr *)&peeraddr, &len);
52         printf("from %s:%d %s\n", inet_ntoa(peeraddr.sin_addr),
53             ntohs(peeraddr.sin_port), buf);
54         sendto(sockfd, buf, n, 0, (struct sockaddr *)&peeraddr, sizeof(peeraddr));
55     }
56     close(sockfd);
57
58     return 0;
59 }

```

13.4 Linux 系统 select IO 复用实验

13.4.1 实验目的

通过 select 来实现网络 I/O 的复用。

13.4.2 实验平台

华清远见开发环境

13.4.3 实验内容

tcp 服务器端在处理客户端连接请求的同时，对已连接客户实现 echo 服务

13.4.4 实验步骤

将【华清远见-嵌入式 ARM 实验箱资料- I -FS6818：程序源码\Linux 应用实验源码】目录拷贝到虚拟机共享目录【D:\share\】下。

(注意：此步骤已经在“Linux Shell 编程”章节中完成，如果没有完成请参照此章节。如已经完成可省略此步骤)

将 select IO 部分代码拷贝到虚拟机 Linux 下。

1、建立相关目录；

```

$ mkdir ~/workdir/linux/application/select_io -p
$ cd ~/workdir/linux/application/select_io

```



```
linux@ubuntu64-vm:~$ mkdir ~/workdir/linux/application/select_io -p
linux@ubuntu64-vm:~$ cd ~/workdir/linux/application/select_io
linux@ubuntu64-vm:~/workdir/linux/application/select_io$
```

2、将代码从共享目录拷入虚拟机 Linux 操作系统下；（可使用【ctrl+空格】切换输入法）

```
$ cp /mnt/hgfs/share/Linux 应用实验源码/19.\ Linux 系统 select\ IO 复用实验/实验代码
/selectecho/*.c .
$ ls
```

```
linux@ubuntu64-vm:~/workdir/linux/application/select_io$ cp /mnt/hgfs/share/Linux应用实验源码/19.
\ Linux系统select\ IO复用实验/实验代码/selectecho/*.c .
linux@ubuntu64-vm:~/workdir/linux/application/select_io$ ls
client.c select_server.c
linux@ubuntu64-vm:~/workdir/linux/application/select_io$
```

3、编译代码：

```
$ gcc client.c -o client
$ gcc select_server.c -o select_server
```

4、执行代码

首先运行服务器程序：

```
$ ./select_server 192.168.100.192 8888 //其中 ip 地址是 ubuntu 系统实际的 ip 地址(如用户未进行更
改，此处默认为 192.168.100.192)
```

另外打开一个终端（Ctrl+Alt+t），进入该目录，运行客户端程序

```
$ cd ~/workdir/linux/application/select_io
$ ./client 192.168.100.192 8888
>hello //向服务器端发送消息
n=64 buf=hello
```

```
linux@ubuntu64-vm:~$ cd ~/workdir/linux/application/select_io
linux@ubuntu64-vm:~/workdir/linux/application/select_io$ ./client 192.168.100.192 8888
>hello
n=64 buf=hello
>
```

13.4.5 实验现象

服务器所在终端会显示：

```
welcome 192.168.100.192 xxxxx
n=64 hello
```

```
linux@ubuntu64-vm:~/workdir/linux/application/select_io$ ./select_server 192.168.100.192 8888
welcome 192.168.100.192 37563
n=64 hello
```



13.4.6 实验代码

client.c

```

1#include <stdio.h>
2#include <unistd.h>
3#include <string.h>
4#include <stdlib.h>
5#include <sys/types.h>
6#include <sys/socket.h>
7#include <errno.h>
8#include <strings.h>
9#include <netinet/in.h>
10#include <arpa/inet.h>
11
12typedef struct sockaddr SA;
13#define N 64
14
15int main(int argc, char *argv[])
16{
17    int sockfd;
18    ssize_t n;
19    struct sockaddr_in servaddr;
20    char buf[N] = {0};
21
22    if (argc < 3)
23    {
24        fprintf(stdout, "usage:%s ip port\n", argv[0]);
25        exit(0);
26    }
27
28    if ((sockfd = socket(PF_INET, SOCK_STREAM, 0)) == -1)
29    {
30        perror("socket");
31        exit(-1);
32    }
33
34    bzero(&servaddr, sizeof(servaddr));
35    servaddr.sin_family = PF_INET;
36    servaddr.sin_port = htons(atoi(argv[2])); // "9000"---9000
37    servaddr.sin_addr.s_addr = inet_addr(argv[1]);
38
39    if (connect(sockfd, (SA *)&servaddr, sizeof(servaddr)) == -1)
40    {

```



```

41     perror("connect");
42     exit(-1);
43 }
44
45 printf(">");
46 while (fgets(buf, N, stdin) != NULL) //abc\n
47 {
48     buf[strlen(buf) - 1] = 0; //abc\0
49     send(sockfd, buf, N, 0);
50
51     bzero(buf, sizeof(buf));
52     n = recv(sockfd, buf, N, 0);
53     printf("n=%d buf=%s\n", n, buf);
54     printf(">");
55 }
56
57 close(sockfd);
58
59 exit(0);
60 }

```

select_server.c

```

1 #include <stdio.h>
2 #include <string.h>
3 #include <stdlib.h>
4 #include <sys/types.h> /* See NOTES */
5 #include <sys/time.h>
6 #include <unistd.h>
7 #include <sys/select.h>
8 #include <sys/socket.h>
9 #include <errno.h>
10 #include <strings.h>
11 #include <netinet/in.h>
12 #include <arpa/inet.h>
13
14 #define N 64
15 typedef struct sockaddr SA;
16
17 int main(int argc, char *argv[])
18 {
19     int listenfd, connfd, maxfd, i;
20     struct sockaddr_in servaddr, peeraddr;
21     socklen_t len;
22     char buf[N] = {0};

```



```

23     fd_set rdfs, bakrdfs;
24     ssize_t n;
25
26     if (argc < 3)
27     {
28         fprintf(stdout, "usage:%s <ip> <port>\n", argv[0]);
29         exit(0);
30     }
31
32     bzero(&servaddr, sizeof(servaddr));
33
34     if ((listenfd = socket(PF_INET, SOCK_STREAM, 0)) == -1)
35     {
36         perror("socket");
37         exit(-1);
38     }
39
40     servaddr.sin_family = PF_INET;
41     servaddr.sin_port = htons(atoi(argv[2]));
42     servaddr.sin_addr.s_addr = inet_addr(argv[1]);
43
44     if (bind(listenfd, (SA *)&servaddr, sizeof(servaddr)) == -1)
45     {
46         perror("bind");
47         exit(-1);
48     }
49
50     listen(listenfd, 5);
51     maxfd = listenfd;
52
53     FD_ZERO(&bakrdfs);
54     FD_SET(listenfd, &bakrdfs);
55
56     len = sizeof(peeraddr);
57     while (1)
58     {
59         rdfs = bakrdfs;
60
61         if (select(maxfd + 1, &rdfs, NULL, NULL, NULL) == -1)
62         {
63             perror("select");
64             exit(-1);
65         }

```



```

66
67     for (i = 0; i <= maxfd; i++)
68     {
69         if (FD_ISSET(i, &rdfs))
70         {
71             if (i == listenfd)
72             {
73                 if ((connfd = accept(i, (SA *)&peeraddr, &len)) == -1)
74                 {
75                     perror("accept");
76                     exit(-1);
77                 }
78                 fprintf(stdout, "welcome %s %d\n",
79                     inet_ntoa(peeraddr.sin_addr),
80                     ntohs(peeraddr.sin_port));
81
82                 FD_SET(connfd, &bakrdfs);
83                 maxfd = (maxfd > connfd) ? maxfd : connfd;
84             }
85             else
86             {
87                 bzero(buf, sizeof(buf));
88                 if ((n = recv(i, buf, N, 0)) == 0)
89                 {
90                     close(i);
91                     FD_CLR(i, &bakrdfs);
92                 }
93                 else
94                 {
95                     printf("n=%d %s\n", n, buf);
96                     send(i, buf, N, 0);
97                 }
98             }
99         }
100     }
101 }
102
103 exit(0);
}

```



第 14 章 Linux 设备驱动开发

任何一个计算机系统的运转都是系统软硬件共同作用的结果，没有硬件的软件是空中阁楼，而没有软件的硬件则只是一堆废铁。硬件是底层基础，是所有软件得以运行的平台，代码最终会落实为软件上的组合逻辑与时序逻辑。软件则实现了具体应用，他按照各种不同的业务需求而设计，完成用户的最终诉求。硬件较固定，软件则很灵活，可以适应各种复杂多变的应用。可以说，计算机系统的软硬件互相成就了对方。

对设备驱动最通俗的解释就是“驱使硬件设备工作”，驱动与底层硬件直接打交道，按照设备具体的工作方式，读写设备的寄存器，完成设备的轮询、中断处理、DMA 通讯，进行物理内存向虚拟内存的映射等。

14.1 内核模块

我们如何将编写的驱动包含在内核中？一种方法是把所有需要的功能都编译到 Linux 内核。这样会导致两个问题，一是生成的内核会很大，二是如果我们增加、删除、修改内核的某个功能的话，不得不重新编译整个内核，然后重新启动整个系统。这对驱动开发者来说基本上是不可接受的，因为驱动程序的特殊性，在驱动开发的初期，需要经常修改驱动的代码，即便是经验丰富的驱动开发者也是如此。

为了弥补这一缺点，Linux 引入了内核模块（后面在不至于引起混淆的情况下简称模块）。简单地说，内核模块就是被单独编译的一段内核代码，他可以在需要的时候动态地加载到内核，从而动态地增加内核的功能。在不需要的时候，可以动态地卸载，从而减少内核的功能，并节约一部分内存（这要求内核配置了模块可卸载的选项才行）。而不论是加载还是卸载，都不需要重新启动整个系统。这种特性使得他非常适合于驱动程序的开发（注意，内核模块不一定是驱动程序，驱动程序也不一定是模块的形式）。驱动开发者可以随时修改驱动的代码，然后仅仅是编译驱动代码本身（而非整个内核），并将新编译的驱动加载到内核进行测试。只要新加入的驱动不会使得内核崩溃，那么可以不用重新启动系统。

14.1.1 最简单的内核模块

```
1#include <linux/module.h>
2#include <linux/kernel.h>
3#include <linux/init.h>
4
5MODULE_LICENSE ("GPL");
6
7static int __init hello_2_init (void)//入口函数，执行 insmod 时执行
8{
9    printk (KERN_INFO "Hello world\n");
10    return 0;
11}
12
13static void __exit hello_2_exit (void)//出口函数，执行 rmmod 时执行
```



```
14{
15    printk (KERN_INFO "Goodbye world\n");
16}
17
18module_init (hello_2_init);
19module_exit (hello_2_exit);
```

这个最简单的内核模块只包含内核模块加载函数、卸载函数，用户可以通过编译生成 ko 后通过 insmod 命令加载到内核中，通过 rmmod 命令可以卸载这个模块，加载时输出 “Hello World”，卸载时输出 “Goodbye World”。

14.1.2 内核模块程序结构

一个 Linux 内核模块主要由如下几个部分组成。

(1) 模块加载函数

当通过 insmod 或 modprobe 命令加载内核模块时，模块的加载函数会自动被内核执行，完成模块的相关初始化工作。

(2) 模块卸载函数

当通过 rmmod 命令卸载某模块时，模块的卸载函数会自动被内核执行，完成与模块卸载函数相反的功能。

(3) 模块许可证声明

许可证声明描述内核模块的许可权限，如果不声明 LICENSE，模块被加载时，将收到内核被污染 (Kernel Tainted) 的警告。

在 Linux 内核模块领域，可接受的 LICENSE 包括 “GPL”、“GPL v2”、“GPL and additional rights”、“Dual BSD/GPL”、“Dual MPL/GPL”。

大多数情况下，内核模块应该遵循 GPL 兼容许可权。Linux 内核模块最常见的是以 MODULE_LICENSE(“GPL v2”)。

(4) 模块参数 (可选)

模块参数是模块被加载的时候可以传递给它的值，它本身对应模块内部的全局变量。

(5) 模块导出符号 (可选)

内核模块可以导出的符号 (symbol，对应于函数或变量)，若导出，其他模块则可以使用本模块中的变量或函数。

(6) 模块作者等信息声明 (可选)

关于这部分的其他部分我们就不多做介绍，相关的代码请参照【华清远见-嵌入式 ARM 实验箱资料-I-FS6818：程序源码\linux 驱动实验源码\基础驱动例程】。

14.1.3 模块的编译

下面是一个简单的 Makefile：

C++ Code



```

1 ifeq ($(KERNELRELEASE),)
2
3 KERNELDIR ?= /home/linux/workdir/6818/Fs6818/kernel/
4 CROSS_COMPILE = arm-linux-
5 ROOTFS ?= /source/rootfs/
6 PWD := $(shell pwd)
7
8 modules:
9     $(MAKE) CROSS_COMPILE=$(CROSS_COMPILE) -C $(KERNELDIR) M=$(PWD) modules
10
11 modules_install:
12     $(MAKE) -C $(KERNELDIR) M=$(PWD) INSTALL_MOD_PATH=$(ROOTFS) modules_install
13
14 clean:
15     rm -rf *.o *~ core .depend *.cmd *.ko *.mod.c .tmp_versions Module* modules*
16
17 .PHONY: modules modules_install clean
18
19 else
20     obj-m := hello.o
21 endif

```

代码被外层的 ifeq...else...endif 语句分为了两部分，第一部分是在 KERNELRELEASE 变量值为空的情况下执行的代码（代码第 1 行至第 16 行），第二部分则与之相反（代码第 17 行至第 20 行）。

KERNELRELEASE 是内核源码树中顶层 Makefile 文件中定义的一个变量，并对其赋值为 Linux 内核源码的版本，该变量会用 export 导出，从而可以在子 Makefile 中使用该变量。

在模块目录下执行 make 操作，将会导致 make 工具对当前目录下的 Makefile 的解释执行。即会解释执行上面的 Makefile 文件，第一次解释执行该 Makefile 时，代码第 1 行的 KERNELRELEASE 变量没有被定义，也没有被赋值，所以 ifeq 条件成立，则解释执行第一部分的内容。第一部分内容中包含了内核源码目录的变量 KERNELDIR 的定义，对应于用户的内核源码目录。ROOTFS 是根文件系统目录的变量，对应于用户的 NFS 挂载的目录。接下来是对当前的模块所在的目录的变量 PWD 定义（代码第 6 行）。Makefile 文件中的第一个目标 modules 为默认目标（代码第 7 行），执行 make 而不跟参数，则会默认生成该目标。而生成该目标就是要执行第 11 行的命令，在代码第 11 行中，\$(MAKE) 相当于 make，主要用于平台的兼容。代码第 11 行的含义是进入到内核源码目录（由“-C \$(KERNELDIR)”指定）编译在内核源码树之外的一个目录（由“M=\$(PWD)”指定）中的模块（由最后的“modules”指定）。

当编译过程折返回来（退出内核源码目录，再次进入模块目录，由“M=\$(PWD)”指定）编译模块时，上述的 Makefile 第二次被解释执行，不过这一次的情况和上一次不同，主要体现在 KERNELRELEASE 变量已经被赋值，并且被导出，导致 ifeq 条件不成立，那么将解释执行 Makefile 的第二部分，即外层 else 和 endif 之间的部分。其中 obj-m 表示将后面跟的目标编译成一个模块。相关的说明请参见内核文档中的 Kbuild 部分。



`modules_install` 目标表示把编译之后的模块安装到指定目录，安装的目录为 `$(INSTALL_MOD_PATH)/lib/modules/$(KERNELRELEASE)`，在没对 `INSTALL_MOD_PATH` 赋值的情况下，模块将会被安装到 `/lib/modules/$(KERNELRELEASE)` 目录下。关于 `modules` 和 `modules_install` 的更详细解释，请在内核源码树下执行 `make help` 命令，查看相应的帮助信息。

`clean` 目标则用于清除 `make` 生成的中间文件。

这里的 `Makefile` 只是众多 `Makefile` 中的一种写法，其他 `Makefile` 的变量设置或者结构可能存在部分差异，用户需要根据实际情况修改自己的 `Makefile` 文件。

14.1.4 内核模块的相关工具

模块相关工具及使用说明如下。

(1) 模块加载

`insmod`：加载指定目录下的一个 `.ko` 到内核。比如加载刚编译好的模块，可以用下面命令：

```
insmod hello.ko
```

(2) 模块信息

`modinfo`：查看模块的信息，在安装了模块（编译时执行 `make modules_install`）并运行 `depmod` 命令后，可以不指定路径和后缀，也可以指定查看某一特定 `.ko` 文件的模块信息，示例如下。

(3) 模块卸载

`rmmod`：如果内核配置为允许卸载模块，那么 `rmmod` 将指定的模块从内核中卸载。示例如下。其中“Goodbye World”是卸载模块时调用了模块清除函数，在模块清除函数中的打印输出。

```
[root@farsight]# insmod hello.ko
[ 2952.766000] Hello world
[root@farsight]# modinfo ./hello.ko
filename:      ./hello.ko
license:       GPL
depends:
vermagic:      3.4.39+ SMP preempt mod_unload ARMv7 p2v8
[root@farsight]# modinfo hello.ko
filename:      hello.ko
license:       GPL
depends:
vermagic:      3.4.39+ SMP preempt mod_unload ARMv7 p2v8
[root@farsight]# rmmod hello
[ 2974.060000] Goodbye world
[root@farsight]#
```

14.1.5 实验一 简单的内核模块实验

【实验原理】

模块的初始化函数和清除函数的名字是固定的，如同 C 的应用程序一样，入口函数基本上都叫 `main`。这对于追求个性化和更想表达函数真实意图的我们来说显得比较呆板了一些。幸亏内核借助于 GNU 的函数别名机制，使得我们可以更灵活地指定模块的初始化函数和清除函数的别名。



```
module_init(hello_2_init);
module_exit(hello_2_exit);
```

上面的 `module_init` 和 `module_exit` 是两个宏，分别用于指定了 `init_module` 的函数别名是 `hello_2_init`，以及 `cleanup_module` 的别名是 `hello_2_exit`。那么我们的模块初始化函数和清除函数就可以用别名来定义了。

函数名可以任意指定又带来了一个新的问题，那就是可能会和内核中已有的函数重名，因为模块的代码最终也是属于内核代码的一部分。C 没有类似于 C++ 的命名空间的概念，为了避免因为重名而带来的重复定义的问题，函数可以加 `static` 关键字修饰。经过 `static` 修饰后的函数的链接属性为内部，从而解决了该问题。这就是几乎所有的驱动程序的函数前都要加 `static` 关键字修饰的原因。

Linux 是节约内存的操作系统的典范，任何可能节约下来的内存都不会放过。上面的模块代码看上去已经足够简单了，但仔细思考，还是会发现一些可以优化的地方。模块的初始化函数有且仅会被调用一次，在调用完成后，该函数不应该被再次调用。所以该函数所占用的内存应该被释放掉，在函数名前加 `__init` 可以达到此目的。`__init` 是把标记的函数放到 ELF 文件的特定代码段，在模块加载这些段时将会单独分配内存，这些函数调用成功后，模块的加载程序会释放这部分内存空间。`__exit` 用于修饰清除函数，和 `__init` 的作用类似，但用于模块的卸载，如果模块不允许卸载，那么这段代码完全就不用加载。

【实验环境】

代码编写编译环境：ubuntu

驱动运行环境：FS6818 相关实验平台

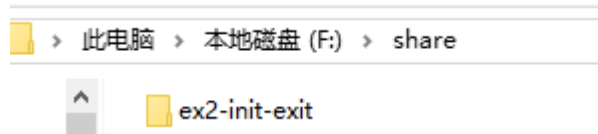
【实验目的】

通过该实验应该掌握的是内核驱动的编写方式，如何编写一个基本的驱动，如何编译一个基本的驱动，如何使用驱动，掌握之后再去编写其它进阶的驱动。

【实验步骤】

1. 按照目标机环境搭建的章节连接好硬件设备，包含串口，串口调试工具以及相应的 NFS 挂载，以下章节驱动实验均需要按照此步骤进行连接。
2. 按照 Linux 系统编译/准备源码章节拷贝内核源码。
3. 准备驱动代码

代码目录【华清远见-嵌入式 ARM 实验箱资料-I-FS6818：程序源码\linux 驱动实验源码\基础驱动例程】中的 `ex2-init-exit`。将该代码拷贝到 windows 下的共享目录文件夹下。



在虚拟机中将该代码拷贝到用户的工作目录下，并进入代码目录内。

```
$ cd /home/linux/workdir
$ cp /mnt/hgfs/share/ex2-init-exit . -a
$ cd ex2-init-exit
```

```
linux@ubuntu64-vm:~/workdir$ cp /mnt/hgfs/share/ex2-init-exit . -a
linux@ubuntu64-vm:~/workdir$ cd ex2-init-exit/
linux@ubuntu64-vm:~/workdir/ex2-init-exit$ ls
hello.c Makefile README
```

4. 修改 Makefile

用户需要进入驱动代码目录下，修改对应的 Makefile，将 Makefile 中的内核源码路径(KERNELDIR)替换为用户的内核源码目录。

```
KERNELDIR ?= /home/linux/workdir/linux-3.4.39/
```

5. 编译驱动代码

```
$ make
$ make modules_install
```

make modules_install 会将代码拷贝到 ROOTFS 变量所在目录的 lib/modules/\$(KERNELDIR)/extra 目录下，对应于 FS6818 平台目录为/source/rootfs/lib/modules/3.4.39-farsight/extra。

```
linux@ubuntu64-vm:/source/rootfs/lib/modules/3.4.39-farsight/extra$ ls
hello.ko
linux@ubuntu64-vm:/source/rootfs/lib/modules/3.4.39-farsight/extra$
```

如果用户不执行 make modules_install，手动拷贝驱动至 NFS 挂载目录。

```
$ cp hello.ko /source/rootfs/
$ cd /source/rootfs/
```

6. 加载驱动（手动拷贝的驱动需要更改为对应的路径），注意以下都是在目标板上操作。

```
# mkdir -p lib/modules/`uname -r` //注意 ` 为键盘 ESC 下方键位
# insmod /lib/modules/3.4.39-farsight/extra/hello.ko
```

```
[root@farsight]# insmod lib/modules/3.4.39-farsight/extra/hello.ko
[ 4621.570000] Hello world
[root@farsight]#
```

卸载驱动

```
# rmmod hello
```

```
[root@farsight]# rmmod hello
[ 4677.095000] Goodbye world
```

14.2 字符设备驱动

Linux 系统中根据驱动程序实现的模型框架将设备的驱动分为了三大类，这三大类驱动的特点分别如



下。

(1) 字符设备驱动：设备对数据的处理是按照字节流的形式进行的，可以支持随机访问，也可以不支持随机访问，因为数据流量通常不是很大，所以一般没有页高速缓存。典型的设备如串口、键盘、帧缓存设备等。以串口为例，串口对收发的数据长度没有具体要求，可以是任意多个字节；串口也不支持 lseek 操作，即不能定位到一个具体的位置进行读写，因为串口按顺序发送或接收数据；串口的数据通常保存在一个较小的 FIFO 中，并且不会重复利用 FIFO 中的数据。再者，帧缓存设备（就是我们通常说的显卡），也是一个字符设备，但他可以进行随机访问，这样我们就能修改某个具体位置的帧缓存数据，从而改变屏幕上的某些确定像素点的颜色。

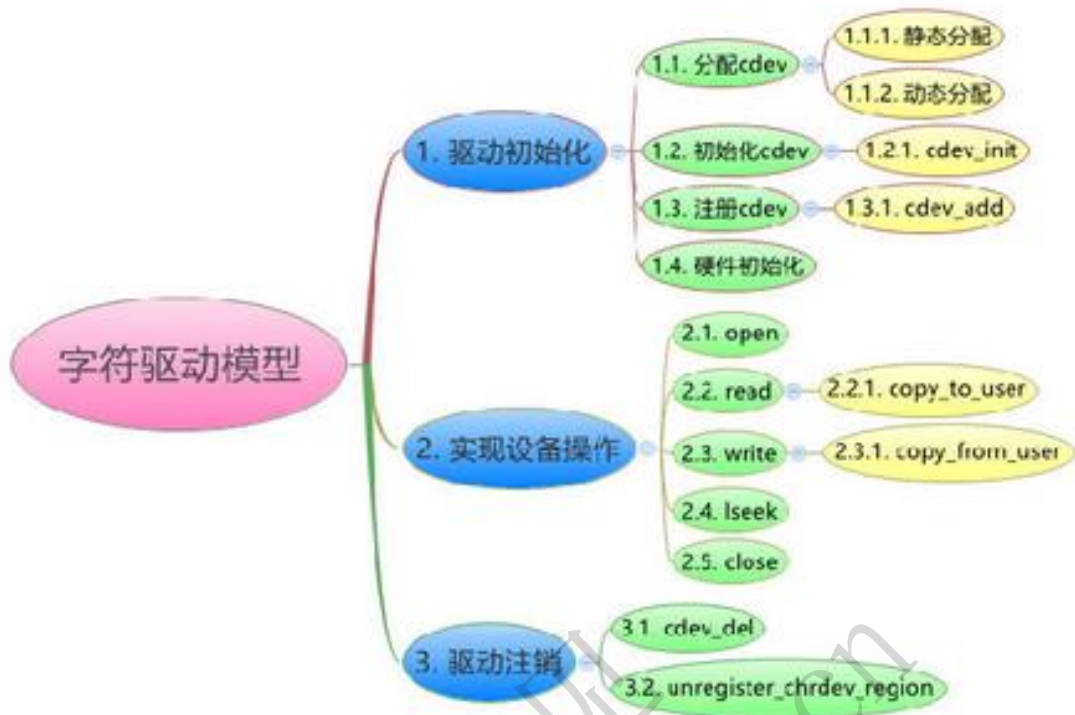
(2) 块设备驱动：设备对数据的处理是按照若干个块进行的，一个块有其固定的大小，比如 4096 字节，那么每次读写的数据就至少是 4096 个字节。这类设备都支持随机访问，并且为了提高效率，可以将之前用到的数据缓存起来，以便下次使用。典型的设备如硬盘、光盘、SD 卡等。以硬盘为例，一个硬盘的最小访问单位是一个扇区，一个扇区通常是 512 字节，那么块的大小就至少是 512 字节。我们可以访问硬盘中的任何一个扇区，也就是说硬盘支持随机访问。因为硬盘的访问速度非常慢，如果每次都去硬盘上获取数据，那么效率会非常低，所以一般将之前从硬盘上得到的数据放在一个叫做页高速缓存的内存中，如果程序要访问的数据是之前访问过的，那么程序会直接从页高速缓存中获得数据，从而来提高效率。

(3) 网络设备驱动：顾名思义，他就是专门针对网络设备的一类驱动，其主要目的是进行网络数据的收发。

以上驱动程序的分类是按照驱动模型框架进行分类的，现实生活中有的设备很难被严格界定为字符设备还是块设备。甚至有的设备同时具有两类驱动，如 MTD（存储技术设备，如闪存）。一个设备的驱动选择上述三类中的哪一类，还要看具体的使用场合和最终的用途。

14.2.1 字符设备驱动框架

字符设备驱动模型如下图所示



字符设备驱动的开发主要分这几步：

1) 驱动初始化

a) 分配 cdev

cdev 结构体主要用来描述字符设备，在驱动中分配 cdev，主要是分配一个 cdev 结构体与申请设备号。

```

1 struct cdev {
2     struct kobject kobj;
3     struct module *owner; /*通常为 THIS_MODULE*/
4     struct file_operations *ops; /*在 cdev_init()这个函数里面与 cdev 结构联系起来*/
5     struct list_head list;
6     dev_t dev; /*设备号*/
7     unsigned int count;
8 };
    
```

其中 cdev 结构体的 dev_t 成员定义了设备号，为 32 位，其中高 12 位为主设备号，低 20 位为次设备号。

b) 初始化 cdev

cdev_init()函数用于初始化 cdev 的成员，并建立 cdev 和 file_operations 之间的连接。

```

1 void cdev_init(struct cdev *, struct file_operations *);
    
```

c) 注册 cdev

cdev_add()函数向系统添加一个 cdev，完成字符设备的注册。

```

1 int cdev_add(struct cdev *, dev_t, unsigned);
    
```

d) 硬件初始化

硬件初始化主要是硬件资源的申请与配置。



2) 实现设备操作

用户空间的程序以访问文件的形式访问字符设备，通常进行 `open`、`read`、`write`、`close` 等系统调用。而这些系统调用的最终落实则是 `file_operations` 结构体中成员函数，它们是字符设备驱动与内核的接口。

a) `Open()`函数

```
1 int(*open)(struct inode *, struct file*);
```

b) `read()`函数

用来从设备中读取数据，成功时函数返回读取的字节数，出错时返回一个负值

```
1 ssize_t(*read)(struct file *, char __user*, size_t, loff_t*);
```

c) `write()`函数

向设备发送数据，成功时该函数返回写入的字节数。如果此函数未被实现，当用户进行 `write()` 系统调用时，将得到 `-EINVAL` 返回值

```
1 ssize_t(*write)(struct file *, const char __user *, size_t, loff_t*);
```

d) `close()`函数

关闭

```
1 int(*release)(struct inode *, struct file*);
```

e) 补充说明

- f) 在 Linux 字符设备驱动程序设计中，有 3 种非常重要的数据结构：`struct file`、`struct inode`、`struct file_operations`。`struct file` 代表一个打开的文件。系统中每个打开的文件在内核空间都有一个关联的 `struct file`。它由内核在打开文件时创建，在文件关闭后释放。其成员 `loff_t f_pos` 表示文件读写位置。`struct inode` 用来记录文件的物理上的信息。因此，它和代表打开文件的 `file` 结构是不同的。一个文件可以对应多个 `file` 结构，但只有一个 `inode` 结构。其成员 `dev_t i_rdev` 表示设备号。`struct file_operations` 一个函数指针的集合，定义能在设备上进行的操作。结构中的成员指向驱动中的函数，这些函数实现一个特别的操作，对于不支持的操作保留为 `NULL`。

- i. 在 `read()` 和 `write()` 中的 `buff` 参数是用户空间指针。因此，它不能被内核代码直接引用，因为用户空间指针在内核空间时可能根本是无效的——没有那个地址的映射。因此，内核提供了专门的函数用于访问用户空间的指针：
- ii. `f_pos` 表示文件读写位置。`struct inode` 用来记录文件的物理上的信息。因此，它和代表打开文件的 `file` 结构是不同的。一个文件可以对应多个 `file` 结构，但只有一个 `inode`

```
1 unsigned long copy_from_user(void *to, const void __user *from, unsigned long count);
```

```
2 unsigned long copy_to_user(void __user *to, const void *from, unsigned long count);
```

结构。其成员 `dev_t i_rdev` 表示设备号。`struct file_operations` 一个函数指针的集合，定义能在设备上进行的操作。结构中的成员指向驱动中的函数，这些函数实现一个特别的操作，对于不支持的操作保留为 `NULL`。

- iii. 在 `read()` 和 `write()` 中的 `buff` 参数是用户空间指针。因此，它不能被内核代码直接引用，因为用户空间指针在内核空间时可能根本是无效的——没有那个地址的映射。因此，内核提供了专门的函数用于访问用户空间的指针：

3) 驱动注销

a) 删除 `cdev`

在字符设备驱动模块卸载函数中通过 `cdev_del()` 函数向系统删除一个 `cdev`，完成字符设



备的注销。

```
1 void cdev_del(struct cdev *);
```

b) 释放设备号

在调用 `cdev_del()` 函数从系统注销字符设备之后，`unregister_chrdev_region()` 应该被调用以释放原先申请的设备号。

```
1 void unregister_chrdev_region(dev_t from, unsigned count)
```

14.2.2 实验二 字符设备驱动编写

【实验原理】

要实现一个字符设备驱动，其中最重要的事就是要构造一个 `cdev` 结构对象，并让 `cdev` 同设备号和设备的操作方法集合相关联，然后将该 `cdev` 结构对象添加到内核的 `cdev_map` 散列表中。在模块的初始化函数中，首先使用 `MKDEV` 宏将主设备号和次设备号合并成一个设备号。构造好设备号之后，代码第 18 行调用 `register_chrdev_region` 将构造的设备号注册到内核中，表明该设备号已经被占用，如果有其他驱动随后要注册该设备号，将会失败。其函数原型如下。

```
int register_chrdev_region(dev_t from, unsigned count, const char *name);
```

该函数一次性可以注册多个连续的号，由 `count` 形参指定个数，由 `from` 指定起始的设备号，`name` 用于标记主设备号的名称。该函数成功返回 0，不成功返回负数，通常是在设备号已经被其他的驱动抢先注册的情况。如果注册出错，则使用 `goto` 语句跳转到错误处理代码处执行，否则初始化函数返回 0。使用 `goto` 函数进行集中错误处理在驱动中非常常见，也非常实用，虽然这和一般的 C 语言编程规则相悖。

在模块卸载时，已注册的号应该从内核中注销，否则再次加载该驱动时，注册设备号操作会失败。代码中调用了 `unregister_chrdev_region` 函数，该函数只有两个形参，和 `register_chrdev_region` 函数的前两个形参的意义一样。

【实验环境】

代码编写编译环境：ubuntu

驱动运行环境：FS6818 相关实验平台

【实验目的】

在掌握了编写基本的模块驱动的前提下，用户需要了解内核中最常见的驱动种类-字符设备驱动，如何创建字符设备，如何操作（读写控制）字符设备，只有用户熟练掌握这一部分后，才能编写基本的设备驱动，例如 LED 灯驱动。



【实验步骤】

系统源码准备、拷贝驱动代码到已有共享目录、修改 Makefile 请参考【简单的内核模块实验】，这里不再进行赘述。

驱动代码为【华清远见-嵌入式 ARM 实验箱资料- I -FS6818：程序源码\linux 驱动实验源码\基础驱动例程】中的 ex7-char-dev-register。

1. 拷贝驱动代码到用户虚拟机工作目录下

```
$ cd /home/linux/workdir
```

```
$ cp /mnt/hgfs/share/ex7-char-dev-register . -a
```

```
$ cd ex7-char-dev-register
```

```
linux@ubuntu64-vm:~/workdir$ cp /mnt/hgfs/share/ex7-char-dev-register . -a
linux@ubuntu64-vm:~/workdir$ cd ex7-char-dev-register/
linux@ubuntu64-vm:~/workdir/ex7-char-dev-register$ ls
char-dev-reg.c  Makefile  README
```

2. 修改完 Makefile 之后，编译驱动

```
$ make
```

```
$ make modules_install
```

```
linux@ubuntu64-vm:~/workdir/ex7-char-dev-register$ make
make CROSS_COMPILE=arm-linux- -C /home/linux/workdir/6818/Fs6818/kernel/ M=/home/linux/w
make[1]: 正在进入目录 `/home/linux/workdir/6818/Fs6818/linux/kernel/kernel-3.4.39'
CC [M] /home/linux/workdir/ex7-char-dev-register/char-dev-reg.o
Building modules, stage 2.
MODPOST 1 modules
CC /home/linux/workdir/ex7-char-dev-register/char-dev-reg.mod.o
LD [M] /home/linux/workdir/ex7-char-dev-register/char-dev-reg.ko
make[1]:正在离开目录 `/home/linux/workdir/6818/Fs6818/linux/kernel/kernel-3.4.39'
linux@ubuntu64-vm:~/workdir/ex7-char-dev-register$ make modules_install
make -C /home/linux/workdir/6818/Fs6818/kernel/ M=/home/linux/workdir/ex7-char-dev-regis
make[1]: 正在进入目录 `/home/linux/workdir/6818/Fs6818/linux/kernel/kernel-3.4.39'
INSTALL /home/linux/workdir/ex7-char-dev-register/char-dev-reg.ko
DEPMOD 3.4.39-farsight
make[1]:正在离开目录 `/home/linux/workdir/6818/Fs6818/linux/kernel/kernel-3.4.39'
```

3. 加载驱动，注意以下都是在目标板上操作。

```
# insmod /lib/modules/3.4.39-farsight/extra/char-dev-reg.ko
```

生成设备文件(主次设备号需与驱动中一致)

```
# mknod /dev/hello c 255 0
```

```
[root@farsight ]# insmod /lib/modules/3.4.39-farsight/extra/char-dev-reg.ko
[ 1914.739000] Registered character driver
[root@farsight ]# mknod /dev/hello c 255 0
[root@farsight ]# ls /dev/hello
/dev/hello
```

如果出现用户使用的主设备号无法使用，请使用如下命令查看当前系统中已使用的主设备号，如有已经使用的请修改程序源码中的主设备号。

```
# cat /proc/devices
```

4. 卸载驱动



```
# rmmod char-dev-reg
```

```
[root@farsight]# rmmod char-dev-reg
[ 9790.359000] char driver cleaned up
```

14.3 GPIO 驱动开发

根据前面的章节我们了解到驱动的种类有三种，平时编写的一般都属于字符类设备驱动，那么在字符设备驱动中又根据操作的设备不同分为 GPIO、PWM、ADC、中断、I2C 等等，这里我们先讲一下 GPIO 类的字符设备驱动编写的方式。

14.3.1 什么是 GPIO？

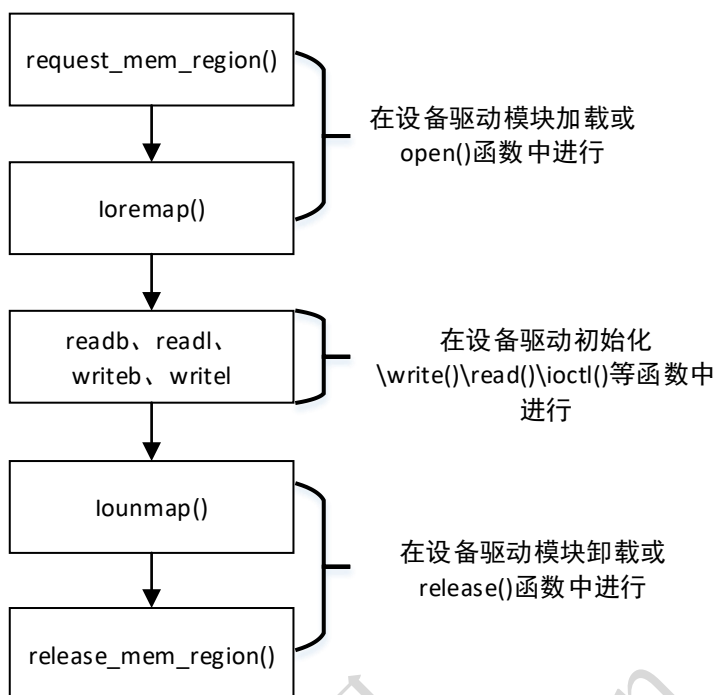
GPIO，英文全称为 General-Purpose IO ports，也就是通用 IO 口。在嵌入式系统中常常有数量众多，但是结构却比较简单的外部设备/电路，对这些设备/电路有的需要 CPU 为之提供控制手段，有的则需要被 CPU 用作输入信号。而且，许多这样的设备/电路只要求一位，即只要有开/关两种状态就够了，比如灯亮与灭。对这些设备/电路的控制，使用传统的串行口或并行口都不合适。所以在微控制器芯片上一般都会提供一个“通用可编程 IO 接口”，即 GPIO。接口至少有两个寄存器，即“通用 IO 控制寄存器”与“通用 IO 数据寄存器”。数据寄存器的各位都直接引到芯片外部，而对数据寄存器中每一位的作用，即每一位的信号流通方向时输入还是输出，则可以通过控制寄存器中对应位独立的加以设置。

14.3.2 如何操作 GPIO？

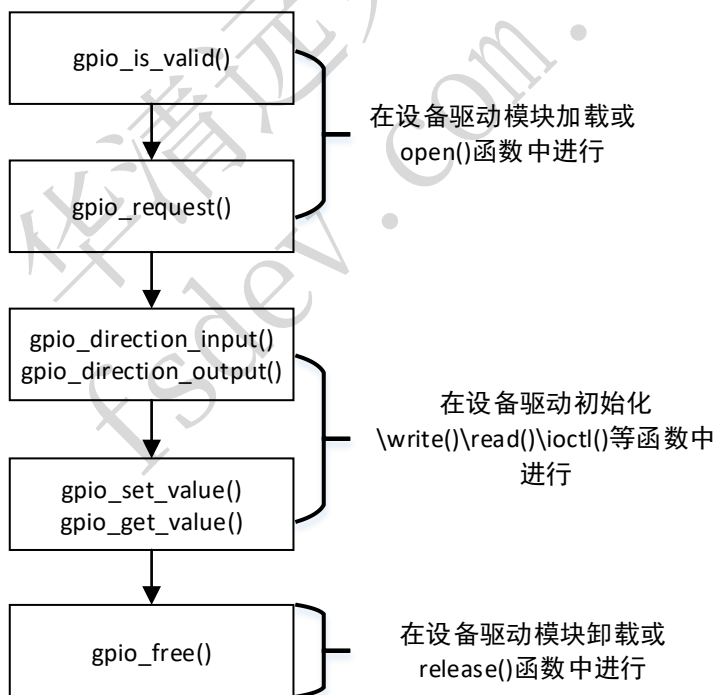
GPIO 端口都会有至少两个寄存器，一类是控制寄存器，一类是数据寄存器，通过控制寄存器可以将 GPIO 口配置为需要的功能（输入输出或者复用功能等），通过数据寄存器可以设置引脚的电平或者获取引脚的电平。

在驱动的开发过程中，有多种方式可以直接操作相应的 IO 引脚，这里我们介绍两种比较常用的方式。

- 获取 IO 的物理寄存器地址后，通过内核中提供的接口将物理地址映射到内核空间，直接按位对映射后的地址进行操作，这种操作比较考验编程人员的位操作熟练度，同时如果操作不恰当，可能会误修改到其它位的值，从而影响改变其它 IO 的状态。



- Linux 内核提供了一个框架统一处理 gpio，使得编程人员能够使用统一的接口来操作 gpio，这里先介绍通用的 gpio 操作函数（这些函数在 2.6 之后的内核中都是可以使用的）。



每一个厂商还会提供一些自定义的 IO 函数供用户使用，这些函数的声明在 [源码] /kernel/arch/arm/mach-s5p6818/include/mach/soc.h 中，实现在 [源码] /kernel/arch/arm/mach-s5p6818/soc/gpio.c 中。

这两种函数理论上都可以使用，前者具有通用性，后者有局限性，但后者使用起来更加方便，用户可以根据自己的需求进行选择。



14.3.3 实验三 LED 灯驱动开发实验

【实验原理】

FS6818 平台板载一个 RGB 三色灯，分别与三个 IO 引脚相连，每一个 IO 引脚控制一种色彩，编程人员需要根据原理图得出硬件与 CPU 相连的引脚，并通过芯片手册得到该引脚的控制位及数据位，获得之后通过如上的两种方式进行相应的 IO 操作。

原理图及芯片手册的路径如下，以后的实验不再赘述。

原理图【华清远见-嵌入式 ARM 实验箱资料- I -FS6818：文档资料\原理图】下的 FS6818 底板原理图与核心板原理图

芯片手册【华清远见-嵌入式 ARM 实验箱资料- I -FS6818：文档资料\芯片手册\FS6818 板级芯片手册】下的 S5P6818X 用户手册

华清远见
fsdev.com.cn

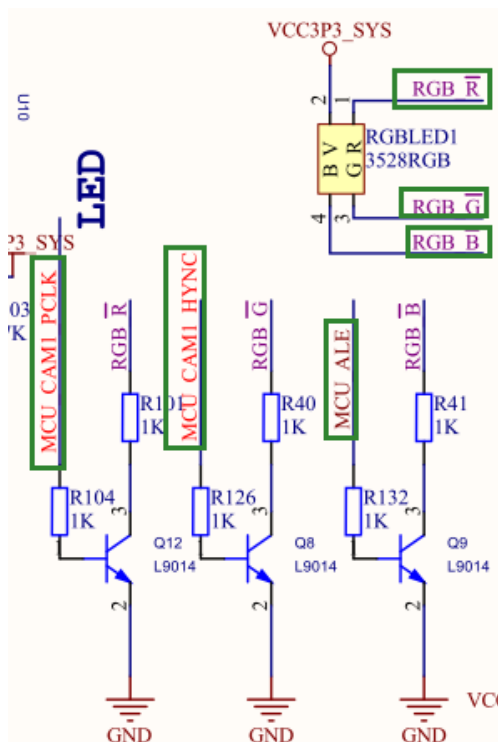


图 1 底板 RGB 连接

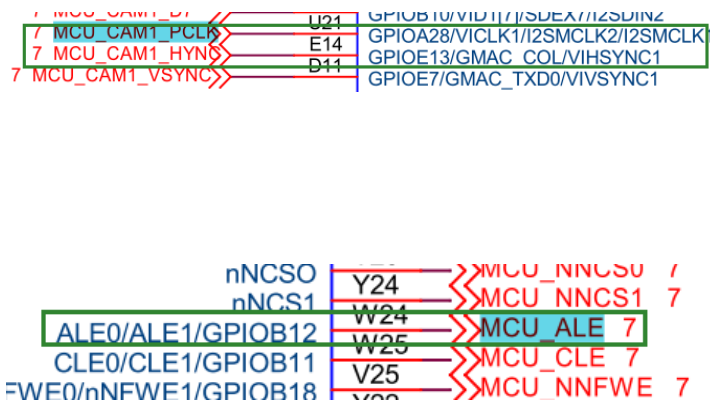


图 2 底板与核心板连接引脚

通过如上的底板 RGB 连接原理分析得知，灯的亮灭是由三极管是否导通控制的，当 MCU_CAM1_PCLK 为高电平时三极管导通，此时 RGB_R 引脚为低电平，此时 RGBLED1 中的 R 导通，此时红灯亮，相反则灭。由右侧的底板与核心板的连接图可以得知 RGB 的 R、G、B 引脚分别与 GPIOA28、GPIOE13、GPIOB12 相连，那么此时我们只需要控制相应的 IO 引脚输出高低电平即可控制灯的色彩。

底板引脚	核心板引脚	内核 IO 声明
MCU_CAM1_PCLK	GPIOA28	PAD_GPIO_A + 28
MCU_CAM1_HYNC	GPIOE13	PAD_GPIO_E + 13
MCU_ALE	GPIOB12	PAD_GPIO_B + 12

该实验的驱动中控制 IO 引脚的方式采用如上的第一种方式。

【实验环境】

代码编写编译环境：ubuntu

驱动运行环境：FS6818 相关实验平台



【实验目的】

实验人员需要了解如何通过原理图获得对应设备的控制方式，如何通过芯片手册查找相应的 IO 引脚，熟练掌握内核驱动中操作 IO 寄存器的方法。

【实验步骤】

系统源码准备、拷贝驱动代码到已有共享目录、修改 Makefile 请参考“简单的内核模块实验”，这里不再赘述。

驱动代码为【华清远见-嵌入式 ARM 实验箱资料- I -FS6818：程序源码\Linux 驱动实验源码\设备驱动例程】中的 fs6818_led。

1. 拷贝驱动代码到用户虚拟机工作目录下

```
$ cd /home/linux/workdir
$ cp /mnt/hgfs/share/fs6818_led . -a
$ cd fs6818_led
```

```
linux@ubuntu64-vm:~/workdir$ cp /mnt/hgfs/share/fs6818_led . -a
linux@ubuntu64-vm:~/workdir$ cd fs6818_led/
linux@ubuntu64-vm:~/workdir/fs6818_led$ ls
fs6818_led.c fs6818_led.h led_test Makefile test.c
```

2. 修改完 Makefile 之后，编译驱动

```
$ make
$ make modules_install
```

```
linux@ubuntu64-vm:~/workdir/fs6818_led$ make
make CROSS_COMPILE=arm-linux- -C /home/linux/workdir/6818/Fs6818/kernel/ M=/home/linux/workdir/6818/Fs6818/linux/kernel/kernel-3.4.39'
make[1]: 正在进入目录 `/home/linux/workdir/6818/Fs6818/linux/kernel/kernel-3.4.39'
CC [M] /home/linux/workdir/fs6818_led/fs6818_led.o
Building modules, stage 2.
MODPOST 1 modules
CC /home/linux/workdir/fs6818_led/fs6818_led.mod.o
LD [M] /home/linux/workdir/fs6818_led/fs6818_led.ko
make[1]:正在离开目录 `/home/linux/workdir/6818/Fs6818/linux/kernel/kernel-3.4.39'
linux@ubuntu64-vm:~/workdir/fs6818_led$ make modules_install
make -C /home/linux/workdir/6818/Fs6818/kernel/ M=/home/linux/workdir/fs6818_led INSTALL=/home/linux/workdir/fs6818_led/fs6818_led.ko
DEPMOD 3.4.39-farsight
make[1]:正在离开目录 `/home/linux/workdir/6818/Fs6818/linux/kernel/kernel-3.4.39'
```

3. 编译测试使用的应用程序

```
$ arm-linux-gcc test.c -o led_test
```

4. 拷贝应用程序到目标板文件系统

```
$ cp led_test /source/rootfs
```

5. 加载驱动，注意以下都是在目标板上操作。

```
# insmod /lib/modules/3.4.39-farsight/extra/fs6818_led.ko
```



```
[root@farsight]# insmod /lib/modules/3.4.39-farsight/extra/fs6818_led.ko
[12397.427000] Led init
```

生成设备文件(主次设备号需与驱动中一致)

```
# mknod /dev/led c 500 0
```

6. 执行测试程序

```
# ./led_test
```

执行测试程序后观察 RGB 灯，红灯、绿灯、蓝灯交替闪烁。

7. 卸载驱动

```
# rmmod fs6818_led
```

14.4 PWM 定时器

定时器 / 计数器简称定时器，其作用主要包括产生各种时标间隔、记录外部事件的数量等，是微机中最常用、最基本的部件之一。

S5P6818 有 5 个 32 位的脉冲宽度调制定时器。这些定时器可以产生 ARM 子系统的内部中断。另外，其中定时器 0、1、2、3 带有作为驱动外部 I/O 信号的脉宽调制功能，Timer0 的 PWM 有一个可选的死区发生器可以控制大电流设备。Timer4 仅供内部定时没有接任何输出引脚。

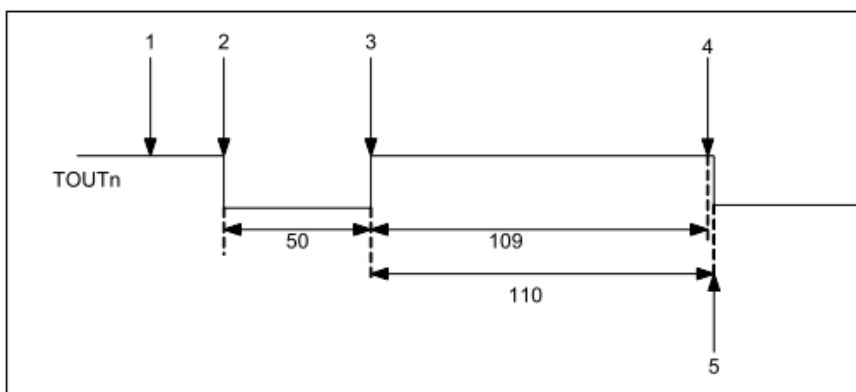
定时器 0 与定时器 1 共用一个 8bit 预分频器，定时器 2、定时器 3 与定时器 4 共用另一个 8bit 预分频器，每个定时器都有一个时钟分频器，时钟分频器有 5 种分频输出（1，1/2，1/4，1/8，1/16）。每个定时器都从时钟分频器接收的时钟信号，时钟分频器从相应的 8bit 预分频器接收时钟信号。可编程 8bit 预分频器根据存储在 TCFG0 和 TCFG1 中的数据对 PCLK 进行分频。

当时钟被使能后，定时器计数缓冲寄存器（TCNTBn）把计数初值下载到递减计数器中。定时器比较缓冲寄存器（TCMPBn）把其初始值下载到比较寄存器中，并将该值和递减计数器的值进行比较。这种基于 TCNTBn 和 TCMPBn 的双缓冲特性使定时器在频率和占空比变化时产生稳定的输出。

每个定时器都有一个专用的由定时器时钟驱动的 32 位递减计数器。当递减计数器的计数值达到 0 的时候，就会产生定时器中断请求来通知 CPU 定时器操作完成。当定时器递减计数器达到 0 的时候相应的 TCNTBn 的值会自动重载到递减计数器中以继续下次操作。然而，如果定时器停止了，比如在定时器运行时清除 TCON 中的定时器使能位，TCNTBn 的值不会被重载到递减计数器中。

TCMPBn 的值用于脉冲宽度调制（PWM）。当定时器的递减计数器的值和比较寄存器的值相匹配的时候，定时器控制逻辑将改变输出电平。因此，比较寄存器决定了 PWM 输出的开关时间。

一个 PWM 波形的占空比控制如下：

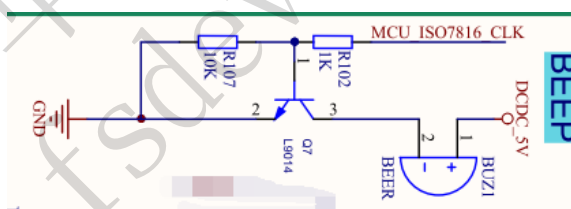


PWM 管脚默认输出高电平，在时刻 1 将递减计数器计数值设为 159，比较计数器比较值设为 109，在时刻 2 启动定时器，PWM 立即输出低电平，在时钟的作用下，计数器开始做减法计数，当计数值减来和比较值一致时（时刻 3），输出翻转，之后一直输出高电平。最后当计数到达 0 后（时刻 4），再完成一次计数，在时刻 5 重新从 159 开始计数，输出再次变成低电平，如此周而复始就形成一个矩形波。波形的周期由计数值决定，占空比由比较值和计数值的比值决定。

【实验原理】

FS6818 平台板载了一个无源蜂鸣器，无源蜂鸣器没有内部驱动电路，工作的理想信号是方波。如果给预直流信号蜂鸣器是不响应的，因为磁路恒定，钼片不能振动发音。在实例中，把驱动方式给为交流驱动（PWM 输出）控制。通过控制 pwm 输出不同的方波来调节蜂鸣器的响度。

硬件连接如下：



蜂鸣器硬件连接图

底板引脚	核心板引脚
MCU_ISO7816_CLK	PWM2

编程人员通过配置 pwm2 输出方波即可。

【实验环境】

代码编写编译环境：ubuntu

驱动运行环境：FS6818 相关实验平台



【实验目的】

掌握如何操作 pwm，如何使 pwm 控制器输出不同占空比波形。

【实验步骤】

系统源码准备、拷贝驱动代码到已有共享目录、修改 Makefile 请参考“简单的内核模块实验”，这里不再赘述。

驱动代码为【华清远见-嵌入式 ARM 实验箱资料- I -FS6818：程序源码\Linux 驱动实验源码\设备驱动例程】中的 fs6818_pwm

1. 拷贝驱动代码到用户虚拟机工作目录下

```
$ cd /home/linux/workdir
$ cp /mnt/hgfs/share/fs6818_pwm -a
$ cd fs6818_pwm
```

2. 修改完 Makefile 之后，编译驱动

```
$ make
$ make modules_install
```

3. 编译测试使用的应用程序

```
$ arm-linux-gcc pwm_music.c -o pwm_music
```

4. 拷贝应用程序到目标板文件系统

```
$ cp pwm_music /source/rootfs
```

5. 加载驱动，注意以下都是在目标板上操作。

```
# insmod /lib/modules/3.4.39-farsight/extra/fs6818_pwm.ko
```

6. 执行测试程序

```
# ./pwm_music
```

执行程序后可以听到蜂鸣器播放音乐

7. 卸载驱动

```
# rmmod fs6818_pwm
```

14.5 patform 设备驱动

在前面的基础上，我们已经能够开发一个功能较完备的字符设备驱动了，但是仔细思考这个驱动，还是会发现一些不足，比如存在下面的一些主要问题。

(1) 设备和驱动没有分离，也就是说设备的信息是硬编码在驱动代码中的，这给驱动程序造成了极大的限制。如果硬件有所改动，那么必然要修改驱动代码（比如对于前面的 LED 驱动，硬件上修改 LED 连接的管脚），这样驱动的通用性将会非常的差。这是最突出的一个问题，必须要很好地解决。

(2) 没有类似于 Windows 系统中的设备管理器，可以方便查看设备和驱动的信息。



- (3) 不能自动创建设备节点。
- (4) 驱动不能自动加载。
- (5) U 盘 SD 卡等不能自动挂载。
- (6) 没有电源管理。

为了解决这些问题，Linux 有相应的设备模型，需要有总线、设备和驱动来描述这一模型，但是有的设备并没有对应的物理总线，比如 LED、RTC 和蜂鸣器等。为此，内核专门开发了一种虚拟总线——platform 总线，用来连接这些没有物理总线的设备或者一些不支持热插拔的设备，以下的 adc 设备驱动使用了这种方式。

14.5.1 平台设备

平台设备是用 struct platform_device 结构来表示的，他的定义如下。

```
struct platform_device {
    const char      *name;
    int             id;
    bool            id_auto;
    struct device    dev;
    u32             num_resources;
    struct resource  *resource;

    const struct platform_device_id *id_entry;

    /* MFD cell pointer */
    struct mfd_cell *mfd_cell;

    /* arch specific additions */
    struct pdev_archdata archdata;
};
```

驱动开发者关心的主要成员如下。

name: 设备的名字，在平台总线的 match 函数中可用于同平台驱动的匹配。

id: 设备的 ID 号，用于区别同类型的不同平台设备。

dev: 内嵌的 struct device。

num_resources: 平台设备使用的资源个数。

resource: 平台设备的资源列表（数组），指向资源数组中的首元素。

id_entry: 用于同平台驱动匹配的 ID，在平台总线的 match 函数中首先尝试匹配该 ID，如果不成功再尝试用 name 成员来匹配。

在平台设备中，最关键的就是设备使用的资源信息的描述，他是实现设备和驱动分离的关键。struct resource 的定义如下。



```
struct resource {
    resource_size_t start;
    resource_size_t end;
    const char *name;
    unsigned long flags;
    struct resource *parent, *sibling, *child;
};
```

驱动开发者关心的主要成员如下。

start: 资源的开始，对于 I/O 内存就是起始地内存地址，对于中断资源就是起始地中断号，对于 DMA 资源就是起始地 DMA 通道号。

end: 资源的结束。

flags: 资源的标志，定义在“include/linux/ioport.h”文件中，最常见如下。

IORESOURCE_MEM: 表示资源的类型是内存资源，也包括 I/O 内存。

IORESOURCE_IRQ: 表示资源的类型是中断资源。

IORESOURCE_DMA: 表示资源的类型是 DMA 通道资源。

资源可以组成一个树形结构，由成员 **parent**、**sibling** 和 **child** 来完成。

对于平台设备及其资源通常存在于 BSP（Board Support Package，板级支持包）文件中，该文件通常包含和目标板相关的一些代码，如下是 adc 平台设备及资源。

```
1 #ifndef CONFIG_ADC_SAMPLE_RATE
2 #define CONFIG_ADC_SAMPLE_RATE    200000 /* 200K */
3 #endif
4 static struct resource fs6818_adc_resource[] = {
5     [0] = DEFINE_RES_MEM(PHY_BASEADDR_ADC, SZ_1K),
6     [1] = DEFINE_RES_IRQ(IRQ_PHY_ADC),
7 };
8 static struct platform_device fs6818_adc_device = {
9     .name          = "fs6818-adc",
10    .id             = -1,
11    .num_resources  = ARRAY_SIZE(fs6818_adc_resource),
12    .resource       = fs6818_adc_resource,
13    .dev            = {
14        .platform_data = CONFIG_ADC_SAMPLE_RATE,
15    },
16};
```

向平台总线注册和注销的平台设备的主要函数如下。

```
int platform_add_devices(struct platform_device **devs, int num);
int platform_device_register(struct platform_device *pdev);
void platform_device_unregister(struct platform_device *pdev);
```

platform_add_devices 用于一次性注册多个平台设备，**platform_device_register** 一次性只注册一个平台



设备。其实，platform_add_devices 是通过多次调用 platform_device_register 来实现的。

platform_device_unregister 用于注销平台设备。

adc 平台设备注册方法。

```
1 platform_device_register(&fs6818_adc_device);
```

当平台总线发现有和平台设备匹配的驱动时，平台总线会调用平台驱动内的一个函数，并传递匹配的平台设备结构地址，从而平台驱动可以从中获取设备的资源信息。关于资源操作的主要函数如下。

```
struct resource *platform_get_resource(struct platform_device *dev, unsigned int type,
unsigned int num);
resource_size_t resource_size(const struct resource *res);
```

platform_get_resource: 从平台设备 dev 中获取类型为 type，序号为 num 的资源。

resource_size: 返回资源的大小，其值为 end - start + 1。

```
1 //获得 platform_device 中描述的地址资源
2 if (!(adc_dev->mem_res = platform_get_resource(pdev,
3     IORESOURCE_MEM, 0))) {
4     err = -ENODEV;
5     goto err1;
6 }
7 //获得 platform_device 中描述的中断资源
8 if (!(adc_dev->irq_res = platform_get_resource(pdev,
9     IORESOURCE_IRQ, 0))) {
10     err = -ENODEV;
11     goto err1;
12 }
```

14.5.2 平台驱动

平台驱动是用 struct platform_driver 结构来表示的，他的定义如下。

```
struct platform_driver {
    int (*probe)(struct platform_device *);
    int (*remove)(struct platform_device *);
    void (*shutdown)(struct platform_device *);
    int (*suspend)(struct platform_device *, pm_message_t state);
    int (*resume)(struct platform_device *);
    struct device_driver driver;
    const struct platform_device_id *id_table;
    bool prevent_deferred_probe;
};
```

驱动开发者关心得主要成员如下。

probe: 总线发现有匹配的平台设备时调用。

remove: 所驱动的平台设备被移除时或平台驱动注销时调用。



shutdown、suspend 和 resume：电源管理函数，在要求设备掉电、挂起和恢复时被调用。内嵌的 struct device_driver 的 pm 成员也有对应的电源管理函数。

id_table：平台驱动可以驱动的平台设备 ID 列表，可用于和平台设备匹配。

向平台总线注册和注销的平台驱动的主要函数如下。

```
platform_driver_register(drv)
void platform_driver_unregister(struct platform_driver *);
```

adc 驱动注册方法

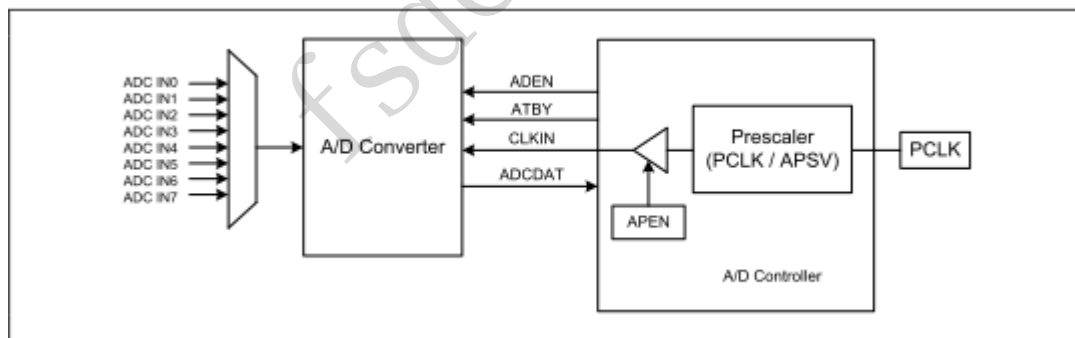
```
1 platform_driver_register(&adc_driver);
```

14.5.3 ADC 驱动编写

【试验原理】

A/D 转换又称模数转换，顾名思义，就是把模拟信号数字化。实现该功能的电子器件称为 A/D 转换器，A/D 转换器可将输入的模拟电压转换为与其成比例输出的数字信号。随着数字技术，特别是计算机技术的飞速发展与普及，在现代控制、通信及检测领域中，对信号的处理广泛采用了数字计算机技术。由于系统的实际处理对象往往都是一些模拟量（如温度、压力、位移、图像等），要使计算机或数字仪表能识别和处理这些信号，必须首先将这些模拟信号转换成数字信号，这就必须用到 A/D 转换器。

S5P6818 提供了一个 28nmCMOS 模拟数字转换器，它具有 8 个输入通道，并可将模拟量转换为 12 位的二进制数，最大的转换的速率为 1MSPS，输入电压的范围为 0 ~ 1.8v，编程人员通过配置相应的寄存器后进行 adc 数据的采集。

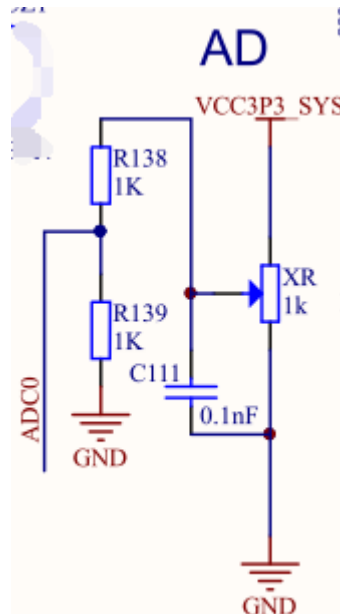


S5P6818 ADC 框图

FS6818 平台板载一个电位器，该电位器可以通过 adc 控制器的通道 0 采集 0-1.8v 电压，经过 adc 控制器转换后得到实际的电压值。该 adc 控制器中的 ADCDAT 寄存器存储了经过 adc 转换后的数据，该数据范围为 0 ~ 2¹²，即 0-4096，电压转化公式为 $X = \text{ADCDAT} * 1.8 / 4096$ ，转化后的 X 就是当前采集到的实际电压值。



硬件连接



电位器连接图

【试验环境】

代码编写编译环境：ubuntu

驱动运行环境：FS6818 相关实验平台

【试验目的】

通过对芯片中 adc 控制器的学习，熟练掌握 linux 内核中 adc 类设备的驱动开发方法，同时能够掌握 platform 平台设备框架。

【实验步骤】

系统源码准备、拷贝驱动代码到已有共享目录、修改 Makefile 请参考“简单的内核模块实验”，这里不再赘述。

驱动代码为【华清远见-嵌入式 ARM 实验箱资料- I -FS6818：程序源码\Linux 驱动实验源码\设备驱动例程】中的 fs6818_adc

1. 拷贝驱动代码到用户虚拟机工作目录下

```
$ cd /home/linux/workdir
$ cp /mnt/hgfs/share/fs6818_adc . -a
$ cd fs6818_adc
```



2. 修改完 Makefile 之后，编译驱动

```
$ make
```

```
$ make modules_install
```

3. 编译应用程序。

```
$ arm-linux-gcc adc_test.c -o adc_test
```

4. 拷贝应用程序到目标板文件系统。

```
$ cp adc_test /source/rootfs
```

5. 加载驱动，注意以下都是在目标板上操作。

```
# insmod /lib/modules/3.4.39-farsight/extra/adc_driver.ko
```

6. 执行测试程序。

```
# ./adc_test
```

输入 P（即选择通道 0）进行电位器的测试。

```
[root@farsight]# ./adc_test
*****
[11647.601000] <adc> register init successfully
please change Sensor type:
A: alcohol sensor(switch AD1)
L: light sensor(switch AD2)
S: smoke sensor(switch AD4)
F: flame sensor(switch AD3)
P: potentiometer(no switch)
*****
input type:P
Vol: 0.33V
```

14.6 I2C 总线驱动

I2C（Intel-Integrated Circuit）是由恩智浦公司开发的一种慢速两线制总线协议。最初总线的速率定为 100KHz，经过发展，后来的速率出现了 400KHz、3.4MHz、1MHz 和 5MHz 的不同等级，目前大多数器件都支持 400KHz。不过速率是可变的，比如一个支持 400KHz 的器件，完全可以工作在 299KHz 这个速率上，只要不超过 400KHz 即可。I2C 总线为那些不需要经常访问，低带宽的设备提供了一种廉价的互联方式。他被广泛地应用在嵌入式系统设备当中。不过 I2C 总线协议有商标上的一些限制，有些厂家为了避免这种限制，使用了另外一种总线协议 SMBus（System Management Bus）。虽然名字不同，但是 SMBus 总线协议其实是 I2C 总线协议的一个子集，绝大多数 I2C 设备都能够在 SMBus 总线上工作。现在的计算机主板上通常使用 SMBus 总线，最常见的就是内存条上的 EEPROM 配置芯片。

I2C 总线由 SCL（串行时钟线）和 SDA（串行数据线）两条线组成。上面连接有主机控制器（Master，Linux 驱动中称为 Adapter）和从设备（Slave，Linux 驱动中称为 Client）。所有的访问操作都是由主机控制器发起的，在同一条总线上可以有多个主机控制器，当多个主机控制器同时对设备发起访问时，由协议的冲突检测和仲裁机制来保证只有一个主机控制器对从设备进行访问。多个从设备是通过从设备的地址来区分的，地址分为 7 位地址和 10 位地址两种，常见的是 7 位地址。下图是具有两个主机控制器的 I2C 总线连接图，总线控制器由微控制器来充当，从设备有门阵列、LCD 驱动器、ADC 和



EEPROM。

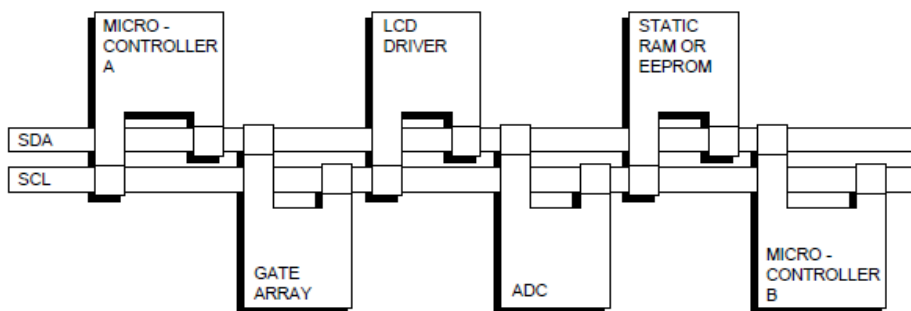


图 I2C 连接图

下面以图示方式来说明 I2C 的总线时序，时序图如下图所示。

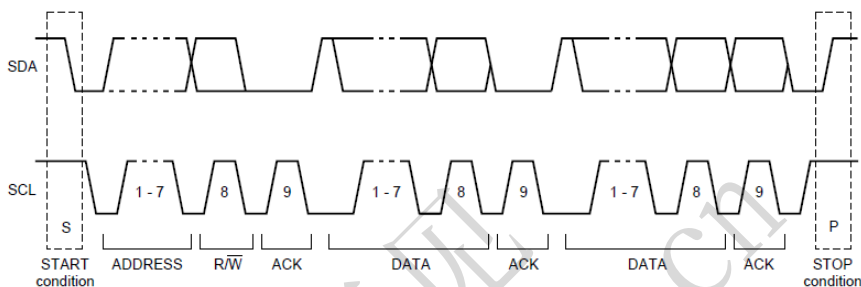


图 I2C 时序图

开始位：当 SCL 为高电平时，SDA 由高电平变为低电平的期间，上图中最左边的 START condition 所标记的区域，这表示主机控制器要开始对从机发起访问了。

地址位：接下来的 7 个时钟周期，主机控制器将会发送从机的 7 位地址（如果是 10 位地址需要分两次发送），如上图中 ADDRESS 所标记的区域。

读/写位：在第 8 个时钟周期，如果 SDA 为高电平则表示接下来要读取从机的数据，如果是低电平则表示主机要写数据到从机，如上图中 R/W 所标记的区域。

应答位：在第 9 个时钟周期由从机进行应答，低电平为 ACK，高电平为 NACK，如果从机响应，应该发 ACK。

数据位：在接下来的若干个周期内，主机可以持续读取数据（如果读/写位为读），或写数据（如果读/写位为写），每次数据传输完成（如时序图中的 DATA 所标记的区域），也要进行应答，是读则由主机控制器来应答，是写则由从机来应答，只是在主机读完最后一个字节的数据后应该以 NACK 来应答。

停止位：当 SCL 为高电平时，SDA 由低电平变为高电平的期间，如上图中最右边的 STOP condition 所标记的区域，这表示主机控制器结束了对从机的访问。

I2C 从设备内部通常有若干个寄存器，每个寄存器都有一个地址，对这些从设备的访问通常是顺序访问的。比如上次从寄存器 2 开始连续访问了 4 个寄存器，那么下一次访问将会从寄存器 6 开始。不过按照下两图的时序，可以实现随机的读和写访问。

Master	S	AD+W		RA		DATA		DATA		P
Slave			ACK		ACK		ACK		ACK	



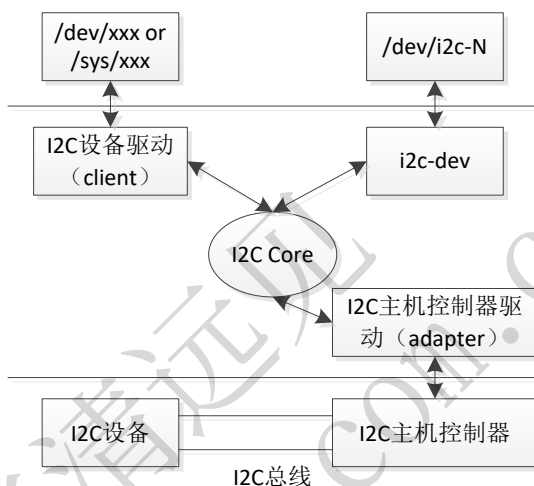
图 随机写访问

Master	S	AD+W		RA		S	AD+R			ACK		NACK	P
Slave			ACK		ACK			ACK	DATA		DATA		

图 随机读访问

随机写访问是在主机发送完从机地址和写标志位，从机应答后，主机继续写寄存器地址，如上图中的 RA 所示。当从机对写入的寄存器地址进行应答后，主机才写入真正的数据。对随机读访问则要复杂一些，在寄存器地址写入，从机应答后，主机需要再次发送开始位，然后发送从机地址和读标志位，之后才是读操作。

I2C 一般具有下图层次结构——



I2C 主机驱动：是 I2C 主机控制器的驱动，一般由 SoC 芯片厂商负责设计实现，用于控制 I2C 主机控制器发出时序信号。

I2C Core：为上层提供统一的 API 接口和对其他模块进行注册和注销等管理。

I2C 设备驱动：调用 I2C Core 提供的统一 API，根据 I2C 设备的访问规范，控制 I2C 主机控制器发出不同的时序信号，对 I2C 设备进行访问。该驱动称为内核层 I2C 设备驱动。

i2c-dev：将 I2C 主机控制器实现为一个字符设备，应用程序可以直接访问 /dev/i2c-N 来访问 I2C 主机控制器，从而对 I2C 设备发起访问，该应用程序称为应用层 I2C 设备驱动。

```

1 struct i2c_board_info __initdata zlg7290_i2c_bdi = {
2     I2C_BOARD_INFO("zlg7290", 0x70>>1);
3 };
  
```

i2c_board_info 用于描述某些信息已知的 I2C 设备，通常用 I2C_BOARD_INFO 宏来描述基本信息，包括驱动的名字和设备的地址。上面的代码定义了一个 i2c_board_info 的数组 mini2440_i2c_devs，里面有一个设备，其对应的 I2C 设备驱动的名字是 zlg7290，设备地址为 0x38，另外可以使用 platform_data 指定其他的一些设备数据，如芯片使用的 IO 资源以及中断资源等等。最后使用 i2c_register_board_info 将该结构体注册到 I2C 总线上。



```

1 struct i2c_board_info {
2     char type[I2C_NAME_SIZE];    /* 初始化 i2c_client.name */
3     unsigned short
4     flags;                        /* 初始化 i2c_client.flags */
5
6     unsigned short
7     addr;                         /* i2c_client.addr */
8     void *platform_data;          /* i2c_client.dev.platform_data */
9     struct dev_archdata
10    *archdata;                    /* i2c_client.dev.archdata */
11    int irq;                       /* i2c_client.irq */
12 };

```

i2c_client 是用来表示一个 I2C 从设备的数据结构，是属于在 Linux 驱动开发中的几个比较重要的数据结构

```

1 struct i2c_client {
2     unsigned short flags;
3     /* div., see below */
4     unsigned short addr;
5     /* chip address - NOTE: 7bit */
6     /* addresses are stored in the
7     */
8     /* _LOWER_ 7 bits
9     */
10    unsigned long timing;
11    /* parameters of timing */
12    char name[I2C_NAME_SIZE];
13    struct i2c_adapter *adapter;
14    /* the adapter we sit on */
15    struct i2c_driver *driver;
16    /* and our access routines */
17    struct device dev;
18    /* the device structure */
19    int irq; /* irq issued by device */
20    struct list_head detected;
21 };

```

i2c_client 和 i2c_board_info 两者都是描述一个 I2C 设备，但是它们使用的地方是不一样的。

i2c_client 应用于设备驱动的各个操作函数的参数中，用于指代当前操作的设备，而且它还包含了 i2c_adapter，指明了设备是挂载在哪个 I2C 总线上的。i2c_client 是系统动态创建的。i2c_board_info，用于在代码中静态声明 I2C 设备，这些声明往往放在 board 相关的文件中。board 的初始化函数会调用 i2c_register_board_info()，将声明的静态 I2C 设备注册到系统的中。这些信息是生成对应的 i2c_client 的重



要依据。

struct i2c_msg, 是 i2c 消息数据结构, 是 i2c 的通讯单位

```
1 struct i2c_msg {
2     __u16 addr;      /* slave address */
3     __u16 flags;
4
5     #define I2C_M_TEN      0x0010 /* this is a ten bit chip address */
6     #define I2C_M_RD      0x0001 /* read data, from slave to master */
7     #define I2C_M_NOSTART  0x4000 /* if I2C_FUNC_PROTOCOL_MANGLING */
8     #define I2C_M_REV_DIR_ADDR 0x2000 /* if I2C_FUNC_PROTOCOL_MANGLING */
9     #define I2C_M_IGNORE_NAK 0x1000 /* if I2C_FUNC_PROTOCOL_MANGLING */
10    #define I2C_M_NO_RD_ACK 0x0800 /* if I2C_FUNC_PROTOCOL_MANGLING */
11    #define I2C_M_RECV_LEN 0x0400 /* length will be first received byte */
12
13    __u16 len;        /* msg length */
14    __u8 *buf;        /* pointer to msg data */
15
16 };
```

i2c_transfer()函数用于进行 I2C 适配器和 I2C 设备之间的一组消息交互, 函数本身不具备驱动适配器物理硬件完成消息交互的能力, 它只是寻找到 i2c_adapter 对应的 i2c_algorithm, 并使用 i2c_algorithm 的 master_xfer()函数真正驱动硬件流程. 成功返回 0

C++ Code

```
1 int i2c_transfer(struct i2c_adapter * adap, struct i2c_msg *msgs, int num)
2 {
3     int ret;
4
5     if (adap->algo->master_xfer) {
6 #ifdef DEBUG
7         for (ret = 0; ret < num; ret++) {
8             dev_dbg(&adap->dev, "master_xfer[%d] %c, addr=0x%02x, "
9                 "len=%d\n", ret, msgs[ret].flags & I2C_M_RD ?
10                 'R' : 'W', msgs[ret].addr, msgs[ret].len);
11         }
12 #endif
13         down(&adap->bus_lock);
14         /* 主要就是这一句, 调用 master_xfer() */
15         ret = adap->algo->master_xfer(adap, msgs, num);
16         up(&adap->bus_lock);
17     }
```



```
18     return ret;
19     } else {
20         dev_dbg(&adap->dev, "I2C level transfers not supported\n");
21         return -ENOSYS;
22     }
23 }
```

14.6.1 MMA8451 三轴加速度传感器实验

【实验原理】

MMA8451 内置了运动和/或自由落体检测功能以及非常灵活的中断配置方案。运动检测经常被用于简单的提醒主处理器移动设备目前正在使用。这可以通过运动检测功能和/或飞思卡尔应用笔记 AN4071 中描述的冲击检测功能来完成。冲击检测功能，可以选择禁用高通滤波器，以达到与运动检测完全一样的效果。这就为用户提供了两个用于运动检测的独立可编程中断，如果需要的话。运动和自由落体检测是一个芯片内部集成的功能，使用中断机制，可以节省整个系统的功耗。此功能在移动/倾斜阈值或自由落体事件发生时唤醒主处理器时。嵌入式的运动检测功能允许用户启用和禁用不同的轴。当事件发生时，将以正或负加速度值给出的事件的方向为。结果：此功能避免了系统处理器连续读取 XYZ 数据并且不断运行的软件算法与阈值数据进行比较。

典型应用有电子指南针应用程序、静态方向检测（纵向/横向，上/下，左/右，后/前）、笔记本电脑，电子阅读器和笔记本电脑翻滚和自由落体检测、实时定向检测（虚拟现实和游戏 3D 用户位置反馈）、实时活动分析（计步器步数计数，硬盘自由落体检测，航位推算 GPS 备份）、便携式产品省电的运动检测（手机，PDA，GPS，游戏的自动休眠和自动唤醒）、冲击和振动监测（机电一体化补偿，运输和保修使用记录）

MMA8451 器件电源通过 VDD 线供电。电源去耦电容器（100 nF 陶瓷加上 4.7 μ F 散装，或一个 4.7 μ F 陶瓷）应尽可能靠近器件的引脚放置。控制信号 SCL，SDA 和 SA0 不能容忍高于 VDDIO + 0.3V 的电压。如果 VDDIO 被移除，控制信号 SCL，SDA 和 SA0 将钳位任何逻辑信号与他们的内部 ESD 保护二极管。两个中断引脚（INT1 和 INT2）的功能，阈值和时序可通过 I2C 进行用户编程接口。SDA 和 SCL I2C 连接是漏极开路的，因此需要使用上拉电阻，

该传感器通过 I2C 进行通信，因此您可以在同一个两个 I2C 引脚上与一堆其他传感器共享该传感器。有一个地址选择引脚，所以你可以让加速度计共享一个 I2C 总线。请注意，此芯片需要重复启动 I2C 支持。

MMA8451 寄存器：

- 1、状态寄存器 STATUS/F_STATUS 地址 0x00 2.
- 2、控制寄存器 CTRL_REG1，地址 0x2a

7	6	5	4	3	2	1	0
ASLP_RATE1	ASLP_RATE0	DR2	DR1	DR0	LNOISE	F_READ	ACTIVE

ASLP_RATE[1:0]: 设置睡眠模式采样频率，默认 00



DR[2:0]: 数据速率选择。默认为 000,800Hz

DR2	DR1	DR0	数据速率 Hz	周期 ms
0	0	0	800	1.25
0	0	1	400	2.5
0	1	0	200	5
0	1	1	100	10
1	0	0	50	20
1	0	1	12.5	80
1	1	0	6.25	160
1	1	1	1.56	640

注意：当设备在睡眠模式下，数据输出速率以 ASLP_RATE[1:0]为准，DR 设置速率无效。

LNOISE :

3、XYZ 数据设置寄存器 XYZ_DATA_CFG，地址 0x0e

7	6	5	4	3	2	1	0
0	0	0	HPF_OUT	0	0	FS1	FS0

HPF_OUT: 高通滤波器数据输出使能。

HPF_OUT=1: 使能;

HPF_OUT=0: 禁止。

FS1:FS0: XYZ 量程设置。

如下表所示:

FS1	FS0	量程
0	0	2g
0	1	4g
1	0	8g
1	1	保留

【实验环境】

代码编写编译环境: ubuntu

驱动运行环境: FS6818 试验箱平台

【实验目的】

掌握 mma8451 三轴传感器的工作原理，掌握 Linux 内核中 i2c 类设备驱动基本编写方法。



【实验步骤】

系统源码准备、拷贝驱动代码到已有共享目录、修改 Makefile 请参考“简单的内核模块实验”，这里不再赘述。

驱动代码为【华清远见-嵌入式 ARM 实验箱资料- I -FS6818：程序源码\Linux 驱动实验源码\设备驱动例程】中的 fs6818_gsensor

1. 拷贝驱动代码到用户虚拟机工作目录下

```
$ cd /home/linux/workdir
$ cp /mnt/hgfs/share/fs6818_gsensor . -a
$ cd fs6818_gsensor
```

2. 修改完 Makefile 之后，编译驱动

```
$ make
$ sudo make modules_install
```

3. 编译应用程序。

```
$ arm-linux-gcc gsensor_test.c -o gsensor_test
```

4. 拷贝应用程序到目标板文件系统。

```
$ cp gsensor_test /source/rootfs
```

5. 加载驱动，注意以下都是在目标板上操作。

```
# insmod /lib/modules/3.4.39-farsight/extra/fs6818_gsensor.ko
```

6. 执行测试程序。

```
# ./gsensor_test
```

执行完测试程序后串口会打印当前三轴状态，如下图所示：

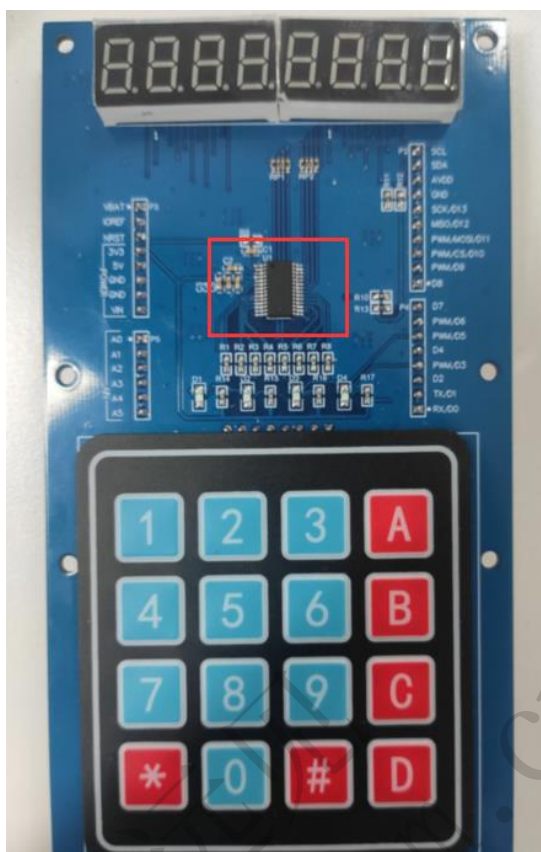
```
[root@farsight]# ./gsensor_test
-----
|      mma8451 test      |
|-----|
mma8451 data: x: -32 y: 688 z: 16008
mma8451 data: x: -32 y: 656 z: 16240
mma8451 data: x: -48 y: 640 z: 16176
```

14.6.2 ZLG7290 驱动开发

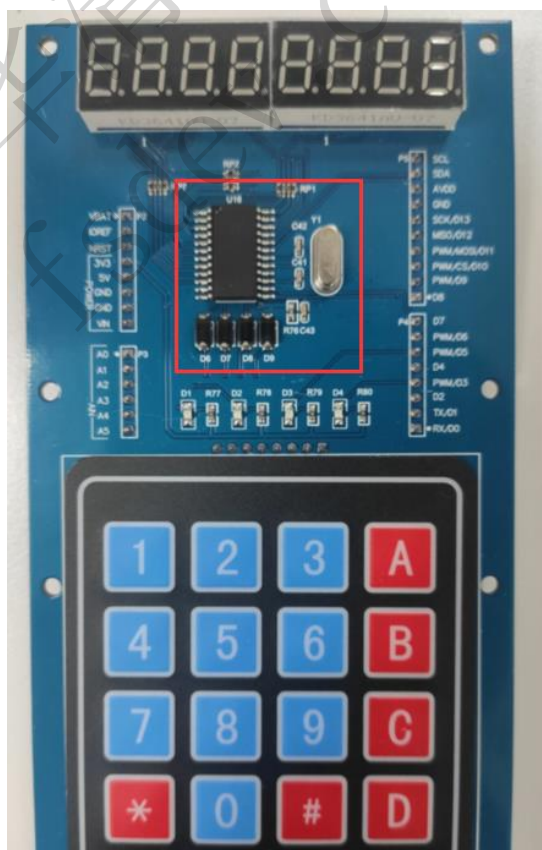
该驱动是针对于 FS6818M4 平台，其它配备 Arduino 按键数码管板的平台无法进行该实验。

说明：对于 2022.2.7 号之前购买的产品使用本章节 zlg7290 驱动开发，在此之后购买的产品使用下一章节 zlg72128 驱动开发。两者区别如下所示

zlg72128:



zlg7290:





【实验原理】

在智能仪表中，经常会用到键盘、数码管等外设。因此，一个稳定、占用系统资源少的人机对话通道设计非常重要。传统的键盘与数码管解决方案，由于键盘与数码管是分离的，因而电路连接比较复杂，不管是独立式键盘还是矩阵式键盘，都会浪费微控制器的端口资源，而且都需要人为进行去抖动处理，且抗干扰性差。而数码管部分，不管是静态显示方式还是动态显示方式，在不进行锁存器扩展的前提下。仍然要占用 8 根 I/O 端口线，这将严重浪费系统的端口资源。

ZLG7290B 是广州周立功单片机发展有限公司自行设计的数码管显示驱动及键盘扫描管理芯片。能够直接驱动 8 位共阴式数码管（或 64 只独立的 LED），同时还可以扫描管理多达 64 只按键。其中有 8 只按键还可以作为功能键使用，就像电脑键盘上的 Ctrl、Shift、Alt 键一样。另外 ZLG7290B 内部还设置有连击计数器，能够使某键按下后不松手而连续有效。采用 I2C 总线方式，与微控制器的接口仅需两根信号线。该芯片为工业级芯片，抗干扰能力强，在工业测控中已有大量应用。

ZLG7290B 可以扫描管理多达 64 个按键，K1~K56 为普通按键，F0~F7 为功能键。普通按键还有连击检测功能。ZLG7290B 内部有 8 个显示缓冲寄存器 DpRam0~DpRam7，（本实验采用的是直接写显示缓冲寄存器，它们地址分别是 10H~17H）它们直接决定数码管显示的内容。ZLG7290B 提供有两种显示控制方式，一种是直接向显存写入字型数据，另一种是通过向命令缓冲寄存器写入控制指令实现自动译码显示。访问这些寄存器需要通过 I2C 总线接口来实现。ZLG7290B 的 I2C 总线器件地址是 70H（写操作）和 71H（读操作）。访问内部寄存器要通过“子地址”来实现。

在这里介绍本实验用到的几个寄存器作用：

系统寄存器 SystemReg（地址：00H）

系统寄存器的第 0 位（LSB）称作 KeyAvi，标志着按键是否有效，0—没有按键被按下，1—有某个按键被按下。SystemReg 寄存器的其它位暂时没有定义。当按下某个键时，ZLG7290B 的 INT 引脚会产生一个低电平的中断请求信号。当读走键值后，中断信号就会自动撤销。而 KeyAvi 也同时予以反映。正常情况下，微控制器只需要判断 INT 引脚就可以了。通过不断查询 KeyAvi 位也能判断是否有键按下，这样就可以节省微控制器的一根 I/O 口线，但是代价是 I2C 总线处于频繁的活动状态，多消耗电流并且不利于抗干扰。（本节采用外部中断判断 INT 引脚，不读该寄存器）

键值寄存器 Key（地址：01H）

如果某个普通键（图 3.1 中的 K1~K56）被按下，则微控制器可以从键值寄存器 Key 中读取相应的键值 1~56。如果微控制器发现 ZLG7290B 的 INT 引脚产生了中断请求，而从 Key 中读到的键值是 0，则表示按下的可能是功能键。键值寄存器 Key 的值在被读走后自动变成 0。

zlg7290 设备信息声明在平台代码中，如下是设备信息以及注册方法。

设备信息描述：



```

1 #include <linux/i2c.h>
2
3 #define ZLG7290_I2C_BUS      (2)
4
5 /* CODEC */
6 static struct i2c_board_info __initdata zlg7290_i2c_bdi = {
7     I2C_BOARD_INFO("zlg7290", 0x70>>1),
8 };
9

```

设备注册：

```

1 i2c_register_board_info(ZLG7290_I2C_BUS, &zlg7290_i2c_bdi, 1);

```

【实验环境】

代码编写编译环境：ubuntu

驱动运行环境：FS6818M4 试验箱平台

【实验目的】

掌握周立功矩阵键盘的工作原理，掌握 Linux 内核中 i2c 类设备驱动基本编写方法。

【实验步骤】

系统源码准备、拷贝驱动代码到已有共享目录、修改 Makefile 请参考“简单的内核模块实验”，这里不再赘述。

驱动代码为【华清远见-嵌入式 ARM 实验箱资料- I -FS6818：程序源码\Linux 驱动实验源码\设备驱动例程】中的 zlg7290

1. 拷贝驱动代码到用户虚拟机工作目录下

```

$ cd /home/linux/workdir
$ cp /mnt/hgfs/share/zlg7290 . -a
$ cd zlg7290

```

2. 修改完 Makefile 之后，编译驱动

```

$ make
$ sudo make modules_install

```

3. 编译应用程序。

```

$ arm-linux-gcc zlg7290_test.c -o zlg7290_test

```



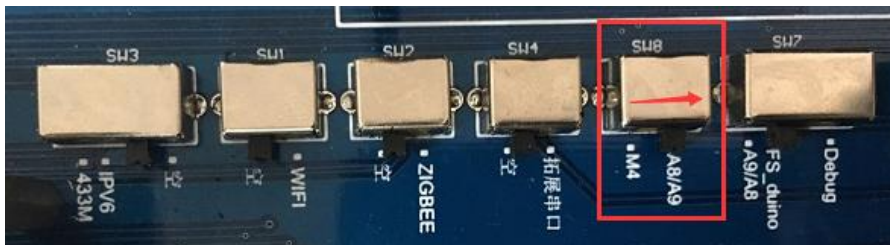
4. 拷贝应用程序到目标板文件系统。

```
$ cp zlg7290_test /source/rootfs
```

5. 加载驱动，注意以下都是在目标板上操作。

```
# insmod /lib/modules/3.4.39-farsight/extra/zlg7290.ko
```

6. 调整试验箱拨码，因为在硬件设计时，将 SW8 作为 MCU 和 CPU 的 i2c 主机切换开关。



7. 执行测试程序。

```
# ./zlg7290_test
```

按下 4*4 键盘上的任意键，数码管会显示该按键值，按 “*” 键可以清楚当前数码管显示的数值。

14.6.3 ZLG72128 驱动开发

该驱动是针对 FS6818M4 平台，其它配备 Arduino 按键数码管板的平台无法进行该实验。

【实验原理】

在智能仪表中，经常会用到键盘、数码管等外设。因此，一个稳定、占用系统资源少的人机对话通道设计非常重要。传统的键盘与数码管解决方案，由于键盘与数码管是分离的，因而电路连接比较复杂，不管是独立式键盘还是矩阵式键盘，都会浪费微控制器的端口资源，而且都需要人为进行去抖动处理，且抗干扰性差。而数码管部分，不管是静态显示方式还是动态显示方式，在不进行锁存器扩展的前提下。仍然要占用 8 根 I/O 端口线，这将严重浪费系统的端口资源。

ZLG72128 是广州周立功单片机科技有限公司自行设计的数码管显示驱动及键盘扫描管理芯片。能够直接驱动 12 位共阴式数码管（或 96 只独立的 LED），同时还可以扫描管理多达 32 只按键。其中有 8 只按键还可以作为功能键使用，就像电脑键盘上的 Ctrl、Shift、Alt 键一样。另外 ZLG72128 内部还设置有连击计数器，能够使某键按下后不松手而连续有效。采用 I2C 总线方式，与微控制器的接口仅需两根信号线。该芯片为工业级芯片，抗干扰能力强，在工业测控中已有大量应用。

ZLG72128 可以扫描管理多达 32 个按键，K1~K24 为普通按键，F0~F7 为功能键。普通按键有连击检测功能。ZLG72128 内部有 12 个显示缓冲寄存器 DispBuf0~DispBuf11，它们直接决定数码管显示的内容。ZLG72128 提供有两种显示控制方式，一种是直接向显存写入字型数据，另一种是通过向命令缓冲寄存器写入控制指令实现自动译码显示。

访问这些寄存器需要通过 I2C 总线接口来实现。ZLG72128 的 I2C 总线器件地址是 60H（写操作）和 61H（读操作）。访问内部寄存器要通过“子地址”来实现。

在这里介绍本实验用到的几个寄存器作用：



系统寄存器 SystemReg（地址：00H）

系统寄存器的第 0 位（LSB）称作 KeyAvi，标志着按键是否有效，0—没有按键被按下，1—有某个按键被按下。SystemReg 寄存器的其它位暂时没有定义当按下某个键时，ZLG72128B 的 INT 引脚会产生一个低电平的中断请求信号。当读走键值后，中断信号就会自动撤销。而 KeyAvi 也同时予以反映。正常情况下，微控制器只需要判断 INT 引脚就可以了。通过不断查询 KeyAvi 位也能判断是否有键按下，这样就可以节省微控制器的一根 I/O 口线，但是代价是 I2C 总线处于频繁的活动状态，多消耗电流并且不利于抗干扰。（本节采用外部中断判断 INT 引脚，不读该寄存器）

键值寄存器 Key（地址：01H）

如果某个普通键（图 3.1 中的 K1~K24）被按下，则微控制器可以从键值寄存器 Key 中读取相应的键值 1~24。如果微控制器发现 ZLG72128 的 INT 引脚产生了中断请求，而从 Key 中读到的键值 0，则表示按下的可能是功能键。键值寄存器 Key 的值在被读走后自动变成 0。

zlg72128 设备信息声明在平台代码中，如下是设备信息以及注册方法。

设备信息描述：

```
1#include <linux/i2c.h>
2
3#define ZLG72128_I2C_BUS      (2)
4
5/* CODEC */
6static struct i2c_board_info __initdata zlg7290_i2c_bdi = {
7    I2C_BOARD_INFO("zlg72128", 0x60>>1),
8
9};
```

设备注册：

```
1    i2c_register_board_info(ZLG72128_I2C_BUS, &zlg72128_i2c_bdi, 1);
```

【实验环境】

代码编写编译环境：ubuntu

驱动运行环境：FS6818M4 试验箱平台

【实验目的】

掌握周立功矩阵键盘的工作原理，掌握 Linux 内核中 i2c 类设备驱动基本编写方法。



【实验步骤】

系统源码准备、拷贝驱动代码到已有共享目录、修改 Makefile 请参考“简单的内核模块实验”，这里不再赘述。

驱动代码为【华清远见-嵌入式 ARM 实验箱资料- I -FS6818：程序源码\Linux 驱动实验源码\设备驱动例程】中的 zlg72128

8. 拷贝驱动代码到用户虚拟机工作目录下

```
$ cd /home/linux/workdir
$ cp /mnt/hgfs/share/zlg72128 . -a
$ cd zlg72128
```

9. 进入 dri 目录下，修改完 Makefile 之后，编译驱动

```
$ make
$ sudo make modules_install
```

10. 进入上级目录下的 app 目录下，编译应用程序。

```
$ make
```

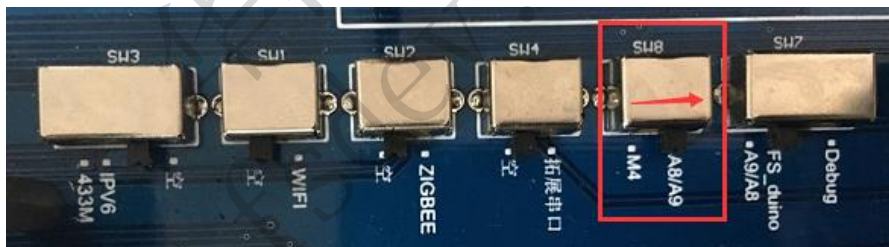
11. 拷贝应用程序到目标板文件系统。

```
$ cp zlg72128_key_test /source/rootfs
```

12. 加载驱动，注意以下都是在目标板上操作。

```
# insmod /lib/modules/3.4.39-farsight/extra/ key-zlg72128.ko
```

13. 调整试验箱拨码，因为在硬件设计时，将 SW8 作为 MCU 和 CPU 的 i2c 主机切换开关。



14. 执行测试程序。

```
# ./zlg72128_key_test
```

按下 4*4 键盘上的任意键，数码管会显示该按键值，按“#”键可以退出程序。